
Saturn 汇编语言 入门

作者

Gilbert Fernandes

编辑与维护

Eric Rechlin

作者

Gilbert Henri Fernandes	Eric Rechlin
1 rue des Gâte Ceps	3212 Winnipeg Drive
F-92210 Saint-Cloud	Bismarck, ND 58503-0453
France	USA
gilbert.fernandes@wanadoo.fr	eric@hpcalc.org
ICQ: 78327407	ICQ: 783944

本书及书中所附示例均为自由作品。您可以为自己或任何其他人打印本书。只要不改动对原作者的署名，即可用任何形式或任何方式传播或复制本书。

用直白的话说：本书免费，请广泛传播！只需确保注明我们的作者身份。

Hewlett-Packard 和 HP 是 Hewlett-Packard Company 的注册商标。

第 3 版：2005 年 7 月 16 日

Copyright 1999-2005 Gilbert Fernandes and Eric Rechlin

请自由分发给所有拥有 HP 计算器的朋友。

前言

汇编语言与大多数其他编程语言非常不同。学习一种常见的编程语言时，最初几课通常会讲解如何编写一个简单的“Hello World”程序。但是，对汇编语言来说，事情没有这么简单。在编写任何汇编语言代码之前，首先必须理解处理器的工作原理，以及操作系统如何与处理器协同工作。此外，即便只是完成向屏幕输出这样简单的任务，也必须学习许多命令。

在本书中，您要先读完四十页介绍性材料，才会真正拿起计算器。不过不必担心，您很快就能开始编程。了解基本概念之后，您将从一个极其简单的程序开始，分成若干小组逐步学习汇编语言命令，也就是“指令”。

由于机器码采用二进制表示，您将学习用来代表这些二进制代码的汇编语言助记符。本书面向所有 HP 48 和 HP 49 系列计算器的用户。我们采用 MASD 语法，因为 49 系列内置 MASD，而 48GX 的 Meta Kernel 也附带 MASD。对于没有 MASD 的用户，可以使用一个名为 HP-ASM 的小型汇编器；它兼容本书采用的 MASD 语法有限子集。这种汇编器语法相当容易掌握，PC 上的 HP Tools 也支持它。Jazz 和 HP Tools 支持的 HP 汇编语言语法与此非常相似，因此两者之间应当可以轻松切换，不过本书不讨论具体差异。

我们认为，能够学习汇编语言的人远比现在更多。这个领域需要更多程序员，而最大的障碍在于：汇编语言编程资料，尤其是 Saturn 汇编资料，实在太难获得。大多数优秀的 Saturn 汇编语言教程都是法文的；唯一一本英文教程是 James Donnelly 的《An Introduction to HP 48 System RPL and Assembly Language Programming》。那是一本很出色的系统 RPL 著作，但其中的汇编语言部分对大多数人来说推进得太快。我们希望本书能够填补 HP 48 和 49 现有文档中的这一空白。

尽管我们算不上经验丰富的程序员，仍愿意分享自己的知识。我们也依然把自己看作新手，因此本书并不是一部定论式的汇编语言入门书。如果您发现任何错误，请不要责骂我们，而请发邮件到下列地址。有些人，例如 49 的三位开发者 Jean-Yves Avenard、Cyrille de Brébisson 和 Gerald Squelart，拥有多年的编程经验，但您不必因此畏缩。每个人都曾经是新手。

本书从处理器如何工作的最基础内容讲起，包括二进制和十六进制教程，最终进入复杂例程的编写。它既可作为面向初学者的入门手册，这些读者可能完全不了解计算机底层运行方式；也可作为经验丰富的程序员使用的参考手册。

我们为本书投入了非常多的时间。如果您遇到问题，欢迎给我们发邮件；我们会尽力提供帮助，并在需要时补充教程。这个在线版本免费提供，请在所有 HP 计算器爱好者中广泛传播。

此外，我们还要感谢下列人士对本书所作的贡献（按字母顺序排列）：Arnaud Amiel、Jean-Yves Avenard、Cyrille de Brébisson、Christoph Giesselink、Chris Griffiths、Eduardo Kalinowski、Jan Kozicki、John H Meyers、Aldiney Oliveira、Mark Power、Gerald Squelart 和 Richard Steventon。

尤其感谢 Christoph Giesselink 和 Eduardo Kalinowski 的帮助，因为他们撰写了完整的章节内容。

本书更新可在 <http://www.hpcalc.org> 获取。

恳请您将反馈发送到我们的电子邮件地址：

Gilbert Fernandes <gilbert.fernandes@wanadoo.fr>

Eric Rechlin <eric@hpcalc.org>

目录

前言	3
第一部分 编程概念	10
1 用户 RPL 与系统 RPL 简介	10
2 汇编语言与机器语言	10
3 二进制与半字节	11
4 从二进制转换为十进制	12
5 最高有效位与最低有效位	13
6 小数	13
7 使用二进制进行计算	14
7.1 加法	14
7.2 减法	14
7.3 负数	14
7.3.1 有符号二进制：第一种方法	14
7.3.2 补码：第二种方法	15
7.4 乘法	16
7.5 除法	17
8 十六进制	17
9 从十六进制转换为十进制	18
10 使用十六进制书写四位一组的数据（半字节）	19
11 真的需要做完这些数值运算吗？	19
12 BCD：“二进制编码的十进制”	20
13 将十进制数转换为 BCD 形式	20
14 普通 BCD 与“扩展 BCD”	21
15 使用 BCD 进行计算	21
15.1 半字节溢出时	21
15.2 需要传递进位时	21
16 逻辑运算	21
16.1 逻辑 OR	22
16.2 逻辑 AND	22
16.3 取反	23
16.4 NOT OR（又称 NOR）	23
16.5 AND NOT（又称 NAND）	23
16.6 Exclusive OR（又称 XOR）	23
17 复习：数制	24
第二部分 编程工具	24
18 使用 HP 48	24
18.1 汇编器	24
18.2 HP-ASM 命令	24
18.3 使用 HP-ASM 汇编代码	25
18.4 HP-ASM 的其他功能	25
18.4.1 链接器	25
18.4.2 指令控制符	26
18.4.3 宏	26
18.5 高速文本编辑器	26
19 使用 HP 49	27
19.1 汇编器	27
19.2 开发库命令	27
19.3 文本编辑器	28
19.4 使用 MASD 汇编代码	28
19.5 MASD 标志设置	29
19.6 MASD 的其他功能	29
19.6.1 链接器	29
19.6.2 指令控制符	29

19.6.3 入口点表.....	30
19.6.4 MASD 高级功能	30
19.7 其他工具	30
20 使用 PC	31
第三部分 Saturn 处理器.....	31
21 Saturn 寄存器.....	31
21.1 处理器寄存器	32
21.1.1 指针寄存器	32
21.1.2 计算字节和 KB.....	32
21.1.3 可寻址空间	33
21.1.4 工作寄存器	33
21.1.5 保存寄存器	33
22 工作寄存器的字段	33
22.1 半字节顺序反转	34
22.2 可用字段	34
22.2.1 W 字段	34
22.2.2 A 字段	35
22.2.3 B 字段.....	35
22.2.4 X 字段.....	35
22.2.5 XS 字段.....	35
22.2.6 M 字段.....	35
22.2.7 S 字段	35
22.2.8 字段汇总.....	35
23 P 寄存器和 WP	36
24 用户定义字段.....	36
25 处理器内部数值的环回处理	36
26 RSTK.....	37
27 标志：ST、HST 和 Carry.....	37
27.1 Carry	37
27.2 ST（状态位）	38
27.3 HST（硬件状态位）	38
28 OUT 和 IN	38
29 处理器如何工作：一条指令的生命周期.....	38
30 什么是处理器周期？	39
31 复习：计算	39
第四部分 Saturn 指令集.....	40
32 入口点及其他说明	40
33 工作寄存器指令	40
33.1 向寄存器装入数值	40
33.1.1 LA、LC.....	41
33.1.2 把寄存器置零	41
33.1.3 改变某一位的值.....	43
33.1.4 交换两个寄存器.....	45
33.1.5 把一个寄存器复制到另一个寄存器	45
33.2 使用寄存器进行运算	46
33.2.1 递增寄存器	46
33.2.2 循环中进位位的特殊之处	47
33.2.3 两个寄存器相加.....	47
33.2.4 寄存器与自身相加	48
33.2.5 给寄存器加常数.....	48
33.2.6 递减寄存器	50
33.2.7 两个寄存器相减.....	50
33.2.8 两个寄存器相乘.....	51
33.2.9 两个寄存器相除.....	52
33.2.10 两个寄存器取模.....	52
33.2.11 减去常数.....	53

33.2.12 反码	53
33.2.13 补码	54
33.2.14 逻辑运算	55
33.2.14.1 逻辑 OR, 也称“逻辑和”	55
33.2.14.2 逻辑 AND, 也称“逻辑积”	56
33.2.14.3 逻辑 XOR	56
33.2.15 左移或右移一个半字节	57
33.2.16 左旋或右旋一个半字节	57
33.2.17 向右移动一位	58
33.2.17.1 在字段内向右移动一位	58
33.2.18 向左移位	58
33.2.19 向右移位	59
34 保存寄存器	60
34.1 把工作寄存器保存到保存寄存器	60
34.2 在指定字段内把工作寄存器保存到保存寄存器	60
34.3 把保存寄存器复制到工作寄存器	61
34.4 在指定字段内把保存寄存器复制到工作寄存器	61
34.5 在保存寄存器与工作寄存器之间交换	62
34.6 在指定字段内交换保存寄存器与工作寄存器	62
35 指针	63
35.1 给 D0 或 D1 赋值	63
35.2 给 D0 或 D1 加值或减值	63
35.3 把 A 或 C 复制到 D0 或 D1	64
35.4 在 A/C 的 A 字段与 D0/D1 之间交换	64
35.5 在 A/C 与 D0/D1 之间交换最低有效的 4 个半字节	68
36 在字段范围内读取内存	68
37 使用数值指定读取长度	69
38 在字段范围内写入内存	69
39 使用数值指定写入长度	69
40 跳转与测试	70
41 相对、绝对和间接跳转	70
41.1 条件跳转: GOC 和 GONC	70
41.2 无条件跳转: GOTO、GOLONG、GOVLNG	71
41.2.1 GOTO	71
41.2.2 GOLONG	71
41.2.3 GOVLNG	72
41.2.4 无条件跳转到 A 或 C 中的地址	72
41.2.5 带交换的无条件跳转	72
41.2.6 无条件间接跳转	72
41.2.7 保存 PC 内容	72
42 调用子例程	72
42.1 GOSUB	73
42.2 GOSUBL	73
42.3 GOSBVL	73
42.4 从子例程返回	73
42.5 根据进位位返回	73
43 测试	73
43.1 与 0 比较	74
43.1.1 是否等于 0?	74
43.1.2 是否不等于 0?	74
43.2 两个寄存器是否相等	74
43.3 两个寄存器是否不相等	75
43.4 小于和大于	75
43.5 小于等于和大于等于	75
43.6 测试某一位	76
44 使用 P 寄存器	76

44.1 P= n	77
44.2 P=P+1	77
44.3 P=P-1	77
44.4 ?P# n.....	77
44.5 ?P= n.....	77
44.6 C=P n.....	77
44.7 P=C n.....	77
44.8 C+P+1	77
44.9 CPEX n.....	77
45 RSTK 栈	77
46 IN 和 OUT 寄存器	77
47 状态位 ST	78
47.1 CLRST.....	78
47.2 C=ST	78
47.3 ST=C	78
47.4 CSTEX	78
47.5 ST=1 n.....	78
47.6 ST=0 n.....	79
47.7 ?ST=0 n 和 ?ST=1 n	79
48 硬件状态位 HST	79
49 Saturn 的 DEC/HEX 模式.....	80
50 中断	80
50.1 禁用全部中断	80
50.2 启用全部中断	81
51 总线相关指令.....	81
51.1 RESET	81
51.2 SREQ?.....	81
51.3 CONFIG	82
51.4 UNCNFG	82
51.5 C=ID	82
51.6 SHUTDN	82
51.7 BUSCx.....	82
52 填充指令	82
53 ARM 系统指令.....	82
54 ARM 增强指令.....	84
第五部分 对象	84
55 对象概述.....	84
56 简单对象.....	85
56.1 实数.....	85
56.2 字符串	86
56.3 全局名称	89
56.3.1 空全局名称与隐藏目录.....	91
56.4 局部名称	92
56.5 用户二进制整数	92
56.6 图形对象	93
56.7 XLIB 名称.....	94
56.8 系统二进制整数	94
56.9 长实数	94
56.10 字符	95
56.11 代码对象	95
56.12 库数据.....	95
56.13 扩展指针	96
56.14 无限精度整数（仅 HP 49）	96
56.15 闪存指针（仅 HP 49）	97
56.16 小字体（仅 HP 49）	97
56.17 字体（仅 HP 49）	97

57 复合对象	98
57.1 复数	98
57.2 数组	98
57.3 列表	99
57.4 RPL 程序	100
57.5 代数表达式	101
57.6 标记对象	102
57.7 单位对象	102
57.8 目录对象	103
57.8.1 HOME 目录	103
57.8.2 子目录	104
57.9 库	105
57.9.1 哈希表	106
57.9.2 消息表	107
57.9.3 链接表	107
57.10 备份	107
57.11 长复数	108
57.12 链接数组	108
57.13 无限精度实数（仅 HP 49）	109
57.14 无限精度复数（仅 HP 49）	110
57.15 符号矩阵（仅 HP 49）	111
58 原始代码对象	111
第六部分 编写程序	111
59 实现循环	111
60 从键盘读取输入	112
60.1 检查一次按键	113
60.2 检查组合按键	114
60.3 读取整个键盘	114
60.4 读取 [ON] 键	115
61 操作堆栈	117
61.1 Drop	118
61.2 Swap	118
62 示例程序：数字之和	119
63 示例程序：完全平方数	120
64 创建用于生成宏的程序	122
64.1 →MACRO.S	122
64.2 本程序使用的 ROM 例程	124
64.2.1 #5023h，也称 Errjmp 或 ERROR_A	124
64.2.2 #679Bh，也称 SAVPTR 或 SAVE_REG	124
64.2.3 #3019h，也称 SKIPOB（Skip Object）	125
64.2.4 #05B7Dh，也称 MAKE\$N 或 RES_STR（RESeRve STRing）	125
64.2.5 #0670Ch，也称 MOVEDOWN 或 COPY DOWN	125
64.2.6 #067D2h，也称 GETPTR 或 GET_REG	126
65 更多内置 ROM 例程	126
66 HP 48 内存管理	127
66.1 菊花链基础	127
66.2 设备配置	127
66.3 RESET、CONFIG、UNCNFG 和 C=ID	128
66.3.1 RESET	128
66.3.2 CONFIG	128
66.3.2.1 大小	128
66.3.2.2 地址	128
66.3.3 UNCNFG	129
66.3.4 C=ID	129
66.4 RESET 后的 CONFIG	130
66.5 UNCNFG 示例	131

66.6 默认配置设置	132
66.6.1 HP 48S/SX 内存配置	132
66.6.2 HP 48G 内存配置	132
66.6.3 HP 48G+ 和 GX（无 RAM 卡）内存配置	132
66.6.4 插槽 1 中有 32KB 时的 HP 48GX 内存配置	133
66.6.5 插槽 1 中有 128KB 时的 HP 48GX 内存配置	133
66.6.6 插槽 1 和插槽 2 中均有 128KB 时的 HP 48GX 内存配置	133
66.7 GX 存储体切换器及其他 ROM 内容	133
66.7.1 在 A19 与 NCE3 之间切换	134
66.7.2 选择存储体	134
66.7.3 插槽 2 中使用 4MB 卡时的无效卡数据	134
67 HP 49 内存管理	136
67.1 概述	136
67.2 硬件细节	137
67.3 A19 与 NCE3 共用	137
67.4 内存控制器	137
67.4.1 NCE1（闪存芯片）	137
67.4.2 HDW	137
67.4.3 NCE2（RAM）	137
67.4.4 CE1（存储体切换器）	137
67.4.5 CE2（RAM）	138
67.4.6 NCE3（RAM / 闪存芯片）	138
67.5 恢复存储体切换器的内容	139
68 图形	139
68.1 概述	139
68.2 主显示区 / 菜单显示区	140
68.3 主显示区域设置	140
68.4 菜单显示区域设置	142
68.5 显示 I/O 寄存器概览	142
#00100h =BITOFFSET（R/W）	142
#00101 =CONTRAST（R/W）	143
#00102 =DTEST（R/W）	143
#00103 =DSPCTL（W/O）	143
#0010B =ANNCTRL（R/W）	143
#0010C =ANNCTRL+1（R/W）	143
#00120 =DISP1CTL（W/O）	143
#00123 =LINENIBS（W/O）	143
#00128 =LINECOUNT（W/O）	143
#00128 =LINECOUNT（R/O）	143
#00129 =LINECOUNT+1（W/O）	144
#00129 =LINECOUNT+1（R/O）	144
#00130 =DISP2CTL（W/O）	144
附录	144
69 延伸阅读	144
69.1 内部开发工具	144
69.2 汇编语言	144
69.3 System RPL	145
69.4 User RPL	145
70 索引	146

第一部分 编程概念

1 用户 RPL 与系统 RPL 简介

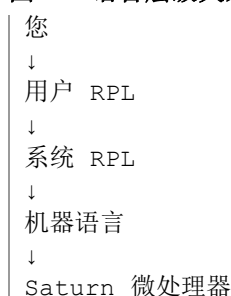
HP 48 或 49 的普通用户可以使用用户 RPL。它容易学习，也是您在 HP 上可以使用的第二种编程语言。

每次执行用户 RPL 命令时，系统都会检查参数，因此它是一种相当安全的语言。使用它时，通常不会丢失任何数据。

如果查看 HP 的内部软件，就会发现每条用户 RPL 命令实际上都被分解为若干更简单的命令；每条命令通常只做一件简单的事情。用户 RPL 在内部被分解为这些基本命令，我们称之为系统 RPL。每条系统 RPL 命令最终都会调用机器码，而机器码是 HP 的处理器 Saturn 唯一能够直接执行的语言。

为了帮助您理解，请看下面的层级关系：

图 1-1 语言层级关系



图例：向下箭头表示上一层最终调用或转换到下一层。

HP 48 为您提供三种开发语言。用户 RPL 安全、易用，但速度最慢。系统 RPL 更快，因为它不做检查；因此，您必须在开始时自行确保所有参数正确，但在随后把数据交给那些“饥饿”的命令时，就不再需要重复检查。最后是汇编语言，它生成机器语言。这是速度最快的代码，由处理器直接执行，当然也让您能够完全控制 HP。

HP 49 系列在最上层还多一种语言，即 HP Basic。它本质上是用户 RPL 的代数形式，但速度更慢。49G+ 和 48GII 在最下层还多一个层级，因为它们没有实体 Saturn 处理器，而是使用 ARM 处理器，并在 ARM 上运行模拟器来模拟 Saturn。可以绕过 Saturn 层，直接在 ARM 处理器上运行 ARM 机器语言，但这超出了本教程的范围。

如果您想学习系统 RPL，掌握用户 RPL 就已经走完了大部分路程。您还需要的是 ROM 中所有系统 RPL 命令的完整列表，这些命令称为“入口点”。

汇编语言与用户 RPL 或系统 RPL 没有直接关系，因此您将在这里学习真正不同的内容。汇编器也不具备系统 RPL 程序员可用的全部命令。

汇编语言本身并不复杂，因为它的命令很简单，只完成非常基础的操作。真正显得复杂的是：要完成比让 HP 死机更复杂的事情，您需要写下大量“基本命令”。

您得到的将是最快的代码，以及睡眠不足，不过这正是程序员的生活。

2 汇编语言与机器语言

Saturn 处理器与其他处理器一样，使用一连串 0 和 1 来工作。由于我们的大脑能力有限，无法直接理解这些序列。*

* Avenard 先生之所以厉害，是因为他通过外科手术把一颗 Saturn 处理器植入了大脑。另一种办法是像 Jazz 和 49G 中部分 CAS 的开发者 Mika Heiskanen 那样，吃掉大量 Saturn 处理器；不过相信我，想只吃一颗相当困难。

因此，我们会把这些 0 和 1 的序列组合起来，形成小的信息包。Saturn 处理器在设计上主要以四位一组的方式处理二进制信息，每个四位包称为一个半字节（nibble）。

稍后我们会进一步学习二进制和十六进制。在计算机上，您主要使用十六进制，不必担心。

因此，机器语言是一系列二进制数字（0 或 1），也就是“位”（bit，这个词由 binary 和 digit 缩合而来）；这些位按四位一组组合成半字节。

在内部，Saturn 处理器只能完成一些非常简单的事情，例如从内存读取若干半字节、写入半字节，以及用读出的半字节进行计算。对我们来说，每条基本指令都像一种命令。汇编语言程序员使用一种称为“助记符”的东西。助记符是什么？

“助记符”是赋予每条基本命令的名称。它非常简短，可以帮助我们记住用途（mnemosis → memory，因此称为 mnemonic）。

我们将使用由助记符构成的汇编语言编写程序。Saturn 能执行的每条命令都有相应的助记符。编译器会把这些助记符翻译成二进制信息，处理器则使用这些位来执行我们的命令。

请记住：

1. 机器语言是 Saturn 处理器唯一能够理解的语言。它由一串位（0 和 1）组成，过于复杂，因此我们不会直接使用它。
2. 我们把这些机器指令改写为简短词语，也就是助记符。Saturn 处理器的每条可用命令都有一个对应的助记符。

请务必不要混淆“汇编语言”和“机器语言”。后者是前者经编译器翻译后的结果形式。

这也正是汇编语言被称为“低级语言”的原因。在汇编语言中，每个助记符对应一条指令。因此，汇编语言是处理器专用的。编写汇编语言程序时，我们先在源文件中写入助记符；之所以称为“源”文件，是因为它是工作的源头。然后使用称为“汇编器”或“编译器”的程序，把助记符翻译为机器语言，也就是一串 0 和 1；在 HP 中，这些二进制数字按四位一组保存。

3 二进制与半字节

下面要做一点数学。别害怕，即使您不擅长数学，也仍然可以编写汇编语言。

如前所述，机器语言由一系列 0 和 1 组成。这是因为处理器内部使用的元件只存在两种状态：允许电子通过或不允许电子通过。在计算机领域，1 通常表示存在电压，0 通常表示不存在电压。

因此，Saturn 处理器把位保存成四位一组的数据包，也就是半字节。这一点必须记住。传统拼写是“nybble”，但近年来“nibble”更常见，所以本书采用字母 i 的拼写。

先从我们熟悉的十进制说起。十进制有十个数字，因此我们能够很自然地使用它。

十进制并不是唯一的数制。水手会使用六十进制，也就是以 60 为基数，因为他们使用小时、分钟和秒。

当您在 HP 上输入十进制数 10 时，屏幕显示“10”。实际上，HP 会在内部把它转换为位，而 Saturn 处理器使用这些位，也就是在二进制下工作。（稍后您会学到，Saturn 处理器既能使用我们的十进制，也能使用自己的二进制，所以它确实是一颗很出色的处理器。）

十进制意味着我们用 10 个符号书写数字：

| 0 1 2 3 4 5 6 7 8 9

在二进制中，我们只有两个符号：

| 0 1

因此，二进制中的零写作 0，一写作 1，那么二呢？

在十进制中，我们用符号“2”表示数值二；在二进制中，则用“10”表示数值二。换句话说，我们通过组合 0 和 1，在二进制中表示各种数值。

二进制与布尔代数联系非常紧密，稍后会作简要介绍。

下面的小表列出了一些十进制数及其对应的二进制形式：

十进制	二进制
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010
...	...

您必须观察 0 和 1 如何组合成数，并能够继续写出后续数值。接下来几页会详细讨论这一点，因为它很重要。这并不容易，所以即使理解起来有困难，也不必气馁。

现在，我们要把位按四位一组保存，因为 Saturn 喜欢这种方式。

四位组成的数据包称为半字节，因此一个半字节由四个位构成。汇编语言程序员在谈论位以及不同位数组合规模时，会使用一些专门术语。

把四个位组合在一起时，称为一个“四位字”。把 32 个位组合在一起时，就成为一个“32 位字”。

四位数据包可以有多少种取值？很简单，用 2 的幂来计算：

$$2^4 = 16$$

所以，四个位组合成一个半字节，并且：

一个半字节可以有 16 种不同的取值。

再算一个：一个 64 位字可以有多少种取值？

$$2^{64} = 18\,446\,744\,073\,709\,551\,616 \approx 1.8 \times 10^{19}$$

也就是大约 18 个“十亿的十亿”。

非常多。

4 从二进制转换为十进制

下面考察两个放在一起的半字节，这称为一个“字节”。

4 位 → 一个半字节

8 位 → 一个字节

我们从右向左给每一位编号，并把这个编号称为该位的“位次”。例如，两个半字节合在一起，也就是一个字节，其内容为 01100101，则有：

位次：7 6 5 4 3 2 1 0

位值：0 1 1 0 0 1 0 1

可以看到，“位次”从零开始，然后从右向左计数。每一位都有一个确定的位次，并且还有自己的另一项属性。

位次用于确定该位的“权重”。这是什么意思？它表示每一位的权重由其位次决定，是以 2 为底、位次为指数的幂。请看：

位次：7 6 5 4 3 2 1 0

位值：0 1 1 0 0 1 0 1

权重： 2^7 2^6 2^5 2^4 2^3 2^2 2^1 2^0

这里所说的权重，就是这一位在十进制中的数值。举例来说，二进制值 00100110 在十进制中是多少？写出来：

位次：7 6 5 4 3 2 1 0

位值：0 0 1 0 0 1 1 0

权重： 2^7 2^6 2^5 2^4 2^3 2^2 2^1 2^0

现在计算每一位的“权重”，就能得到十进制值。具体做法是把每一位的位值乘以其权重，再把所有结果相加：

$$(0 \times 2^7) + (0 \times 2^6) + (1 \times 2^5) + (0 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (0 \times 2^0)$$

也就是：

$$0 + 0 + 32 + 0 + 0 + 4 + 2 + 0 = 38$$

请用 HP 验证：依次按 **MTH**、**BASE**、**BIN**，然后输入 #00100110b。现在按 **DEC**，得到：

#38d

确实如此。

现在，您已经能够把二进制数转换成十进制数。先把各个位写下来，从右到左、从零到最后一位标出位次；然后写出每一位的“权重”，就可以计算十进制值。

一个半字节的最大值是多少？两个半字节呢？

两个半字节的情况如下：

位次：7 6 5 4 3 2 1 0

位值：1 1 1 1 1 1 1 1

权重： 2^7 2^6 2^5 2^4 2^3 2^2 2^1 2^0

也就是：

$$(1 \times 2^7) + (1 \times 2^6) + (1 \times 2^5) + (1 \times 2^4) + (1 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)$$

即：

$$128+64+32+16+8+4+2+1=255$$

所以，一个字节，也就是两个半字节，可以表示从 0 到 255，共 256 种不同的值。

请记住：0 也是一个取值。

一个半字节的情况如下：

位次：	3	2	1	0
位值：	1	1	1	1
权重：	2^3	2^2	2^1	2^0

也就是：

$$(1 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)$$

即：

$$8+4+2+1=15$$

所以，一个半字节可以表示从 0 到 15，共 16 种不同的值。

要得到一个二进制整数的十进制形式，就把每一位的值乘以 2 的‘位次’次幂，再把所有结果相加。

5 最高有效位与最低有效位

这一点很重要，请不要跳过。

如前所示，每一位都有自己的权重。最右边一位的位次为零，最左边一位的位次最高；如果一共有 n 位，则最高位次为 $n-1$ 。

这意味着每一位对最终数值的贡献权重都不同。

最右边的位称为最低有效位，即 LSB。

最左边的位称为最高有效位，即 MSB。

在一个半字节中，LSB 和 MSB 的位置如下：

位次：	3	2	1	0
	MSB			LSB

在一个字节中，LSB 和 MSB 的位置如下：

位次：	7	6	5	4	3	2	1	0
	MSB							LSB

6 小数

这一部分稍微难一些。

一个小数有两部分：小数分隔符之前的整数部分，以及小数分隔符之后的小数部分。小数分隔符可能是小数点，也可能是逗号，具体取决于地区。

但是，怎样用位来表示小数呢？

我们要给每一位赋予一个负指数权重。

原来是：

位次：	3	2	1	0
位值：	1	1	1	1
权重：	2^3	2^2	2^1	2^0

而对小数则是：

位次：	3	2	1	0
位值：	1	1	1	1
权重：	2^{-3}	2^{-2}	2^{-1}	2^{-0}

请注意：

$$2^{-3}=1/2^3$$

$$2^{-2}=1/2^2$$

二进制小数 0.0110 的十进制值是多少？

位次：	3	2	1	0
位值：	0	1	1	0
权重：	2^{-3}	2^{-2}	2^{-1}	2^{-0}

也就是：

$$(0 \times 2^{-3}) + (1 \times 2^{-2}) + (1 \times 2^{-1}) + (0 \times 2^{-0})$$

即：

$$0 + 0.25 + 0.5 + 0 = 0.75$$

不过请注意，反向转换并不总是可行。例如，0.6 近似等于 2^{-1} (0.5) 加 2^{-4} (0.0625) 再加 2^{-5} (0.03125)，也就是 0.59375，但并不完全相等。十进制中的大多数小数都没有精确的二进制等价形式。使用更多位可以得到更好的近似值，但通常很难找到精确表示。

较低端的现代 HP 计算器，例如 30S，以及 TI 计算器，都采用这种二进制方法表示小数，不过它们使用大量位，使近似结果在合理范围内尽可能准确。较高端的 HP 计算器以及所有较早型号采用另一种方法来保持数值精确，也就是 BCD；我们会在第 12 章介绍它。

7 使用二进制进行计算

本节将介绍如何对两个二进制值进行加、减、乘、除。这会帮助您理解 Saturn 的工作方式。

7.1 加法

请遵循以下规则：

```
0 + 0 = 0 → 无进位
0 + 1 = 1 → 无进位
1 + 0 = 1 → 无进位
1 + 1 = 0 → 有进位
```

您以前大概一直觉得 $1 + 1 = 2$ 是理所当然的。

下面是一个示例：

```
  0101
+ 1101
-----
 10010
```

遇到 $1 + 1$ 时，我写下 0，并把进位 1 移到左侧。这里从右向左计算，每次把两个位相加。

7.2 减法

请遵循以下规则：

```
0 - 0 = 0
0 - 1 = -1 → 结果为负
1 - 0 = 1
1 - 1 = 0
```

因此，需要先说明怎样在二进制中表示负数。

7.3 负数

负数有多种表示方式。下面介绍两种必须掌握的重要方法。您会在自己的程序中选用其中一种；具体选择取决于您是想让处理器替您计算，还是让它完全按您的方式工作，从而驾驭这台机器。

7.3.1 有符号二进制：第一种方法

这里要使用最高有效位。现在您应该明白，我为什么先介绍 MSB 和 LSB。

当 MSB 为零时，数值为正；当 MSB 为一时，数值为负。

其余所有可用位用于编码数值本身。

这种表示称为“有符号二进制”，也是我最喜欢的一种表示方式，别问为什么。

如果使用一个半字节，正零为：0000

负零为：1000

是的，使用有符号二进制时，既有 +0，也有 -0。

如果还记得前面的内容，一个半字节可表示 0 到 15，共 16 种可能的值。这里仍然有 16 种可能值，但不能再用 0 到 15 表示，因为 MSB 被用来标明正负。因此，一个半字节的取值范围如下：

二进制	十进制
正数：从 0000 到 0111	正数：从正零到 7
负数：从 1000 到 1111	负数：从负零到 -7

如果使用一个字节，也就是两个半字节：

二进制	十进制
正数：从 00000000 到 01111111	正数：从正 0 到 127
负数：从 10000000 到 11111111	负数：从负 0 到 -127

请记住：这里有两个零，一个负零和一个正零。

7.3.2 补码：第二种方法

当一个字中的所有位都被反转时，不论这个字有多大，得到的都是它的“反码”。0 变成 1，1 变成 0。这一点并不难。

```
0101 变为 1010
1100 变为 0011
0010 变为 1101
```

但“补码”才是一个字真正的补数，它等于反码加一：

$$\text{补码} = \text{反码} + 1$$

示例：01101110 的补码是多少？

首先反转所有位，得到：10010001

再加一：

```
 10010001
+         1
-----
 10010010
```

所以，01101110 的补码是 10010010。

处理补码二进制数要更复杂一些。请集中注意力；它并不复杂，但必须保持清醒才能理解。

下面学习补码数的编码方式。我们用一个字节，也就是 8 位，来说明。

起始值为 0000 0000。如果不断加一，数值就会逐步上升。

但是，如果从 0000 0000 中减去 1，会发生什么？

得到：

```
0000 0000 - 1 = 1111 1111
```

可以看到，使用补码时，负值的 MSB 为 1。如果 MSB 为 0，数值就是正数。

有趣的是：

```
正数从 0000 0000 到 0111 1111，也就是十进制的 0 到 127。
负数从 1000 0000 到 1111 1111，也就是十进制的 -128 到 -1。
```

看明白了吗？

注意：这里使用 8 位，但也可以使用更少的位，例如一个半字节的 4 位；也可以使用更多的位，例如 64 位，那就有很多位可用了。

这里没有负零和正零。

抽泣！抽泣！

（“抽泣”表示一个人快要哭出来时发出的声音。）

不过，还有更重要的事情。

不能执行 1000 0000 - 1。

为什么？因为这会从未值跳到正值。

同样，也不能执行 0111 1111 + 1，因为这会从正值跳到负值；除非重新定义数学，否则把 +1 加到正值上不应当得到负值。

您必须理解并掌握有符号二进制与补码之间的差异。第一种表示有助于记忆和理解第二种表示。我就是这样记忆的，而且很有效，因为第一种表示连同其中的 +0 和 -0 都非常容易记住。

使用补码稍微复杂一些。下面是一个示例：怎样使用 8 位补码计算十进制的 8 - 11？

```
8  为 0000 1000
11 为 0000 1011
```

为了减去 11，我要加上它的负值，因此：

8-11 变为 8+(-11)

求 0000 1011 的补码，需要反转所有位并加一。详细过程如下：

0000 1011 --反转各位→ 1111 0100

再加一，得到：1111 0101

现在把两个值相加：

```
0000 1000
+ 1111 0101
-----
1111 1101
```

可以看到，这里的 MSB 等于 1，因此可知结果为负数。不过，不能直接使用这个最终值。必须先求它的补码，记住它是负数，再在前面加上负号。

1111 1101 的补码为：0000 0011

最终结果为：-3

请记住！

像上面的示例一样，用补码进行计算时，不能直接使用最终结果。必须：

3. 计算“绝对值”。
4. 根据 MSB 的值加上符号。

7.4 乘法

我不喜欢这一部分，因为有时会把自己绕晕，但又必须讲。进行乘法时，要使用加法和移位。

要把一个值乘以另一个值，最慢的方法是不断把这个数与自身相加，直到得到所需的乘积；当然，如果只乘以一个很小的数，例如 3，则另当别论。这种方法会消耗处理器时间。Avenard 先生会说：“这是一件非常非常糟糕的事，我的朋友。”

可以通过移位和加法完成乘法。下面是一个示例：

```
0010
x 0011
----
0010
+ 0010
+ 0000
+ 0000
-----
00000110
```

第一行 0010 对应 0010 × 1；第二行也对应 0010 × 1；后两行对应乘以 0。

虽然写得不算漂亮，但确实有效。

可以看到，这里做了四次乘法。由于左边两位都是零，可以简化为下一页所示形式。

```
10
* 11
----
10
10
----
110
```

进行乘法时，我按以下步骤操作：

5. 从右向左读取乘数，也就是第二个数。
6. 遇到值为 1 的位时，复制被乘数，也就是第一个数。
7. 遇到 0 时，写入若干个 0。这很简单。
8. 每查看第二个数中的下一位，就把位向左移。请看上面的计算，每一行都向左错开一个位置。
9. 最后，按已经学会的方法把所有数值相加。

请记住：只需从右向左逐位查看第二个数，也就是乘数，并进行移位；然后把所有数值相加，遇到进位时按规则处理。

虽然 Saturn 处理器不能直接做乘法，却能够快速完成移位。

把一个值左移一位，相当于乘以 2；右移一位，相当于除以 2。

乘法和除法还可以优化。如前所述，每遇到一个 0，就需要左移一位。要快速乘以某个值，可以统计其中有多少个 0，然后执行相同次数的左移。

7.5 除法

感谢 Eduardo Kalinowski 对本节的帮助。他最初用巴西式长除法编写了本节，但我改写为美式长除法，希望这种写法更常见。

好，开始吧。首先请看前面的乘法：它与十进制乘法完全一样，只是简单得多，因为您只需记住四种情况（ 0×0 、 0×1 、 1×0 、 1×1 ），而不用记十进制乘法中的 100 种情况。仔细观察过程，您会发现步骤相同：先用第二个数的最后一位乘第一个数，再用倒数第二位相乘，依此类推，最后把所有结果相加。这就像十进制乘法，只不过只使用 1 和 0。二进制除法也与十进制除法完全相同。

用 `#10101b` (21) 除以 `#111b` (7) 练习。结果应为 `11` (3)，没有余数。

首先像普通除法一样写出两个数。我不知道世界其他地方怎样书写，但在美国写成这样：

```
      -----
111 | 10101
```

然后看除数 `111` 在被除数第一位中能“容纳”几次。因为在二进制中最多只能容纳一次，所以只需检查它能否容纳。`111` 无法放入 `1`，因此写下一个 0：

```
      0
      -----
111 | 10101
```

接着取被除数的两位，检查是否能容纳。仍然不能，因此再写一个 0：

```
      00
      -----
111 | 10101
```

现在取前三位，仍然无法容纳，所以再写一个 0。然后取四位 `1010`，此时可以容纳 `111`。因此，在商中写 1，并减去 `111`，求出部分余数：

```
      0001
      -----
111 | 10101
    -111
    ----
      11
```

部分余数 `11` 显然太小，不能再减去 `111`，因此把下一位移下来：

```
      0001
      -----
111 | 10101
    -111
    ----
      111
```

现在要用原除数 `111` 除下面的数 `111`，得到 1，把它写到顶部，余数为零：

```
      00011
      -----
111 | 10101
    -111
    ----
      111
    -111
    ----
      000
```

已经没有更多位可以移下来，因此计算结束。忽略前导零，顶部的商为 `11`，即十进制的 3，符合预期；底部的余数为 0。请注意，我们同时也学会了模，也就是余数函数的工作方式。

正如您已经看到的，如果还没读，请现在读一遍，整个过程与普通除法完全相同，只是所有运算都在二进制中进行。

8 十六进制

十进制使用基数 10，二进制使用基数 2，而十六进制使用基数 16。为什么要使用基数 16？如前所述，Saturn 喜欢把位按四位一组保存，这称为一个半字节。

前面已经学到，一个半字节最多可以有 16 种不同的值。下面列出这些值的二进制形式：

十进制	二进制	十六进制
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4

十进制	二进制	十六进制
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

使用十进制时，我们有十个符号，并把它们组合起来表示数字。前面也学习了二进制，它把 0 和 1 组合起来形成数字。

十六进制需要 16 个数字符号，因此必须找到 16 个符号来表示它们。我们先使用十进制中的前十个数字 0 到 9，再使用字母表的前六个字母：A、B、C、D、E、F。

然后，把这些数字和字母组合起来表示数值。这里很方便的一点是，在十六进制模式下，四个位可以用一个符号表示。

十六进制适用于所有现代计算机。具有八个符号的八进制在处理某些老式计算机时可能有用，但对任何较新的计算机系统来说，只需要十六进制。

9 从十六进制转换为十进制

拿到一个十六进制数时，可以像处理二进制数一样，为其中各位标出“位次”。只是指数不同，得到的权重也更大。

来看一个由五个半字节组成的十六进制数。为什么是五个？因为 Saturn 处理器用五个半字节为内存中的每个半字节指定位置。要得到一个可用作地址的数值，从内存读取信息或向该处写入内存，都需要五个半字节。

假设十六进制数为 #ABCDE，要得到它的十进制值，可以写成：

位次：	4	3	2	1	0
十六进制：	A	B	C	D	E
权重：	16^4	16^3	16^2	16^1	16^0

这很简单：与二进制相同，我们给每个半字节赋予一个权重。

十六进制中位次为 n 的半字节，其“权重”为：

$$16^n$$

示例：十六进制数 #1AB0F 的十进制值是多少？

写成：

位次：	4	3	2	1	0
十六进制：	1	A	B	0	F
权重：	16^4	16^3	16^2	16^1	16^0

现在，只需把每个半字节乘以 16 的‘位次’次幂：

$$\text{数值} = (1 \times 16^4) + (A \times 16^3) + (B \times 16^2) + (0 \times 16^1) + (F \times 16^0)$$

不要因为这里出现字母而不知所措：由于使用十六进制，数字中经常会包含字母。

如果您更容易理解， $(B \times 16^2)$ 实际上就是 (11×16^2) 。忘记这些对应关系时，可以随时回看前面的表格。
计算各项后得到：

$$(1 \times 65536) + (10 \times 4096) + (11 \times 256) + 0 + 15 = 109\,327$$

因此：

$$\#1AB0Fh = 109\,327$$

您可以用 HP 验证。输入数值 109327，然后依次按 **MTH**、**BASE**、**R→B**。

R→B 命令把实数转换为二进制整数。请确保当前使用十六进制记法；**HEX** 菜单字段中应当有一个实心方点。如果没有，请按 **HEX**，让二进制整数以十六进制值显示。

可以试着用 **R→B** 把实数转换为十六进制数，再用 **B→R** 把二进制整数转换回实数。

10 使用十六进制书写四位一组的数据（半字节）

怎样用十六进制数字书写半字节？一个半字节有四个位，可表示 16 种不同的值；十六进制也有 16 个符号，因此只需把这四个位转换成十六进制形式。

如果位数多于四位呢？例如，有一个 12 位二进制数，需要转换为十六进制。

诀窍是从右向左，把位分成每四位一组。

例如，使用数值 0111 1011 0101。

为了避免复杂而乏味的计算，不一次处理全部 12 位，而是把它们分成四位一组。然后，把每个四位数据包转换为十六进制值：

```
0111 → 7
1011 → B
0101 → 5
```

前面已经给出了表格；不过符号并不多，您也可以写出 0 到 15，再写出它们的二进制形式和十六进制值。

因此可知：

```
0111 1011 0101（二进制）是 #7B5h
```

用这种方法，可以轻松转换很大的二进制数。

但是，如果位数不是四的倍数怎么办？没有问题：在数值左侧补足若干个 0，使总位数成为四的倍数。

示例：把二进制数 011101 转换为十六进制。

先按四位一组拆分，但显然左侧位数不足：

```
01 1101
```

在最左边的一组前面补两个 0，使总位数成为四的倍数：

```
01 1101 变为 0001 1101
```

因为补的是前导零，数值并没有改变。

现在可以把每个四位组转换为十六进制形式：

```
0001 → 1
1101 → D
```

所以：

```
011101（二进制）→ #1Dh
```

11 真的需要做完这些数值运算吗？

当然，我们有 HP，可以用现成的功能在二进制与十六进制之间任意转换。但理解位怎样编码、怎样用来表示数字仍然很重要。

随着知识和编程能力的提高，您会发现一些逻辑函数非常有用；这些函数将在下下个部分介绍。它们可以使用我们称为“掩码”的东西来检查某些值。

“掩码”是一组位，之后会用它检查一个或多个位的值。

开始读取 HP 键盘上的按键时，我会说明掩码有什么用。

我刚开始学习时也不喜欢位和各种转换。您可能也不想学，因为 HP 不是可以替您完成这些工作吗？

错了。这样只会让您比平常更加困惑，并最终把事情全部弄错。最好学会在头脑中完成这些处理。

如果还有任何疑惑，请回头重读前几节。即使不擅长数学，您也能够理解。

12BCD：“二进制编码的十进制”

您可能同样不喜欢学习这一部分。不过最好掌握它，尤其是在需要从堆栈弹出实数时。

Saturn 处理器使用称为半字节的四位数据包。前面已经学到，它用一种很方便的十六进制格式保存半字节。因此，当要求处理器把两个值相加时，它会把这些值视为十六进制数：如果要求它计算 9 加 9，Saturn 给出的不会是“18”，而是“12”，因为 18=#12h。

不过，我们人类，不论您究竟是什么，更习惯十进制；Saturn 处理器也被设计成能够处理十进制数。

我说过，这是一颗很出色而且相当强大的处理器，虽然体积很小，也很难吞下去。咕咚，我今天已经吃掉第二颗了。

当要求 Saturn 用十进制数进行计算时，它会使用一种称为 BCD 的特殊模式。

BCD 表示 Binary Coded Decimal，即“二进制编码的十进制”。

一个半字节有四个位，可以表示 16 种值。但使用 BCD 模式时，数字仍保持十进制的形式。

请留意下表中 BCD 编码数字的方式。保持冷静，很快就会发现它并不困难。

十进制	半字节（四个位）	BCD
0	0000	0000
1	0001	0001
2	0010	0010
3	0011	0011
4	0100	0100
5	0101	0101
6	0110	0110
7	0111	0111
8	1000	1000
9	1001	1001
10	1010	0001 0000
11	1011	0001 0001
12	1100	0001 0010
13	1101	0001 0011
14	1110	0001 0100
15	1111	0001 0101

可以看到，在数值小于十时，半字节位值与 BCD 值之间没有变化。

但是，达到十时，BCD 不能再只用一组四个位继续增加，因此必须再增加一组四个位。我们在新的的一组四个位中重新从头计数，就像在前一组中所做的一样。请留意从 9 到 10 的变化，以及之后怎样继续到 15。

使用 BCD 时，每个十进制数字占用四个位。四个位可用于从零数到 9；但“10”由两个数字组成，因此必须用八个位按 BCD 编码这个值。

可以看到，它并不复杂。BCD 也许看起来很奇怪，但这并不是此处需要关注的重点。

13 将十进制数转换为 BCD 形式

要把十进制数转换为 BCD 形式，只需从左向右读取该数，并写出每个数字的二进制形式：

如果数值是 934，它的 BCD 形式是什么？

```
| 9 → 1001
| 3 → 0011
| 4 → 0100
```

因此，934 的 BCD 形式是：

```
| 1001 0011 0100
```

看到了吗？一点也不困难。BCD 看似复杂，实际上并不是。

必须谨慎：BCD 二进制形式与普通二进制形式并不相同。

下面仍以 934 为例，列出它的 BCD 二进制形式和普通二进制形式：

BCD: 1001 0011 0100

二进制: 0011 1010 0110

不要把两者混在一起，否则会彻底糊涂。

14 普通 BCD 与“扩展 BCD”

前面介绍的 BCD 是常见形式，称为“压缩 BCD”（Compacted BCD）。还有另一种形式，称为“扩展 BCD”（Extended BCD）。

扩展 BCD 不再为每个数字使用四个位，而是使用八个位。例如，数值 11 的编码如下：

压缩 BCD: 0001 0001

扩展 BCD: 00000001 00000001

使用压缩 BCD 时，一个字节，也就是两个半字节，可以表示 0 到 99；而扩展 BCD 只能表示 0 到 9。

所以，Saturn 处理器使用的 BCD 不仅方便，而且聪明。

现在看看您的 HP，告诉它您爱它。

15 使用 BCD 进行计算

保存数字很好，能够处理这些数字就更好了。不过，由于 BCD 的编码方式，有些半字节取值是不允许的。

Saturn 会提供很大帮助，因为它不仅能够保存 BCD 数字，还能对它们进行计算。我们只需让处理器进入 BCD，也就是十进制模式，而不是十六进制模式，然后使用它即可。

使用 BCD 时有两件重要的事情：半字节发生溢出，以及出现进位并需要传递。

15.1 半字节溢出时

如果在 BCD 中计算 16 加 16，会发生什么？

```
0001 0110    (16 的 BCD 形式)
+ 0001 0110
-----
0010 1100
```

问题在于：使用 BCD 时，1100 是不允许出现的值。

15.2 需要传递进位时

两个值为 1 的位相加时会产生进位，必须把一个位移到左边，并在当前位写 0。例如：

0001 + 0001 = 0010

但是，使用 BCD 时，进位不能像通常那样随意向左传递。

大多数数学指令都能识别 BCD，并正确处理这些问题，但并非所有指令都具备这种能力。

16 逻辑运算

这一部分很重要。我们会遇到 OR、AND 以及 XOR 等运算。（Julien Meyer，也就是 SunHP，曾为 XOR 函数写过一篇祷文，因为它在游戏和图形处理中非常有用；他每天晚上都会向 XOR 祈祷。）

从最底层来看，处理器完全使用逻辑运算设计而成。这里仅讨论一些简单运算，但把许多逻辑运算组合起来，处理器就能完成非常复杂的任务。

想一想 HP 上的 COS 键。按下它时，RPL 操作系统会先检查是否存在参数，再检查参数是否合法，最后调用一个或多个机器语言运算符。处理器随之把位从一处移动到另一处，并对它们执行逻辑运算。这非常奇妙。

George Boole（1815-1864）在 1847 年出版了一本名为《Analyse mathématique de la logique》的书，标题大致可译为《数学的逻辑分析》。他最重要的工作涉及逻辑。我们把一些语言表述转换为代数组件，以便为计算中的思维过程建立规则，这称为布尔代数。

在布尔代数中，有两个值：0 和 1。

0 称为“假”（值 false）

1 称为“真”

由于讨论的是逻辑，必须理解布尔代数中的假与真。每个值都有一个用途：0 表示假，1 表示真。

电子学广泛使用我们敬爱的 Boole 提出的这一概念。找一张他的画像，再加上 Newton 和 Einstein，一起挂在墙上；否则您永远成不了好程序员。

因此，我们要使用位 0 和 1。

0 表示“假”，在电子学中通常对应没有电压。

1 表示“真”，通常对应电路中存在电压。

因为这里使用位，所以有两个可处理的值：

0 是 1 的“补数”
1 是 0 的“补数”

所谓补数，就是把位反转。这里说的是“反码”（one's complement），也称“受限补数”；它不是前面用于保存负值的“补码”（two's complement）。如果感到困惑，请回看第一部分。

有三种基本逻辑运算。只需其中两种，就能构造出其他所有逻辑运算；或许更准确地说，是我们使用和已经发现的所有逻辑运算。

法文名称	英文名称	使用的符号
ET	AND	·
OU	OR	+
NON	NOT	上横线

一个值经过 NOT 取反后，会在其名称上方画一条横线，例如：

$$\text{NOT } A = \bar{A}$$

在法语中，这个值称为“A-barre”；在英语中称为“A-bar”。有时也写作 A' 或 $\sim A$ ，但本文始终使用 A-bar。

16.1 逻辑 OR

它也可以称为“逻辑加法”。

下面给出一种称为“真值表”的表格。每种逻辑运算都有自己的真值表。左侧用 A、B、C 等字母表示每个输入位；右侧放置一个 S，表示结果；如果存在多个输出，则会放置多个结果。

OR 在两个位之间进行。第一个位是 A，第二个位是 B，S 是这两个位执行 OR 后的结果。

OR 真值表

A	B	S
0	0	0
0	1	1
1	0	1
1	1	1

逻辑 OR 也可以写成：

$$S = A + B$$

注意：逻辑 OR 不是学习二进制数时介绍的加法。

0101 + 0011 = 1000
0101 OR 0011 = 0111

现在可以看出，只要 A 或 B 中有一个为 1，或者两者都为 1，OR 的结果就是 1。

16.2 逻辑 AND

它也称为“逻辑乘积”，但并不是乘法。

表达式写作：

$$S = A \cdot B$$

AND 真值表

A	B	S
0	0	0
0	1	0
1	0	0
1	1	1

只有 A 和 B 都为 1 时，结果才是 1。

16.3 取反

只需把位反转。表达式写作：

$$S=\bar{A}$$

请注意，字母上方有一条横线。

INVERT 真值表

A	S
0	1
1	0

16.4 NOT OR（又称 NOR）

这是把 NOT 与 OR 组合起来。执行 NOR 时，只有全部输入均为零，输出才置为 1。

NOR 的表达式为：

$$S=\bar{A}\bar{B}$$

NOR 真值表

A	B	S
0	0	1
0	1	0
1	0	0
1	1	0

16.5 AND NOT（又称 NAND）

这是把 AND 与 NOT 组合起来。使用 NAND 时，它是 NOR 的反运算。换句话说，除非两个输入都置为 1，否则输出始终为 1。

NAND 的表达式为：

$$S=\bar{A}\bar{B}$$

NAND 真值表

A	B	S
0	0	1
0	1	1
1	0	1
1	1	0

16.6 Exclusive OR（又称 XOR）

XOR 广泛用于图形处理。A 或 B 中有一个为 1 时，结果为 1，但必须恰好只有一个为 1。如果 A 和 B 都置为 1，则结果为零。

其表达式为：

$$A \text{ XOR } B=(A \text{ OR } B) \text{ AND } (\bar{A} \text{ OR } \bar{B})$$

XOR 真值表

A	B	S
0	0	0
0	1	1
1	0	1
1	1	0

这很有用，因为图形对象在内存中就是一组位。要绘制图形时，可以把它的位与屏幕上的现有图形做 XOR。如果在同一位置再次执行 XOR，图形对象就会消失。

稍后会再次介绍，请耐心等待。

17 复习：数制

有些人说，他们要么没有阅读数制部分，要么读得太快。我理解，因为自己刚开始学习时也做过同样的事。但是，最低限度的知识仍然不可缺少。必须能够识别十六进制和二进制形式。以后介绍掩码及其构造方法时，需要同时使用二进制和十六进制信息，因为我们会先写出各个位来构造所需掩码，再转换为十六进制，最后把它装入 C 或 A，用来测试另一个寄存器的值。

您不必掌握如何把数值从二进制转换为十六进制之类的全部转换细节，但我认为，知道每一位的权重很重要。

快速复习一下：一个半字节有四个位，每个位都有权重。不论共有多少个位，它们都从右向左编号。

第二部分 编程工具

在开始编写代码之前，必须先讨论汇编语言开发所需的工具。早期程序员只能把汇编程序写在纸上，再把汇编指令翻译成十六进制形式，最后由此建立 Code 对象字符串。

今天有了更好的工具。您可以把程序写入字符串，再由称为“汇编器”的编译器完成代码检查，毕竟每个人有时都会出错，并生成 Code 对象。

18 使用 HP 48

要在 HP 48 系列计算器上进行汇编语言开发，需要安装额外软件。必须安装一个汇编器；出于性能考虑，还建议使用替代文本编辑器。本节列出的工具适用于 HP 48S、48SX、48G、48G+ 和 48GX。

18.1 汇编器

我们试用过多种汇编器。Jazz 很有吸引力，功能也极其强大，但对多数用户来说体积太大。Jazz 还采用不同的语法，因此不太适合按照本书示例学习的初学者；本书使用的是更容易获得的汇编器所采用的语法。我们希望 G 用户也能编程，本书面向所有人，其中包括 S 和 G 型号的用户。

这里使用 HP-ASM 1.1 语法。这个工具是 Phong Nguyen 的 ASM Flash 面向 G/GX 的改编版；据 Jean-Yves 所说，Phong Nguyen 编写它时只有 8 岁。在本文中，HP-ASM 与 ASM Flash 是同义名称。

HP-ASM 的速度非常快。Jazz 的 ASS 命令慢好几倍；HP-ASM 半秒即可完成的工作，ASS 可能需要数秒。这虽然不是决定性差异，但很令人印象深刻。

HP 48G、48G+ 和 48GX 使用的 HP-ASM 可从下列位置获得：

<http://www.hpcalc.org/details.php?id=1666>

→ <http://www.hpcalc.org/hp48/programming/asm/hpasmgx.zip>

HP 48S 和 48SX 使用的 ASM Flash 可从这里获得：

<http://www.hpcalc.org/details.php?id=1659>

→ <http://www.hpcalc.org/hp48/programming/asm/asmflash.zip>

[hpasmgx.zip](#) 是一个 ZIP 文件，其中包含两个适用于 G 系列的 HP-ASM 版本。如果使用 S 或 SX，应改为获取 ASM Flash。

HP-ASM 有 1.0 和 1.1 两个版本。1.0 版大小为 8606 字节，工作良好；1.1 版大小为 9124 字节。1.1 版新增了一些指令，也可能还有其他内容。

这个库包含一个汇编器，可以根据源代码建立 Code 对象。本教程介绍 1.1 版命令，但 1.0 版命令非常相似，例如使用 →ASM 而不是 ASM。1.0 与 1.1 的大小只相差约 600 字节，因此建议使用 1.1。

如果您的 GX 有大量空闲且未占用的 RAM，可以选择 Jazz。它提供许多附加功能，例如反汇编器、调试器和系统 RPL 支持。本书不使用 Jazz 的语法，因为它略有不同。G 或 S 型号没有足够空间使用 Jazz。本书并非只为 GX 用户编写，而是面向所有人，因此将使用 HP-ASM 语法。

HP-ASM 语法与 Meta Kernel 的 MASD 汇编器兼容，因此本教程也适用于 Meta Kernel。此外，MASD 还为程序员提供了一些很实用的功能。如果您拥有 Meta Kernel，请阅读其手册以了解更多信息。Meta Kernel 用户也可以阅读下面的 49 章节，因为 49 内含 Meta Kernel，所以 49 的文档大部分同样适用。

18.2 HP-ASM 命令

下面是 HP-ASM 库菜单中的命令：

10. **ASM:** 以字符串为参数, 返回汇编后的对象, 可以是 Code 对象, 也可以是包含十六进制代码的字符串。
 11. **ED:** 它不是编辑器。不过, 如果您犯了错误, 按 ED 后会编辑源字符串, 并把光标移到发现错误的那一行开头。ED 会调用 HP 提供的标准编辑器。如果懂一点系统 RPL, 可以修改 ED, 让它调用您想用的编辑器, 但这超出了本书范围。
 12. **OPT:** 用于选择选项。按下菜单键时, 该菜单键的内容会在两个选项之间切换。
- **CODE/HEXA:** 选择 CODE 时, 汇编器生成 Code 对象; 选择 HEXA 时, 汇编器把包含所生成代码对应十六进制值的字符串压入堆栈。
 - **0-15/1-16:** 选择 0-15 时, 半字节从 0 到 15 编号; 否则从 1 到 16 编号。这会影响 P 寄存器的用法, 例如 `P=n`、`CPEX`、`C=P`、`P=C`; 也会影响 `CBIT`、`ABIT` 等位操作助记符, 以及涉及 P 和 HS 的测试。汇编语言从 0 开始计数, 因此强烈建议选择 0-15, 也就是把寄存器的第一个半字节编号为 `#0h`。建议始终保持 0-15 设置。
 - **HP/PC:** 此选项决定使用哪一组运算符符号。HP 的扩展 ASCII 比较符号与 PC 的扩展 ASCII 不兼容。选择 HP 时使用 HP 字符; 选择 PC 时使用与 PC 兼容的双字符标准 ASCII 符号: `#`、`>=` 和 `<=`。例如, 选择 HP 时, “不等于”符号是带斜线的等号; 选择 PC 时则是井号 `#`。
 - **JMP/UNJ:** 选择 JMP 时, 汇编器检查并计算所有跳转。选择 UNJ 时, 不计算指向标签的跳转, 因此也不会验证这些跳转。建议让 JMP 保持选中。只有在编译从较大项目中截取的一小段代码、希望计算它占用多少字节而又不想让错误跳转中止汇编时, 才选择 UNJ。
 - **ON/OFF:** 编译非常大的程序时, 可以关闭屏幕, 使编译速度提高约 13%。选择 OFF 后, 汇编期间屏幕会关闭; 选择 ON 则保持开启。汇编大型源代码时, 建议关闭屏幕。
 - **EXIT:** 退出 OPT 菜单。

18.3 使用 HP-ASM 汇编代码

HP-ASM 或 Jazz、J-ASM、ASM Flash、MASD 等其他汇编器生成的 Code 对象速度很快, 因为 HP 的 Saturn 处理器会直接执行它们。

我们不直接用位编程, 因为那会非常困难。汇编语言使用助记符, 处理器的每条可用指令都有一个对应助记符。HP-ASM 等汇编器把这些助记符翻译成机器语言, 而机器语言正是 Code 对象内部以十六进制形式保存的内容。HP 48 的 RPL 无法控制 Code 对象运行时发生的事情, 因此必须把代码写好, 否则就只能一边耗尽电池, 一边恢复丢失的程序; 至于有一张或多张 RAM 卡的人, 可能会庆幸自己有所准备。

HP-ASM 语法不是 HP 官方语法, 而是法国程序员沿用了一段时间的语法。DEV 和 TC 等其他工具也使用这种语法, 因此它已经成为历史的一部分。

您要把代码写在一个字符串中。HP 内置编辑器速度不快, 而且对内存的使用方式糟糕得令人难以置信, 因此建议使用外部编辑器。MiniWriter 在 G 型号上很好用。

交给汇编器的字符串中, 每行必须只有一条指令。可以使用百分号 `%` 添加注释。从 `%` 开始直到回车符之前的所有内容都被视为注释。

文本文件中的最后一个符号应为“at”符号 `@`; 汇编器据此知道源代码已经结束。这个符号由 **ALPHA**、**Right-Shift**、**ENTER** 输入。

汇编器区分大小写, 这意味着 `Label` 与 `LABEL` 不同。命名标签时, 应全部使用大写或全部使用小写。请选择一种书写风格并始终坚持, 以免混淆。

要转到下一行, 请按 **Right-Shift**、**.**。ALPHA 模式的状态无关紧要, 因为两种状态下的操作相同。

18.4 HP-ASM 的其他功能

下面是 ASM Flash 所附英文文档的摘要。

18.4.1 链接器

这是一个很实用的功能。如果把源代码分成若干部分, 可以要求编译器分别编译各部分, 再生成最终的 Code 对象。如果有两个源文件 ONE 和 TWO, 只需在程序中写入:

```
'ONE
'TWO
@
```

' 符号用于把 HOME 目录, 也可能是当前目录, 中某个文件的内容放入源代码。

如果发现错误, 汇编器会说明错误位置。

18.4.2 指令控制符

指令控制符是 **!** 符号后跟一个 OPT 值。如果希望在 HP 选项开启的情况下编译代码，只需写入：

```
| !HP
```

只要插入 **!**，再跟上 OPT 菜单中的名称，就能从源代码字符串中临时开启或关闭任何选项。这可以出现在源代码的任何位置；例如，如果愿意，一部分代码可以使用 PC 比较运算符，另一部分则使用 HP 比较运算符。

18.4.3 宏

这些不完全是程序员通常所说的宏，但仍然很实用。您可以编译代码并重复使用已经编译的结果，也可以在程序中插入 GROB 或任何类型的数据。我们需要编写一个程序来创建可用作宏的对象；开始学习图形部分时，这会很有用。该程序见后文。

18.5 高速文本编辑器

HP 48 系列内置编辑器不太适合编辑大型对象，因此应使用替代编辑器来编辑 ASM 源代码。

如果使用装有 Meta Kernel 的 48GX，就已经拥有一个不错的文本编辑器。它内置在 Meta Kernel 环境中，与 StringWriter 相似。否则，需要单独安装文本编辑器。

StringWriter (24 KB) 很好用，但对 G 型号来说体积太大，因此可以改用较小的 MiniWriter (5 KB)。MiniWriter 更小，G 和 S 型号都能轻松容纳。

如果 GX 有充足 RAM，StringWriter 提供的功能更多，但字体也更大。这两个编辑器都有一个很方便的选项：按 **MTH** 后，会列出所有以 * 开头的行；HP-ASM 把这些行视为标签。在源代码中跳转到任意标签，是一项极其必要的功能。

此外，TED (12 KB) 具有许多与 StringWriter 相同的功能，但使用小字体。

StringWriter 可从这里下载：

<http://www.hpcalc.org/details.php?id=129>

→ <http://www.hpcalc.org/hp48/apps/editors/strwrt44.zip>

MiniWriter 可从这里下载：

<http://www.hpcalc.org/details.php?id=114>

→ <http://www.hpcalc.org/hp48/apps/editors/minwrt12.zip>

TED 可从这里下载：

<http://www.hpcalc.org/details.php?id=131>

→ <http://www.hpcalc.org/hp48/apps/editors/ted32.zip>

注意：MiniWriter 和 TED 都使用小字体，并且需要 UFL。UFL 是一个字体库，使每个程序不必各自内置一套字体而浪费内存空间。MiniWriter 和 TED 都不附带 UFL，因此必须从下列位置另行下载：

<http://www.hpcalc.org/details.php?id=162>

→ <http://www.hpcalc.org/hp48/utils/fonts/uf1102.zip>

必须同时把 MiniWriter (或 TED) 和 UFL 库下载到 HP，否则编辑器无法工作。使用 MiniWriter 时，不需要名为 `strwadp.lib` 的库。请把下列文件发送到 HP：

uf11.lib (位于“uf1102.zip”压缩包中)

以及下列二者之一：

minwrt12 (位于“minwrt12.zip”压缩包中)

或

ted.lib (位于“ted32.zip”压缩包中)

我个人更喜欢 StringWriter，因为较大的字体更容易阅读。如果使用 G 或 S 型号，需要尽量减少内存占用，则应使用 MiniWriter。

19 使用 HP 49

HP 49 系列已经内置开始开发所需的一切。49G 和 49G+ 都包含来自 Meta Kernel 的 MASD 汇编器，它与本书使用的 HP-ASM 语法兼容。它们还带有一个同样来自 Meta Kernel 的高速文本编辑器；这是 HP 48 系列上最好的文本编辑器之一。

48GII 虽然属于 49 系列，却没有 MASD 汇编器，也没有适用于它的第三方汇编器软件。因此，就本书编写时的情况而言，不建议用它进行汇编语言编程。不过，它确实带有反汇编器和其他一些实用开发工具；如果有人把汇编器移植过去，它也许仍能为开发者所用。

19.1 汇编器

49 系列有一个隐藏的开发库，即库 256。这个库不会出现在库菜单中，而且默认不附加；在附加之前，无法通过输入命令名称来执行其中的命令。

要进入开发库，只需输入：

```
| 256 MENU
```

然后可以按几次 NXT，查看所有可用命令。要附加开发库，使您能够直接输入命令名称来执行它们，请输入：

```
| 256 ATTACH
```

如果设置系统标志 -86，开发库会在每次暖启动后自动附加。可用下列命令设置系统标志 -86：

```
| -86 SF
```

可以用 ASM 命令运行 MASD 汇编器。如果汇编时出错，错误代码列表会放到堆栈上。可以使用 ER 命令显示错误，并跳转到代码中引发错误的相应位置。如果附加库 257，还可以使用 ASM2 命令；它会调用 ASM，并在发生错误时自动调用 ER。它实质上等同于下面的用户 RPL 程序：

```
| << IFERR ASM THEN ER END >>
```

开发库中还有许多其他实用工具，其中大部分将在下一节介绍。

19.2 开发库命令

除汇编器外，49 系列还有大量可能对程序员有用的其他命令。它们位于包含汇编器的同一个库 256 中。

随着 HP 49 ROM 的演进，开发库逐步增加了更多功能，因此建议把计算器升级到最新 ROM，以确保功能完整。

下面列出 ROM 2.00 的开发库命令：

命令	说明
→H	把栈 1 中的对象转换为其十六进制表示。
H→	把栈 1 中的十六进制字符串转换为它所表示的对象。
→A	返回栈 1 中对象的内存地址。
A→	返回栈 1 中内存地址所指向的对象。警告：危险！
A→H	把栈 1 中的内存地址转换为等价的十六进制字符串。
H→A	使用栈 1 中十六进制字符串的前 5 个数字生成内存地址。
→CD	把栈 1 中的十六进制字符串转换为 Code 对象。
CD→	把栈 1 中的 Code 对象转换为其十六进制表示。

命令	说明
S→H	把栈 1 中的 ASCII 字符串转换为等价的十六进制字符串。
H→S	把栈 1 中的十六进制字符串转换为等价的 ASCII 字符串。
→LST	把栈 1 中的复合对象转换为由其组成部分构成的列表。
→ALG	把栈 1 中的复合对象转换为代数对象。
→PRG	把栈 1 中的复合对象转换为可执行程序对象。
COMP→	把栈 1 中的复合对象转换为代数对象。
→RAM	在内存中为栈 1 中的对象创建一个新副本，即使它是 ROM 对象也如此。反汇编内置 ROM 命令时很有用。
SREV	反转栈 1 中的字符串。
POKE	把原始十六进制值写入内存；起始地址放在栈 2，十六进制字符串放在栈 1。警告：危险！

命令	说明
PEEK	从内存读取原始十六进制值；起始地址放在栈 2，要读取的半字节数放在栈 1，两者均为用户二进制整数。
APEEK	读取栈 1 所给地址中保存的地址。
R~SB	在栈 1 的实数与系统二进制整数之间双向转换，也接受整数。
SB~B	在栈 1 的系统二进制整数与用户二进制整数之间双向转换。
LR~R	在栈 1 的长实数与实数之间双向转换，也接受整数。
S~N	在栈 1 的字符串与名称之间双向转换。
LC~C	在栈 1 的长复数与复数之间双向转换。
ASM→	反汇编栈 1 中的 Code 对象，返回等价字符串。要用 ASM 重新编译该字符串，必须清除系统标志 -71，也就是显示/隐藏地址标志。
BetaTesting	以字符串形式返回 HP 49 的 Beta 测试人员列表。
CRLIB	建立库。具体细节超出本文范围，但可从其他资料获得更多信息。
CRC	为给定字符串、库或备份生成 CRC，结果为系统二进制整数。
MAKESTR	建立长度由栈 1 指定的字符串；根据需要重复字符串 ABCDEFG\ n，直到达到该长度。
SERIAL	返回包含内部序列号的字符串；该序列号不同于计算器背面的序列号，格式为 HP49 Serial Number: COYWWXXXXX。
ASM	使用 MASD 汇编器汇编栈 1 中的字符串。既接受系统 RPL，也接受汇编语言。
ER	解析栈 1 中的错误代码列表，并在栈 2 的字符串中显示错误。
→S2	系统 RPL 反编译器。接收栈 1 中的对象，返回包含等价系统 RPL 源代码的字符串，可用 ASM 重新编译。
XLIB~	接收栈 2 中的库编号和栈 1 中的命令编号，建立代表该命令的 XLIB 对象；也可以反向转换。输入数字可以是整数、实数或二进制整数。
ARM→	把栈 1 中的 ARM Code 对象反汇编为等价 ARM 源代码；或者反汇编从栈 2 中内存地址开始、到栈 1 中内存地址结束的 ARM 代码。注意：这是仅适用于 49G+ 的 ARM 代码，并不是本书介绍的 Saturn 代码。
POKEARM	把原始十六进制值写入 ARM 内存；起始地址放在栈 2，十六进制字符串放在栈 1。警告：危险！
PEEKARM	从 ARM 内存读取原始十六进制值；起始地址放在栈 2，要读取的半字节数放在栈 1，两者均为用户二进制整数。

19.3 文本编辑器

虽然 49 系列已经内置高效、快速的文本编辑器，仍然有改进空间。Carsten Dominik 和 Peter Geelhoed 编写了一个名为 Emacs 的工具，试图复现著名 Unix 同名文本编辑器的部分功能，并在内置文本编辑器基础上加以增强。可从下列位置下载：

<http://www.hpcalc.org/details.php?id=3940>

→ <http://www.hpcalc.org/hp49/apps/editors/emacs211a.zip>

对大多数用户来说，内置文本编辑器已经足够；但对希望使用更强大功能的人而言，Emacs 很值得占用这部分内存。Emacs 增加了命令补全、直接从编辑器反编译 ROM 入口以便检查 49 内部代码、键盘宏、正则表达式、书签等大量功能。

19.4 使用 MASD 汇编代码

我们不直接用位编程，因为那会非常困难。汇编语言使用助记符，处理器的每条可用指令都有一个对应助记符。HP-ASM 等汇编器把这些助记符翻译成机器语言，而机器语言正是 Code 对象内部以十六进制形式保存的内容。编写汇编语言时，我们拥有对计算器的完全控制，因此必须把代码写好，否则很可能出现 Memory Clear 错误。您要把代码写在一个字符串中。HP-ASM 在 48 系列上的用法以及本文全部示例都要求每行只有一条指令；MASD 则允许每行写任意多条指令。可以用百分号 % 添加注释。百分号由 ALPHA、Left-Shift、1 输入。从 % 开始直到行尾的所有内容都被视为注释。

文本文件中的最后一个符号必须是“at”符号 @，并且独占新的一行；汇编器据此知道源代码已经结束。这个符号由 ALPHA、Right-Shift、ENTER 输入。

编译器区分大小写，这意味着 `Label` 与 `LABEL` 不同。命名标签时，最好全部使用大写或全部使用小写。请选择一种书写风格并始终坚持，以免混淆。

要转到下一行，请按 **Right-Shift**、`.`。ALPHA 模式的状态无关紧要，因为两种状态下的操作相同。

19.5 MASD 标志设置

开发库使用几个标志配置自身行为：

13. **-71**：设置后，MASD 反汇编器不会在汇编代码中显示额外内存地址。要让反汇编后的代码能够重新汇编，必须设置此标志。
14. **-86**：设置后，每次暖启动都会自动附加开发库。
15. **-92**：设置后，MASD 汇编器默认进入系统 RPL 模式，而不是汇编语言模式。

就本书而言，建议设置标志 -71 和 -86，但清除标志 -92。

19.6 MASD 的其他功能

19.6.1 链接器

这是一个很实用的功能。如果把源代码分成若干部分，可以要求编译器分别编译各部分，再生成最终的 Code 对象。如果有两个源文件 ONE 和 TWO，只需在程序中写入：

```
'ONE
'TWO
@
```

' 符号用于把当前目录或目录层级中一直向上直到 HOME 的其他目录内某个文件的内容放入源代码。也可以指定文件的完整路径，不论它位于 HOME 层级还是端口中，例如：

```
'H/PROGS/MATH/ONE
'2/TWO
@
```

被链接的文件还可以继续链接其他文件，但链接总数最多为 64 个；路径层级中，被链接文件的深度不能超过 16 层。

19.6.2 指令控制符

指令控制符由 **!** 符号和随后的文本值组成，用于在编译源代码时即时改变 MASD 设置。开始学习时不必使用任何指令控制符，但它们有时很有用。MASD 文档包含完整说明；下面仅作简要概述。其中一些只适用于 ROM 2.00 或更高版本的 49G+。

指令控制符	说明
<code>!1-16</code> 或 <code>!0-15</code>	把半字节编号为 1-16 或 0-15。详见前面的 HP-ASM 选项说明；默认值为 0-15。
<code>!CODE</code> 或 <code>!NO CODE</code>	决定是生成带有序言 <code>\$02DCC</code> 的 Code 对象，还是生成不带序言的普通对象。
<code>!DBGON</code> 或 <code>!DBGOFF</code>	决定是否从用于调试的 <code>DISP</code> 和 <code>DISPKEY</code> 宏生成代码。
<code>!RPL</code> 、 <code>!ASM</code> 或 <code>!ARM</code>	分别切换到系统 RPL、Saturn 汇编或 ARM 汇编模式。
<code>!STAT</code>	显示或更新编译统计信息。
<code>!DBGINF</code>	让 MASD 生成调试信息，放在一个特殊格式的字符串中，并置于堆栈上已编译对象之后。
<code>!ABSOLUTE address</code>	从给定地址开始，切换到绝对寻址模式。
<code>!ABSADR address</code>	处于绝对寻址模式时，加入空指令，使代码从给定地址继续。
<code>!EVEN</code>	处于绝对寻址模式时，如果当前地址不是偶数地址，则在编译时报告错误。
<code>!FL=1.n</code> 或 <code>!FL=0.n</code>	分别设置或清除编译指令控制符标志 <code>n</code> 。
<code>!?FL=1.n</code> 或 <code>!?FL=0.n</code>	如果编译指令控制符标志 <code>n</code> 分别为已设置或已清除，则编译该行剩余部分。
<code>!ADR</code>	生成源代码中使用的所有常数和标签列表，而不是生成已编译对象。
<code>!PATH+ path</code>	把给定路径加入链接器的自动搜索路径。
<code>!JAZZ</code> 或 <code>!MASD</code>	分别决定使用 Jazz 风格还是 MASD 风格的局部变量。

只需把指令控制符单独写在一行，就能从源代码字符串中临时开启或关闭相应选项。许多指令控制符只在源代码顶部使用，但根据需要，其中一些也可以出现在源代码的任何位置。

19.6.3 入口点表

正如用助记符表示构成机器语言操作的数字一样，也可以用助记符表示计算器 ROM 中常用例程的地址。MASD 允许安装入口点表来保存这些助记符，并在编译源代码时自动查找正确的内存地址。入口点表很大，约 90KB；不过，如果愿意，所附工具可以生成一个较小的自定义入口点表，只包含您需要的入口点。

软件可从下列位置下载：

<http://www.hpcalc.org/details.php?id=3245>

→ <http://www.hpcalc.org/hp49/pc/entries/extable.zip>

所需文件为 `extable.hp`，其中包含 HP 49 系列支持的全部入口点。只需把它安装到 49 的任意端口即可使用。安装后，它还会自动附加前面介绍的开发库。

19.6.4 MASD 高级功能

MASD 具有许多高级能力，例如把代码放入由花括号 { 和 } 包围的代码块，以及在一行写入多条指令。使用 MASD 时，还可以在指令与其作用字段之间使用点号 .，而不是空格；一行有多条指令时，这能让代码更清楚。例如，`C=0 W` 可以写成 `C=0.W`。

还可以去掉操作与常数之间的空格，例如把 `D0=D0+ 5` 写成 `D0=D0+5`。如果等号右侧的第一个寄存器与等号左侧被赋值的寄存器相同，还可以省略等号左侧的寄存器以及等号本身；换句话说，`D0=D0+5` 可以进一步简化为 `D0+5`。

为了保持与功能较少的汇编器兼容，本书不会使用这些功能；MASD 文档中有详细说明，可从下列地址获得：

<http://www.hpcalc.org/details.php?id=2986>

→ <http://www.hpcalc.org/hp49/docs/programming/masddocs.zip>

19.7 其他工具

还有一些附加工具可以让程序员的工作更轻松。

Nosy 是一款优秀工具。它允许您在 ROM 代码中浏览并检查任意内容。您可以给它一个命令名称，查看实现该命令的代码，并从系统 RPL 命令一路跳转到下层汇编语言代码。甚至还能在 ROM 中搜索某个例程的使用示例。学习汇编语言的最佳方法之一，就是查看现有代码示例；而 49 的 ROM 是高效代码的杰作，没有比它更好的学习材料。

Nosy 由 Jurjen N.E. Bos 编写，可从下列位置下载：

<http://www.hpcalc.org/details.php?id=4323>

→ <http://www.hpcalc.org/hp49/programming/misc/nosy.zip>

另一个包含大量实用工具的库是 Wolfgang Rautenberg 编写的 OT49。OT49 并非专门为汇编语言程序员设计；实际上，它更多面向用户 RPL 和系统 RPL 程序员，但其中有许多任何人都会觉得方便的工具。使用 OT49，可以轻松检查按键功能、切换设置、压缩和解压对象、快速删除对象、快速建立库等。

OT49 可从下列地址下载：

<http://www.hpcalc.org/details.php?id=3397>

→ <http://www.hpcalc.org/hp49/utils/interface/ot49.zip>

对愿意尝试几款尚未完成但仍然有用的程序的开发者来说，Pierre Tardy 编写了一个原始调试器和一个替代 49 反汇编器。

调试器 Debug49 是对内置 DISP 宏的改进。它增加了多个屏幕，可查看一部分寄存器、接下来的几条指令和内存，还能逐步执行程序。虽然开发似乎已经停止，但它的功能仍然很完善，而且完整源代码按 GPL 提供，因此任何人都可以从作者停下的地方接手。Debug49 可从这里下载：

<http://www.hpcalc.org/details.php?id=4715>

→ <http://www.hpcalc.org/hp49/programming/misc/debug49.zip>

反汇编器 xASM-> 旨在让汇编语言反汇编输出更易阅读，同时速度远高于内置汇编器。它尽可能使用紧凑的 MASD 助记符，并输出宏和入口点助记符，使反汇编结果更清楚。xASM-> 不提供源代码，但已经基本完成，仍可能很有用。可从这里下载：

<http://www.hpcalc.org/details.php?id=4456>

→ <http://www.hpcalc.org/hp49/programming/misc/xasmtto.zip>

20 使用 PC

本书主要面向在计算器上进行汇编语言编程的读者，但我们知道，也有许多人更愿意在 PC 上开发。这里不详细讨论 PC 工具，只对现有软件作简要概述。

最初，所有 PC 开发都使用一组面向 DOS 计算机的程序，称为 HP Tools。它发布在 Joe Horn 的 Goodies Disk 4 中，并使用所谓的“HP 语法”助记符。本书使用 HP-ASM/MASD 语法，因此需要使用 HP Tools 3.0 系列。该系列由 Hewlett-Packard 和 Jean-Yves Avenard 编写，同时提供源代码和二进制版本，可从下列位置获得：

<http://www.hpcalc.org/details.php?id=4263>

→ <http://www.hpcalc.org/hp48/pc/programming/hptools-src-3.0.8.tar.gz>

<http://www.hpcalc.org/details.php?id=4264>

→ <http://www.hpcalc.org/hp48/pc/programming/hptools-3.0.8-win32.zip>

HP Tools 文档包含在源代码压缩包中。上面只列出了 32 位 Windows 的二进制文件，但源代码应当也能在任何 Unix 兼容系统上编译，包括 Linux 和 Mac OS X。

对 Windows 计算机而言，还有一个友好得多的系统 Debug4x，由 Hewlett-Packard、Cyrille de Brébisson 和 William Graves 编写。

Debug4x 包含汇编语言开发所需的一切，功能还不止于此。完整安装约占 45MB 磁盘空间，二进制文件和源代码可从下列位置获得：

<http://www.hpcalc.org/details.php?id=5441>

→ <http://www.hpcalc.org/hp49/pc/programming/debug4x.zip>

<http://www.hpcalc.org/details.php?id=5455>

→ <http://www.hpcalc.org/hp49/pc/programming/debug4xsource.zip>

它包含项目管理器、编辑器、调试器和详尽帮助。最重要的是，它附带 48 和 49 模拟器，甚至对 49G+ 提供有限支持。由于功能很多，建议使用高分辨率屏幕；即使在 1024×768 分辨率下，界面仍会显得相当拥挤。

其汇编器使用 MASD 语法，与本书相同。唯一的区别是，代码必须以 `CODEM` 开始，并以 `ENDCODE` 结束，而不是用 `@` 结束。这样，即使既没有 48 也没有 49，仍然可以生成适用于这两种计算器的代码。

Debug4x 最强大的部分是调试器。您可以轻松地在代码中设置断点，并在断点处检查寄存器内容；然后逐步执行代码、观察寄存器值，在运行期间添加新断点，甚至编辑寄存器内容。如果经常犯一些不经意的错误，而我们都会如此，例如装入新值前忘记清空寄存器，那么这个功能极其宝贵。

另一个优点是，使用模拟器时，不必担心程序崩溃后丢失内存内容，而且代码运行速度可以明显更快。遗憾的是，模拟器毕竟不是真实硬件；它偶尔会误导程序员，虽然这种情况非常少见。因此，最终仍必须在真实计算器上测试。

第三部分 Saturn 处理器

21 Saturn 寄存器

下面是 Saturn 处理器内部结构的示意图：

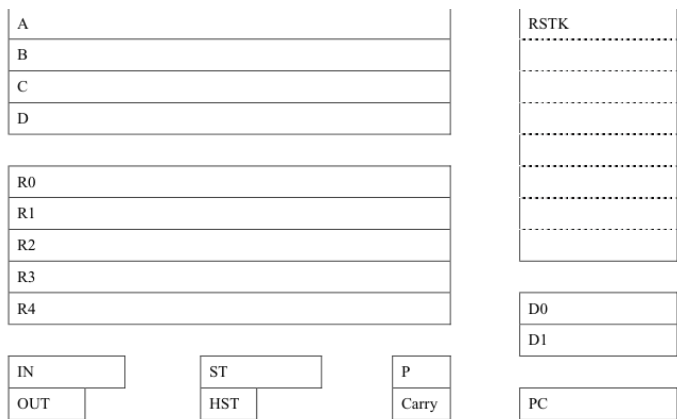


图 21-1 Saturn 处理器寄存器布局

图例：A-D 为工作寄存器；R0-R4 为保存寄存器；D0、D1 为指针寄存器；RSTK 为返回栈；PC 为程序计数器。

如上图所示，处理器内部包含若干部分，每一部分都位于各自的框中，并有自己的名称，例如 D0、D1、RSTK 等。请注意，该图并非严格按比例绘制，不过大部分部件的相对大小比较接近实际情况。

这些都是 Saturn 处理器的内部部件，用来让处理器完成您要求的工作。

首先，处理器在 HP 计算器内部位于何处？它不仅与内存相连并处理内存中存储的信息，还与键盘和蜂鸣器相连。因此，我们有两个重要部分：第一是处理器，也称 CPU，即 Central Processing Unit（中央处理单元）；第二是内存。

简单来说，汇编程序从内存或输入设备读取信息，用简单操作对其进行修改，通常是逻辑运算或代数运算，然后把结果写回内存或输出设备。

因此，Saturn 处理器的工作就是执行指令，也就是运行程序，并处理内存中的数据。

21.1 处理器寄存器

处理器内部有一些小型区域，它们与内存相似，也能以半字节为单位存储信息。寄存器分为三类：有些称为“指针”，有些直接称为“寄存器”，另一些称为“保存寄存器”。

21.1.1 指针寄存器

Saturn 处理器有两个指针寄存器：D0 和 D1。

每个指针寄存器都能存储 20 位。将 20 除以 4，得到由 4 位组成的数据包，也就是半字节，结果为 5。这意味着，从硬件角度看，Saturn 有两个指针寄存器，每个寄存器都能容纳 5 个半字节，以便“指向”内存中的某个位置。

例如，如果要读取地址 ABCDE 中存储的一个半字节，就可以用下面的汇编指令将该地址装入 D0 或 D1：

| D0= ABCDE

或：

| D1= ABCDE

此后再读取半字节时，读取的就是内存地址 ABCDE 处的数据。

因此，一个地址由 5 个半字节组成，Saturn 处理器可以“指向”RAM 或 ROM 中从 #00000h 到 #FFFFFFh 的任意半字节。这样共有 1 048 576 个半字节，每个半字节为 4 位。后文还会进一步介绍内存。

另有一个指针寄存器叫作 PC。PC 表示 Program Counter（程序计数器）。它是一个非常重要的寄存器，其中保存当前正在执行的指令在内存中的地址。处理器靠它确定自己当前位于何处，以及接下来要去哪里。

21.1.2 计算字节和 KB

把刚才的数值 1 048 576 除以 1024。为什么要这样做？

在计算机以及大多数具有内存的设备中，我们需要说明可用“空间”的大小。

16. 您已经知道，4 位组成 1 个半字节。

17. 2 个半字节共同组成 1 个字节。

字节是一种基本单位。随着时间推移，计算机的内存越来越大，因此我们使用指数来表示这些日益庞大的数值，但原理并没有改变。

不断将 2 相乘，可以得到下列数值：

幂	数值
2^1	2
2^2	4
2^3	8
2^4	16
2^5	32
2^6	64
2^7	128
2^8	256
2^9	512
2^{10}	1024

1024 对我们非常重要，因为这里采用二进制。

前缀“kilo”通常表示其后单位的 1000 倍。例如，1 千克等于 1000 克。但在表示内存时，采用的是 1024，而不是 1000。

这一点很重要：1 千字节不是 1000 个字节，而是 1024 个字节。

因此，我们先有“字节”，即 2 个半字节。由字节可构成千字节，缩写为 KB；1 KB 等于 1024 字节。

在 48 上，通过 `Left-Shift`、`VAR`、`BYTES` 运行 `BYTES`；在 49 上，则通过 `Left-Shift`、`PRG`、`MEM`、`BYTES` 运行 `BYTES`。HP 会在栈 2 给出对象的字节大小，在栈 1 给出对象的 CRC。CRC 也称“校验和”，是对对象执行某种数学运算后得到的最终结果。CRC 的数值较短，因此多个不同对象可能算出相同的 CRC；不过发生这种情况的概率足够低，所以 CRC 仍是一种可靠手段，可用来比较两个看起来相同的对象是否确实相同。

要确定一个对象占用多少 KB，请运行 `BYTES` 命令，再把所得字节数除以 1024。

如果您使用的是带 32 KB 内存的 48G，那么它有 $32 * 1024$ 字节 RAM，即 32768 字节。再将该数乘以 2，可得 48G 的 RAM 半字节总数：65536。

21.1.3 可寻址空间

Saturn 能访问多大的内存空间？我们知道，处理器有两个指针寄存器 D0 和 D1，每个寄存器中都能保存一个 5 半字节的值。5 个半字节共有 1 048 576 种不同取值。由于 HP 的 Saturn 以半字节为单位工作，因此它可以访问 1 048 576 个半字节，也就是 524 288 字节。把这个数除以 1024，得到多少？正好是 512 KB。

因此，Saturn 在执行过程中的任一时刻只能管理 512 KB 内存。后文会介绍一些管理超过 512 KB 内存的技巧。这称为可寻址空间。

21.1.4 工作寄存器

这些寄存器有时也称计算寄存器，是处理器实际进行运算的地方。如果要把内存一个区域中的值与另一个区域中的值相加，必须：

18. 将第一份和第二份数据装入工作寄存器；如果没有第二份数据，则只装入第一份；
19. 对数据进行运算；
20. 按需要处理所得结果。

因此，如果要对数值进行加减、移位、逻辑运算等操作，就必须先把数值装入工作寄存器。在 Saturn 中，这些工作寄存器称为 A、B、C 和 D。

所以一共有 4 个工作寄存器。每个寄存器的宽度为 64 位，即 16 个半字节。

也就是说，Saturn 处理器能够在一个工作寄存器内、用一步操作处理 64 位数据。以它所处的年代而言，这是一款相当不错的处理器，尽管十年之后它可能就不再显得那么强大。

21.1.5 保存寄存器

共有 5 个保存寄存器：R0、R1、R2、R3 和 R4。

这些寄存器用于在处理器内部暂存数据，从而释放一个或多个工作寄存器，让它们接收新数据。如果要腾出工作空间，把半字节从工作寄存器移动到保存寄存器，通常比把它们写到内存中的其他位置更好。这是因为，保留在处理器内部的数据操作速度比与处理器外部内存交换数据更快。

当然，每个保存寄存器都与工作寄存器大小相同：64 位，即 16 个半字节。因此，每个保存寄存器都能保存任意一个工作寄存器的完整内容。

到这里，我们已经知道：

21. A、B、C 和 D 是工作寄存器；处理器在这些寄存器内处理半字节。
22. 5 个保存寄存器中的任意一个，都可以短时间保存一个或多个工作寄存器的内容；它们是临时区域。
23. D0 和 D1 用于指向 Saturn 处理器能够访问的任意地址，地址由 5 个半字节组成。
24. PC 也是指针寄存器，但它只指向当前指令；处理器在运行机器语言代码时会不断更新它。
25. 当 D0 或 D1“指向”某个内存区域时，可以在该地址与工作寄存器之间读取或写入半字节。

22 工作寄存器的字段

我们已经知道，工作寄存器能够保存半字节。每个工作寄存器的大小为 64 位，也就是 16 个半字节。

很多时候，我们并不需要处理寄存器中的全部 16 个半字节。也许只需要处理 5 个、2 个，甚至仅 1 个半字节。

所谓字段，就是工作寄存器的一部分。例如，寄存器宽 16 个半字节时，可以把其中一个宽 5 个半字节的区段称为该工作寄存器的一个“字段”。

Saturn 处理器已经预定义了若干字段，便于我们使用。

在开始学习字段之前，先介绍 Saturn 处理器一个看起来有些奇怪的特性：半字节顺序反转。请特别注意这一部分。

22.1 半字节顺序反转

这一点对有些读者而言可能比较复杂。Saturn 在内存与寄存器之间读写时，会反转半字节的顺序。这里所说的“读取半字节”，是指把半字节从某个内存地址移动到工作寄存器；“写入”则是把半字节从工作寄存器移动到某个内存地址。

工作寄存器最多可以存储 16 个半字节。如下图所示，寄存器中的每个半字节都有编号。编号 0 是第一个半字节，也就是 LSB 半字节；编号 #Fh 是最后一个半字节，也就是 MSB 半字节。编号顺序从右向左。

任意工作寄存器的 16 个半字节如下：

| F E D C B A 9 8 7 6 5 4 3 2 1 0

请注意，编号从右向左，因此寄存器的“起点”是最右侧半字节，“终点”则是最左侧半字节，即编号 #Fh。

尽管大多数读者从左向右阅读，Saturn 却从 LSB 半字节开始移动数据，也就是先移动编号 0，再移动编号 1，依此类推。

因此，Saturn 处理器从右向左“读取”和“写入”。对 Saturn 而言，最右侧半字节才是第一个半字节。这称为“小端序”架构，x86 PC 也是如此。另一方面，Macintosh 的 68000 和 PowerPC 970，以及 SPARC，采用“大端序”架构，其中 MSB 位于左侧，LSB 位于右侧。某些 UNIX 系统，例如 Alpha、PowerPC 架构上的系统，以及 ARM 架构，可以切换为任一种方式运行。

以寄存器 C 为例。它宽 16 个半字节，因此可以设想 C 如下：

| F E D C B A 9 8 7 6 5 4 3 2 1 0
| 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

现在再看内存中的下列内容：

位移	内存地址	半字节
0	00100	1B01C
+5	00105	022F0
+10	0010A	00033

如果从地址 00100 开始，把 15 个半字节读入 C 寄存器，C 将包含下列数值：

| F E D C B A 9 8 7 6 5 4 3 2 1 0
| 0 3 3 0 0 0 0 F 2 2 0 C 1 0 B 1

编号 #Fh 的半字节没有被修改，但可以看出 Saturn 是从右向左装入半字节的。内存中若为 1B01C 022F0 00033，工作寄存器中就会得到 33000 0F220 C10B1。这里删去了编号 #Fh 的半字节，因为本例只处理 15 个半字节。请记住，读取寄存器内容时要从右向左看。

22.2 可用字段

下面的图非常重要，您应当熟记，最好能凭记忆画出来。这样就不必每次都去查找这些信息。

半字节编号： F E D C B A 9 8 7 6 5 4 3 2 1 0
W 字段： [-----]
S 字段： [S]
M 字段： [-----]
X 字段： [-----]
A 字段： [-----]
XS 字段： [XS]
B 字段： [---]

在本文中，可能会看到 B(A) 这样的写法，它表示寄存器 B 的 A 字段。其他程序员有时写成 B[A]、B.A 或 Ba；在别处看到不同记法时，请不要混淆。

22.2.1 W 字段

这个字段很简单：它就是整个 64 位寄存器。

如果给每个半字节编号，因为共有 16 个半字节，所以编号从 0 到 15。W 包含全部 16 个半字节：

| F E D C B A 9 8 7 6 5 4 3 2 1 0
| <----- W ----->

W 表示 word，有时也解释为 wide。

如果希望一条指令处理工作寄存器的全部 16 个半字节，就使用 W 字段。

22.2.2 A 字段

这是使用最频繁的字段，宽 5 个半字节。

A 表示 address。

它的范围如下：

```
| F E D C B A 9 8 7 6 5 4 3 2 1 0
|                                     <--- A --->
```

请记住，第一个半字节编号为 0，而不是 1。因此，A 字段的 5 个半字节依次为 0、1、2、3、4。

不要忘记，顺序是从右向左。

22.2.3 B 字段

这是另一个非常有用的字段：字节字段。

两个半字节组成一个字节，所以 B 字段宽 2 个半字节：

```
| F E D C B A 9 8 7 6 5 4 3 2 1 0
|                                     <-B->
```

还记得 MSB 和 LSB 吗？可以说，B 字段是工作寄存器 LSB 的一部分，也可以把它看成该寄存器的最低有效字节。

22.2.4 X 字段

X 字段宽 3 个半字节。它与处理器的另一个部件 OUT 配合时非常有用。

OUT 用于检测按键，或通过 HP 的蜂鸣器发声。OUT 接收 3 个半字节的数据，因此 X 字段会与 OUT 配合使用；它还有其他许多用途。

```
| F E D C B A 9 8 7 6 5 4 3 2 1 0
|                                     <-- X -->
```

X = eXponent，参见下面的 M 字段。

22.2.5 XS 字段

XS 字段是寄存器的第 3 个半字节，也就是 X 字段的 MSB。

XS = eXponent Sign。

22.2.6 M 字段

这是 Mantissa 字段，包括半字节 3 到 14。

该字段源于 HP-71 的 Saturn 处理器表示数字的方式：S 用于符号，M 用于 Mantissa，X 用于 eXponent。把实数压入堆栈时仍会使用这种格式。

现在您就知道这些字母的来历了。

```
| F E D C B A 9 8 7 6 5 4 3 2 1 0
| <----- M ----->
```

22.2.7 S 字段

S 字段是工作寄存器最后一个半字节，即编号 15。

22.2.8 字段汇总

请记住以下几点：

- 26. 工作寄存器可以划分为多个字段。
- 27. 指令经常只作用于某个字段。
- 28. 这里介绍的字段大小都是固定的。

各字段大小如下：

字段	大小	含义
W	64 位，即 16 个半字节	Whole register，整个寄存器
S	4 位，即 1 个半字节	Sign，符号
M	48 位，即 12 个半字节	Mantissa，尾数
X	12 位，即 3 个半字节	eXponent，指数
A	20 位，即 5 个半字节	Address，地址
XS	4 位，即 1 个半字节	eXponent Sign，指数符号

字段	大小	含义
B	8 位，即 2 个半字节	Byte，字节

23 P 寄存器和 WP

上文介绍了 7 个预定义字段。另有一个 WP 字段，其大小可由程序员通过 P 寄存器自定义。

先介绍 P。这个寄存器只能存储 1 个半字节，但它非常特殊，其取值范围为 0 到 F。

29. 可以把 P 用作字段。如果 P 中保存数值 n，那么 P 字段就是工作寄存器中编号为 n 的半字节。例如，若 $P=2$ ，使用 P 字段时就会读写编号 2 的 1 个半字节，也就是寄存器中的第 3 个半字节，因为编号从 0 开始。

30. P 定义 WP 的大小。WP 表示 Wide-P，它包含从第一个半字节到编号 n 的半字节。例如，若 $P=5$ ，WP 就包含半字节 0 到 5，因此用 WP 可以处理 6 个半字节。

WP 适合读取长度可变的信息，例如读取二进制整数。

请记住：

31. 如果要读取寄存器中不是第一个的某个半字节，请把它的位置编号装入 P，然后用 P 字段读取。例如，要读取编号 8 的半字节，就先执行 $P=8$ ，再用 P 字段读取该处的 1 个半字节。后文会给出具体方法。

32. P 定义 WP 字段的大小，范围从 0 到 n，其中 n 是 P 的值。如果要读写 m 个半字节，就执行 $P=(m-1)$ ，再把 WP 用作字段。

下面是关于 P 值的一条极其重要的规则。

程序开始时 $P=0$ ；程序结束时，必须保证 P 仍被设为 0。如果修改过 P 的值，退出前必须恢复为 0。

为什么？因为向寄存器装入数据时，P 会决定从哪个半字节开始装入。

有指令可把半字节装入 C 或 A 寄存器，但没有指令能直接装入 B 或 D。如果要把半字节装入 B 或 D，必须先装入 C 或 A，再把这些半字节复制到 B 或 D。

若 P 为 0，并把 #A3Bh 装入 C，将得到：

```
F E D C B A 9 8 7 6 5 4 3 2 1 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 B 3 A
```

但若 P 为 2，再把 #A3Bh 装入 C：

```
F E D C B A 9 8 7 6 5 4 3 2 1 0
0 0 0 0 0 0 0 0 0 0 0 0 B 3 A 0 0
```

若 P 为 E，并装入 #A3Bh：

```
F E D C B A 9 8 7 6 5 4 3 2 1 0
3 A 0 0 0 0 0 0 0 0 0 0 0 0 0 0 B
```

怎么会这样？

这 3 个半字节从半字节 E 开始装入。由于要装入 3 个半字节，其中两个进入 #Eh 和 #Fh，最后一个则绕回工作寄存器的开头。数值会沿寄存器环回。

后文会更详细地解释这一点。

24 用户定义字段

在采用 ARM 处理器模拟 Saturn 的“Apple”平台计算器中，也就是写作本书时的 49G+ 和 48GII，“Saturn+”架构为 Saturn 增加了额外功能，其中包括用户定义字段。开发者可以创建一个 64 位掩码，并将其赋给某个用户定义字段。对该字段执行的任何操作，都只会影响掩码所指定的位。这些字段称为 F1 到 F7。

要使用这些字段，请先把如下掩码装入 C：

```
F E D C B A 9 8 7 6 5 4 3 2 1 0
F F C 0 0 0 0 0 0 0 0 0 0 0 3 F F
```

然后调用 7 条“设置字段”指令之一，例如 SETFLD7。此后，字段 F7 只会影响上面定义的非零位。

25 处理器内部数值的环回处理

重要内容，如有需要请阅读两遍。

使用字段时，我们会限定参与处理的半字节数量。

B 字段处理 2 个半字节。所有指令，例如给数值加上某个值，都只会作用于这 2 个半字节，而不会影响寄存器剩余的 14 个半字节。

如果某寄存器的 B 字段包含 #FFh，此时再加 1，会发生什么？请记住，我们使用的是 B 字段，因此结果不能从 #FFh 变成 #100h，因为后者需要 3 个半字节。

Saturn 会这样处理：

#FFh + 1 = #00h，前提是操作字段为 B；同时，称为 carry 的进位标志会被设为 1。

carry 本身是一个标志。标志是处理器内部可供测试的指示量，后文会详细说明。

如果使用 A 字段，且 A 中包含 #0000h，减去 1 会在 A 字段中得到 #FFFFFFh，同时进位标志被设为 1。

如果某寄存器的 X 字段含有 #FEAh，再加上 #20h，X 字段会得到 #00Bh。

请注意，对寄存器执行加法或减法等操作时，如果因为在字段中“绕回”而丢失信息，例如对 B 字段执行 #FFh + 1 得到 #00h，处理器会把进位位设为 1，以发出警告。

还有一种操作也可能导致信息丢失：位移。

开始介绍具体指令时会说明这一点。本书将完整介绍每条指令，包括编码方式、占用的处理器周期数等。

把目前学到的 Saturn 处理器知识汇总如下：

33. 4 个工作寄存器：A、B、C 和 D。
34. 5 个保存寄存器：R0、R1、R2、R3 和 R4。
35. P：可访问寄存器中的任意半字节，并定义 WP 字段的大小。
36. WP：范围从半字节 0 到半字节 (n-1)，其中 n 是 P 中存储的值。
37. D0 和 D1：指针寄存器，可指向 RAM/ROM 中从 #0000h 到 #FFFFFFh 的任意半字节。
38. PC：包含当前正在执行的指令。

接下来需要学习 RSTK、ST、HST、OUT 和 IN。

26 RSTK

RSTK 表示 Return STack，即返回栈。

首先，什么是“栈”？作为 HP 用户，您很可能已经在使用 HP 时用过堆栈。由于内存结构的特点，栈在计算机中使用非常广泛。将来 3D 芯片问世，而且 HP 吞并 Intel 和 Microsoft 之后，情况或许会改变；不过在那之前，请耐心等待，并替我们保守这个秘密。

RSTK 是一个 LIFO 栈。LIFO 表示 Last In First Out，即后进先出。

与 HP 的堆栈一样，如果把一个数压入 RSTK，最先弹出的将是最后压入的那个数。

换一种说法，它就像把硬币从管子的一端塞进去，再从同一个开口取出。另有一种栈称为 FIFO，即 First In First Out，先进先出；它像一根两端开口的管子，硬币从一端放入，从另一端取出，最先进入的硬币也最先被取回。

RSTK 采用 LIFO，因此类似于 HP 的 RPL 堆栈。不过它的容量有限：共有 8 层，每层大小固定为 5 个半字节。

子例程会使用 RSTK。从汇编语言程序调用子例程时，处理器会记录当前位置，把它存入 RSTK，然后跳转到子例程。当子例程要返回调用点时，它从返回栈弹出“返回地址”，并跳转到该地址。由于保存的是地址，RSTK 每层只需要 5 个半字节。

每次从栈中弹出一个地址后，顶部一层都会被 0 替代。此外，这个栈具有“环回行为”，所以即使发生栈溢出，仍可以读到最后 8 个地址。例如，依次向 RSTK 写入 1、2、3、4、5、6、7、8、9，随后可以依次读出 9、8、7、6、5、4、3、2 和 0。也就是说，先读出仍保留的 8 层，再读出一个 0；随着各层下移，0 会填入栈中。

因此，RSTK 包含 8 个 5 半字节地址。HP 的中断系统被调用时会占用 2 层，有时还会占用第 3 层。因此，除非禁止所有中断，否则程序自身只能使用 5 层。

RSTK 很适合快速保存临时信息。后文还会继续讨论。

27 标志：ST、HST 和 Carry

标志是特殊的位，只能取 1 或 0。标志置位时，其值为 1；标志清零时，其值为 0。

27.1 Carry

Carry 本身就是一个标志位。每当某项运算发生进位，例如加法、减法，或者寄存器发生溢出时，进位位就会被置位，表示发生了进位或溢出。

这里所说的溢出已在上文解释过。使用工作寄存器的某个字段时，如果加上或减去一个数值使结果在字段中“绕回”，进位位就会被设为 1。例如，若 C 的 A 字段包含 #FFFFFFh，给该字段加 1 后，C 的 A 字段会变成 #0000h，同时进位位被设为 1。

每次执行测试时，若测试为真，进位位就会置位；若测试为假，进位位就会清零。
也就是说，每次执行测试，处理器都会明确设置或清除进位位。随后再测试进位位，以决定下一步操作。

27.2 ST（状态位）

ST 标志由 4 个半字节组成，因此共有 16 位可用，每一位都是一个标志。
程序员可以使用标志 0 到 11。标志 12 到 15 具有特殊含义，误用可能造成危险。
位 12 到 15 的含义如下：

位	含义
12	强制唤醒请求，覆盖 DeepSleep
13	发生中断时置位
14	有中断待处理时置位
15	中断启用时为 1，中断禁用时为 0

ST 位 0 到 11 可以按您的需要赋值。之后可以逐位测试，并由程序自行规定每一位的含义。
位 12 到 15 由 Saturn 处理。后文有时会使用位 15，但不要随意改动其他几位。

27.3 HST（硬件状态位）

该寄存器宽 1 个半字节，也就是 4 位。Saturn 用这 4 位处理特殊硬件事件。各位如下：

位	名称与含义
0	XM, eXternal Module missing, 外部模块缺失
1	SB, Sticky Bit, 粘滞位
2	SR, Service Request, 服务请求
3	MP, Module Pulled, 模块被拔出

在原始 Saturn 上，我们只能主动设置其中一个位：XM。后文会使用一条名为 RTNSXM 的指令，即 ReTurN and Set XM。ARM 架构计算器上的 Saturn+ 则允许设置或清除任意一个 HST 位。
SB 非常有用：在寄存器中移位时，如果某个位从 MSB 一侧移出，或从 LSB 一侧移出，Saturn 会把 SB 置为 1，警告移位过程中发生了信息丢失。也就是说，如果边界处值为 1 的位被移出字段，就会发生这种情况。由于它是粘滞位，一旦变为 1，就会一直保持为 1，直到手动清零。

28 OUT 和 IN

Saturn 处理器能够与键盘交换信息，也能产生声音。处理器内部有两个寄存器用于此目的：OUT 和 IN。它们的大小固定如下：

- 39. OUT 宽 3 个半字节，即 12 位。
- 40. IN 宽 4 个半字节，即 16 位。

如果要检测某个按键，先根据数值表把一个值装入 C 等寄存器，再把它发送到 OUT 寄存器。随后，键盘中的一条或多条线路会被通电；如果有按键按下，结果会写入 IN 寄存器。读取该值后，即可判断是否有按键被按下，以及具体是哪一个按键。

29 处理器如何工作：一条指令的生命周期

Saturn 如何执行一条指令？

我们知道，处理器使用 PC 寄存器，也就是程序计数器，确定自己当前“位于”何处。处理器会不断更新该计数器，以便知道到哪里取下一条要执行的指令。

处理器执行一条指令的周期如下：

- 41. PC 寄存器告诉处理器下一条待执行指令的位置。
- 42. 该地址被送上 Saturn 处理器的总线。总线是一组把处理器、RAM、ROM、输入/输出部件等连接在一起的线路。
- 43. Saturn 把待执行指令接收到内部寄存器。随后必须对指令进行译码，因为处理器需要用专门的电路理解指令。

44. 负责译码的电路完成工作后，会把指令交给处理器中负责安排命令顺序的单元。该单元根据石英晶体产生的频率执行命令。石英晶体以 32 kHz 工作，再经倍频生成时钟频率。HP 48S/SX 的频率约为 2 MHz，HP 48G/G+/GX 和 49G 的频率约为 4 MHz。HP 48GII 和 49G+ 的频率可变，最高分别为 48 MHz 和 75 MHz；它们以可变速度模拟 Saturn，等效速度通常分别约为 8 MHz 和 12 MHz，但运行原生代码时快得多。
45. 负责安排命令顺序的单元通过调用 ALU（Arithmetical Logical Unit，算术逻辑单元）发出操作命令。ALU 负责逻辑运算，例如取补、逻辑 OR/AND/NOT，以及算术运算，例如减法和加法。
46. 结果存入处理器的某个寄存器。PC 随后递增，指向下一条指令。

然后重复这一过程。

这就引出了下面的概念。

30 什么是处理器周期？

当我们要求处理器完成某项工作时，即使在我们看来很简单，例如给寄存器中的数值加 1，处理器实际执行的过程也更复杂。它必须取指、译码，再把指令交给负责执行的单元，等等。

因此，每个步骤又被分解成基本操作，每个基本操作在一个周期中完成。例如，有的指令需要 11 个周期，有的则需要更多或更少。

每个周期就是一个“状态”，也可以理解为来自处理器石英晶体的一次脉冲。

以处理器频率约为 4 MHz 的 HP 48G 为例。每秒会产生 400 万次脉冲，而每条指令都需要预定数量的周期才能完成。

请注意，本书列出的指令周期数并非绝对精确值，因为要得到完全准确的数值几乎不可能，需要使用示波器。不过这些数值应当足够接近。周期数来自 Meta Kernel 文档，感谢 Jean-Yves，它们应与真实数值相当接近。仅在 49G+ 上提供的 Saturn+ 指令没有列出周期数；这些指令运行的是 ARM 代码，本来就不一定遵循 Saturn 的周期规则。

此外，周期数还可能取决于指令位于奇数还是偶数内存地址。有些指令给出小数周期数。对这类指令，如果它位于偶数地址，就向下取整，即取 FLOOR；如果位于奇数地址，就向上取整，即取 CEIL。若给出两个由逗号分隔的周期数，则在从偶数地址读取时加上第二个数的 FLOOR，从奇数地址读取时加上第二个数的 CEIL。由于代码地址通常具有随机性，落在奇数或偶数地址的概率相同，因此直接使用小数周期数，就能相当准确地估算执行时间。

31 复习：计算

这里有一项重要限制：可用的数位数量有限。这不仅是因为每个工作寄存器只有 16 个半字节，也就是 64 位，还因为计算时经常使用比 W 更小的字段。

一个字节由 2 个半字节组成，因此其最大值为 #FFh。

负数不太容易处理，但可以使用二进制补码。小数同样如此；我最早用来学习汇编语言的书曾说，汇编语言无法使用小数。事实上，我们既可以使用负指数，也可以固定小数点位置，还可以采用其他技巧，Sophie 就解释过其中一些方法（没错，也有女生在 HP 上用汇编语言编程）。

还必须理解数值的环回处理。每个字段只能容纳有限数量的半字节，所以在加上或减去常数时，字段内的数值可能越过 #0h，并设置进位标志。这对循环很有用，包括什么也不做的等待循环，以及重复执行某段代码的循环。

如果使用 X 字段，就有 3 个半字节可用于编码数值，取值范围为 #000h 到 #FFFh。给 #FFFh 加 #1h 会怎样？数值变成 #000h，同时进位位被置位。对 #000h - #1h 也是如此。

当数值从一个范围端点越过 #0h 到达另一端时，进位位会被置位，以警告发生了上溢或下溢。

如果您对移位不太熟悉，只需记住：

47. 向右移动 1 位，相当于除以 2。
48. 向右移动 1 个半字节，相当于除以 16。
49. 向左移动 1 位，相当于乘以 2。
50. 向左移动 1 个半字节，相当于乘以 16。

或者更简单地记为：

```
| CSRB f
```

将字段 f 除以 2。

```
| CSR f
```

将字段 f 除以 16。

```
| C=C+C f
```

将字段 f 乘以 2。

```
| CSL f
```

将字段 f 乘以 16。

如果您现在还不理解这些指令，不必担心。这里给出的示例都使用 C 寄存器；若要使用其他寄存器，需要修改第一个字母。下一部分会详细讲解。可以看到，要把一个寄存器或其中一部分乘以 2，只要把它与自身相加即可。

第一次没有完全理解也很正常。

第四部分 Saturn 指令集

32 入口点及其他说明

本书将介绍的一些指令会调用计算器 ROM 中的代码。ROM 中的代码位于称为“入口点”的内存地址。有些入口点在所有计算器中都相同，但许多入口点在 HP 48 和 HP 49 上的地址不同。还有一些称为不受支持的入口点，它们的地址可能随 ROM 版本而变化；为保证兼容性，最好避免使用这些入口点。

如果计算器中安装了入口点表，那么带 Meta Kernel 的 HP 48 上的 MASD，以及 HP 49 上的 MASD，都允许输入入口点名称而不是地址，MASD 会自动把名称转换为地址。本书所有示例都会在源代码中写出口点名称，并在名称后的注释中列出地址，因为并非每位读者都能使用入口点表。

本部分列出的一些指令在操作码后带有加号 +。这些指令只适用于采用 ARM 的计算器，因为它们只存在于 Saturn+ 指令集中。它们也不列执行时间。不要尝试在普通 Saturn 计算器上使用这些指令。

33 工作寄存器指令

33.1 向寄存器装入数值

能够把数值装入工作寄存器非常有用。只有两条相关指令，并且只能把数值装入 A 和 C 寄存器。

LA，即 Load in A，用于把数值装入 A 寄存器；LC 用于把数值装入 C 寄存器。

例如，要把 #ABCDE 装入 C 寄存器，请写：

```
| LC ABCDE
```

代码运行后，C 寄存器如下，每个半字节都已移入寄存器：

```
| F E D C B A 9 8 7 6 5 4 3 2 1 0  
| 0 0 0 0 0 0 0 0 0 0 0 0 E D C B A
```

如前所述，寄存器从右向左读取，LA 和 LC 也按这一顺序工作。请注意，寄存器通常并不为 0；最后一个装入半字节之后的原有内容会保留下来。后文会通过示例说明。

工作寄存器 A 和 C 都宽 16 个半字节，因此无法向其中任意一个装入超过 16 个半字节。

下面看看超出限制会发生什么。把一个空字符串放到堆栈上并进行编辑，输入：

```
| LC 1234567890ABCDEF1  
| @
```

可通过 **ALPHA**、**Right-Shift**、**ENTER** 输入 @。必须包含 @，用来标明源文件结束。

现在输入 ASM；如果使用 HP-ASM 1.0，则输入 →ASM，以编译文件。

由于试图装入过多半字节，您会得到错误。

如果在 48 上使用 HP-ASM，堆栈中会出现：

```
| 3: "LC 12345678...  
| 2: <0h>  
| 1: "LC 12345678...
```

栈 3 中的字符串是原始字符串，栈 2 包含表示错误所在行的系统二进制对象，栈 1 包含发生错误的那一行。

此时可以使用 HP-ASM 的 ED 命令编辑源代码；它会把编辑器光标移动到出错行，但 ED 使用的是 HP 内置编辑器。

可以把 ED 自定义为使用任意编辑器，不过这需要一些库修改和 System RPL 编程知识。

在完成 ED 自定义之前，遇到错误时通常可以丢弃两个栈项，再用您常用的编辑器编辑原字符串。

如果在带 Meta Kernel 的 48 或 49 上使用 MASD，堆栈中会出现：

```
| 2: "LC 12345678...  
| 1: {{ ✖ 10108h ...
```

栈 2 中的字符串是原始字符串，栈 1 包含错误列表。此时可用 ER 命令编辑源代码并跳到错误位置。

使用 LC 或 LA，可以向 A 或 C 装入 1 到 16 个半字节。没有向 B 或 D 直接装入数值的指令，所以必须先把数值装入 A 或 C，再移动到 B 或 D。

使用 LA 或 LC 时，必须考虑 P 的值。程序开始时 P 为 0。您可以改变它，但退出程序时必须把它恢复为 0。

若 $P=0$ ，向 A 或 C 装入数据时，会从寄存器的半字节 0 开始。若 $P=4$ ，则从半字节 4 开始装入。

例如，当 $P=4$ 时把 ABCDE 装入 C，结果如下：

```
| F E D C B A 9 8 7 6 5 4 3 2 1 0
| 0 0 0 0 0 0 0 E D C B A 0 0 0 0
```

因此，如果修改了 P 的值，例如为了读取寄存器的半字节 n 或使用 WP，就必须留意 LA 和 LC 指令。

此外，装入数值会覆盖寄存器中原有的值。假设 C 原本包含：

```
| F E D C B A 9 8 7 6 5 4 3 2 1 0
| 3 A 8 B B 3 8 2 0 0 0 A F 0 F C
```

向其中装入 3 个 0 后，C 将包含：

```
| F E D C B A 9 8 7 6 5 4 3 2 1 0
| 3 A 8 B B 3 8 2 0 0 0 A F 0 0 0
```

装入数值时，寄存器中已有的半字节不会被“挪动”，因此原值会被覆盖。

前面还介绍过半字节会在寄存器中环回装入。如果把 P 设为 E，再用 LC ABCDEF 装入 6 个半字节，会得到：

```
| F E D C B A 9 8 7 6 5 4 3 2 1 0
| B A 0 0 0 0 0 0 0 0 0 0 F E D C
```

每次列出一条指令时，都会给出 5 项信息：

1. 助记符的名称，这里是 LA 和 LC。
2. 它在内存中的编码方式，即十六进制操作码。
3. 可以使用或实际使用的字段。
4. 是否影响进位位。若影响，运算后进位位必定被明确设为 1 或 0。
5. 处理器完成该指令所需的大致周期数。

33.1.1 LA、LC

助记符	操作码	字段	Carry	周期
LA $n..1$	8082 $n\ 1...n$	取决于 P 值	否	$7.5+1.5n$
LC $n..1$	3 $n\ 1...n$	取决于 P 值	否	$3+1.5n$

1 是第一个被装入的半字节，n 是最后一个。上文已经给出了半字节装入方式的示例。即使出现环回装入，LA 和 LC 也不会修改进位位。

可以看到，LC 比 LA 更快，而且在内存中编码所需的半字节更少。通常这并不重要，除非某段代码必须非常快，例如与屏幕显示有关的代码。如果需要速度，请优先使用 LC。

n 表示被移动的半字节数量。下面的代码：

```
| LC 02A2C
```

会把 5 个半字节移入 C 寄存器，需要 $3+1.5*5$ 个周期，也就是 10.5 个周期。

33.1.2 把寄存器置零

下面介绍如何把寄存器中的 1 到 16 个半字节置为 0。指令的第一个字母是寄存器名称 A、B、C 或 D，后接等号 = 和 0。

要清空整个 C 寄存器，也就是全部半字节，只需写：

```
| C=0 W
```

请记住，W 表示 word 或 wide，它是作用于整个寄存器的字段，也就是全部 16 个半字节。

第 22.2 节介绍的字段如下：

```
| 半字节编号： F E D C B A 9 8 7 6 5 4 3 2 1 0
| W 字段：      [-----]
| S 字段：      [S]
| M 字段：      [-----]
| X 字段：                        [-----]
| A 字段：                        [-----]
| XS 字段：                        [XS]
| B 字段：                        [---]
```

因此，如果要把寄存器 B 的 X 字段全部置零，应使用：

B=0 X

助记符	操作码	字段	Carry	周期
A=0 f	D0 (A 字段) Ab0 (P、WP、XS、S、M、B、W)	A; P、WP、XS、S、M、B、W	否	8; 4.5+n
B=0 f	D1 (A 字段) Ab1 (P、WP、XS、S、M、B、W)	A; P、WP、XS、S、M、B、W	否	8; 4.5+n
C=0 f	D2 (A 字段) Ab2 (P、WP、XS、S、M、B、W)	A; P、WP、XS、S、M、B、W	否	8; 4.5+n
D=0 f	D3 (A 字段) Ab3 (P、WP、XS、S、M、B、W)	A; P、WP、XS、S、M、B、W	否	8; 4.5+n

f 是什么？它就是字段。

A 字段需要单独处理，因为处理器使用 A 字段非常频繁，所以它有专门的十六进制编码形式。

还可以看到，“操作码”列通常列出十六进制数字，但有时也包含其他字母。必须根据字段查表，才能确定相应字母应替换成什么值。

例如，要执行 A=0 S，指令编码为 Ab0。根据下表可确定 b 的值：

字段	a 值	b 值
P	0	8
WP	1	9
XS	2	A
X	3	B
S	4	C
M	5	D
B	6	E
W	7	F

由于编码中出现的是 b，所以查 b 列。找到 S 后可见其值为 C，因此 A=0 S 的编码是 AC0。

49G+ 新增的用户定义字段操作使用 f。对应关系如下：

字段	f 值
F1	8
F2	9
F3	A
F4	B
F5	C
F6	D
F7	E

除非要编写反汇编器或汇编器，否则通常不需要这些信息；但若要编写，现在已经有了必要资料。

与往常一样，n 表示被清零的半字节数量。

现在开始编写代码。

下面编写一段把整个 C 寄存器置零后退出的代码。HP-ASM 要求每行只能有一条指令，所以写完一条完整指令后必须换行，如下所示：

```
C=0 W
A=DAT0 A
D0=D0+ 5
PC=(A)
@
```

如前所述，MASD 允许一行写多条指令，但本书中的所有代码都坚持每行一条，以获得最大的兼容性。

我知道这里有些指令尚未介绍，不过先编译并运行它。如果没有出错，会得到一个 Code 对象。

按 EVAL 运行。C 寄存器的 W 字段会被清零，然后代码结束。

下面解释最后 3 条指令。

```
A=DAT0 A
```

您已经知道，Saturn 处理器有两个指针寄存器 D0 和 D1。PC 也是指针寄存器，但这里不用。如果 D0 指向某个区域，那么 DAT0 就表示该处的数据；D1 同理。如果 D1 指向 #ABCDE，就用 DAT1 取得该处的半字节。

假设内存中有以下内容：

地址	半字节
#80120h	02A2C00005

如果要从 #80120h 读取 3 个半字节并放入 A，应写：

```
D1= 80120
A=DAT1 X
```

首先让 D1 指向 #80120h，再把 3 个半字节，也就是 X 字段，读入 A。

如果要使用 C 和 D0 读取 10 个半字节，应写：

```
D0= 80120
C=DAT0 10
```

字段位置也可以直接写数值。

现在解释下面三条指令：

```
A=DAT0 A
D0=D0+ 5
PC=(A)
```

当程序取得控制权时，HP 已经占用了若干寄存器和指针：

56. D1 指向堆栈的栈 1。
57. D0 指向下一条要执行的 RPL 对象。
58. 寄存器 B 的 A 字段指向 RPL 返回栈。
59. 寄存器 D 的 A 字段包含以 5 半字节块为单位的空闲内存量。

程序开始时，这些数值都在使用。有时必须“保存”某些寄存器值，因为一旦丢失 B 或 D0，就会出问题，HP 可能停止响应，甚至崩溃。

D0 指向下一条要执行的 RPL 对象。准备退出时，必须把下一个 RPL 对象的地址读入 A：

```
A=DAT0 A
```

此时 A 包含该地址，也就是从 D0 所指位置读取的 5 个半字节；A 字段本身正好宽 5 个半字节。

读完之后，必须把 D0 向后移动 5 个半字节。把 D0 递增到下一个对象非常重要。当前对象即将执行，所以之后从 D0 读取的指令必须读到正确位置。因此，刚读取了 5 个半字节，就要把 D0 向后移动 5 个半字节：

```
D0=D0+ 5
```

最后一条指令通过 PC 指针跳转到该地址。Saturn 处理器需要确定接下来从哪里继续时，会读取 PC。因此，让 PC 指向 A，执行便会继续。请记住，下面是通常用于“返回 RPL”的代码：

```
A=DAT0 A
D0=D0+ 5
PC=(A)
```

大多数程序都可以用它退出。

随着学习更多 Saturn 指令，后文还会给出其他短代码片段，例如 DROP、SWAP 等。

33.1.3 改变某一位的值

可以改变 A 或 C 寄存器前 4 个半字节中的任意一位，但不能直接对 B 或 D 这样做。前面已经介绍过，一位等于 1 时称为置位，等于 0 时称为清零。

对 A 寄存器使用 ABIT 指令，对 C 寄存器使用 CBIT 指令。要把某一位置位，使用：

```
ABIT=1 n
CBIT=1 n
```

要把某一位清零，使用：

```
ABIT=0 n
CBIT=0 n
```

其中 n 是被设为 1 或 0 的位编号，取值范围为 0 到 15。

注意： 有些人不喜欢从 0 开始计数，而喜欢从 1 开始。HP-ASM 和 MASD 允许选择计数方式，前文对两者的概述中已作说明。本书始终使用 0 到 15；如果您选择 1 到 16，就必须相应调整源代码。

助记符	操作码	字段	Carry	周期
ABIT=0 n	8084n	位编号 n	否	7.5
ABIT=1 n	8085n	位编号 n	否	7.5
CBIT=0 n	8088n	位编号 n	否	7.5
CBIT=1 n	8089n	位编号 n	否	7.5

示例：如果希望在代码启动时点亮闹钟指示符，应怎样做？

首先要知道，指示符其实只是 RAM 某个区域中存储的若干位。把这些位设为 1 可以打开指示符，设为 0 则可以关闭。

下面就介绍这些位及其特殊数值。

在地址 #0010Bh，即入口点 ANNCTRL，有两个半字节，也就是一个字节，用于控制指示符：

位编号	功能
0	左移位指示符，1=开，0=关
1	右移位指示符，1=开，0=关
2	alpha 指示符，1=开，0=关
3	闹钟指示符，1=开，0=关
4	忙碌指示符，1=开，0=关
5	传输指示符，1=开，0=关
6	保留供将来使用
7	是否显示指示符，1=开，0=关

因此，如果要打开闹钟指示符，就必须读取 #0010B 处的字节，把其中的位 3 设为 1，然后退出。

现在来完成它。

首先，需要让 D0 或 D1 指向该地址。但 HP 正在使用 D0 和 D1：D0 指向下一条要执行的 RPL 对象，D1 指向堆栈的栈 1。

所以必须“保存”D1 或 D0 的值，并在退出前恢复。

这里使用 C 读取该字节，用 A 保存 D1 的值，从而让 D1 可以暂时指向其他位置。

要移动 D1 的值，需要使用一条尚未介绍的指令 AD1EX。它把 D1 中的 5 个半字节移到 A，同时把 A 中的 5 个半字节移到 D1。执行后，D1 的原值会保存在 A 中。这条指令用于交换 D1 与 A 的内容，后文还会详细说明。

首先，把 D1 的值移到 A：

```
| AD1EX
```

然后，让 D1 指向 #0010Bh，并读取该处的一个字节：

```
| D1= 0010B
| C=DAT1 B
```

为什么使用 B？B 是最低有效字节，因此该指令读取 D1 所指内存位置的前两个半字节并放入 C。现在必须改变 C 中位 3 的值。可以使用刚学到的指令：

```
| CBIT=1 3
```

再把它写回内存：

```
| DAT1=C B
```

读写数据的指令将在后文解释，不过现在应该已经能够理解它的工作方式。

随后恢复 D1 的旧值：

```
| D1=A
```

再以通常的方式返回 RPL：

```
| A=DAT0 A
| D0=D0+ 5
| PC=(A)
```

因此，打开闹钟指示符并返回 RPL 的完整代码如下：

```
| AD1EX
```

```

% 入口点名称为 =ANNCTRL
D1= 0010B
C=DAT1 B
CBIT=1 3
DAT1=C B
D1=A
A=DAT0 A
D0=D0+ 5
PC=(A)
@

```

现在可以编译并运行。代码执行得非常快，必须仔细观察，也许还要多运行几次，才能看到闹钟指示符短暂闪亮。

33.1.4 交换两个寄存器

可以交换两个寄存器的值。既可以交换整个寄存器，也可以只交换其中较小的字段。

助记符	操作码	字段	Carry	周期
ABEX f	DC (A) AbC (其他字段)	A; P、WP、XS、S、M、B、W	否	8; 4.5+n
BCEX f	DD (A) AbD (其他字段)	A; P、WP、XS、S、M、B、W	否	8; 4.5+n
ACEX f	DE (A) AbE (其他字段)	A; P、WP、XS、S、M、B、W	否	8; 4.5+n
CDEX f	DF (A) AbF (其他字段)	A; P、WP、XS、S、M、B、W	否	8; 4.5+n

n 是两个寄存器之间交换的半字节数量。

注意：ACEX 与 CAEX 当然表示同一件事。HP-ASM 和 MASD 要求交换两个寄存器时按字母顺序书写。因此必须写 ACEX f，而不能写 CAEX f，因为汇编器不会识别后者。

我们知道没有 LD 指令，所以不能把数值直接装入 D。可以先把半字节装入 C 或 A，再与 D 交换，例如：

```

LC 124
CDEX X

```

这会交换 C 和 D 之间的 3 个半字节，从而把 #124h“移入”D。如果要交换两个完整寄存器，当然应使用 W 字段。

33.1.5 把一个寄存器复制到另一个寄存器

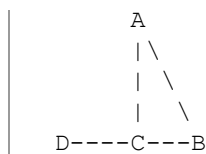
除了交换两个寄存器，还可以直接用一个寄存器中的半字节覆盖另一个寄存器。相关指令如下：

助记符	操作码	字段	Carry	周期
A=B f	D4 (A) Ab4 80B084f +	A; P、WP、XS、S、M、B、W; F1-F7	否	8; 4.5+n; 未列
B=A f	D5 (A) Ab5 80B081f +	A; P、WP、XS、S、M、B、W; F1-F7	否	8; 4.5+n; 未列
C=B f	D6 (A) Ab6 80B086f +	A; P、WP、XS、S、M、B、W; F1-F7	否	8; 4.5+n; 未列
C=D f	D7 (A) Ab7 80B08Df +	A; P、WP、XS、S、M、B、W; F1-F7	否	8; 4.5+n; 未列
A=C f	D8 (A) Ab8 80B088f +	A; P、WP、XS、S、M、B、W; F1-F7	否	8; 4.5+n; 未列
B=C f	D9 (A) Ab9 80B089f +	A; P、WP、XS、S、M、B、W; F1-F7	否	8; 4.5+n; 未列
C=A f	DA (A) AbA 80B082f +	A; P、WP、XS、S、M、B、W; F1-F7	否	8; 4.5+n; 未列
D=A f	80B083a + 80B083f +	A、P、WP、XS、S、M、B、W; F1-F7	否	未列

助记符	操作码	字段	Carry	周期
D=B f	80B087a + 80B087f +	A、P、WP、XS、S、M、B、W；F1-F7	否	未列
D=C f	DB (A) AdB 80B08Bf +	A；P、WP、XS、S、M、B、W；F1-F7	否	8；4.5+n；未列
A=D f	80B08Ca + 80B08Cf +	A、P、WP、XS、S、M、B、W；F1-F7	否	未列
B=D f	80B08Da + 80B08Df +	A、P、WP、XS、S、M、B、W；F1-F7	否	未列

指令很多。

仔细观察可以发现，在普通 Saturn 上，寄存器 D 只能与 C 之间传送数据。



这张图适用于所有涉及 A、B、C 和 D 的指令，请记住它。B 可以与 A 或 C 配合，A 可以与 B 或 C 配合，C 可以与 A、B 或 D 配合，所以 D 只能与 C 配合。

Saturn+ 补充了缺失的指令，因此上述任意寄存器都可以复制到其他任意寄存器。与前文说明一致，这些新增指令在操作码右侧以加号 + 标记。

33.2 使用寄存器进行运算

33.2.1 递增寄存器

递增就是给寄存器加 1。字段决定参与运算的半字节数量。例如，若 C 中有 #FFh，执行 C=C+1 B 后，C 中得到 #00h，同时进位位置 1，因为发生了溢出。如果改用 X 字段，则得到 #100h。

助记符	操作码	字段	Carry	周期
A=A+1 f	E4 (A) Ba4 818f00 +	A；P、WP、XS、S、M、B、W；F1-F7	是；Saturn+ 形式否	8；4.5+n；未列
B=B+1 f	E5 (A) Ba5 818f10 +	A；P、WP、XS、S、M、B、W；F1-F7	是；Saturn+ 形式否	8；4.5+n；未列
C=C+1 f	E6 (A) Ba6 818f20 +	A；P、WP、XS、S、M、B、W；F1-F7	是；Saturn+ 形式否	8；4.5+n；未列
D=D+1 f	E7 (A) Ba7 818f30 +	A；P、WP、XS、S、M、B、W；F1-F7	是；Saturn+ 形式否	8；4.5+n；未列

示例：要实现一种“等待”循环，先选择一个寄存器，例如 C。然后选择一个字段，但不要选 W 这样太大的字段。这里使用 A。先把 #FFFFFFh 装入 A，再逐次减 1，直到减到 0。

为了这个示例，下面的写法：

```
C=0 A
C=C-1 A
```

比下面的写法更合适：

```
LC FFFFFF
```

为什么？

60. 第二种方法需要 7 个半字节编码，即 34FFFFFF，耗时 8 个周期。

61. 第一种方法只需要 4 个半字节，即 D2CE，但耗时更多，为 14 个周期。

可见，体积更大的代码有时反而更快；如果要节省空间，可能就必须付出更多周期。如果需要高速运行，应选择合适的编码，不要盲目节省内存。

由于这里希望得到较短的代码，所以使用：

```
C=0 A
C=C-1 A
```

要构成循环，需要标签。标签所在行以 * 开头，编译器据此知道可以“分支”到该标签。

这里还要使用一条新指令：

```
| GONC
```

GONC 表示 Go if No Carry。

完整代码如下：

```
| C=0 A
| C=C-1 A
| *LOOP
| C=C-1 A
| GONC LOOP
| A=DAT0 A
| D0=D0+ 5
| PC=(A)
| @
```

首先，需要让 C 中得到 #FFFFFFh。因此用 A 字段把 C 清零，再减 1，得到 #FFFFFFh。

代码中还加入了名为 LOOP 的标签。标签行先写星号 *，再写标签名称。这里使用 LOOP，您也可以使用其他名称。

循环内部只需从 C(A) 中减去 1。

C(A) 表示“C 寄存器的 A 字段”。

GONC 会一直分支到 LOOP，直到进位位被设为 1。进位位何时置 1？当 C(A) 到达 0 后，下一次 C=C-1 A 会让 C(A) 从 #00000h 变为 #FFFFFFh，同时把进位位置 1。由于进位位已经为 1，GONC 不再跳转到 LOOP。

随后程序结束。

请编译并运行上面的源代码。它会等待一段时间，然后退出。

如果希望循环持续时间更短，可以使用 X 或 B 字段。

也可以换一种写法：不检查进位位是否置位，也就是是否发生负向下溢，而检查 C(A) 是否等于 0，并循环到它等于 0 为止。不过，使用进位位的循环代码更短、速度也更快。

33.2.2 循环中进位位的特殊之处

在循环中使用进位位时，请注意循环会一直重复到发生上溢或下溢。在上例中，C(A) 到达 0 时仍会再循环一次，因为此时进位位尚未置 1。

看看下面的代码：

```
| LC F
| *LOOP
| C=C-1 P
| GONC LOOP
```

LOOP 会执行 F + 1 次，也就是 16 次，而不是 15 次。因为 C(P) 到达 0 时还会继续循环；只有下溢发生、#0h 变成 #Fh 时，进位位才会置位。

如果希望某段代码循环 4 次，而又使用进位位，就应向计数寄存器装入 3，而不是 4：

```
| LA 3
| *LOOP
| % 在这里插入需要重复 4 次的代码
| A=A-1 P
| GONC LOOP
```

如果这一点难以理解，可以在 comp.sys.hp48 新闻组中寻求帮助。所有内容都应当真正弄清楚：基础学得扎实，代码才能写得稳妥。

33.2.3 两个寄存器相加

这里把一个寄存器的内容加到另一个寄存器，并按所用字段处理相应数量的半字节。进位位当然会受到影响。例如，在 P=0 时，用 P 字段把 #Eh 与 #3h 相加，会设置进位位。

助记符	操作码	字段	Carry	周期
A=A+B f	C0 (A) Aa0 80B184f +	A; P、WP、XS、S、M、 B、W; F1-F7	是; Saturn+ 形式否	8; 4.5+n; 未列
B=B+C f	C1 (A) Aa1 80B189f +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
C=C+A f	C2 (A) Aa2 80B182f +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
D=D+C f	C3 (A) Aa3 80B18Bf +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
B=B+A f	C8 (A) Aa8 80B181f +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
C=C+B f	C9 (A) Aa9 80B186f +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
A=A+C f	CA (A) AaA 80B188f +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
C=C+D f	CB (A) AaB 80B18Ef +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
D=D+A f	80B183a + 80B183f +	A、P、WP、XS、S、M、 B、W; F1-F7	否	未列
D=D+B f	80B187a + 80B187f +	同上	否	未列
A=A+D f	80B18Ca + 80B18Cf +	同上	否	未列
B=B+D f	80B18Da + 80B18Df +	同上	否	未列

请注意，上表没有列出 $A=B+A$ f 之类的指令，因为在这个例子中，它与 $A=A+B$ f 完全相同。当一个操作码可以由多个助记符表示时，本书只列出寄存器按字母顺序排列的形式。不过，HP-ASM 和 MASD 都允许使用替代写法。

33.2.4 寄存器与自身相加

这可用来把寄存器内容乘以 2；若使用 W 以外的字段，则只把寄存器的一部分乘以 2。与前面一样，如果在字段 f 中与自身相加时发生溢出，进位位会受到影响：

助记符	操作码	字段	Carry	周期
A=A+A f	C4 (A) Aa4 80B180f +	A; P、WP、XS、S、M、 B、W; F1-F7	是; Saturn+ 形式否	8; 4.5+n; 未列
B=B+B f	C5 (A) Aa5 80B185f +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
C=C+C f	C6 (A) Aa6 80B18Af +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
D=D+D f	C7 (A) Aa7 80B18Ff +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列

33.2.5 给寄存器加常数

下面的指令可以使用字段 f，把一个常数加到寄存器内容上。这里不能使用所有字段：在十进制模式下，不能使用 S、XS、WP 或 P。

这是 Saturn 的一个错误。

Saturn 在十进制模式下存在一个错误：凡是用大于 1 的常数递增或递减寄存器，并使用 S、XS、WP 或 P 字段的指令，都会作用于整个寄存器，而不是只作用于指定字段。在十六进制模式下，一切按预期工作。

换言之，如果某个半字节溢出并设置进位位，进位会继续传播到寄存器的所有位。例如，假设 A 寄存器如下：

```
| F E D C B A 9 8 7 6 5 4 3 2 1 0
| F F F F F F F F F F F F F F F 2
```

执行 `A=A+4 XS` 后，得到：

```
| F E D C B A 9 8 7 6 5 4 3 2 1 0
| 0 0 0 0 0 0 0 0 0 0 0 0 0 2 F 3
```

再看一个例子：

```
| F E D C B A 9 8 7 6 5 4 3 2 1 0
| F F F F F F F F F F F F 1 F E F F
```

执行 `A=A+4 XS` 后，得到：

```
| F E D C B A 9 8 7 6 5 4 3 2 1 0
| F F F F F F F F F F F F 2 0 2 F F
```

解决方法：

如果使用 S、XS、P，或者在 `P=0` 时使用 WP，只处理 1 个半字节，而又要加上大于 1 的数，就必须按需要多次执行加 1。

例如：

```
| A=A+2 S
```

这样不能正常工作。若进位位置位，寄存器内容就会丢失。

```
| A=A+1 S
```

```
| A=A+1 S
```

这样就没有问题。即使发生溢出并设置进位位，其他内容仍能保持正确。

下面的写法同样有问题：

```
| A=A-2 S
```

应改为：

```
| A=A-1 S
```

```
| A=A-1 S
```

请记住： 使用只处理 1 个半字节的字段时，如果要加上或减去大于 1 的数，必须执行多条 `+1` 或 `-1`，以避免进位错误。

ARM 架构计算器中的 Saturn+ 模拟处理器已经修复了这个错误。不过，如果希望代码向后兼容，仍应遵守上述规则。

执行 `A=A+3 f` 给寄存器加 3 时，如果要确定它的十六进制编码，必须从常数中减 1。因此，`A=A+3 A` 编码为 `818F02`：编码中是 2 而不是 3，因为这里存储的是 `c-1`。只有在阅读给寄存器加常数的机器码时才需要关注这一点。

编写程序时，只要写 `A=A+3 A`，汇编器会自动编码为 `818F02`，无需手动处理 `c-1`。

Gerald Squelart 提供了一个技巧：要给单半字节字段设置某个值，可以写：

```
| A=0 S
```

```
| A=A+2 S
```

这样能够工作，因为不会产生跨字段进位。

助记符	操作码	字段	Carry	周期
<code>A=A+c f</code>	<code>818F0(c-1) (A)</code> <code>818a0(c-1) (W、M、X、B)</code> <code>818f0(c-1) + (F1-F7)</code>	A; W、M、X、B; F1-F7	是; Saturn+ 形式否	<code>8+n</code>
<code>B=B+c f</code>	<code>818F1(c-1)</code> <code>818a1(c-1)</code> <code>818f1(c-1) +</code>	同上	是; Saturn+ 形式否	<code>8+n</code>
<code>C=C+c f</code>	<code>818F2(c-1)</code> <code>818a2(c-1)</code> <code>818f2(c-1) +</code>	同上	是; Saturn+ 形式否	<code>8+n</code>
<code>D=D+c f</code>	<code>818F3(c-1)</code> <code>818a3(c-1)</code> <code>818f3(c-1) +</code>	同上	是; Saturn+ 形式否	<code>8+n</code>

n 表示被加数所在对象的半字节数量。

$$2 < c < 16$$

如果只加 1，其编码方式与这里不同，参见 33.2.1 节。不过，在编写程序时，汇编器会自动选择正确指令。

早期程序员必须区分“某值加 1”和“某值加 c”，其中 c 在 2 到 16 之间。现在有了汇编器，只需写出 1 到 16 的任意加数，其余工作由汇编器完成。

下面还有一个实用技巧。

每个半字节都有相应的“权重”，因此可以改变 P，再给寄存器中的特定半字节加值，从而达到远大于单纯加常数 c 的效果。半字节 #2h，也就是寄存器中的第 3 个半字节，其权重为 16^2 。所以每向该半字节加 1，整个数值就增加 16^2 ，即 256。

因此，有时给寄存器的半字节 n 加一个值，比给整个寄存器加一个大数更合适，但并非任何情况下都如此。

33.2.6 递减寄存器

递减就是减去 1，原理很简单。

助记符	操作码	字段	Carry	周期
A=A-1 f	CC (A) AaC 818f80 +	A; P、WP、XS、S、M、B、W; F1-F7	是; Saturn+ 形式否	8; 4.5+n; 未列
B=B-1 f	CD (A) AaD 818f90 +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
C=C-1 f	CE (A) AaE 818fA0 +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
D=D-1 f	CF (A) AaF 818fB0 +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列

如果试图从值为 0 的任意字段减去 1，进位位就会置位。

33.2.7 两个寄存器相减

即从一个寄存器中减去另一个寄存器。

助记符	操作码	字段	Carry	周期
A=A-B f	E0 (A) Ba0 80B284f +	A; P、WP、XS、S、M、B、W; F1-F7	是; Saturn+ 形式否	8; 4.5+n; 未列
B=B-C f	E1 (A) Ba1 80B289f +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列

助记符	操作码	字段	Carry	周期
C=C-A f	E2 (A) Ba2 80B282f +	A; P、WP、XS、S、M、B、W; F1-F7	是; Saturn+ 形式否	8; 4.5+n; 未列
D=D-C f	E3 (A) Ba3 80B28Bf +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
B=B-A f	E8 (A) Ba8 80B281f +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
C=C-B f	E9 (A) Ba9 80B286f +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
A=A-C f	EA (A) BaA 80B288f +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
C=C-D f	EB (A) BaB 80B28Ef +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
A=B-A f	EC (A) BaC	A; P、WP、XS、S、M、B、W	是	8; 4.5+n

助记符	操作码	字段	Carry	周期
B=C-B f	ED (A) BaD	同上	是	8; 4.5+n
C=A-C f	EE (A) BaE	同上	是	8; 4.5+n
D=C-D f	EF (A) BaF	同上	是	8; 4.5+n
D=D-A f	80B283a + 80B283f +	A、P、WP、XS、S、M、 B、W; F1-F7	否	未列
D=D-B f	80B287a + 80B287f +	同上	否	未列
A=A-D f	80B28Ca + 80B28Cf +	同上	否	未列
B=B-D f	80B28Da + 80B28Df +	同上	否	未列

与往常一样，n 是半字节数量；发生下溢时，进位位会置位。

33.2.8 两个寄存器相乘

这些乘法指令是 ARM 架构计算器新增的，因此 HP 48 系列和 49G 不支持。对这些计算器，必须按 7.4 节所述手工完成。也可以调用例程 `=MUL#`；它在 48 和 49 上的地址均为 `#03991h`，会把 A(A) 与 C(A) 相乘，并把乘积放入 B(A)。

助记符	操作码	字段	Carry	周期
B=B*A f	80B381a + 80B381f +	A、P、WP、XS、S、M、B、W; F1-F7	否	未列
C=C*A f	80B382a + 80B382f +	同上	否	未列
D=D*A f	80B383a + 80B383f +	同上	否	未列
A=A*B f	80B384a + 80B384f +	同上	否	未列
B=B*B f	80B385a + 80B385f +	同上	否	未列

助记符	操作码	字段	Carry	周期
C=C*B f	80B386a + 80B386f +	A、P、WP、XS、S、M、B、W; F1-F7	否	未列
D=D*B f	80B387a + 80B387f +	同上	否	未列
A=A*C f	80B388a + 80B388f +	同上	否	未列
B=B*C f	80B389a + 80B389f +	同上	否	未列
A=A*A f	80B380a + 80B380f +	同上	否	未列
C=C*C f	80B38Aa + 80B38Af +	同上	否	未列
D=D*C f	80B38Ba + 80B38Bf +	同上	否	未列
A=A*D f	80B38Ca + 80B38Cf +	同上	否	未列
B=B*D f	80B38Da + 80B38Df +	同上	否	未列
C=C*D f	80B38Ea + 80B38Ef +	同上	否	未列

助记符	操作码	字段	Carry	周期
D=D*D f	80B38Fa + 80B38Ff +	同上	否	未列

33.2.9 两个寄存器相除

这些除法指令也是 ARM 架构计算器新增的，因此 HP 48 系列和 49G 不支持。对这些计算器，必须按 7.5 节所述手工完成。也可以调用例程 `=IntDiv`；它在 48 和 49 上的地址均为 `#03F24h`，会用 A(A) 除以 C(A)，并把被除数放入 C(A)。

助记符	操作码	字段	Carry	周期
B=B/A f	80B481a + 80B481f +	A、P、WP、XS、S、M、B、W； F1-F7	否	未列
C=C/A f	80B482a + 80B482f +	同上	否	未列
D=D/A f	80B483a + 80B483f +	同上	否	未列
A=A/B f	80B484a + 80B484f +	同上	否	未列
B=1 f	80B485a + 80B485f +	同上	否	未列
C=C/B f	80B486a + 80B486f +	同上	否	未列
D=D/B f	80B487a + 80B487f +	同上	否	未列
A=A/C f	80B488a + 80B488f +	同上	否	未列
B=B/C f	80B489a + 80B489f +	同上	否	未列
A=1 f	80B480a + 80B480f +	同上	否	未列
C=1 f	80B48Aa + 80B48Af +	同上	否	未列
D=D/C f	80B48Ba + 80B48Bf +	同上	否	未列

助记符	操作码	字段	Carry	周期
A=A/D f	80B48Ca + 80B48Cf +	A、P、WP、XS、S、M、B、W； F1-F7	否	未列
B=B/D f	80B48Da + 80B48Df +	同上	否	未列
C=C/D f	80B48Ea + 80B48Ef +	同上	否	未列
D=1 f	80B48Fa + 80B48Ff +	同上	否	未列

请注意，对其中一些指令，本书列出了更实用的助记符。

33.2.10 两个寄存器取模

这些取模指令用于取得除法余数，同样是 ARM 架构计算器新增的，HP 48 系列和 49G 不支持。对这些计算器，必须按 7.5 节所述手工完成。也可以调用例程 `=IntDiv`；它在 48 和 49 上的地址均为 `#03F24h`，会用 A(A) 除以 C(A)，并把余数放入 A(A)。

助记符	操作码	字段	Carry	周期
B=B%A f	80B581a + 80B581f +	A、P、WP、XS、S、M、B、W； F1-F7	否	未列
C=C%A f	80B582a + 80B582f +	同上	否	未列

助记符	操作码	字段	Carry	周期
D=D%A f	80B583a + 80B583f +	同上	否	未列
A=A%B f	80B584a + 80B584f +	同上	否	未列
B=B%B f	80B585a + 80B585f +	同上	否	未列
C=C%B f	80B586a + 80B586f +	同上	否	未列
D=D%B f	80B587a + 80B587f +	同上	否	未列
A=A%C f	80B588a + 80B588f +	同上	否	未列
B=B%C f	80B589a + 80B589f +	同上	否	未列
A=A%A f	80B580a + 80B580f +	同上	否	未列
C=C%C f	80B58Aa + 80B58Af +	同上	否	未列
D=D%C f	80B58Ba + 80B58Bf +	同上	否	未列
A=A%D f	80B58Ca + 80B58Cf +	同上	否	未列
B=B%D f	80B58Da + 80B58Df +	同上	否	未列
C=C%D f	80B58Ea + 80B58Ef +	同上	否	未列
D=D%D f	80B58Fa + 80B58Ff +	同上	否	未列

A=A%A、B=B%B、C=C%C 和 D=D%D 看起来没有什么实际用途，但计算器确实支持，所以仍列在这里。

33.2.11 减去常数

助记符	操作码	字段	Carry	周期
A=A-c f	818F8(c-1) (A) 818a8(c-1) (W、M、X、B) 818f8(c-1) + (F1-F7)	A; W、M、X、B; F1-F7	是; Saturn+ 形式否	8+n
B=B-c f	818F9(c-1) 818a9(c-1) 818f9(c-1) +	同上	是; Saturn+ 形式否	8+n
C=C-c f	818FA(c-1) 818aA(c-1) 818fA(c-1) +	同上	是; Saturn+ 形式否	8+n
D=D-c f	818FB(c-1) 818aB(c-1) 818fB(c-1) +	同上	是; Saturn+ 形式否	8+n

由于进位错误，这里不能使用全部字段。若要减去 *c*，且 *c* > 1，就必须按需要多次减 1，以避开该错误。还可以使用前面介绍的技巧，通过改变 P 来操作特定半字节。利用每个半字节的权重，可以减去较大的数值。

33.2.12 反码

注意： 反码运算的结果取决于 Saturn 当前处于十进制模式还是十六进制模式。切换模式的指令见后文。请记住，程序开始时处理器一定处于十六进制模式，退出时也必须恢复到十六进制模式。

助记符	操作码	字段	Carry	周期
A--A-1 f	FC (A) BbC 80B680f +	A; P、WP、XS、S、M、 B、W; F1-F7	是; Saturn+ 形式否	8; 4.5+n; 未列
B--B-1 f	FD (A) BbD 80B685f +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
C--C-1 f	FE (A) BbE 80B68Af +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
D--D-1 f	FF (A) BbF 80B68Ff +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
B--A-1 f	80B681a + 80B681f +	A、P、WP、XS、S、M、 B、W; F1-F7	否	未列
C--A-1 f	80B682a + 80B682f +	同上	否	未列
D--D-1 f	80B683a + 80B683f +	同上	否	未列
A--B-1 f	80B684a + 80B684f +	同上	否	未列
C--B-1 f	80B686a + 80B686f +	同上	否	未列
D--B-1 f	80B687a + 80B687f +	同上	否	未列

助记符	操作码	字段	Carry	周期
A--C-1 f	80B688a + 80B688f +	A、P、WP、XS、S、M、B、W; F1-F7	否	未列
A--B-1 f	80B689a + 80B689f +	同上	否	未列
D--C-1 f	80B68Ba + 80B68Bf +	同上	否	未列
A--D-1 f	80B68Ca + 80B68Cf +	同上	否	未列
B--D-1 f	80B68Da + 80B68Df +	同上	否	未列
C--D-1 f	80B68Ea + 80B68Ef +	同上	否	未列

如果所处理的寄存器及字段为 0，执行 `r--r-1 f`，其中 r 表示寄存器，就会设置进位位。

MASD 还允许把 `r--r-1 f` 写成 `r=~r f`。例如，`A--A-1 B` 可以写成 `A=~A B`。可通过 **ALPHA**、**Right-Shift**、**1** 输入 ~。

33.2.13 补码

与前面一样，结果取决于 Saturn 当前模式。这里会把所有位取反并加 1：

助记符	操作码	字段	Carry	周期
A--A f	F8 (A) Bb8 80B780f +	A; P、WP、XS、S、M、B、 W; F1-F7	是; Saturn+ 形式否	8; 4.5+n; 未列
B--B f	F9 (A) Bb9 80B785f +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
C--C f	FA (A) BbA 80B78Af +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列

助记符	操作码	字段	Carry	周期
D=-D f	FB (A) BbB 80B78Ff +	同上	是; Saturn+ 形式否	8; 4.5+n; 未列
B=-A f	80B781a + 80B781f +	A、P、WP、XS、S、M、B、W; F1-F7	否	未列
C=-A f	80B782a + 80B782f +	同上	否	未列
D=-D f	80B783a + 80B783f +	同上	否	未列
A=-B f	80B784a + 80B784f +	同上	否	未列
C=-B f	80B786a + 80B786f +	同上	否	未列
D=-B f	80B787a + 80B787f +	同上	否	未列
A=-C f	80B788a + 80B788f +	同上	否	未列
A=-B f	80B789a + 80B789f +	同上	否	未列
D=-C f	80B78Ba + 80B78Bf +	同上	否	未列

助记符	操作码	字段	Carry	周期
A=-D f	80B78Ca + 80B78Cf +	A、P、WP、XS、S、M、B、W; F1-F7	否	未列
B=-D f	80B78Da + 80B78Df +	同上	否	未列
C=-D f	80B78Ea + 80B78Ef +	同上	否	未列

33.2.14 逻辑运算

33.2.14.1 逻辑 OR，也称“逻辑和”

逻辑 OR 使用符号 !。OR 在字段 f 范围内逐位执行，结果写入等号左侧的寄存器：

助记符	操作码	字段	Carry	周期
A=A!B f	0EF8 (A) 0Ea8 0Ef8 +	A; P、WP、XS、S、M、B、W; F1-F7	否	11; 6+n; 未列
B=B!C f	0EF9 (A) 0Ea9 0Ef9 +	同上	否	11; 6+n; 未列
C=C!A f	0EFA (A) 0EaA 0EfA +	同上	否	11; 6+n; 未列
D=D!C f	0EFB (A) 0EaB 0EfB +	同上	否	11; 6+n; 未列
B=B!A f	0EFC (A) 0EaC 0EfC +	同上	否	11; 6+n; 未列
C=C!B f	0EFD (A) 0EaD 0EfD +	同上	否	11; 6+n; 未列
A=A!C f	0EFE (A) 0EaE 0EfE +	同上	否	11; 6+n; 未列

助记符	操作码	字段	Carry	周期
C=C!D f	0EFF (A) 0EaF 0EfF +	同上	否	11; 6+n; 未列

33.2.14.2 逻辑 AND，也称“逻辑积”

使用符号 &。

助记符	操作码	字段	Carry	周期
A=A&B f	0EF0 (A) 0Ea0 0Ef0 +	A; P、WP、XS、S、M、B、W; F1-F7	否	11; 6+n; 未列
B=B&C f	0EF1 (A) 0Ea1 0Ef1 +	同上	否	11; 6+n; 未列
C=C&A f	0EF2 (A) 0Ea2 0Ef2 +	同上	否	11; 6+n; 未列

助记符	操作码	字段	Carry	周期
D=D&C f	0EF3 (A) 0Ea3 0Ef3 +	A; P、WP、XS、S、M、B、W; F1-F7	否	11; 6+n; 未列
B=B&A f	0EF4 (A) 0Ea4 0Ef4 +	同上	否	11; 6+n; 未列
C=C&B f	0EF5 (A) 0Ea5 0Ef5 +	同上	否	11; 6+n; 未列
A=A&C f	0EF6 (A) 0Ea6 0Ef6 +	同上	否	11; 6+n; 未列
C=C&D f	0EF7 (A) 0Ea7 0Ef7 +	同上	否	11; 6+n; 未列

33.2.14.3 逻辑 XOR

逻辑 XOR 使用脱字符 ^。这些 XOR 指令是 ARM 架构计算器新增的，因此 HP 48 系列和 49G 不支持。对这些计算器，必须按 16.6 节所述手工完成。

助记符	操作码	字段	Carry	周期
B=B^A f	80BA81a + 80BA81f +	A、P、WP、XS、S、M、B、W; F1-F7	否	未列
C=C^A f	80BA82a + 80BA82f +	同上	否	未列
D=D^A f	80BA83a + 80BA83f +	同上	否	未列
A=A^B f	80BA84a + 80BA84f +	同上	否	未列
B=B^B f	80BA85a + 80BA85f +	同上	否	未列
C=C^B f	80BA86a + 80BA86f +	同上	否	未列
D=D^B f	80BA87a + 80BA87f +	同上	否	未列
A=A^C f	80BA88a + 80BA88f +	同上	否	未列
B=B^C f	80BA89a + 80BA89f +	同上	否	未列

助记符	操作码	字段	Carry	周期
A=A^A f	80BA80a + 80BA80f +	同上	否	未列
C=C^C f	80BA8Aa + 80BA8Af +	同上	否	未列
D=D^C f	80BA8Ba + 80BA8Bf +	同上	否	未列
A=A^D f	80BA8Ca + 80BA8Cf +	同上	否	未列
B=B^D f	80BA8Da + 80BA8Df +	同上	否	未列
C=C^D f	80BA8Ea + 80BA8Ef +	同上	否	未列
D=D^D f	80BA8Fa + 80BA8Ff +	同上	否	未列

33.2.15 左移或右移一个半字节

有 4 条指令可向左移动 1 个半字节，相当于把数值乘以 16；另有 4 条指令可向右移动 1 个半字节，相当于把数值除以 16。

向左移时，所有半字节都向左移动一位，最后一个半字节，也就是编号 #Fh，会丢失；编号 #0h 的半字节填入 0。向右移时，所有半字节都向右移动一位，最后一个半字节，也就是编号 #Fh，填入 0；第一个半字节，也就是编号 #0h，会丢失。

助记符先写寄存器字母，再写表示 Shift 的 S，最后写方向字母：R 表示右，L 表示左。

例如，让 C 寄存器右移的助记符由 C + S + R 构成，即 CSR f。

助记符	操作码	字段	Carry	周期
ASL f	F0 (A) Bb0	A; P、WP、XS、S、M、B、W	否	8; 4.5+n
BSL f	F1 (A) Bb1	同上	否	8; 4.5+n
CSL f	F2 (A) Bb2	同上	否	8; 4.5+n
DSL f	F3 (A) Bb3	同上	否	8; 4.5+n
ASR f	F4 (A) Bb4	同上	否	8; 4.5+n
BSR f	F5 (A) Bb5	同上	否	8; 4.5+n
CSR f	F6 (A) Bb6	同上	否	8; 4.5+n
DSR f	F7 (A) Bb7	同上	否	8; 4.5+n

这里不会影响进位位，但会影响 SB，即粘滞位。如果在左侧或右侧丢失的半字节数值非零，SB 就会置位，表示发生了值得注意的信息损失。

33.2.16 左旋或右旋一个半字节

要旋转一个半字节，助记符先写寄存器字母，再写表示 Shift 的 S，然后写方向 R 或 L，最后写表示 Circling 的 C。

因此有：

助记符	操作码	字段	Carry	周期
ASLC	810	全部	否	22.5
BSLC	811	全部	否	22.5
CSLC	812	全部	否	22.5

助记符	操作码	字段	Carry	周期
DSLC	813	全部	否	22.5
ASRC	814	全部	否	22.5
BSRC	815	全部	否	22.5
CSRC	816	全部	否	22.5
DSRC	817	全部	否	22.5

向右旋转时，编号 #1h 的半字节内容移入编号 #0h，而编号 #0h 的半字节移到编号 #Fh。

例如，右旋前某寄存器的内容为：

```

F E D C B A 9 8 7 6 5 4 3 2 1 0
3 A 8 B B 3 8 2 0 0 0 A F 0 F C

```

右旋后变为：

```

F E D C B A 9 8 7 6 5 4 3 2 1 0
C 3 A 8 B B 3 8 2 0 0 0 A F 0 F

```

这里没有信息丢失，因为半字节 #0h 移到了寄存器另一端，成为半字节 #Fh。

原文说明，仅当编号 #0h 的半字节中各位均为 0，而它被移到编号 #Fh 时，Sticky Bit 才会置位。

33.2.17 向右移动一位

向右移动一位相当于除以 2。助记符依次由寄存器字母、表示 Shifting 的 S、表示右方向的 R，以及表示 Bit 的 B 组成。

助记符	操作码	字段	Carry	周期
ASRB	81C	全部	否	21.5
BSRB	81D	全部	否	21.5
CSRB	81E	全部	否	21.5
DSRB	81F	全部	否	21.5

这里会把全部 64 位向右移动。虽然进位位不变，但如果编号 #0h 的位被移出且原值为 1，Sticky Bit 就会置位。请注意，没有向左移动一位的对应指令。

33.2.17.1 在字段内向右移动一位

这些指令与前面的指令相似，但这里要指定字段，所以移位只在所选字段内部发生。只需在助记符后加字段字母，例如 ASRB A。

助记符	操作码	字段	Carry	周期
ASRB f	819F0 (A) 819a0 819f0 +	A; P、WP、XS、S、M、B; F1-F7	否	13.5; 8.5+n; 未列
BSRB f	819F1 (A) 819a1 819f1 +	同上	否	13.5; 8.5+n; 未列
CSRB f	819F2 (A) 819a2 819f2 +	同上	否	13.5; 8.5+n; 未列
DSRB f	819F3 (A) 819a3 819f3 +	同上	否	13.5; 8.5+n; 未列

这些指令把字段 f 的值除以 2。

进位位不会改变，但如果字段右端被移出的位等于 1，SB 就会置位。字段左端填入 0。

在原始 Saturn 上，同样没有在字段内向左移动一位的指令。

33.2.18 向左移位

这些向左移位指令把小于号左侧的寄存器向左移动 n 位，其中 n 是小于号右侧寄存器中的值。它们是 ARM 架构计算器新增的，因此 HP 48 系列和 49G 不支持。

助记符	操作码	字段	Carry	周期
B=B<A f	80B881a + 80B881f +	A、P、WP、XS、S、M、B、W； F1-F7	否	未列
C=C<A f	80B882a + 80B882f +	同上	否	未列
D=D<A f	80B883a + 80B883f +	同上	否	未列
A=A<B f	80B884a + 80B884f +	同上	否	未列
B=B<B f	80B885a + 80B885f +	同上	否	未列
C=C<B f	80B886a + 80B886f +	同上	否	未列
D=D<B f	80B887a + 80B887f +	同上	否	未列
A=A<C f	80B888a + 80B888f +	同上	否	未列
B=B<C f	80B889a + 80B889f +	同上	否	未列
A=A<A f	80B880a + 80B880f +	同上	否	未列
C=C<C f	80B88Aa + 80B88Af +	同上	否	未列
D=D<C f	80B88Ba + 80B88Bf +	同上	否	未列
A=A<D f	80B88Ca + 80B88Cf +	同上	否	未列
B=B<D f	80B88Da + 80B88Df +	同上	否	未列
C=C<D f	80B88Ea + 80B88Ef +	同上	否	未列
D=D<D f	80B88Fa + 80B88Ff +	同上	否	未列

33.2.19 向右移位

这些向右移位指令把大于号左侧的寄存器向右移动 n 位，其中 n 是大于号右侧寄存器中的值。它们是 ARM 架构计算器新增的，因此 HP 48 系列和 49G 不支持。

助记符	操作码	字段	Carry	周期
B=B>A f	80B981a + 80B981f +	A、P、WP、XS、S、M、B、W； F1-F7	否	未列
C=C>A f	80B982a + 80B982f +	同上	否	未列
D=D>A f	80B983a + 80B983f +	同上	否	未列
A=A>B f	80B984a + 80B984f +	同上	否	未列

助记符	操作码	字段	Carry	周期
B=B>B f	80B985a + 80B985f +	A、P、WP、XS、S、M、B、W； F1-F7	否	未列
C=C>B f	80B986a + 80B986f +	同上	否	未列
D=D>B f	80B987a + 80B987f +	同上	否	未列

助记符	操作码	字段	Carry	周期
A=A>C f	80B988a + 80B988f +	同上	否	未列
B=B>C f	80B989a + 80B989f +	同上	否	未列
A=A>A f	80B980a + 80B980f +	同上	否	未列
C=C>C f	80B98Aa + 80B98Af +	同上	否	未列
D=D>C f	80B98Ba + 80B98Bf +	同上	否	未列
A=A>D f	80B98Ca + 80B98Cf +	同上	否	未列
B=B>D f	80B98Da + 80B98Df +	同上	否	未列
C=C>D f	80B98Ea + 80B98Ef +	同上	否	未列
D=D>D f	80B98Fa + 80B98Ff +	同上	否	未列

34 保存寄存器

在这里会进一步看到，寄存器 A 和 C 对处理器确实非常重要。

只有 A 和 C 能与 5 个保存寄存器配合使用：R0、R1、R2、R3 和 R4。

在下面这些助记符中，等号左侧寄存器接收右侧寄存器的值，右侧寄存器本身不变。

34.1 把工作寄存器保存到保存寄存器

助记符	操作码	字段	Carry	周期
R0=A	100	全部	否	20.5
R1=A	101	全部	否	20.5
R2=A	102	全部	否	20.5
R3=A	103	全部	否	20.5
R4=A	104	全部	否	20.5
R0=C	108	全部	否	20.5
R1=C	109	全部	否	20.5
R2=C	10A	全部	否	20.5
R3=C	10B	全部	否	20.5
R4=C	10C	全部	否	20.5

34.2 在指定字段内把工作寄存器保存到保存寄存器

助记符	操作码	字段	Carry	周期
R0=A f	81AF00 (A) 81Aa00 81Af00 +	A; P、WP、XS、S、M、B、W; F1-F7	否	14; 9+n; 未列
R1=A f	81AF01 (A) 81Aa01 81Af01 +	同上	否	14; 9+n; 未列
R2=A f	81AF02 (A) 81Aa02 81Af02 +	同上	否	14; 9+n; 未列

助记符	操作码	字段	Carry	周期
R3=A f	81AF03 (A) 81Aa03 81Af03 +	同上	否	14; 9+n; 未列
R4=A f	81AF04 (A) 81Aa04 81Af04 +	同上	否	14; 9+n; 未列
R0=C f	81AF08 (A) 81Aa08 81Af08 +	同上	否	14; 9+n; 未列
R1=C f	81AF09 (A) 81Aa09 81Af09 +	同上	否	14; 9+n; 未列
R2=C f	81AF0A (A) 81Aa0A 81Af0A +	同上	否	14; 9+n; 未列
R3=C f	81AF0B (A) 81Aa0B 81Af0B +	同上	否	14; 9+n; 未列
R4=C f	81AF0C (A) 81Aa0C 81Af0C +	同上	否	14; 9+n; 未列

示例：希望使用 D1，但需要先保存它的值：

```
CD1EX
R0=C A
```

这就是保存方法。

34.3 把保存寄存器复制到工作寄存器

不能直接对保存在保存寄存器中的值进行运算，因此需要用下列指令取回这些信息：

助记符	操作码	字段	Carry	周期
A=R0	110	全部	否	20.5
A=R1	111	全部	否	20.5
A=R2	112	全部	否	20.5
A=R3	113	全部	否	20.5
A=R4	114	全部	否	20.5
C=R0	118	全部	否	20.5
C=R1	119	全部	否	20.5
C=R2	11A	全部	否	20.5
C=R3	11B	全部	否	20.5
C=R4	11C	全部	否	20.5

34.4 在指定字段内把保存寄存器复制到工作寄存器

助记符	操作码	字段	Carry	周期
A=R0 f	81AF10 (A) 81Aa10 81Af10 +	A; P、WP、XS、S、M、B、W; F1-F7	否	14; 9+n; 未列
A=R1 f	81AF11 (A) 81Aa11 81Af11 +	同上	否	14; 9+n; 未列
A=R2 f	81AF12 (A) 81Aa12 81Af12 +	同上	否	14; 9+n; 未列

助记符	操作码	字段	Carry	周期
A=R3 f	81AF13 (A) 81Aa13 81Af13 +	同上	否	14; 9+n; 未列
A=R4 f	81AF14 (A) 81Aa14 81Af14 +	同上	否	14; 9+n; 未列
C=R0 f	81AF18 (A) 81Aa18 81Af18 +	同上	否	14; 9+n; 未列
C=R1 f	81AF19 (A) 81Aa19 81Af19 +	同上	否	14; 9+n; 未列
C=R2 f	81AF1A (A) 81Aa1A 81Af1A +	同上	否	14; 9+n; 未列
C=R3 f	81AF1B (A) 81Aa1B 81Af1B +	同上	否	14; 9+n; 未列
C=R4 f	81AF1C (A) 81Aa1C 81Af1C +	同上	否	14; 9+n; 未列

34.5 在保存寄存器与工作寄存器之间交换

这些指令非常实用。助记符先写工作寄存器字母，再写保存寄存器名称，最后写表示 EXchange 的 EX。

助记符	操作码	字段	Carry	周期
AR0EX	120	全部	否	20.5
AR1EX	121	全部	否	20.5
AR2EX	122	全部	否	20.5
AR3EX	123	全部	否	20.5
AR4EX	124	全部	否	20.5
CR0EX	128	全部	否	20.5
CR1EX	129	全部	否	20.5
CR2EX	12A	全部	否	20.5
CR3EX	12B	全部	否	20.5
CR4EX	12C	全部	否	20.5

34.6 在指定字段内交换保存寄存器与工作寄存器

助记符	操作码	字段	Carry	周期
AR0EX f	81AF20 (A) 81Aa20 81Af20 +	A; P、WP、XS、S、M、B、W; F1-F7	否	14; 9+n; 未列
AR1EX f	81AF21 (A) 81Aa21 81Af21 +	同上	否	14; 9+n; 未列
AR2EX f	81AF22 (A) 81Aa22 81Af22 +	同上	否	14; 9+n; 未列
AR3EX f	81AF23 (A) 81Aa23 81Af23 +	同上	否	14; 9+n; 未列
AR4EX f	81AF24 (A) 81Aa24 81Af24 +	同上	否	14; 9+n; 未列

助记符	操作码	字段	Carry	周期
CR0EX f	81AF28 (A) 81Aa28 81Af28 +	同上	否	14; 9+n; 未列
CR1EX f	81AF29 (A) 81Aa29 81Af29 +	同上	否	14; 9+n; 未列
CR2EX f	81AF2A (A) 81Aa2A 81Af2A +	同上	否	14; 9+n; 未列
CR3EX f	81AF2B (A) 81Aa2B 81Af2B +	同上	否	14; 9+n; 未列
CR4EX f	81AF2C (A) 81Aa2C 81Af2C +	同上	否	14; 9+n; 未列

35 指针

D0 和 D1 只能容纳 5 个半字节，因为所有地址都用 5 个半字节编码。因此，它们称为指针：根据自身的数值，它们会“指向”内存中的某个特定半字节。

A 和 C 同样具有特殊地位：只有这两个寄存器可以与 D0 和 D1 配合使用。

35.1 给 D0 或 D1 赋值

可以向 D0 或 D1 装入 2、4 或 5 个半字节。若只装入 2 个，另外 3 个保持不变；若只装入 4 个，最后 1 个保持不变。

只需写：

```
D0= ..
D1= ..
```

只装入 2 或 4 个半字节也许看起来有些奇怪。实际上，有时可以先装入一个 5 半字节的值，之后只修改需要改变的部分。

只需写 `D0= ..`，其中 `..` 为 2、4 或 5 个半字节。D1 的写法相同。

对 HP-ASM 而言，等号后的空格非常重要；没有这个空格，HP-ASM 就无法识别指令。MASD 不要求该空格，但本书始终保留，以维持 HP-ASM 兼容性。

十六进制编码取决于装入的半字节数量：

助记符	操作码	字段	Carry	周期
D0= yz	19zy	前 2 个半字节	否	6
D0= wxyz	1Azyxw	前 4 个半字节	否	9
D0= vwxyz	1Bzyxwv	前 5 个半字节	否	10.5
D1= yz	1Dzy	前 2 个半字节	否	6
D1= wxyz	1Ezyxw	前 4 个半字节	否	9
D1= vwxyz	1Fzyxwv	前 5 个半字节	否	10.5

可以看到，输入 `D1= 00120` 后，Code 对象编码为 `1F02100`，要装入 D1 的半字节顺序被反转了。

无论您是喜欢还是不喜欢 Saturn，这种编码方式都很难让人毫无感觉。

35.2 给 D0 或 D1 加值或减值

并非总要重新向 D0 或 D1 装入数值，因为可以直接移动指针。还记得下面的代码吗？

```
A=DAT0 A
D0=D0+ 5
PC=(A)
```

第二行就是递增 D0 的指令；若改用减号 `-`，D0 就会递减。

注意不要忘记符号与数值之间的空格。应写 `D0=D0+ 5`，不能写 `D0=D0+5`。如果漏掉空格，HP-ASM 会报告错误；MASD 则允许省略空格。

可以给 D0 或 D1 加上或减去 1 到 16。若要减去更大的值，必须按需要重复执行指令。

助记符	操作码	字段	Carry	周期
D0=D0+ n	16 (n-1)	整个寄存器	是	8.5
D0=D0- n	18 (n-1)	整个寄存器	是	8.5
D1=D1+ n	17 (n-1)	整个寄存器	是	8.5
D1=D1- n	1C (n-1)	整个寄存器	是	8.5

n 是 1 到 16 的整数。进位位会受到影响。例如，若指针寄存器中为 #FFFFFFh，再加一个值就会因溢出而设置进位位。减法同样可能影响进位位。请注意，无论计算器是否处于十进制模式，这些指令都始终按十六进制执行。

35.3 把 A 或 C 复制到 D0 或 D1

A 字段很有用：它正好有 5 个半字节，与 D0 和 D1 相同。可以让 D0 或 D1 指向 A 或 C 中的 5 个半字节，方法如下：

助记符	操作码	字段	Carry	周期
D0=A	130	D0 的 A 字段	否	9.5
D0=C	134	D0 的 A 字段	否	9.5
D1=A	131	D1 的 A 字段	否	9.5
D1=C	135	D1 的 A 字段	否	9.5

也可以只复制 4 个半字节，让第 5 个保持不变。也就是说，D0 或 D1 中的 MSB 半字节保留原值，A 或 C 中的 4 个半字节覆盖其余 4 个半字节。

这些助记符与前面相似，只是在末尾增加字母 S。

助记符	操作码	字段	Carry	周期
D0=AS	138	仅 4 个半字节	否	8.5
D0=CS	13C	仅 4 个半字节	否	8.5
D1=AS	139	仅 4 个半字节	否	8.5
D1=CS	13D	仅 4 个半字节	否	8.5

35.4 在 A/C 的 A 字段与 D0/D1 之间交换

前面已经介绍了在保存寄存器和工作寄存器之间交换的指令。下面这些指令用于在 A 或 C 的 A 字段与 D0 或 D1 之间交换。

这些指令非常实用。例如，要检查某个对象的序言，可以用交换指令修改 D0 或 D1，同时暂存它们的原值。先列出指令，再给出示例。

助记符	操作码	字段	Carry	周期
AD0EX	132	5 个半字节	否	9.5
AD1EX	133	5 个半字节	否	9.5
CD0EX	136	5 个半字节	否	9.5
CD1EX	137	5 个半字节	否	9.5

每个对象都以序言开头，序言是一个 5 半字节的二进制数。堆栈中保存的不是对象本身，而是对象地址。

为什么？

程序开始时，HP 已经在使用 D0、D1、B 和 D。D0 指向下一条要使用的 RPL 对象，D1 指向堆栈的栈 1，D 表示以 5 半字节块为单位的空闲内存，也就是 D 的每个单位代表 5 个可用半字节，B 则指向 RPL 返回栈。

程序开始后，可以使用这些指针，但退出前必须恢复原值。有两种方法：保存所有寄存器并在退出前恢复，或者由程序自己临时保存。

程序可以调用 ROM 中的子例程，其中许多非常有用。若要保存全部指针，只需写：

| GOSBVL 0679B

该入口点名为 =SAVPTR，在 48 和 49 上相同。如果使用 MASD 且已装入入口点表，也可以直接按入口点名称调用：

```
| GOSBVL =SAVPTR
```

可以看到，使用入口点名称时，只需以名称替换地址。不过，HP-ASM 不支持这种写法；若未安装入口点表，也不能使用。

后文会进一步学习这些跳转。调用该例程后，就可以任意使用 D0 或 D1，因为地址 #0679Bh 处的例程会保存 D0、D1、B 和 D。以后介绍这些值保存在哪里时，还会说明 HP 为什么用 RAM 的某些区域保存重要数值。

退出前，可以用下列代码恢复指针：

```
| GOSBVL 067D2
```

该入口点名为 =GETPTR，在 48 和 49 上也相同。

不过，如果只执行非常简单的操作，可能不必保存全部指针。

下面看一个示例。

我们要编写一段 Code 程序：仅当堆栈的栈 1 中是字符串时，才丢弃它。首先要学会执行 DROP，然后还要学会判断堆栈对象的类型。

我们知道 D1 指向堆栈的栈 1。堆栈中保存的不是对象，而是对象地址。这样更高效：执行 SWAP 时，只需要交换栈 1 和栈 2 中的地址。对象本身不移动，因此无论对象多大，只移动 10 个半字节就能交换两个栈项，速度都一样。

这也解释了一些看似奇怪的现象。有时堆栈中有两个对象，修改其中一个后，另一个也会随之改变。

原因是执行 DUP 时复制的是对象地址，而不是对象本身。某些命令会直接修改栈 1 中的对象；由于两个地址相同，栈 2 中引用的同一个对象也会被修改。

这也是存在 NEWOB 命令的原因：它会复制对象本身，并把新对象的地址放到堆栈上。

因此，如果堆栈上有 3 个对象，堆栈实际包含：

内存地址	内容
D1+15	00000
D1+10	第 3 个对象的地址
D1+5	第 2 个对象的地址
D1	第 1 个对象的地址

程序开始时，D1 指向第 1 个对象的地址。若要让 D1 指向下一个对象的地址，执行：

```
| D1=D1+ 5
```

若使用 C=DAT1 f 或 A=DAT1 f，就会从栈 2 中的对象读取指定半字节。

要执行 DROP，可以让 D1 移到栈 2 中的对象，并释放 5 个不再使用的内存半字节。

DROP 因而可以写成：

```
| D1=D1+ 5
```

```
| D=D+1 A
```

第一条指令让 D1 指向堆栈中的下一个对象；第二条把 D 加 1，因为 D 以 5 半字节块为单位表示空闲 RAM。每个堆栈地址占用 5 个半字节，所以加 1 就释放了 5 个半字节的 RAM。

但在不检查堆栈内容的情况下不能随便执行 DROP，否则可能看到“Try To Recover Memory?”消息。汇编语言很强大，但一个错误就可能破坏内存。

下面是一段不检查堆栈是否为空的 DROP：

```
| D1=D1+ 5
```

```
| D=D+1 A
```

```
| A=DAT0 A
```

```
| D0=D0+ 5
```

```
| PC=(A)
```

```
| @
```

不要忘记 @，它告诉汇编器源代码在哪里结束。

可以看到，代码把 D1 向后移动了 5 个半字节，于是原来的栈 2 变成栈 1，同时释放 5 个半字节的内存。随后的 3 条指令是返回 RPL 的标准写法：读取下一条 RPL 对象的地址 A=DAT0 A，更新 D0，即 D0=D0+ 5，再继续执行下一条 RPL 对象，即 PC=(A)。

编译这段代码并把它存入变量。然后在堆栈中放入 3 或 4 个对象并运行。每次运行都会丢弃栈 1 中的对象。务必不要在堆栈为空时运行。

下面做得更完善一些：先检查堆栈是否为空；若不为空，再执行 DROP。

首先读取堆栈 1 中的地址。如果该地址等于 0，堆栈就是空的。由于堆栈中存储的是地址，因此必须检查这里的 5 个半字节是否全为 0。

读取组成堆栈第 1 个对象地址的 5 个半字节：

```
C=DAT1 A
```

此时，C 包含堆栈第 1 个对象的地址；若堆栈为空，则为 #00000h。接下来需要测试 C 是否等于 #00000h，并使用 A 字段：

```
?C=0 A
```

完成测试后，必须告诉汇编器测试为真，也就是 C=0，或为假，也就是 C 不等于 0 时应做什么。

使用的助记符是 GOYES，后接标签。

因此写成：

```
?C=0 A
GOYES QUIT
...
*QUIT
...
```

如果 C 的 A 字段等于 0，就跳转到标签 QUIT。请记住，标签所在行以 * 开头。

如果 C 不等于 0，就从 GOYES 后的下一行继续执行。

完整代码如下：

```
% 读取栈 1 中的内容
C=DAT1 A
% 如果等于 0，就跳转到 QUIT
?C=0 A
GOYES QUIT
% 否则，栈 1 中有对象
% 把 D1 向后移动 5 个半字节
D1=D1+ 5
% 释放 5 个半字节的内存
D=D+1 A
% 在这里退出
*QUIT
% 读取下一条 RPL 对象的地址
A=DAT0 A
% 更新 D0
D0=D0+ 5
% 跳转到该对象
PC=(A)
% 源代码结束
@
```

可以看到，使用 % 符号可添加注释。

程序员用注释记录各段代码的作用。把自己编写的代码交给别人时，注释也非常有用。汇编器会忽略 % 之后直到行尾的所有内容。

也可以把注释和代码按下面的方式分段排版：

```
% 检查栈 1 是否为空

C=DAT1 A
?C=0 A
GOYES QUIT

% DROP

D1=D1+ 5
D=D+1 A

% 在这里退出

*QUIT
A=DAT0 A
```

```
D0=D0+ 5
PC=(A)
```

@

应当在源代码中加入注释。这样有助于随时了解代码位置；数周后重新打开旧源代码时，也不必再次研究每一段代码的用途。

现在可以编写最初想要的代码：如果栈 1 中是字符串，就丢弃它。

与前面一样，首先读取堆栈栈 1 中对象的地址。若为 0，则退出：

```
C=DAT1 A
?C=0 A
GOYES QUIT
```

然后必须确认栈 1 中的对象是字符串。HP 上的每个对象开头都有一个 5 半字节序言，因此只需检查序言即可确定对象类型。

这里需要让 D0 或 D1 指向该对象，但又不能丢失 D0 或 D1 的原值，所以可以使用交换指令：

```
CD1EX
```

此时 C 包含 D1 的原值，而 D1 包含 C 的原值。由于 C 原本保存栈 1 对象的地址，所以 D1 现在指向该对象。

序言是对象最前面的 5 个半字节，因此使用 A 读取 5 个半字节：

```
A=DAT1 A
D1=C
```

此时已经读出对象序言，并把 D1 恢复为先前保存在 C 中的数值。

这里使用 CD1EX，让 D1 暂时指向其他位置，同时不丢失其原值，因为原值保存在 C 中。随后用 A 读取序言，以便 C 继续保存 D1 的原值。

接下来要把 A 与字符串序言比较。字符串序言为 #02A2Ch，所有字符串都以这 5 个半字节开头。下一部分会介绍所有对象，包括它们的序言和编码方式。

这里只需把 #02A2Ch 装入 C，再用 A 字段比较 A 与 C。如果 A=C，就可以丢弃对象；否则退出。

下面是完整代码：仅当堆栈栈 1 中是字符串时才执行 DROP。

```
% 读取栈 1 的地址
C=DAT1 A
% 是否等于 0？若堆栈为空，则退出
?C=0 A
GOYES QUIT
% 让 D1 指向对象，C 保存旧 D1
CD1EX
% 读取前 5 个半字节，也就是序言
A=DAT1 A
% 恢复 D1
D1=C
% 把字符串序言 #02A2Ch 装入 C
LC 02A2C
% # 表示不等于；检查 A 是否不等于 C
?A#C A
% 若不同，就不是字符串，直接退出
GOYES QUIT
% 在这里执行 DROP，把 D1 加 5
D1=D1+ 5
% 释放 5 个半字节的 RAM
D=D+1 A
% 在这里退出
*QUIT
A=DAT0 A
% 常规 RPL 跳转
D0=D0+ 5
PC=(A)
@
```

请试着运行，它会按预期工作。

如果堆栈为空，代码什么也不做。如果堆栈中是字符串，就丢弃该字符串；若不是字符串，则什么也不做。

注意：如果 HP-ASM 报告无法理解 ?A#C A，说明 OPT 中设置的是 HP。在 HP 模式下，不等号是带斜线的等号；在 PC 模式下，不等号为 #。

HP 的不等号可通过 **ALPHA**、**Right-Shift**、**1** 输入；**#** 可通过 **Right-Shift**、**Divide** 输入。

下面继续介绍指令。

35.5 在 A/C 与 D0/D1 之间交换最低有效的 4 个半字节

下面的指令交换工作寄存器 A 或 C 与 D0 或 D1 的 4 个 LSB 半字节，也就是前 4 个半字节。MSB 半字节保持不变：

助记符	操作码	字段	Carry	周期
AD0XS	13A	仅 4 个半字节	否	8.5
AD1XS	13B	仅 4 个半字节	否	8.5
CD0XS	13E	仅 4 个半字节	否	8.5
CD1XS	13F	仅 4 个半字节	否	8.5

这些指令与前面的交换指令相似，只是这里只交换 4 个半字节。

36 在字段范围内读取内存

我们已经知道如何修改 D0 或 D1，让它指向所需的内存位置，也知道如何用 D0/D1= .. 向 D0 或 D1 装入数值。本章只使用寄存器 A 和 C，并详细介绍前文已经用过的 DAT0 和 DAT1 指令。

下面的代码把 D1 所指位置的 16 个半字节复制到 C：

```
C=DAT1 W
```

下面的代码把工作寄存器 A 中的 12 个半字节，也就是 M 字段，复制到 D0 所指的数据位置：

```
DAT0=A M
```

助记符	操作码	字段	Carry	周期
A=DAT0 A	142	A	否	23.5, 3.5
A=DAT0 B	14A	B	否	19.5
A=DAT0 f	152a 152f +	P、WP、X、XS、S、W、M、B； F1-F7	否	20+n, 1+n/2；未列
A=DAT1 A	143	A	否	23.5, 3.5
A=DAT1 B	14B	B	否	19.5
A=DAT1 f	153a 153f +	P、WP、X、XS、S、W、M、B； F1-F7	否	20+n, 1+n/2；未列
C=DAT0 A	146	A	否	23.5, 3.5
C=DAT0 B	14E	B	否	19.5
C=DAT0 f	156a 156f +	P、WP、X、XS、S、W、M、B； F1-F7	否	20+n, 1+n/2；未列
C=DAT1 A	147	A	否	23.5, 3.5
C=DAT1 B	14F	B	否	19.5
C=DAT1 f	157a 157f +	P、WP、X、XS、S、W、M、B； F1-F7	否	20+n, 1+n/2；未列

接收半字节的寄存器位于等号 = 左侧。

被移动的不是指针本身的值，而是指针 D0 或 D1 所指位置的半字节。

例如，要读取 #00100h 处的 5 个半字节，可以写：

```
D0= 00100  
C=DAT0 A
```

D0 中保存 #00100h，而 C 中保存内存地址 #00100h 处的 5 个半字节。

从上表可以看出，字段 A 和 B 有专门的十六进制编码形式，也就是 Code 对象内部使用的形式。从 D0 读取 2 个半字节，即 B 字段时，可以在 Code 对象中用两种不同方式编码。因此，助记符 C=DAT0 B 可以编码为 14E，也可以编码为 1566。

代码中使用 14E 更短也更快：1566 需要 22 个周期，而 14E 只需要 19.5 个周期。Saturn 处理器在读写数据时对 A 和 B 字段做了某种专门优化；可以说，它“偏好”这两个字段。

37 使用数值指定读取长度

也可以用数值代替字段。因此，除了：

| C=DAT0 A

还可以写：

| C=DAT0 5

两者有什么区别？它们在 Code 对象中的编码方式不同。此外，C=DAT0 A 需要 23.5 个周期，而 C=DAT0 5 需要 24 个周期，所以这里使用 A 更好。不过，C=DAT0 16 又优于 C=DAT0 W，因为前者需要 35 个周期，后者需要 36 个周期。虽然只相差一个周期，但若把该指令放在循环中执行 100 次，就能少用 100 个周期。

这样计算指令周期会稍微复杂一些，但如果要编写尽可能快的例程，就应根据周期数选择指令。

助记符	操作码	字段	Carry	周期
A=DAT0 n	15A (n-1)	n 个半字节	否	19+n
A=DAT1 n	15B (n-1)	n 个半字节	否	19+n
C=DAT0 n	15E (n-1)	n 个半字节	否	19+n
C=DAT1 n	15F (n-1)	n 个半字节	否	19+n

38 在字段范围内写入内存

下面把工作寄存器 A 或 C 中的半字节复制到内存：

助记符	操作码	字段	Carry	周期
DAT0=A A	140	A	否	19.5
DAT0=A B	148	B	否	16.5
DAT0=A f	150a 150f +	P、WP、X、XS、S、W、M、B；F1-F7	否	19+n；未列
DAT1=A A	141	A	否	19.5
DAT1=A B	149	B	否	16.5
DAT1=A f	151a 151f +	P、WP、X、XS、S、W、M、B；F1-F7	否	19+n；未列
DAT0=C A	144	A	否	19.5
DAT0=C B	14C	B	否	16.5
DAT0=C f	154a 154f +	P、WP、X、XS、S、W、M、B；F1-F7	否	19+n；未列
DAT1=C A	145	A	否	19.5
DAT1=C B	14D	B	否	16.5
DAT1=C f	155a 155f +	P、WP、X、XS、S、W、M、B；F1-F7	否	19+n；未列

可以看到，字段 A 和 B 都有专门且经过优化的指令。A 适合处理地址；B 宽 2 个半字节，常用于处理字节。后文还会介绍，字符也是用 2 个半字节编码的。

39 使用数值指定写入长度

与前面一样，这里不用字段，而直接给出要写入的半字节数量：

助记符	操作码	字段	Carry	周期
DAT0=A n	158 (n-1)	n 个半字节	否	18+n
DAT1=A n	159 (n-1)	n 个半字节	否	18+n
DAT0=C n	15C (n-1)	n 个半字节	否	18+n

助记符	操作码	字段	Carry	周期
DAT1=C n	15D(n-1)	n 个半字节	否	18+n

假设要写入 1 个半字节：

```
DAT0=A P
```

如果 P 不等于 0，例如它正用于其他目的，可以写：

```
DAT0=A 1
```

这会写入 1 个半字节，并在 Code 对象中编码为 1580，也就是 158 加上 1-1 得到的 0。

现在应该不难理解了。

40 跳转与测试

如前所述，每条指令都位于内存中的某个地址，所以程序内部可以进行跳转。通常，程序指令按顺序逐条执行；但有时，在测试之后，或者仅仅因为希望代码从其他位置继续，就需要跳转到程序的另一部分。

跳转也称为“分支”，因为执行流程会“分支”到内存中的其他位置。

这里使用 PC 寄存器。您已经知道，PC 保存下一条要执行的指令，所以修改 PC 就能改变指令执行流程。

下面讨论三种跳转：

62. **相对跳转。** 从当前位置向后或向前跳转 n 个半字节。我们未必知道要跳到内存中的哪个绝对地址，但知道目标在当前位置之前或之后多少个半字节。这里引入一个新词：偏移量。偏移量是表示位移的数值。它是有符号数，可以为正或为负，因此既能向后跳，也能向前跳。例如，要向前跳 100 个半字节，偏移量就是 +100。
63. **绝对跳转。** 跳到某个明确地址。地址是 5 半字节数，例如 #ABCDE，所以绝对跳转就是跳到地址 ABCDE 并从那里继续执行。
64. **间接跳转。** 这种方式更复杂。它不是直接跳到给定地址，而是先在一个地址处读取另一个地址，再跳到读出的地址。例如，绝对跳转到 ABCDE 会直接到达内存地址 ABCDE；间接跳转则先读取 #ABCDE 处的 5 个半字节，再跳到这 5 个半字节所表示的地址。

在代码中使用标签跳转时，编译器会自动生成相应编码。向前跳转时，它直接编码偏移量；向后跳转时，则对地址使用补码，表示负偏移，从而跳回内存中的较早位置。

例如，假设代码中有下列跳转：

```
?C=0 A
GOYES QUIT
...
*QUIT
```

编译器会先生成省略号所代表的代码，再计算偏移量，使 GOYES 跳到 *QUIT 行之后编码的代码：

内存地址	Code 对象中的编码	指令
#n	8AAzy	?C=0 A
#n + ...	...	...
#n + yz	剩余代码	*QUIT 标签

编译器会计算偏移量，并用两个半字节 yz 对其编码。?C=0 A 会在 Code 对象中编码为 811zy。

如果跳转方向向后，编译器会用补码编码跳转值。

为什么这一点很重要？

因为测试后的跳转在 Code 对象中只用两个有符号半字节编码，所以当偏移量过大时，有时无法完成跳转。此时必须改写代码。后文会说明具体方法。

41 相对、绝对和间接跳转

41.1 条件跳转：GOC 和 GONC

条件跳转不会无条件发生，而是由另一个状态决定；这里决定跳转的是进位位。

有两条指令可以根据进位状态跳转：GOC 和 GONC。

65. GOC 表示 Go On Carry。
66. GONC 表示 Go On No Carry，也可理解为 Not Carry。

示例：假设从 A(A)，也就是工作寄存器 A 的 A 字段中减去 16。如果进位位置位，说明 A 原来的值小于 16，即 0 到 15。下面的代码检查是否发生下溢：

```
C=C-16 A
GOC ERROR
...
*ERROR
% 发生下溢
```

只有在进位位置位时，GOC 才会跳转。如果进位位未置位，就从 GOC ... 后的下一行继续执行。

只有在进位位未置位时，GONC 才会跳转；若进位位已置位，则不会跳转。

注意： 这些检查不会改变进位位。它们只检查进位位是否置位，不会修改其状态。跳转后，进位位仍保持原值，直到另一条会修改进位位的指令将其清除。

助记符	操作码	条件	周期
GONC label	5yz	进位位已清零	4.5 或 12.5
GOC label	4yz	进位位已置位	4.5 或 12.5

这里给出两个周期值：不跳转需要 3 个周期，发生跳转需要 10 个周期。因此，如果事先知道哪种结果更常见，最好安排为较少发生跳转的测试形式。

可以看到，使用两个半字节 yz 编码相对跳转。也就是说，共有 8 位用于编码偏移量。该数值以补码形式表示有符号数，因此 GONC 和 GOC 最多可向后跳 128 个半字节，或向前跳 127 个半字节。

如果标签位于前方超过 127 个半字节，或后方超过 128 个半字节，汇编器会报告跳转过远。

因此，下面的代码：

```
GOC ifCarry
% 无进位时在这里继续
```

可以改写为：

```
GONC noCarry
GOTO ifCarry
*noCarry
% 无进位时在这里继续
```

GOTO 多用 1 个半字节编码跳转，因此总共使用 3 个半字节。如果汇编器仍报告跳转过远，就使用 GOVLNG；它用 5 个半字节编码跳转地址，所以可以跳到内存中的任意位置。请记住，所有地址最多就是 5 个半字节。

请记住，GOC 和 GONC 只能向前跳 127 个半字节，或向后跳 128 个半字节。

如果需要更远的跳转，就必须改变测试结构，使用 GOTO、GOLONG 或 GOVLNG，即 GO Very LoNG。下面会介绍这些指令。

41.2 无条件跳转：GOTO、GOLONG、GOVLNG

“无条件”表示不进行任何检查，跳转一定发生，因此也可以称为“盲跳转”。

GOTO、GOLONG 和 GOVLNG 的区别之一，是它们分别用 3、4 或 5 个半字节编码跳转。

41.2.1 GOTO

助记符	操作码	条件	周期
GOTO label	6xyz	总是跳转	14

GOTO 会让程序从源代码中的 LABEL 位置继续执行，需要 14 个周期。由于使用 3 个半字节编码跳转，它可以从 GOTO 所在位置向后跳 2048 个半字节，或向前跳 2047 个半字节。

41.2.2 GOLONG

助记符	操作码	条件	周期
GOLONG label	8Cwxyz	总是跳转	17

这是范围更大的跳转，用 4 个半字节编码，因此可以从 GOLONG 所在位置向后跳 32768 个半字节，或向前跳 32767 个半字节。

编写代码时，先尝试 GOTO。如果汇编器报告跳转过远，再试 GOLONG。如果仍然过远，就使用 GOVLNG；它使用 5 个半字节，不会有这个问题。这样可以尽量选择占用半字节最少、周期也最少的指令。

41.2.3 GOVLNG

助记符	操作码	条件	周期
GOVLNG label	8Dvwxyz	总是跳转	18.5

由于所有地址都用 5 个半字节编码，这里可以直接跳到内存中的任意位置。编码的不是偏移量，而是绝对地址。

41.2.4 无条件跳转到 A 或 C 中的地址

这里使用两条指令：PC=A 和 PC=C。它们直接修改 PC 寄存器，把 A 或 C 装入 PC。下一条指令将从 A 或 C 中给出的地址读取。

助记符	操作码	目标	周期
PC=A	81B2	寄存器 A 的 A 字段	26, 3.5
PC=C	81B3	寄存器 C 的 A 字段	26, 3.5

这两条指令把 A 或 C 中的 5 个半字节直接移入 PC。

41.2.5 带交换的无条件跳转

这里不是把 A 或 C 的 A 字段复制到 PC，而是交换两者的值。PC 让指令流从 A 或 C 的 A 字段所给地址继续，而 PC 的旧值则被存入 A 或 C。

助记符	操作码	目标	周期
APCEX	81B6	寄存器 A 的 A 字段	19
CPCEX	81B7	寄存器 C 的 A 字段	19

41.2.6 无条件间接跳转

下面就是前面提到的间接跳转。先读取 A 或 C 所给地址处的 5 个半字节，再把它们装入 PC，从而跳到读出的地址。

助记符	操作码	目标	周期
PC=(A)	808C	寄存器 A 的 A 字段	26, 3.5
PC=(C)	808E	寄存器 C 的 A 字段	26, 3.5

例如，如果 A 或 C 中为地址 #ABCDE，而该地址处的 5 个半字节为 #FGHIJ，执行就会从 #FGHIJ 继续。

41.2.7 保存 PC 内容

下面两条指令不用于执行跳转，但适合在跳转前保存 PC 的值：

助记符	操作码	目标	周期
A=PC	81B4	寄存器 A 的 A 字段	11
C=PC	84B5	寄存器 C 的 A 字段	11

42 调用子例程

创建子例程通常是为了在同一段代码需要多次调用时节省空间。如果某段例程要反复调用，而又希望节省空间，就应把它放进子例程。子例程本质上是一个标签及其后的代码。需要时可调用它，执行完成后再返回调用位置，继续运行程序。

跳转到子例程时，处理器把跳转指令后一条指令的地址装入 RSTK，也就是 Return STack。随后从跳转目标继续执行，直到遇到返回指令。

遇到返回指令后，返回地址被装入 PC，程序便从子例程“返回”，并继续执行。

它与普通跳转的区别在于，必须记录一个地址，以便稍后返回。

RSTK 有 8 层，每层宽 5 个半字节，但正常情况下只能使用其中 5 层。这意味着可以嵌套 5 层子例程；如果禁止全部中断，则可以使用 8 层，后文会进一步说明。

有多种返回指令。有些只执行返回，有些返回时还会修改进位位，例如置位或清零。

调用子例程通常采用相对跳转，所以编码的是偏移量；GOSBVL，即 GOSuB Very Long，例外，它直接编码一个 5 半字节地址。

42.1 GOSUB

GOSUB 用 3 个半字节编码跳转偏移量，因此可以从 GOSUB 所在位置向后调用 2048 个半字节以内的子例程，或向前调用 2047 个半字节以内的子例程。

助记符	操作码	RSTK 使用量	周期
GOSUB label	7xyz	1 层	15

42.2 GOSUBL

GOSUBL 表示 GOSUB Long。这里用 4 个半字节编码偏移量，因此可以向后调用 32768 个半字节以内的子例程，或向前调用 32767 个半字节以内的子例程。

助记符	操作码	RSTK 使用量	周期
GOSUBL label	8Ewxyz	1 层	18

42.3 GOSBVL

这里直接编码地址的 5 个半字节：

助记符	操作码	RSTK 使用量	周期
GOSBVL label	8Fvwxyz	1 层	19.5

42.4 从子例程返回

可使用以下指令：

助记符	操作码	作用	周期
RTN	01	普通返回	11
RTNSC	02	返回并设置进位位	11
RTNCC	03	返回并清除进位位	11
RTI	0F	重新允许中断	11
RTNSXM	00	返回并设置 XM 位	11

RTN 从 RSTK 中弹出返回地址，并跳回调用子例程的指令之后。

RTNSC 和 RTNCC 在返回的同时设置或清除进位位。因此，可以让子例程通过进位位状态向调用者传递信息。

RTI 会重新允许中断。以后学习中断如何工作，以及 HP ROM 中的例程如何禁止和重新允许中断时，会更清楚地理解这条指令；后文还会对这些代码进行反汇编。

RTNSXM 会返回并设置 XM，即 eXternal Module missing 位。

42.5 根据进位位返回

可以根据进位位的值决定是否返回。相关指令如下：

助记符	操作码	条件	周期
RTNC	400	进位位置位时	4.5 或 12.5
RTNNC	500	进位位清零时	4.5 或 12.5

RTNC 只在进位位置位时返回。若进位位清零，测试耗时 4.5 个周期；若进位位置位，返回耗时 12.5 个周期。

RTNNC 只在进位位清零时返回。不执行返回时需要 4.5 个周期，执行返回时需要 12.5 个周期。

43 测试

执行测试时，会把一个寄存器与 0 或另一个寄存器比较。

下文会为周期数给出两个值。第一个是测试完成但不跳转时使用的周期数，第二个是测试完成且发生跳转时使用的周期数。

每条测试指令都以问号 ? 开头，后接工作寄存器 A、B、C 或 D，再接运算符 =、>、< 等。

测试指令之后的下一行必须是一条动作指令。它可以是使用 GOYES 的跳转，也可以在使用 RTNYES 时从子例程返回。

43.1 与 0 比较

43.1.1 是否等于 0?

这里检查工作寄存器是否等于 0。判断相等使用符号 =。

助记符	操作码	字段	Carry	周期
?A=0 A	8A8 yz	A	是	13.5/21.5
?A=0 f	9a8 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?B=0 A	8A9 yz	A	是	13.5/21.5
?B=0 f	9a9 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?C=0 A	8AA yz	A	是	13.5/21.5
?C=0 f	9aA yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?D=0 A	8AB yz	A	是	13.5/21.5
?D=0 f	9aB yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n

在生成的 Code 对象中，yz 是测试为真时用于跳转的有符号偏移量。

如果测试为真，进位位就会置位。

测试结果为假时，这些指令需要 8.5+n 个周期，其中 n 是参与比较的字段半字节数量；测试结果为真并发生跳转时，需要 16.5+n 个周期。

这些指令之后可以使用 GOYES label 或 RTNYES。GOYES 会把执行流程跳转到代码中的指定标签；RTNYES 会从子例程返回。

偏移量用两个半字节编码，并且是有符号数，所以最多能跳多远？向后 128 个半字节，或向前 127 个半字节。

43.1.2 是否不等于 0?

表示“不等于”的符号取决于 HP-ASM 选项：选择 HP 时，它是带斜线的等号；选择 PC 时，它是井号 #。MASD 两种都支持。强烈建议使用 #，这样可以在 PC、Mac 或 Linux 系统上编写代码并发送到 HP，而不必反复查找和替换特殊符号。

助记符	操作码	字段	Carry	周期
?A#0 A	8AC yz	A	是	13.5/21.5
?A#0 f	9aC yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?B#0 A	8AD yz	A	是	13.5/21.5
?B#0 f	9aD yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?C#0 A	8AE yz	A	是	13.5/21.5
?C#0 f	9aE yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?D#0 A	8AF yz	A	是	13.5/21.5
?D#0 f	9aF yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n

测试为真时进位位置位，否则清零。

43.2 两个寄存器是否相等

相等时测试为真，不相等时为假。

助记符	操作码	字段	Carry	周期
?A=B A	8A0 yz	A	是	13.5/21.5
?A=B f	9a0 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?B=C A	8A1 yz	A	是	13.5/21.5
?B=C f	9a1 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?A=C A	8A2 yz	A	是	13.5/21.5
?A=C f	9a2 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n

助记符	操作码	字段	Carry	周期
?C=D A	8A3 yz	A	是	13.5/21.5
?C=D f	9a3 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n

43.3 两个寄存器是否不相等

助记符	操作码	字段	Carry	周期
?A#B A	8A4 yz	A	是	13.5/21.5
?A#B f	9a4 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?B#C A	8A5 yz	A	是	13.5/21.5
?B#C f	9a5 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?A#C A	8A6 yz	A	是	13.5/21.5
?A#C f	9a6 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?C#D A	8A7 yz	A	是	13.5/21.5
?C#D f	9a7 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n

43.4 小于和大于

助记符	操作码	字段	Carry	周期
?A>B A	8B0 yz	A	是	13.5/21.5
?A>B f	9b0 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?B>C A	8B1 yz	A	是	13.5/21.5
?B>C f	9b1 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?A<C A	8B2 yz	A	是	13.5/21.5
?A<C f	9b2 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?C<D A	8B3 yz	A	是	13.5/21.5
?C<D f	9b3 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?A<B A	8B4 yz	A	是	13.5/21.5
?A<B f	9b4 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?B<C A	8B5 yz	A	是	13.5/21.5
?B<C f	9b5 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?A>C A	8B6 yz	A	是	13.5/21.5
?A>C f	9b6 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?C>D A	8B7 yz	A	是	13.5/21.5
?C>D f	9b7 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n

43.5 小于等于和大于等于

助记符	操作码	字段	Carry	周期
?A>=B A	8B8 yz	A	是	13.5/21.5
?A>=B f	9b8 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n

助记符	操作码	字段	Carry	周期
?B>=C A	8B9 yz	A	是	13.5/21.5
?B>=C f	9b9 yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?A<=C A	8BA yz	A	是	13.5/21.5
?A<=C f	9bA yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n

助记符	操作码	字段	Carry	周期
?C<=D A	8BB yz	A	是	13.5/21.5
?C<=D f	9bB yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?A<=B A	8BC yz	A	是	13.5/21.5
?A<=B f	9bC yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?B<=C A	8BD yz	A	是	13.5/21.5
?B<=C f	9bD yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?A>=C A	8BE yz	A	是	13.5/21.5
?A>=C f	9bE yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n
?C>=D A	8BF yz	A	是	13.5/21.5
?C>=D f	9bF yz	P、WP、X、XS、S、W、M、B	是	8.5+n/16.5+n

与往常一样，每条测试指令之后都可以接 GOYES 或 RTNYES。测试为真时执行跳转。

43.6 测试某一位

下面的指令用于测试 C 或 A 中某一位的值。只能测试工作寄存器最前面的 16 位，也就是 4 个半字节。

若测试为假并继续执行，需要 12 个周期；若测试为真并执行跳转，需要 21 个周期。

Code 对象中使用两个半字节 #yz 编码相对跳转偏移量，因此从测试位置最多可向前跳 128 个半字节，或向后跳 127 个半字节。

助记符	操作码	字段	Carry	周期
?ABIT=0 n	8086n yz	位编号 n	是	12.5/20.5
?ABIT=1 n	8087n yz	位编号 n	是	12.5/20.5
?CBIT=0 n	808An yz	位编号 n	是	12.5/20.5
?CBIT=1 n	808Bn yz	位编号 n	是	12.5/20.5

44 使用 P 寄存器

P 寄存器只能容纳 1 个半字节。程序开始时，它的值为 0。P 不仅决定装入指令 LC 和 LA 的工作方式，还定义 WP 寄存器的宽度，即从半字节 0 到半字节 P。

只要循环内部不使用 LC、LA 或与 WP 有关的指令，P 寄存器经常可以用作循环计数器。除了可以给 P 赋值，还可以递增或递减它。

还可以测试 P 寄存器、让它与 C 寄存器交换等。

下面列出助记符，随后逐条解释：

助记符	操作码	字段	Carry	周期
P= n	2n	P 的半字节	否	3
P=P+1	0C	P 的半字节	是	4
P=P-1	0D	P 的半字节	是	4
?P# n	88n yz	P 的半字节	是	7.5 或 15.5
?P= n	89n yz	P 的半字节	是	7.5 或 15.5

助记符	操作码	字段	Carry	周期
C=P n	80Cn	P 的半字节	否	8
P=C n	80Dn	P 的半字节	否	8
C+P+1	809	P 的半字节	是	9.5
CPEX n	80Fn	P 的半字节	否	8

44.1 P= n

不要忘记等号 = 后的空格。该指令把数值 n 赋给 P。

44.2 P=P+1

把 P 的值加 1。**注意：** 无论 Saturn 当前处于十六进制模式还是十进制模式，递增都按十六进制执行。发生溢出时，进位位置位。

44.3 P=P-1

把 P 的值减 1。**注意：** 无论 Saturn 当前处于十六进制模式还是十进制模式，递减都按十六进制执行。发生下溢时，进位位置位。

44.4 ?P# n

检查 P 是否不等于 n。Code 对象中使用两个有符号半字节编码相对跳转的偏移量。RTNYES 编码为 #00h；其他任何值都作为有符号偏移量处理，与前面的测试相同。测试为真时，进位位置位。

44.5 ?P= n

检查 P 是否等于 n。测试为真时，进位位置位。

44.6 C=P n

把 P 寄存器的值写入工作寄存器 C 的编号 n 半字节。n 的范围当然是 0 到 15。

44.7 P=C n

把工作寄存器 C 的编号 n 半字节写入 P。

44.8 C=P+1

把 C(A)、P 和 1 相加，并把结果写入 C 的 A 字段。这条指令看起来有些奇怪，但会非常有用。无论 Saturn 当前处于什么模式，该指令也始终按十六进制执行。

44.9 CPEX n

该指令交换工作寄存器 C 的编号 n 半字节与 P 寄存器。

45 RSTK 栈

RSTK 是返回栈，子例程运行时，返回地址保存在这里。也可以用 RSTK 暂时保存工作寄存器 C，但退出前不要忘记弹出已保存的值。它比使用保存寄存器更快，因此很实用。

助记符	操作码	字段	Carry	周期
RSTK=C	06	C 的 A 字段	否	9
C=RSTK	07	C 的 A 字段	否	9

C=RSTK 还很适合在代码中嵌入数据，例如：

```
GOSUB AfterData
INCLUDE MyDataGoesHere
*AfterData
% C 现在指向嵌入的数据
C=RSTK
```

46 IN 和 OUT 寄存器

处理器的输入和输出都通过 IN 和 OUT 寄存器进行。X 字段在这里非常有用，因为 OUT 寄存器宽 3 个半字节。IN 寄存器宽 4 个半字节。

共有 4 条指令：

助记符	操作码	字段	Carry	周期
OUT=CS	800	C 的半字节 #0h	否	5.5
OUT=C	801	C 的 X 字段	否	7.5

助记符	操作码	字段	Carry	周期
A=IN	802	A 的 4 个半字节	否	8.5
C=IN	803	C 的 4 个半字节	否	8.5

另一个错误

除非 A=IN 或 C=IN 指令位于偶数地址，否则它们不能正常工作。

因此，源代码中不要直接写 A=IN 或 C=IN，而应始终调用 ROM 中执行 A=IN 或 C=IN 后返回程序的子例程。

应使用：

67. 用 GOSBVL =CINRTN 代替 C=IN。=CINRTN 在 HP 48 上为 01160，在 HP 49 上为 00212。

68. 用 GOSBVL =AINRTN 代替 A=IN。=AINRTN 在 HP 48 上为 0115A，在 HP 49 上为 0020A。

这些例程包含：

```
C=IN
RTN
```

以及：

```
A=IN
RTN
```

ARM 架构计算器中的 Saturn+ 模拟处理器已经修复了这个错误，但为了保持向后兼容，最好仍避开它。

47 状态位 ST

程序中使用许多标志。ST 内部有 4 个半字节，也就是 $4 * 4 = 16$ 个标志；但其中一些已经由 Saturn 使用。在熟练掌握之前，应避免随意操作位 12、13 和 14。位 15 会用于启用或禁用中断，后文很快就会介绍。

这些状态位与 RPL 中使用的标志相似。

下面先列出指令，随后逐条解释：

助记符	操作码	字段	Carry	周期
CLRST	08	ST 的前 3 个半字节	否	7
C=ST	09	C 的 X 字段	否	7
ST=C	0A	C 的 X 字段	否	7
CSTEX	0B	C 的 X 字段	否	7
ST=0 n	84n	ST 的位 n	否	5.5
ST=1 n	85n	ST 的位 n	否	5.5
?ST=0 n	86n yz	ST 的位 n	是	8.5 或 16.5
?ST=1 n	87n yz	ST 的位 n	是	8.5 或 16.5

47.1 CLRST

清除 ST 的前 3 个半字节。最后 1 个半字节不清除，因为它包含处理器使用的特殊数值。

47.2 C=ST

把 ST 的 3 个半字节复制到 C。

47.3 ST=C

把 C 中的 3 个半字节复制到 ST。

47.4 CSTEX

交换 C 与 ST 的 3 个半字节。

47.5 ST=1 n

把 ST 的位 n 设为 1。

47.6 ST=0 n

清除 ST 的位 n，也就是把它设为 0。

47.7 ?ST=0 n 和 ?ST=1 n

这两条指令用于检查 ST 寄存器中单个位的值，可以测试它是置位还是清零。需要时，可在指令后使用 GOYES 和 标签，或使用 RTNYES。

48 硬件状态位 HST

共有 11 条指令可以影响 HST 位。HST 中有用的位如下：

- 69. SR, Service Request, 服务请求。
- 70. SB, Sticky Bit, 粘滞位。
- 71. MP, Module Pulled, 模块被拔出。
- 72. XM, eXternal Module missing, 外部模块缺失。

助记符	操作码	字段	Carry	周期
CLRHST	82F	HST 的 4 位	否	4.5
XM=0	821	XM 位	否	4.5
SB=0	822	SB 位	否	4.5
SR=0	824	SR 位	否	4.5
MP=0	828	MP 位	否	4.5
HST=0 n	82n	1 个或多个位	否	4.5
?XM=0	831	XM 位	是	7.5 或 15.5
?SB=0	832	SB 位	是	7.5 或 15.5
?SR=0	834	SR 位	是	7.5 或 15.5
?MP=0	838	MP 位	是	7.5 或 15.5
?HST=0 n	83n	1 个或多个位	是	7.5 或 15.5

需要记住的内容不少。

CLRHST 会清除 HST 的 4 个位。也可以用 XM=0 或 SB=0 等指令只清除 HST 中的一个位。HST 共 4 位，因此有 4 条分别清除各位的指令。

还可以用下面的公式，把 HST 中某些位设为 0：

$$\text{value} = \text{XM} + (2 * \text{SB}) + (4 * \text{SR}) + (8 * \text{MP})$$

在这个公式中，1 表示清除相应位，0 表示不清除。若希望：

$$\begin{aligned} \text{XM} &= 0 \\ \text{SB} &= 0 \\ \text{SR} &= 1 \\ \text{MP} &= 1 \end{aligned}$$

应计算：

$$\begin{aligned} \text{value} &= 0 + (2 * 0) + (4 * 1) + (8 * 1) \\ &= 0 + 0 + 4 + 8 \\ &= 12 \end{aligned}$$

因此，执行 HST=0 12 就能只清除 SR 和 MP 位。请注意，不存在 HST=1 n 指令。

可以分别测试 4 个可用位，也可以用 ?HST=0 n 结合上面的同一公式，测试 HST 中某些位或全部位是否为 0。在这种情况下，公式中的 1 同样表示应检查该位是否清零。

这些测试指令之后，若测试为真，可以使用 GOYES 或 RTNYES。

测试为假并继续执行时需要 6 个周期；测试为真并发生跳转时需要 13 个周期。若当前位于子例程内部，动作既可以是跳转到标签，也可以用 RTNYES 返回。

例如，编写一段代码：如果 XM 位为 1，就从子例程返回。先计算：

$$\begin{aligned} \text{value} &= 1 + (0 * 2) + (0 * 4) + (0 * 8) \\ &= 1 \end{aligned}$$

于是代码可以写成：

```
?HST=0 1
RTNYES
```

如果希望在 `XM=1` 且 `SR=1` 时跳转到标签 `REACT`，先计算：

```
value = 1 + (0 * 2) + (1 * 4) + (0 * 8)
       = 1 + 4
       = 5
```

代码如下：

```
?HST=0 5
GOYES REACT
```

要在工作寄存器执行位旋转或半字节旋转后测试 `SB`，即 `Sticky Bit`，可以写：

```
?HST=0 2
GOYES RESULT_LOSS
```

其他情况以此类推。

49 Saturn 的 DEC/HEX 模式

Saturn 处理器可以在两种模式下工作：十六进制模式，即基数 16；或十进制模式，即基数 10。前文已经介绍过，十进制模式使用压缩 BCD。

请记住，BCD 用 4 位编码每个十进制数字。可回看前面数十页的相关说明。

助记符	操作码	字段	Carry	周期
SETHEX	04	十六进制模式	否	4
SETDEC	05	压缩 BCD 模式	否	4

50 中断

感谢 Christoph Giesselink 对本章的贡献。

助记符	操作码	字段	Carry	周期
INTOFF	808F	禁用键盘中断	否	7
INTON	8080	启用键盘中断	否	7
RSI	80810	复位中断	否	8.5

什么是中断？通常，CPU 会逐条执行指令。但有时需要对外部事件作出反应，例如键盘输入。一种做法是不断读取键盘输入端口，这称为“轮询”。更理想的方式是让外部事件暂停 CPU 当前执行流程，调用一个指定子例程处理事件，然后返回原应用程序，这称为“中断”处理。由中断调用的代码称为“中断处理程序”。

HP 48 和 49 只有一个中断处理程序，它会在地址 `#0000F` 处继续执行。中断可以由不同来源触发：键盘、定时器、串行端口等。其中一些可屏蔽，一些不可屏蔽。键盘、定时器或串行端口等可屏蔽中断源可以禁用，但 **ON** 键等不可屏蔽中断源总会生效。不过，中断不可再次触发：进入中断服务例程后，在用 `RTI` 命令离开之前，不会再发生其他中断。这由 CPU 内部的“interrupt in service”标志控制。发生中断时，该标志置位，PC 被设为 `#0000F`；`RTI` 命令会清除该标志。

50.1 禁用全部中断

有时需要禁用全部中断，包括不可屏蔽中断。但唯一绝对不会再发生中断的位置，是中断服务例程内部。因此，需要先触发一次中断，并在“中断服务中”状态下继续执行。

可执行：

```
% 禁用全部中断
ST=0 15
```

通常，中断发生时，HP 操作系统会检查 `ST` 寄存器的位 15。如果该位已清零，操作系统就把 `ST` 的位 14 置位，表示有中断待处理，并立即通过 `RTN` 返回。此时“interrupt in service”标志已经置位，所以之后不会再发生中断。如果希望避免进入关键区后再发生首次调用，就必须在进入不允许中断的关键区之前主动触发一次中断。

为简化操作，可以调用：

```
% =DisableIntr 在 48 上为 01115，在 49 上为 26791
GOSBVL =DisableIntr
```

这段代码会清除 `ST` 的位 15，通过 `INTOFF` 禁用键盘中断，并触发一次中断。

50.2 启用全部中断

要启用全部中断，最好调用 `RTI` 指令，它会清除“interrupt in service”标志。如果存在待处理中断，也就是 `ST` 的位 14 已置位，还应复位中断系统。

要启用中断系统，可通过 `GOSUB REINT` 调用下列子例程：

```
*REINT
% 允许全部中断
ST=1 15
% 是否有中断待处理？
?ST=0 14
% 没有，则继续
GOYES NOINT
% 复位中断系统
RSI
*NOINT
% 清除 interrupt in service 标志
RTI
```

为简化操作，可以调用：

```
% =AllowIntr 在 48 上为 010E5，在 49 上为 26767
GOSBV L =AllowIntr
```

这段代码会设置 `ST` 的位 15，通过 `INTON` 启用键盘中断；若有中断待处理，则复位中断系统；最后用 `RTI` 清除“interrupt in service”标志。

51 总线相关指令

后文会更深入地讨论总线，但在学习总线的全部细节时，可以把本章作为参考。总线与处理器相连；模块、内存配置等方面还有许多内容要学习，这一部分可能比较难。

Saturn 只有一条总线。通常，处理器可能分别有数据总线和指令总线；Saturn 只有一条，所以它是“多路复用”的。这里的指令可用于配置或取消配置特定内存模块，使某些部分可见、另一些部分不可见。

有 3 条指令不用于 HP 48：`BUSCB`、`BUSCC` 和 `BUSCD`。

助记符的含义是：`BUS` 表示 Bus，`C` 表示 Command，最后一个字母表示具体命令。

不过，HP 49G+ 和 48GII 上的一些 Saturn+ 指令以 `BUSCC` 的操作码为基础，因此许多新增 Saturn+ 指令以 `80B` 开头。

助记符	操作码	字段	Carry	周期
RESET	80A	-	否	7.5
SREQ?	80E	-	否	9.5
CONFIG	805	C 的 A 字段	否	13.5
UNCNFG	804	C 的 A 字段	否	14.5
C=ID	806	C 的 A 字段	否	13.5
SHUTDN	807	-	否	6.5
BUSCB	8083	-	否	10
BUSCC	80B	-	否	8.5
BUSCD	808D	-	否	10

51.1 RESET

执行该命令后，所有可配置模块都会被设为未配置状态。ROM 不会被取消配置，因为它没有配置管理器。

51.2 SREQ?

这条指令似乎没有使用。它原本用于在最早实现 Saturn 架构的 HP-71 上发出 `POLL` 总线命令，并把系统总线状态锁存到 `C(0)`。

51.3 CONFIG

用于配置内存模块。本书第 66 节会作说明。

51.4 UNCNFG

取消配置基地址位于 C 的 A 字段中的内存模块。

51.5 C=ID

把当前内存模块的 ID 复制到 C 的 A 字段。其内容会随模块状态而变化，后文关于“被覆盖内存（Covered memory）”的部分会作说明。

51.6 SHUTDN

让计算器进入低功耗模式。只有少数必要部件保持供电，HP 会休眠，直到以下任一条件将其唤醒：

- 73. 定时器 1 或 2：对应定时器控制寄存器中的唤醒位 WKE 和 MSB 均已置位。
- 74. IN 寄存器中出现非零值，即识别到按键按下。
- 75. 串行接收缓冲区发出中断。

51.7 BUSCx

这些指令不用于 HP 48，也不用于采用传统 Saturn 的 HP 49。BUSCC 指令的操作码在 48GII 和 49G+ 中被用作许多新增 Saturn+ 指令的前缀。

52 填充指令

有 3 条指令可以作为“填充”，因为它们不执行任何操作。它们唯一的实际用途，是在使用绝对寻址编写代码时，如果删除了某些代码，就用这些指令填补空隙，使各入口点仍位于原地址。绝对寻址表示，所有到代码内部入口点的跳转都是指向特定内存地址的绝对跳转，而不是相对跳转。这样性能更好，但会让代码维护变得非常困难，也会严重限制最终程序在计算器上的安装方式。

这些 NOP，即 no operation 指令，分别有 3、4 和 5 个半字节的长度。

HP-ASM 和 MASD 都不识别这些助记符。如果确实必须使用，可以在操作码前加美元符号 \$，直接输入操作码。

助记符	操作码	字段	Carry	周期
NOP3	420	-	否	3 或 10
NOP4	6300	-	否	11
NOP5	64000	-	否	11

53 ARM 系统指令

采用 ARM 架构的计算器，包括 49G+ 和 48GII，提供了额外的 Saturn+ 指令，主要用于与底层 ARM 架构交互。HP 没有发布这些指令的详细资料，但下表列出新增指令，以及从 comp.sys.hp48 新闻组收集到的少量信息。

助记符	操作码	说明
RPL2	80B00 +	返回 RPL，等效于 $A=DAT0.A\ D0+5\ PC=(A)$ 。
OFF	80B01 +	关闭计算器。按 ON 后继续执行。
CONFIGD	80B03 +	使用 C(A) 中的地址配置一个包含 16 行状态栏的 4 KB 块；地址必须是 4K 的整数倍。
REMON	80B05 +	启用计算器远程控制，即 ON-R 。通过 USB 端口传输屏幕内容并接收键盘输入。
ACCESSSD	80B06 +	访问 SD 卡。把卡的可用空间写入 A 的低 8 个半字节；失败时写入与错误消息对应的负数，例如未插卡。受 P 影响。
GETTIME	80B11 +	由于定时器不再位于 I/O RAM，可用此指令访问定时器。C(W) 返回自由运行定时器，D(W) 返回倒计时定时器，B(W) 返回从 0 开始向上计数的定时器。调用前必须禁用全部中断，否则计算器会崩溃。
UNCNFGD	80B13 +	取消配置 16 行状态栏并刷新。
REMOFF	80B15 +	禁用计算器远程控制，即 ON-S 。

助记符	操作码	说明
PORTTAG?	80B16 +	在 A(A) 中返回与 DAT1 指定标签名称对应的端口号 0 到 3；标签按长度后接字符的形式存储。
SETTIME	80B21 +	使用 C(W) 中的 tick 数设置实时时钟。
REFRESHD	80B23 +	刷新 16 行状态栏。
RESETOS	80B31 +	执行 ON-A-F 复位。
SETLNED	80B33 +	把状态栏设为 C(A) 行并刷新屏幕。使用 C=0 可禁用新状态栏。
AUTOTEST	80B41 +	执行 ON-D 菜单中的诊断测试。
SETOFFD	80B43 +	把状态栏显示偏移量设为 C(X) & 0x7FF 字节。
BEEP2	80B50 +	使用蜂鸣器。C(A) = time (ms), D(A) = frequency (Hz), 并要求 P=0。
NATIVE?	80B51 +	如果后续两个半字节表示的原生函数未定义, 则设置进位位; 如果已定义, 则清除进位位。
MOVEDN	80B60 +	MOVEDOWN 的高速原生等效指令。C(A) 给出半字节数量, D0 给出源起始地址, D1 给出目标起始地址。
OUTBYT	80B65 +	通过红外发送 A(B) 中的字节。
MOVEUP	80B70 +	MOVEUP 的高速原生等效指令。C(A) 给出半字节数量, D0 给出源结束地址, D1 给出目标结束地址。
SERIAL	80B71 +	用 10 字节 ASCII 编码把 49G+ 的序列号写入 DAT1。
CRTMP	80B80 +	CREATETEMP 的高速原生等效指令。大小由 C(A) 给出。
HST=1.n	80B82h +	设置 HST 的单个位, 其中 XM=1、SB=2、SR=4、MP=8。将各值相加可一次设置多个位; 助记符中的十进制 n 会编码为相应十六进制值。
XM=1	80B821 +	设置 XM, 是 HST=1.1 的替代助记符。
SB=1	80B822 +	设置 SB, 是 HST=1.2 的替代助记符。
SR=1	80B824 +	设置 SR, 是 HST=1.4 的替代助记符。
MP=1	80B828 +	设置 MP, 是 HST=1.8 的替代助记符。
?HST=1.n	80B83h +	检查 HST 的单个位是否置位, 其中 XM=1、SB=2、SR=4、MP=8。将各值相加可同时检查多个位; 助记符中的十进制 n 会编码为相应十六进制值。
?XM=1	80B831 +	检查 XM 是否置位, 是 ?HST=1.1 的替代助记符。

助记符	操作码	说明
?SB=1	80B832 +	检查 SB 是否置位, 是 ?HST=1.2 的替代助记符。
?SR=1	80B834 +	检查 SR 是否置位, 是 ?HST=1.4 的替代助记符。
?MP=1	80B838 +	检查 MP 是否置位, 是 ?HST=1.8 的替代助记符。
HSCREEN	80B92 +	把屏幕高度返回到 A(A)。
KEYDN	80BA0 +	读取键盘, 使用 C(A)。
WSCREEN	80BA2 +	把屏幕宽度返回到 A(A)。
GOSLOW	80BB0 +	等待 (C(A) / 183) 毫秒。
MIDAPP?	80BB2 +	在 HP 49G+ 上清除进位位, 在 48GII 上设置进位位。
BIGAPP?	80BC2 +	在 HP 49G+ 上设置进位位, 在 48GII 上清除进位位。
SETFLDn	80BF7h +	为指令定义新字段 F1 到 F7, 其中 n = 1-7。字段掩码放在 C(W) 中。助记符中的 n 为 1 到 7, 在操作码中变为 8 到 F, 以替换 h。
ARMSYS	80BFE +	把 ARM PC 设为 C(B) 中的 32 位值; 执行前先清除位 0 和 1, 确保地址位于 4 字节边界。
ARMSAT	80BFF +	调用 C(A) 指定 Saturn 地址处的 ARM 代码。

Saturn+ 还支持一些其他 80Bxx 指令, 但它们尚无助记符, 也完全没有文档; 除在 comp.sys.hp48 新闻组中零星出现外, 几乎没有其他信息。

54 ARM 增强指令

ARM 架构计算器又增加了少量指令，为常用操作提供原生 ARM 替代实现。写作本书时，对它们所知甚少。新增指令列于下表。它们似乎没有助记符，所以使用时必须手工输入操作码，并在前面加 \$。本表主要供关注 ARM 架构计算器底层工作方式的读者参考。

助记符	操作码	说明
无可助记符	81B1 +	出现在返回 RPL 循环 <code>A=DAT0.A D0+5 PC=(A)</code> 的开头，用于返回 RPL。
无可助记符	81B8, 81B9, 81BA + 81BB, 81BC +	这些指令显然都与更快的 SKIPOB 等效实现有关。
无可助记符	81BD +	用于 SEMI 的最开头，以运行更快的替代代码。
无可助记符	81BE +	用于 DOCOL 的最开头，以运行更快的替代代码。

第五部分 对象

55 对象概述

对象是 HP 48 和 HP 49 的核心组成部分。放入堆栈的所有内容都是对象，而且它们都采用相同的编码方式：开头是一个用于标识对象的 5 半字节前导码，随后是所谓的对象“主体”。从某种意义上说，对象就像一个外壳，数据装在其中。

什么是前导码？它不仅用于标识对象，实际上还包含一个用于执行该对象的代码例程地址。因此，前导码既标识某项内容，也负责执行它。

有些对象在求值时只会将自身放到堆栈上，例如字符串。对字符串执行 `EVAL` 后，它仍留在堆栈上。但是，如果把一个全局名称放到堆栈上并执行 `EVAL`，与该全局名称对应的变量内容就会被放到堆栈上。此时，全局名称的前导码得到求值，并把变量内容压入堆栈。

下面将逐一研究 HP 48 和 HP 49 的各种对象类型，因此这一部分相当长。

您可能知道，命令 `TYPE` 会根据堆栈中的对象类型返回一个值。`TYPE` 值共有 28 个，但很快就会看到，这还不够。在这 28 种对象类型中，只有一部分能够从 RPL 使用。

下面介绍的一些对象是复合对象，例如 `HOME` 目录对象或库对象（其中最有意思的一种），它们由其他对象组成；另一些对象则是简单对象，例如字符串。

下表列出各对象类型的名称、User RPL `TYPE` 命令给出的值、System RPL 代码、前导码，以及它属于简单对象还是复合对象：

对象类型名称	TYPE	代码	前导码	类别
实数	0	1	02933	简单
复数	1	2	02977	复合
字符串	2	3	02A2C	简单
数组	3/4	4	029E8	复合
列表	5	5	02A74	复合
全局名称	6	6	02E48	简单
局部名称	7	7	02E6D	简单
RPL 程序	8	8	02D9D	复合
代数表达式	9	9	02AB8	复合
符号对象	N/A	A	N/A	二者均可
二进制整数	10	B	02A4E	简单
图形对象	11	C	02B1E	简单
标记对象	12	D	02AFC	复合
单位对象	13	E	02ADA	复合
XLIB 名称	14	OF	02E92	简单

对象类型名称	TYPE	代码	前导码	类别
目录对象	15	2F	02A96	复合
库	16	8F	02B40	复合
备份	17	9F	02B62	复合
（未使用）	18	N/A	N/A	N/A
（未使用）	19	N/A	N/A	N/A
系统二进制整数	20	1F	02911	简单
长实数	21	3F	02955	简单
长复数	22	4F	0299D	复合

对象类型名称	TYPE	代码	前导码	类别
链接数组	23	5F	02A0A	复合
字符	24	6F	029BF	简单
代码对象	25	7F	02DCC	简单
库数据	26	AF	02B88	简单
扩展指针（仅 49）或 EXT1	27	BF	02BAA	简单
小字体（仅 49）	27	DF	026FE	简单
EXT3	27*	N/A	02BEE	N/A
EXT4	27*	EF	02C10	N/A
无限精度实数（仅 49）	27**	N/A	0263A	复合
无限精度复数（仅 49）	27**	N/A	02660	复合
闪存指针（仅 49）	27	N/A	026AC	简单
Aplet（仅 49）	27***	N/A	026D5	复合
无限精度整数（仅 49）	28	FF	02614	简单
符号矩阵（仅 49）	29	N/A	02686	复合
字体（仅 49）或 EXT2	30	CF	02BCC	简单

* 这些类型由 HP 保留，以备将来需要时使用。

** HP 49 的内置软件未使用这些类型，但某些第三方软件支持它们。

*** 此类型从未在 HP 49 中完整实现，也没有任何已知软件使用它。

56 简单对象

56.1 实数

前导码：02933

结束码：无

大小：21 半字节

TYPE：0

内存中的结构：

字段	长度
前导码（02933）	5 半字节
指数	3 半字节
尾数	12 半字节
符号	1 半字节

把它与寄存器的 S、M 和 X 字段进行比较，您是否注意到它们完全相同？

这是一种典型且经常使用的对象类型。实数分为三个字段：尾数、符号和指数。尾数是数值本身，指数是 10^n 中的 n ；符号用一个半字节编码，正数为 #0h，负数为 #9h。

尾数使用 12 位数字，并在内存中采用压缩 BCD（Binary Coded Decimal，二进制编码十进制）编码。指数也使用 BCD 编码，但其编码方式会随符号而变化。

如果指数为正，这 3 个半字节直接采用 BCD 编码。

如果指数为负，则按下式编码：

| 1000 - ABS(exponent)

ABS 表示指数的绝对值，其取值范围为 0 至 499。

例如，编码 2.79233×10^{-9} 。首先是前导码 #33920h（请记住，它必须反向存放）。随后是指数：这里指数为 $1000 - \text{ABS}(-9)$ ，即 $1000 - 9 = 991$ ，因此反向存放后为 #199。尾数是 279233，又因为它在内存中也要反向编码，所以为 #000000332972h。最后放置符号 #0h。

内存中的内容为：

| #33920 199 000000332972 0h

其中的空格仅为便于阅读，实际实数对象中没有这些空格。

| **注意！** Saturn 在内存中会颠倒半字节的顺序。因此，这里每个字段在内存中都是反向的。

56.2 字符串

前导码： 02A2C

结束码： 无

大小： 12 半字节起

TYPE： 2

内存中的结构：

字段	长度
前导码 (02A2C)	5 半字节
大小	5 半字节
字符 1	2 半字节
...	...
字符 n	2 半字节

字符串主体的大小等于用于编码大小本身的 5 个半字节，再加上每个字符 2 个半字节。与其他所有对象一样，计算大小时不包含前导码。

每个字符使用 2 个半字节并按 ASCII 编码，ASCII 将在下文介绍。大小包括除前导码以外的所有内容，因此长度为 5 个半字节加 $2 * n$ 个半字节，即 $5 + 2n$ 。

ASCII 是用于编码字符的表，其名称是 American Standard Code for Information Interchange（美国信息交换标准代码）的缩写。它用八位，也就是一个字节或两个半字节，来编码数值。八位最多可表示 2^8 ，即 256 个不同的值。

实际上，最初的 ASCII 表只使用 7 位，因此只有 128 个不同符号。现在使用的是扩展 ASCII，它使用全部八位。

前 32 个字符具有通信或打印方面的特殊含义。这些字符称为“不可打印字符”，包括换页、制表以及设备间通信所用的其他代码。前 128 个字符是标准的，后 128 个字符则会随国家和计算机平台而变化，例如 DOS、Windows、Macintosh、西里尔字符集、欧洲字符集和西方字符集等。

在 HP 48 上，可以按 [Right-Shift] [PRG] 查看 ASCII 表。最初会看到字符 128 至 191；使用 -64 可以查看此前的 64 个字符，使用 +64 可以查看此后的 64 个字符。在 HP 49G 上，按 [Right-Shift] [CAT] 可打开 ASCII 表；在其他 HP 49 系列计算器上则必须使用 [Right-Shift] [EVAL]。在所有 HP 49 计算器上，都可以用方向键移动来查看任意字符。

因此，字符 0 至 127 是标准字符，因为它们属于最初的 7 位 ASCII 标准。字符 128 至 255 会随平台（PC、Mac 等）和字母表（罗马、西里尔、汉字等）而变化。

例如，字符串 “HP” 的编码由字符串前导码 #02A2Ch 加对象主体组成。请记住，这里有两个字符，占用四个半字节。大小使用五个半字节编码，值为 #9h 个半字节；最后是字符的四个半字节 #4850。因此，字符串对象编码如下：

| #C2A20 90000 8405h

请注意，与其他内容一样，每个字符的两个半字节也会反向存放。

#48h 是 #72d，即字符“H”。

#50h 是 #80d，即字符“P”。

可以用汇编语言操作字符串。下面编写一个程序，把字符串中的所有字母转换为大写。

首先，用 [Right-Shift] [PRG] 查看 CHARS 表，并注意字符 A 与 a 之间的固定距离为 32。如果已有一个小写字符，也就是值在 97 至 122（含端点）之间的字符，只需减去 32 就能得到对应的大写字符值。这很简单。

这个示例很有意思：它包含一个使用进位标志的循环，您将看到如何计算循环值，其中还包括若干测试。它是一个不错而又不太复杂的示例。

把 HP 变成《Star Trek》相位器、利用红外二极管惊人发热能力的程序稍后才会出现；请耐心等待，因为 HP 似乎不愿公开那段秘密代码。

代码已加注释：

```
% --- 堆栈为空时报告“Too Few Arguments” ---

C=DAT1 A
% 读取栈 1 对象的地址
?C#0 A
% 如果栈 1 非空，即不等于零.....
GOYES STRING?
% 检查它是否为字符串
LA 00201
% 将 #201h 装入 A
*ERROR
% 如果栈 1 中没有字符串，就需要跳到此标签
GOVLNG =Errjmp
% =Errjmp 位于 05023；该例程产生错误 #201h: “Too Few Arguments”

% --- 检查是否为字符串 ---

*STRING?
% 标签
CD1EX
% 让 D1 指向对象
A=DAT1 A
% 读取其前导码
D1=C
% 恢复 D1
LC 02A2C
% 把字符串前导码装入 C
?A=C A
% 如果对象是字符串
GOYES MAIN
% 跳到 MAIN
LA 00202
% 否则装入错误代码 #202h
GOTO ERROR
% 该代码表示“Bad Argument Type”；跳到 ERROR，那里包含通往 ROM 中 DOERROR 例程的 GOVLNG

% --- 准备循环 ---

*MAIN
% 类似主例程
GOSBVL =SAVPTR
% =SAVPTR 位于 0679B，用于保存寄存器 D1、D0、B 和 D
C=DAT1 A
% 读取字符串地址
D1=C
% 指向字符串
D1=D1+ 5
```

```

% 越前导码并指向大小字段的半字节
C=DAT1 A
% 读取字符串大小
C=C-5 A
% 去掉大小字段自身的长度

D=C A
% 将结果存入 D
% --- 计算循环值 ---

% D 中保存的是半字节数。每个字符占 2 个半字节，因此要把 D(A) 除以二；这里使用 DSRB A，即右移一位实现除以二。

DSRB A
% 除以二
D=D-1 A
% 再减去一，因为将使用进位循环
GOC END
% 如果进位标志置位，说明字符串为空；跳到末尾退出
D1=D1+ 3
% 向前移动 3 个半字节而不是 5 个；这样进入 LOOP 时，其中的 D1=D1+ 2 会在第一次进入时指向第一个字符，在继续循环时则指向下一个字符

% --- 循环 ---

*LOOP
D1=D1+ 2
% 指向字符
C=DAT1 B
% 把字符读入 C
LA 61
% #61h 即 #97d
?C<A B
% 如果小于 97，则跳过
GOYES CHECK
LA 7A
% #7Ah 即 #122d
?C>A B
% 如果大于 122，则跳过
GOYES CHECK

% 此时得到的是小写字符；减去 32，即可得到相应的大写字符

C=C-16 B
C=C-16 B
% 字符值减 32

DAT1=C B
% 把结果写回字符串

*CHECK
D=D-1 A
% 是否还有另一个字符？
GONC LOOP
% 如果有，则跳到 LOOP

% --- 从这里退出 ---

*END

GOSBVL =GETPTR

```

```
% =GETPTR 位于 067D2，用于恢复寄存器 D0、D1、B 和 D
GOVLNG =LOOP
% =LOOP 在 HP 48 上位于 2D564，在 HP 49 上位于 05149；它会返回 RPL
@
```

现在放入一个类似 “aBcDeFGhij” 的字符串并运行代码，将得到 “ABCDEFGHIJ”。这个代码对象只有 84 个半字节，很小。

如果得到的大小不同，代码可能有误。

作为练习，请编写一个作用相反的程序：把字符串中的所有字符转换为小写。

56.3 全局名称

前导码： 02E48

结束码： 无

大小： 7 半字节起

TYPE： 6

内存中的结构：

字段	长度
前导码 (02E48)	5 半字节
大小	2 半字节
字符 1	2 半字节
...	...
字符 n	2 半字节

每个字符用 ASCII 编码，每个字符占两个半字节。在堆栈上，全局名称存放在一对单引号 (' ') 中。

例如，'HP48' 的编码包括前导码 #02E48h、表示字符数量的 #04h（最多 255 个字符），最后是每个字符各两个半字节，即：

```
| #84E20 40 84054383
```

HP 默认不允许创建任意名称，那么何不创建一些“无效”的全局名称？

下面的程序以字符串为参数，并把全局名称放到堆栈上。字符串对象与全局名称对象之间的主要区别在于：全局名称的长度用两个半字节编码，因此最多只能包含 255 个字符；字符串的长度则用五个半字节编码。

注意： 以下程序不检查输入字符串的长度。请勿向它提供超过 255 个字符的字符串。

如果输入空字符串，将得到一个空全局名称；否则，程序会把一个内容与字符串相同的全局名称放到堆栈上。

```
C=DAT1 A
% 读取栈 1 的内容
R0=C A
% 保存到 R0，以便以后需要时使用
?C#0 A
% 如果栈 1 中有内容
GOYES TEST
% 检查它是否为字符串
LA 00201
% 否则产生错误“Too Few Arguments”
*ERROR
% 未提供字符串时跳到处
GOVLNG =Errjmp
% =Errjmp 位于 05023；它使用 A(A) 中的错误代码产生错误
*TEST
CD1EX
% 指向对象
A=DAT1 A
% 读取其前导码
D1=C
% 恢复 D1，使其指向堆栈的栈 1
LC 02A2C
% 02A2C 是字符串前导码
```

```

?A=C A
% 如果是字符串，则跳转
GOYES ALLOCATE
% 分配内存
LA 00202
% 否则使用错误 #202h
GOTO ERROR
% "Bad Argument Type"
*ALLOCATE
GOSBVL =SAVPTR
% =SAVPTR 位于 0679B，用于保存寄存器 D0、D1、B 和 D
C=R0 A
% C 中是栈 1 对象的地址
D1=C
% 让 D1 指向对象
D1=D1+ 5
% 指向字符串大小
C=DAT1 A
% A(A) 中为字符串大小
C=C-5 A
% 去掉大小字段本身的长度
CSRB A
% 再除以二；此时 C(A) 中是字符串的字符数
R1=C A
% 把字符数保存到 R1(A)
C=C+C A
% 乘以二，得到为了预留内存所需的半字节数
C=C+7 A
% 加 7，其中全局名称前导码占 5 半字节，长度字段占 2 半字节
GOSBVL =GETTEMP
% =GETTEMP 位于 039BE；它分配 C(A) 个半字节，如果 RAM 不足则产生“Insufficient Memory”错误。
它还会检查是否另有 5 个空闲半字节，从而保证能把预留对象压入堆栈
CD0EX
% D0 指向新预留的对象，此时 C 中为其地址
R2=C A
% 把地址保存到 R2(A)，因为最后将把 R2(A) 压入堆栈
D0=C
% D0 指向新预留的内存
LC 02E48
% 02E48 是全局名称前导码
DAT0=C A
% 写入前导码
D0=D0+ 5
% 再向前移动 5 个半字节
C=R1 B
% 恢复此前计算出的大小
DAT0=C B
% 把它写入全局名称的长度字段
C=C-1 B
% 减一；如果进位标志置位
GOC PUSH
% 则字符串为空，直接把空全局名称压入堆栈
A=R0 A
% 恢复字符串地址
D1=A
% 让 D1 指向字符串
D1=D1+ 8
% 向前移动 8 个半字节。为什么不是 10？因为循环内还有 D1=D1+ 2，最终恰好相当于 D1=D1+ 10

*LOOP
% 此循环每次复制一个字符

```

```

D1=D1+ 2
% 向前移动 2 个半字节；第一次进入循环时到达第一个字符，之后每次到达下一个字符
D0=D0+ 2
% 在新预留的对象中向前移动 2 个半字节
A=DAT1 B
% 从字符串读取一个字符
DAT0=A B
% 把该字符写入全局名称
C=C-1 B
% 计数器减 1
GONC LOOP
% 循环，直到进位标志置位

*PUSH
% 在这里把全局名称压入堆栈
GOSBVL =GETPTR
% =GETPTR 位于 067D2，用于恢复寄存器 D0、D1、B 和 D
D=D-1 A
% 从空闲 RAM 计数中减去 5 个半字节
D1=D1- 5
% 在堆栈中压入一个对象
C=R2 A
% 恢复全局名称地址
DAT1=C A
% 把它压入堆栈
A=DAT0 A
% 读取下一个要执行的 RPL 对象
D0=D0+ 5
% 更新 D0 指针，使其指向下一个对象
PC=(A)
% 跳转并继续程序流

@
% 源代码结束

```

请用空字符串试一试。在 `HOME` 中对得到的空全局名称执行 `EVAL`，将进入隐藏目录。下面介绍隐藏目录。

56.3.1 空全局名称与隐藏目录

您可以创建具有特殊功能的空全局名称：对它执行 `EVAL` 将进入 `HOME` 的隐藏目录。该目录默认包含三个对象：`Alarms`、`UserKeys` 和 `UserKeys.CRC`。第一个对象包含闹钟列表，第二个对象包含所有键盘赋值的列表，最后一个对象则是一个四半字节二进制数，即 `UserKeys` 的 `CRC`。有一条规则：永远不要改变该目录中变量的顺序。

当您位于 `HOME` 内的某个文件夹中时，可以用这个空全局名称隐藏变量列表中位于它后面的所有变量。例如，如果有四个变量，并在第二个变量之后存储一个空全局名称，那么最后两个变量将变得“不可见”，但仍可以提取或使用它们的值。

您可能希望亲自看看效果。

首先，只需按 `[Right-shift][']`，确保当前位于 `HOME`。然后在堆栈上输入 `#15777h` (HP 48) 或 `#272FEh` (HP 49)，务必包含末尾的 `h`，再输入 `SYSEVAL [ENTER]`。此时空全局名称已在堆栈上。只需执行 `EVAL` 即可进入其中。

不在 `HOME` 时，可以用空名称隐藏变量。存储在空全局名称之后的所有内容都会消失。

请不要忘记：如果重新排列不可见目录中的内容，将会发生内存丢失。这已经是第二次警告。

要退出不可见目录，在 HP 48 上执行 `[Left-shift][']` (`UPDIR`) 或 `[Right-shift][']` (`HOME`)；在 HP 49 上执行 `[Left-shift-held][']` (`UPDIR`)。

想隐藏某个目录的内容吗？进入所需目录，在 HP 48 上输入 `#15777h SYSEVAL [EVAL]`，或在 HP 49 上输入 `#272FEh SYSEVAL [EVAL]`，把空全局名称放到堆栈上；再把类似 1 的值放到堆栈上，并按 `[SWAP][STO]` 存储。现在便有了一个值为 1 的空全局名称。这个值本身并不重要，重要的是变量已不再可见。要再次看到它们，请重新把空全局名称放到堆栈上并对其执行 `PURGE`。

56.4 局部名称

前导码: 02E6D

结束码: 无

大小: 7 半字节起

TYPE: 7

内存中的结构:

字段	长度
前导码 (02E6D)	5 半字节
大小	5 半字节
字符 1	2 半字节
...	...
字符 n	2 半字节

可以看到, 前导码是局部名称与全局名称之间的唯一差别。但是, 局部名称不像全局名称那样始终可用。

56.5 用户二进制整数

二进制整数有两种: 一种大小固定为 15 半字节, 另一种大小不受限制 (当然仍受可用内存限制)。HP 只允许直接创建较小的二进制整数, 但可以手工创建较大的类型。

前导码: 02A4E

结束码: 无

大小: 11 半字节起

TYPE: 10

内存中的结构:

字段	长度
前导码 (02A4E)	5 半字节
大小	5 半字节
数字 1	1 半字节
...	...
数字 n	1 半字节

输入以 # 开头的数时, HP 会把它转换为当前使用的进制。如果当前使用 HEX 进制, 输入数字时不加最后的字符 (十六进制的 h), HP 就会把它视为十六进制数。如果在末尾加上字母 (十六进制为 h、十进制为 d、二进制为 b、八进制为 o), HP 会自动把它转换为当前使用的进制。

这里每个数字占一个半字节。

例如, 编码 #BAh:

如前表所示, 前导码为 #02A4Eh。大小为 #00015h, 因为 HP 不允许使用其他大小; 下面将说明如何绕过这一限制。数值为 #AB00000000000000h, 因此得到:

```
#E4A20 51000 AB00000000000000
```

下面的代码以一个二进制数作为输入, 运行后会把它具有指定数字位数的该二进制数放到堆栈上, 前提是 RAM 足够。甚至可以创建 200 位的二进制数。

以下程序创建一个 85 位二进制数:

```
GOSBVL =SAVPTR
% =SAVPTR 位于 0679B, 用于保存寄存器 D0、D1、B 和 D
LC 0005A
% 预留 90 个半字节
GOSBVL =GETTEMP
% =GETTEMP 位于 039BE, 用于分配内存
CD0EX
% 保存对象地址
R0=C A
```

```

% 保存到 R0 (A)
D0=C
% 恢复 D0
LC 02A4E
% 二进制数前导码
DAT0=C A
% 写入前导码
D0=D0+ 5
% 向前移动 5 个半字节
LC 00055
% 大小为 90-5=85, 即 #55h
DAT0=C A
% 写入对象的大小区域
GOSBVL =GETPTR
% =GETPTR 位于 067D2, 用于恢复寄存器 D0、D1、B 和 D
D=D-1 A
% 从空闲 RAM 中减去 5 个半字节
D1=D1- 5
% 把对象压入堆栈
C=R0 A
% 对象地址位于 R0 (A)
DAT1=C A
% 将它压入堆栈
A=DAT0 A
% 从这里按通常方式退出
D0=D0+ 5
PC=(A)
@

```

56.6 图形对象

前导码: 02B1E

结束码: 无

大小: 22 半字节起

TYPE: 11

内存中的结构:

字段	长度
前导码 (02B1E)	5 半字节
大小	5 半字节
行数	5 半字节
列数	5 半字节
...	像素

这是一种非常常用的对象。图片或 GROB (GROb Object, 图形对象) 包含像素, 更准确地说, 是包含表示像素的数据。图形对象首先包含前导码 #02B1Eh, 随后是整个对象的大小, 但大小不计前导码的五个半字节。再后面的两个五半字节字段编码行数和列数。

像素数据随后出现, 按从左上角到右下角的顺序依次编码, 并以字节为单位组合在一起。GROB 中的数据使用字节而不是半字节编码, 因此每个字节编码八位, 也就构成 GROB 的八个像素。值为 1 的位表示点亮像素, 值为 0 的位表示熄灭像素。“pixel”一词是“picture element”的缩写。

重要: 在每个字节中, 像素按反向半字节编码。不要忘记, 图片必须以完整字节编码, 也就是半字节数量必须是 2 的倍数。因此, 每个 GROB 的像素数据必须包含偶数个十六进制数字。每个半字节编码四个像素, 所以每个 GROB 的编码宽度必须以八个像素为单位补齐。

如果 GROB 的尺寸不是八个像素的倍数, 也不必担心。只需用零“填充”数据, 也就是在末尾添加足够数量的零, 使其达到八的倍数。甚至可以用少量非零值来填充 GROB, 从而在对象中隐藏少量数据。

例如, 假设希望在同一行依次得到一个熄灭像素、一个点亮像素、一个熄灭像素、一个点亮像素、一个熄灭像素, 随后三个点亮像素。以 1 表示点亮像素、0 表示空白像素, 并添加空格以便阅读, 屏幕数据如下:

| 0101 0111

这里有八个像素，也就是两个半字节，因此无需担心添加尾随零。编码时必须颠倒每个半字节的内容，所以像素应编码为 1010 1110，即 #AEh。在开头依次加入前导码 #02B1Eh、大小 #19d（即 #13h 个半字节）、高度 #1h 和宽度 #4h 后，图形编码如下：

| #E1B20 31000 10000 40000 AE00

请注意，末尾添加了两个零，而且 GROB 的大小也随之调整。这样做只是为了让长度成为 #13h 而不是 #11h，以便提醒您半字节必须反向。此外，这也说明图像内可以包含额外数据而不会造成问题。当然，如果去掉这两个零，GROB 会少占一个字节的内存。

Meta Kernel 和 HP 49 用户请注意：Meta Kernel 和 HP 49 会在图形末尾添加一个值为零的字节，表示它是灰度 GROB，因此在堆栈上显示时，该 GROB 的大小末尾会带有一个 g。只需去掉这两个零并修正长度，g 就会消失。

作为复习，上例中的大小 #13h（#19d）是这样计算的：大小字段占 5 半字节，宽度字段占 5 半字节，高度字段占 5 半字节，本例的像素数据占 4 半字节。

56.7 XLIB 名称

前导码：02E92

结束码：无

大小：11 半字节

TYPE：14

内存中的结构：

字段	长度
前导码（02E92）	5 半字节
库 ID	3 半字节
命令编号	3 半字节

HP 48 和 HP 49 使用这种对象引用库中的命令。虽然您看到的可能是库命令名称，但计算器内部使用的是该对象。例如，当堆栈上出现 DTAG 时，它实际上是一个 XLIB 对象；计算器在屏幕上显示时把它转换成命令名称。该对象包含库编号和命令编号。当命令具有“用户”形式，也就是可键入的名称时，屏幕会显示该名称；否则，堆栈上会显示 XLIB ...。

可以用命令 LIBEVAL 间接使用 XLIB。LIBEVAL 接受一个六位二进制值 #... 作为参数；前三位是库编号，后三位是命令编号，二者都采用十六进制。

56.8 系统二进制整数

前导码：02911

结束码：无

大小：10 半字节

TYPE：17

内存中的结构：

字段	长度
前导码（02911）	5 半字节
数值	5 半字节

这是一个非常简单的对象。HP 的 RPL 操作系统大量使用它；其中包含一个用五个半字节，也就是 20 位编码的二进制整数。该数值会按当前启用的进制显示，但在内存中始终采用十六进制编码。

56.9 长实数

长实数也称为“扩展实数”。

前导码：02955

结束码：无

大小：26 半字节

TYPE：21

内存中的结构：

字段	长度
前导码（02955）	5 半字节
指数	5 半字节

字段	长度
尾数	15 半字节
符号	1 半字节

长实数比标准实数更精确。用户无法直接访问这种数，但 HP 在内部计算时会用它取得更高精度。它的编码方式与标准实数相似，只是某些字段更长。与实数对象一样，这里也使用 BCD。详细信息请参阅标准实数一节。

56.10 字符

前导码：029BF

结束码：无

大小：7 半字节

TYPE：24

内存中的结构：

字段	长度
前导码（029BF）	5 半字节
字符的 ASCII 代码	2 半字节

字符是 HP 上最简单的对象之一。对于这种对象，前导码实际上比所编码的数据还大。每个对象都包含一个 HP 能够显示的字符。字符 0 至 128 是标准 ASCII 字符，HP 在更高的编号中还定义了一些特殊字符。这里每个字符占两个半字节，因此能够编码扩展 8 位 ASCII 表的全部 256 个字符。由于原始 ASCII 表只有 7 位，只有前 128 个字符是“通用”的。

56.11 代码对象

前导码：02DCC

结束码：无

大小：10 半字节起

TYPE：25

内存中的结构：

字段	长度
前导码（02DCC）	5 半字节
大小	5 半字节
Saturn 机器代码	不定

这是 HP-ASM、ASM Flash、MASD、Jazz 以及其他代码编译器生成的对象类型。

由于它与字符串具有相同结构，有些人会用代码对象隐藏字符串或其他对象。只需更改前导码即可；这很简单，现在您已经知道怎么做了。

56.12 库数据

前导码：02B88

结束码：无

大小：10 半字节起

TYPE：26

内存中的结构：

字段	长度
前导码（02B88）	5 半字节
大小	5 半字节

字段	长度
数据	不定

HP 向程序员提供这种对象，使需要保存数值的库能够把信息保存在独立对象中。程序员可以自行决定在该对象中放入哪些数据以及采用何种方式，因此它是一种方便的信息存储方法，可视为一种 HP Cookie。

一种常见用途是存储供其他程序调用的代码片段。由于这些代码片段不会检查堆栈，这种方法可以防止用户“意外”运行这些代码对象。您应该能够轻松编写一个用于启动此类对象的程序。其目的就是保护数据。

56.13 扩展指针

前导码： 02BAA

结束码： 无

大小： 15 半字节

TYPE： 27

内存中的结构：

字段	长度
前导码（02BAA）	5 半字节
对象地址	5 半字节
程序地址	5 半字节

下一部分将说明，某些内存区域会被其他区域覆盖，但有时仍需要指向被覆盖的内容。HP 在内部使用这种对象实现该目的。ROM 中有一些例程用于“显露”所需区域，稍后将介绍这些例程。在这里，对象就是希望访问的内容，程序则是与对象地址关联的 ROM 例程；它会按需取消配置和重新配置内存模块。

56.14 无限精度整数（仅 HP 49）

前导码： 02614

结束码： 无

大小： 11 半字节起

TYPE： 28

内存中的结构：

字段	长度
前导码（02614）	5 半字节
大小	5 半字节
数字 1	1 半字节
...	...
数字 n	1 半字节
符号（0 除外）	1 半字节

无限精度整数也称为 ZINT。其主体大小等于用于编码大小本身的 5 个半字节，加上每个数字 1 个半字节，再加上 1 个符号半字节。数值 0 没有符号半字节。与其他所有对象一样，计算大小时不包含前导码。

每个数字用 1 个半字节编码。大小包括除前导码以外的所有内容，因此总长度为 5 个半字节加 n 个数字半字节，再加 1 个符号半字节，即 $6+n$ 。所以数值 0 的长度为 6 个半字节。

正数的符号为 0，负数的符号为 9。

56.15 闪存指针（仅 HP 49）

前导码：026AC

结束码：无

大小：12 半字节

TYPE：27

内存中的结构：

字段	长度
前导码（026AC）	5 半字节
指向表 A 的指针	3 半字节
指向表 B 的指针	4 半字节

该对象用作内存映射工具，使 HP 的开发人员能够生成新的 ROM 版本，同时保持各例程的入口点不变。第一个指针长 3 个半字节，指向系统 RAM 中地址 #86037h 处的一个表，以下称为表 A。表 A 中的每个值长 5 个半字节，因此先把第一个指针乘以 5，得到表 A 内的地址。系统使用该地址处的值配置闪存，以运行相应例程；可以把这个值视为闪存中的存储体编号。

第二个指针长 4 个半字节，指向地址 #40222h 处的另一个表，以下称为表 B。表 B 中的每个值也长 5 个半字节，但这些值是闪存指针所引用的具体例程地址。可以把这个值视为命令编号。

56.16 小字体（仅 HP 49）

前导码：026FE

结束码：无

大小：1548 半字节

TYPE：27

内存中的结构：

字段	长度
前导码（026FE）	5 半字节
大小	5 半字节
小字体 ID 编号	2 半字节
小字体图形	1536 半字节

由于内置约束固定，小字体对象相对简单。前导码之后是大小，其值始终为 #607h 个半字节，即 #1543d，因为小字体对象中没有任何可变长度内容。下一个数是两半字节的小字体 ID 编号，必须在 #0h 至 #FFh（#255d）之间。

最后是字体图形本身。每个字符分别存储为高 6 像素、宽 4 像素的图形。每个图形中的像素存储方式与前述 GROB 相同，占 6 个半字节。共有 256 个字符，因此小字体的整个图形部分占 1536 个半字节。

56.17 字体（仅 HP 49）

前导码：02BCC

结束码：无

大小：4130 半字节起

TYPE：30

内存中的结构：

字段	长度
前导码（02BCC）	5 半字节
大小	5 半字节

字段	长度
字体高度（像素）	2 半字节
字体 ID 编号	2 半字节
字体名称大小	2 半字节
字体名称	长度不定
字体名称大小	2 半字节
字体图形	4096 半字节

字体对象比小字体对象稍复杂。前导码和大小之后是字体高度，它用两个半字节存储。该值应在 6 至 8 像素之间。下一个数是字体 ID 编号。系统中的每种字体都必须具有唯一 ID 才能访问，该 ID 必须在 #0h 至 #FFh (#255d) 之间。

接下来是字体名称的大小。它也占两个半字节，而且至少应为 8，因为 HP 49 要求名称至少有 8 个字符，才能正确显示字体。大小之后是名称本身；与其他内容一样，字符必须反向排列，每个字符占两个半字节。奇怪的是，名称后面还会再次出现名称大小，并再次占用两个半字节。

最后是字体图形本身。无论字体大小如何，每个字符都分别存储为高 8 像素、宽 8 像素的图形。每个图形中的像素存储方式与前述 GROB 相同，占 16 个半字节。共有 256 个字符，因此字体的整个图形部分占 4096 个半字节。

57 复合对象

57.1 复数

前导码：02977

结束码：无

大小：37 半字节

TYPE：1

内存中的结构：

字段	长度
前导码 (02977)	5 半字节
实部指数	3 半字节
实部尾数	12 半字节
实部符号	1 半字节
虚部指数	3 半字节
虚部尾数	12 半字节
虚部符号	1 半字节

这就是复数对象。实际上，表示复数只需把两个实数放在一起。第一个数是复数的实部，第二个数是虚部，每个数都用 16 个半字节编码。

例如，编码 (100,1000)。

首先是前导码 #77920h，仍按惯例反向存放；随后是实部指数 #200h，再后是实部尾数 #000000000001，然后是符号 #0h。接下来是虚部，其指数编码为 #300h，尾数为 #000000000001h，最后符号为 #0h。因此，该复数在内存中的存储形式如下：

```
| #77920 200 000000000001 0 300 000000000001 0
```

57.2 数组

前导码：029E8

结束码：无

大小：27 半字节起

TYPE：3/4

内存中的结构:	
字段	长度
前导码 (029E8)	5 半字节
大小	5 半字节
对象类型	5 半字节
维数	5 半字节
维度 1	5 半字节
...	...
维度 n	5 半字节
对象 1 的内容	长度不定
...	...
最后一个对象的内容	长度不定

数组对象可以包含任意数量的同类型对象。只有一个维度时称为“向量”；有两个维度时称为“矩阵”。数组也可以超过两个维度，因为有五个半字节可用于编码数组的维度。

数组通常用于把一组实数或复数保存在一起。您可以使用任意数量的维度存储任意类型的对象，但 HP 只能管理一维或二维数组。

前导码是 #029E8h。在对象内部，必须先定义数组中要保存哪种对象。然后必须给出整个数组的大小；该大小取决于所收集的对象类型，当然也取决于要编码的对象数量。

您将看到，在对象中编码长度很有用：HP 可以利用它“跳过”对象。大多数对象内部都保存自身大小，但有些对象没有，因此“结束码”对于寻找这些对象的末尾非常有用。

这里有趣的一点是，即使收集十个实数，数组中也不会存储十个实数前导码：前导码只存一次。这就是对象类型使用五个半字节编码的原因，也是一个数组只能保存一种对象类型的原因。确定数组内部的对象类型后，再用五个半字节编码维数：向量为 #00001h，矩阵为 #00002h。维数之后，必须说明每个维度中有多少对象。对于矩阵，要编码行数和列数。

完成这些信息后，就可以开始编码数组中的对象。数值的存储方式随对象类型而异。因为已经记录了对象类型，所以无需再存储各对象的前导码。

需要注意的是，无论数组大小和对象类型如何，即使某些部分“为空”，仍会占用内存。

例如，创建一个包含若干名称的字符串数组：

```
[ [ HP48S HP48G ]
```

首先是数组前导码 #029E8h，像往常一样，它在内存中反向编码。总大小为 #00032h，稍后会说明原因；该值占五个半字节。接下来必须说明其中包含哪种对象，也就是字符串，因此使用 #02A2Ch（五个半字节）。数组只有一个维度，所以必须写入 #00001h（又五个半字节）；其中有两个元素，所以必须写入 #00002h（再五个半字节）。

最后编码两个字符串。每个字符占两个半字节，因此每个字符串的字符数据占十个半字节，前面还要加五个半字节表示长度。包括长度字段在内，每个字符串占 15 个半字节，所以长度为 #0000Fh，反向存放后显示为 #F0000h。因此，编码该对象共需 50 个半字节，另加前导码；这也解释了长度为何是 #32h，因为 #50d = #32h。编码如下：

```
[ #8E920 23000 C2A20 10000 20000 F00008405438335 F00008405438374
```

HP 的堆栈不会显示字符串数组的内容，只会显示 `Array of String`。不过，如果正在运行 Meta Kernel，它会显示实际数组。

57.3 列表

前导码: 02A74
结束码: 0312B
大小: 10 半字节起
TYPE: 5

内存中的结构：

字段	长度
前导码（02A74）	5 半字节
对象 1	长度不定
...	...
最后一个对象	长度不定
结束码（0312B）	5 半字节

列表对象很有意思：它可以包含任意类型的对象，数量只受 RAM 容量限制。实现方法是用前导码标记开头、用结束码标记末尾，而每个对象连同其自身前导码都位于二者之间。ROM 中经常能见到这种结构。

空列表写作 { }，长度为 10 个半字节：它具有前导码和结束码，但二者之间没有内容。

例如，编码列表 { 10.2 "dX" }。

这里选用了前文已经介绍过的两种对象：实数和字符串。方法相当简单：先分别编码每个对象，再把二者放在列表的前导码和结束码之间：

```
| 10.2 编码为 #33920 100 000000000201 0
| "dX" 编码为 #C2A20 90000 4685
```

因此，列表编码为：

```
| #47A20 3392010000000000002010 C2A20900004685 B2130
```

57.4 RPL 程序

前导码：02D9D

结束码：无

大小：20 半字节起

TYPE：7

内存中的结构：

字段	长度
前导码（02D9D）	5 半字节
<< User RPL 开始（48：2361E；49：389B9）	5 半字节
...	...
>> User RPL 结束（48：23639；49：389D4）	5 半字节
结束码（0312B）	5 半字节

创建 RPL 程序时，得到的就是这种对象。仔细观察会发现，它与列表相似，也包含前导码和结束码。在 RPL 程序内部，对象一个接一个地排列，但每一项可能只是对象地址，而不是对象本身。例如，假设创建以下 User RPL 程序：

```
| << OFF >>
```

在该 RPL 对象中并不会保存让 HP 关机的代码，而只保存一个指向代码的地址。OFF 代码位于该地址。当 HP 显示 RPL 程序时，它会把每个地址解码成相应名称，如果该地址具有名称的话。

User RPL 程序 << OFF >> 在内存中的编码如下：

HP 48	HP 49	System RPL
D9D20	D9D20	::
E1632	9B983	x<<
E13A1	C0593	xOFF
93632	4D983	x>>
B2130	B2130	;

这里的半字节会反向存放，Saturn 在处理内存时总是如此。最前和最后的五个半字节属于 RPL 对象：#02D9Dh 是前导码，#0312Bh 是结束码。

在对象本身内部有 <<, 其地址在 HP 48 上为 #2361Eh, 在 HP 49 上为 #389B9h; 还有 >>, 其地址在 HP 48 上为 #23639h, 在 HP 49 上为 #389D4h。两个定界符之间是一个地址: HP 48 上为 #1A31Eh, HP 49 上为 #3950Ch, 它就是 OFF 命令。

可以验证这一点: 在 HP 48 上输入 #1A31Eh SYSEVAL [ENTER], 或在 HP 49 上输入 #3950Ch SYSEVAL [ENTER], HP 就会关机。

每次在 RPL 程序中放入命令时, HP 实际放入的都是该命令的地址。

这里的 << 和 >> 在 RPL 对象内部建立了一个块。这些“块”甚至可以相互嵌套。

编写 RPL 程序时, << 和 >> 本身并非真正必需, 但前导码和结束码不可缺少, 否则 RPL 程序无效, 也就无法执行。即使输入 << << OFF >> >>, HP 也会采用不同编码: 其中仍有 << 和 >>, 但为了确保内部的 << OFF >> 不会执行, 系统会在其前后分别插入一些内容。

本书并不是 System RPL 教程, 因此不再讨论这些内容。如果希望进一步了解 RPL 的奥秘, 有许多优秀教程可供参考。推荐 James Donnelly 的 *An Introduction to HP 48 System RPL and Assembly Language Programming* (ISBN: 1-879828-04-9, Armstrong Publishing Company 出版)、Eduardo Kalinowski 的 *Programming in System RPL*, 以及 Hewlett Packard 的 *RPLMAN.DOC*。可以按 ISBN 搜索并购买第一本书, 后两份资料的 PDF 可从以下位置下载:

```
| http://www.hpcalc.org/details.php?id=5142
| → http://www.hpcalc.org/hp49/docs/programming/progsysrpl_pdf.zip
| http://www.hpcalc.org/details.php?id=1745
| → http://www.hpcalc.org/hp48/docs/programming/rpl-pdf.zip
```

57.5 代数表达式

这种对象类型也称为“符号对象”。

前导码: 02AB8

结束码: 0312B

大小: 10 半字节起

TYPE: 9

内存中的结构:

字段	长度
前导码 (02AB8)	5 半字节
对象 1	长度不定
...	...
对象 n	长度不定
结束码 (0312B)	5 半字节

这是 RPL 中通常使用的代数表达式。它同时包含前导码和结束码。对象内容采用 RPL 表示法编码; 虽然它在堆栈上看起来不像 RPL, 但实际上就是 RPL。您已经知道, 由于它按 RPL 对象编码, 该对象内部没有 + 号或其他所需命令的字符; 每个数学符号都使用其 ROM 地址编码, 地址长度为五个半字节。

例如, 编码 'A+2':

如上表所示, 前导码是 021B8。接下来必须编码全局名称 A: 前导码是 02E48, 大小是 01 (使用两个半字节), 字符 A 的 ASCII 代码是 41。如果对此感到困惑, 请参阅全局对象编码一节。之后是 + 命令, 使用其 ROM 地址编码, 在 HP 48 上为 #2A2DEh, 在 HP 49 上为 #2F961h; 再后面是数值 2, 使用其 ROM 地址编码, 在 HP 48 上为 #1AB67h, 在 HP 49 上为 #39B58h。最后是上表中的结束码 #0312Bh。因此, 完整对象在 ROM 中反向存放后为:

HP 48:

```
| #8BA20 84E201014 ED2A2 76BA1 B2130
```

HP 49:

```
| #8BA20 84E201014 169F2 85B93 B2130
```

57.6 标记对象

前导码: 02AFC

结束码: 无

大小: 不定

TYPE: 12

内存中的结构:

字段	长度
前导码 (02AFC)	5 半字节
标签大小	2 半字节
标签第一个字符	2 半字节
...	...
标签第 n 个字符	2 半字节

字段	长度
对象	不定

标记对象就是带有标签的对象。先创建一个最多 256 个字符的标签，然后紧接着放置对象，整体封装在“标记”对象类型之中。

创建标记对象时，只需先放入标记前导码，然后放入用于标记对象的字符串大小，再放入标签字符，最后按通常方式编码被标记的对象。标签字符串的大小用两个半字节编码，因此最多为 256 个字符。

例如，用 “Ten” 标记实数 10，编码如下：

```
| #CFA20 30 4556E6 33920 100 000000000001 0
```

可以看到，开头是标记前导码 #02AFCh，随后是标签所用的字符数 #03h，当然需要反向存放；再后面编码组成标签的字符 ASCII 值，Ten 先变为 #6E6554h，然后再反向存放。最后按前文所述编码实数 10。

57.7 单位对象

前导码: 02ADA

结束码: 0312B

大小: 不定

TYPE: 13

内存中的结构:

字段	长度
前导码 (02ADA)	5 半字节
对象	不定
单位	不定
结束码 (0312B)	5 半字节

这种对象由前导码和结束码界定，二者之间有两个值。第一个值是要附加单位的对象，例如 '2 m^2' 中的实数 2。第二个值是用于编码单位的对象。+ 或 * 等符号使用其 ROM 地址编码，各种单位字母则用字符编码。

例如，编码每秒 300 000 000 米。该速度略高于光速；根据 HP 的常数库，光速约为每秒 299 792 458 米。

首先放入前导码 #02ADAh。然后放入对象，即实数 300 000 000，其编码为：

```
| #33920 800 000000000003 0
```

最后编码单位。这一点很重要：必须使用 RPN 表示法。您知道 HP 48 和 HP 49 是基于 RPL 的计算器，因此这并不意外。

使用 RPN 后，单位“米每秒”表示为：

```
| m s /
```

这里有两个字符 m 和 s。它们是两个字符串，根据前面关于字符串编码的知识，分别为：

```
| #C2A20 70000 D6
```

```
| #C2A20 70000 37
```

接下来是除号 /，用于把 m 除以 s；之后是乘号 *，用于把数值乘以单位。

在末尾插入结束码后，最终得到：

```
#ADA20（前导码）
#339200800000000000030（数值 300000000）
#C2A2C70000D6（m）
#C2A2C7000037（s）
#86B01（48）或 #957D2（49）（/）
#68B01（48）或 #777D2（49）（*）
#B2130（结束码）
```

57.8 目录对象

目录有两类：[HOME](#) 目录和其他所有目录。[HOME](#) 通常称为“根目录”。

库会“附加”到目录。[HOME](#) 可以附加多个库，但每个子目录只能附加一个库。下面分两种情况说明：一种是 [HOME](#) 目录，另一种是子目录。这是一种非常复杂的对象，请仔细阅读。

57.8.1 HOME 目录

前导码：[02A96](#)

结束码：无

大小：57 半字节起

TYPE：15

内存中的结构：

字段	长度
前导码（02A96）	5 半字节
已附加库的数量	3 半字节
第一个已附加库的编号	3 半字节
第一个库的哈希表地址	5 半字节
第一个库的消息表地址	5 半字节
...	...
最后一个已附加库的编号	3 半字节
最后一个库的哈希表地址	5 半字节
最后一个库的消息表地址	5 半字节
最后一个对象的偏移量	5 半字节
00000	5 个空半字节
对象 1 名称的字符数	2 半字节
对象 1 名称的字符 1	2 半字节
...	...
对象 1 名称的字符 n	2 半字节

字段	长度
对象 1 名称的字符数	2 半字节
对象 1	不定
对象 1 区域的大小	5 半字节
...	...
最后一个对象名称的字符数	2 半字节
最后一个对象名称的字符 1	2 半字节
...	...
最后一个对象名称的字符 n	2 半字节
最后一个对象名称的字符数	2 半字节

字段	长度
最后一个对象	不定

首先是 `HOME` 前导码。其后是一个三半字节值，表示当前附加到 `HOME` 的库数量，也就是 HP 内的库。即使没有自行安装任何库，HP 48 和 HP 49 的 ROM 中仍内置了附加到 `HOME` 的库。三个众所周知的库分别是：编号 2，包含 `ASR`、`RL`、`RLB` 等；编号 171，包含 `XVOL`、`YVOL`、`VERSION` 等；编号 1792，包含 `IF`、`THEN`、`ELSE` 等。即使 HP 看起来完全为空，这三个库也始终处于附加状态。

接下来，每个已附加库占 13 个半字节。前三个半字节是库编号，随后五个半字节是库哈希表的地址，后文会说明它，但结构相当复杂；最后五个半字节是库消息表的地址。

哈希表用于快速访问命令；如果消息表存在，则用于查找库中的自定义错误消息。这并不是最好的优化方法，但仍很有意思。

随后是一个五半字节地址，它是指向目录内最后一个对象的偏移量。这种设计非常有用，也非常巧妙。

然后，目录内的对象一个接一个地编码。对于每个对象，先列出名称长度；由于全局名称的长度用两个半字节编码，这里也使用两个半字节。随后给出名称的各个字符。如果您熟悉 Pascal 编程，会发现这种变量命名约定与 Pascal 相似。Pascal 语言先编码大小，再编码数据；C 语言则先编码数据，末尾跟一个空值，而没有大小字段。在这里，预先保存长度非常有用。

对象名称的最后一个字符之后是对象本身；对象之后的五个半字节给出对象 1 所占的整个空间，从编码名称长度的两个半字节开始，一直到对象末尾。

每个对象的结构都相同：先是名称长度，再是组成名称的字符，然后是对象，最后是整个结构所占的空间。

注意 1： 如果某个库的哈希表或消息表位于无法直接访问的被覆盖内存中，这里会出现扩展指针。扩展指针对象和被覆盖内存稍后介绍。如果某个库没有消息表，则会看到 `#00000h`。

注意 2： 以下编号可用作库 ID：

编号范围	Hewlett-Packard 建议的用途
<code>#000h</code> 至 <code>#100h</code>	ROM 中的 HP 库（保留）
<code>#101h</code> 至 <code>#200h</code>	RAM 中的 HP 库（保留）
<code>#201h</code> 至 <code>#300h</code>	非 HP 库（保留）
<code>#301h</code> 至 <code>#6FFh</code>	用户库（可用）
<code>#700h</code> 至 <code>#7FFh</code>	HP 自用（保留）

可以看到，HP 上附加的三个库 { `#700h` `#002h` `#0ABh` } 中，第一个位于 HP 自用的区域，可能用于应用卡；另两个位于为 ROM 常驻库保留的区域。

57.8.2 子目录

内存中的结构：

字段	长度
前导码 (02A96)	5 半字节
附加到目录的库编号	3 半字节
最后一个对象的偏移量	5 半字节
00000	5 个空半字节
对象 1 名称的字符数	2 半字节
对象 1 名称的字符 1	2 半字节
...	...
对象 1 名称的字符 n	2 半字节
对象 1 名称的字符数	2 半字节
对象 1	不定
对象 1 区域的大小	5 半字节
...	...
最后一个对象名称的字符数	2 半字节

字段	长度
最后一个对象名称的字符 1	2 半字节
...	...
最后一个对象名称的字符 n	2 半字节
最后一个对象名称的字符数	2 半字节
最后一个对象	不定
最后一个对象区域的大小	5 半字节

可以看到，一个子目录只能附加一个库。如果没有附加库，则使用 `#7FFh` 作为已附加库编号。

注意： 目录对象内部保存一个偏移量，因而能够快速、轻松地找到目录中的最后一个对象。第一个对象之前有五个空半字节；每个对象之后还会有一个五半字节值，用于给出该对象占用的总长度，不过最后一个对象没有该信息。还可以找到各对象编码名称的字符数。

57.9 库

注意！ 这是一种非常复杂的对象。

前导码： `02B40`

结束码： 无

大小： 不定

TYPE： 16

内存中的结构：

字段	长度
前导码 (02B40)	5 半字节
大小 (不含 CRC)	5 半字节
库名称大小	2 半字节
库名称的第一个字符	5 半字节
...	...
库名称的最后一个字符	2 半字节
库名称大小	2 半字节
库编号	3 半字节
哈希表偏移量	5 半字节
消息表偏移量	5 半字节
链接表偏移量	5 半字节
Config 对象偏移量	5 半字节
消息表数组	不定
链接表	不定
XLIB 1 (类型)	1 或 3 半字节
库编号	3 半字节
命令编号	3 半字节
XLIB 1 对象	不定
XLIB 2 (类型)	1 或 3 半字节
库编号	3 半字节
命令编号	3 半字节
XLIB 2 对象	不定
...	...
XLIB n (类型)	1 或 3 半字节

字段	长度
库编号	3 半字节

字段	长度
命令编号	3 半字节
XLIB n 对象	不定
隐藏对象 1	不定
...	...
最后一个隐藏对象	不定
Config 对象	不定
CRC	4 半字节

库的结构不仅相当复杂，而且还会变化。库对象内部有四种结构非常特殊的对象：哈希表、消息表、链接表和 Config 对象。

如前所示，库内的四个偏移量正是用于这些对象。

如果库没有名称，第一个长度字段为零，也不会有第二个长度字段。如果有名称，则依次出现名称长度、名称字符，然后再次出现名称长度。没有名称的库不会在执行 `[Right-shift] [2]` 时显示。

哈希表用于提高访问库命令的速度。消息表包含库的所有自定义错误消息，链接表则引用库中的所有对象。库内部使用偏移量：如果偏移量为 `#00000h`，说明要查找的对象不存在。

下面将讨论这四种对象类型。不过，库还有一项重要之处：其中有两类对象，一类是可见对象，称为 XLIB；另一类是隐藏对象。对于库中的每个可见对象，都可以找到：

- 76. 类型，用 1 至 3 个半字节编码；
- 77. 库编号，占 3 个半字节；
- 78. 命令编号，占 3 个半字节。

CRC 由 HP 内部的硬件 CRC 电路计算。对象的 CRC 从大小字段开始计算到对象末尾，不包含前导码。CRC 自身不属于参与 CRC 计算的对象内容。计算完毕后，再把 CRC 添加到库中。库的计算大小既不包含前导码的半字节，也不包含 CRC 的半字节。

57.9.1 哈希表

该表用于加快访问库命令的速度。它非常简单，因此性能提升并不惊人，但这个思路很好。命令按名称长度分组，长度从 1 到 16。命令名称可以超过 16 个字符，但这种命令会与 16 字符长的命令归为一组。

哈希表的内存结构如下：

前导码： `02A4E`

结束码： 无

大小： 不定

TYPE： 无

类别 1：名称长度为 1 个字符的命令

...

类别 n：名称长度为 n 个字符的命令

内存中的结构：

字段	长度
前导码 (02A4E)	5 半字节
哈希表大小	5 半字节

字段	长度
第一个类别的偏移量	5 半字节
...	...
最后一个类别的偏移量	5 半字节

字段	长度
名称列表大小	2 半字节
单字符命令名称的大小	2 半字节
第一个单字符命令	2 半字节
命令编号	2 半字节
第二个单字符命令	2 半字节
命令编号	2 半字节
...	...
第 n 个单字符命令	2 半字节
双字符命令名称的大小	2 半字节
第一个双字符命令	4 半字节
命令编号	2 半字节
...	...
命令 1 的偏移量	5 半字节
...	...
最后一个命令的偏移量	5 半字节

没有可见名称的命令不会列入哈希表。每个库命令在库中都有名称和编号，这些信息会在哈希表中排序。命令编号紧跟在名称之后：第一个单字符命令后面就是它的编号，后续依此类推。首先按名称长度排序，然后在每个名称之后放置命令在库中的编号，但只包括可见命令。接下来是哈希表偏移量，它有两类：一类根据命令名称长度查找命令，另一类使用哈希表末尾的偏移量查找命令。

57.9.2 消息表

消息表有两种形式：可以是一维数组，也可以是带索引的一维数组。

如果遇到数组，前导码为 **029E8**，其中先给出库内消息数量，随后是所有消息。每条消息开头都有自身长度，采用 Pascal 字符串编码，之后是每个字符的 ASCII 字节。

如果遇到索引数组，前导码为 **02A0A**，其中先给出消息数量，再列出指向各消息的偏移量。每条消息前同样有长度，仍采用 Pascal 字符串编码，随后每个字符占一个 ASCII 字节。

57.9.3 链接表

链接表就是一个简单的二进制整数，其中编码了库内各对象的地址。可以把这个二进制整数划分成每块五个半字节。第一块是前导码 **02A4E**，随后是长度；与往常一样，该长度不包括前导码本身。

此后每五个半字节就是一个对象地址。因此，这里只是一个地址表，库中的每个对象都有一个对应地址。

57.10 备份

前导码： **02B62**

结束码： 无

大小： 不定

TYPE： 17

内存中的结构：

字段	长度
前导码（02B62）	5 半字节
大小	5 半字节
对象名称的字符数	2 半字节
名称的第一个字符	2 半字节
...	...
名称的最后一个字符	2 半字节
对象	不定

字段	长度
系统二进制整数前导码 (02911)	5 半字节
0	1 半字节
CRC	4 半字节

备份对象内部可以放置多个对象。HP 自身只会放入一个对象，并在其后附加一个隐藏的系统二进制整数；该系统二进制整数的第一个半字节为 0，另外四个半字节包含对象的 CRC。CRC 用于确认对象未被损坏。这里所用系统二进制整数的五个半字节中，第一个始终为零，因为 CRC 只用四个半字节编码。

57.11 长复数

前导码：0299D

结束码：无

大小：47 半字节

TYPE：22

内存中的结构：

字段	长度
前导码 (0299D)	5 半字节
实部指数	5 半字节
实部尾数	15 半字节
实部符号	1 半字节
虚部指数	5 半字节
虚部尾数	15 半字节
虚部符号	1 半字节

长复数相当于两个扩展实数组合在一起，第一个是实部，第二个是虚部。HP 在内部使用这种对象来获得更高精度。

57.12 链接数组

前导码：02A0A

结束码：无

大小：32 半字节起

TYPE：23

内存中的结构：

字段	长度
前导码 (02A0A)	5 半字节
大小	5 半字节
对象大小	5 半字节
维数	5 半字节
维度 1	5 半字节
...	...
维度 n	5 半字节
指向第一个对象的指针	5 半字节
...	...
指向最后一个对象的指针	5 半字节
第一个对象的内容	长度不定
...	...
最后一个对象的内容	长度不定

链接数组是一种特殊数组。链接数组内部包含若干指针，每个指针都是通往相应编码对象的捷径。如果大型数组中包含大量相同值，这类数组可以节省许多内存空间。例如，指针长 5 个半字节，实数长 16 个半字节；如果一个 100x100 数组中只有五种不同的实数值，使用链接数组就能节省大量空间。

57.13 无限精度实数（仅 HP 49）

前导码：0263A

结束码：无

大小：17 半字节起

TYPE：27

内存中的结构：

字段	长度
前导码（0263A）	5 半字节
尾数大小	5 半字节
尾数数字 1	1 半字节
...	...
尾数数字 n	1 半字节
符号（0 除外）	1 半字节
指数大小	5 半字节
指数数字 1	1 半字节
...	...
指数数字 n	1 半字节

字段	长度
符号（0 除外）	1 半字节

无限精度实数就是前导码后跟两个无限精度整数，用于按科学记数法表示实数。第一个整数包含实数的全部数字，构成尾数；第二个整数是指数，用于确定小数点的位置。这两个整数中的数字反向存放，因此最低有效位在前，最高有效位在后。

HP 49 ROM 中的任何软件都不使用这种对象，也不能通过在命令行中键入来生成它，但堆栈能够显示它。要使无限精度实数真正有用，必须安装第三方软件，例如 Gjermund Skailand 和 Thomas Rast 编写的 LongFloat 库。

由于这种格式允许用无限多种方式表示同一个数，例如 1E-2 与 10E-3 相同，LongFloat 库生成实数时会通过用户变量 DIGITS 决定尾数所用的数字数量，但这并不是格式本身的强制要求。另一个有趣的细节是：LongFloat 库表示整数 0 时，指数使用 -DIGITS，也就是 DIGITS 值的相反数；尾数则使用 DIGITS 个零，而且符号为正。

LongFloat 文档提供了更多详细信息。可从以下位置下载 LongFloat：

| <http://www.hpcalc.org/details.php?id=5363>
| → <http://www.hpcalc.org/hp49/math/numeric/longfl36.zip>

下面创建一个简单示例。要表示无限精度实数 6432.2，先写入前导码 0263A 并反转其半字节，得到：

| #A3620

现在计算尾数的大小。大小字段本身长 5 个半字节，数值长 5 个半字节，小数点不计入其中，符号长 1 个半字节。合计为 #11d，即 #0000Bh。把它的半字节反向，并放在前导码之后：

| #A3620 B0000

为了表示 6432.2 的尾数，必须把它变成 5 位整数，所以得到 64322。暂时不用担心这使数值大了一个数量级，稍后会修正。反转数字后得到 22346，再在末尾加上符号半字节 0。到目前为止得到：

| #A3620 B0000 22346 0

为了修正创建尾数时乘上的 10^1，必须建立 10^-1 的指数，也就是在指数中存储 -1。要建立 -1，仍要先计算其大小。大小字段始终长 5 个半字节，数值 1 长 1 个半字节，符号长 1 个半字节，合计为 #7d。把它转换为十六进制，本例中数值恰好不变；补零并像往常一样反向，然后放到末尾，得到：

| #A3620 B0000 22346 0 70000

现在反转数值 1 的数字，结果当然不变，再在后面放入符号；负数的符号为 9。于是，完整长实数如下：

| #A3620 B0000 22346 0 70000 1 9

57.14 无限精度复数（仅 HP 49）

前导码：02660

结束码：无

大小：29 半字节起

TYPE：27

内存中的结构：

字段	长度
前导码（02660）	5 半字节
实部尾数大小	5 半字节
实部尾数数字 1	1 半字节
...	...
实部尾数数字 n	1 半字节
实部符号（0 除外）	1 半字节
实部指数大小	5 半字节
实部指数数字 1	1 半字节
...	...
实部指数数字 n	1 半字节
实部符号（0 除外）	1 半字节
虚部尾数大小	5 半字节
虚部尾数数字 1	1 半字节
...	...
虚部尾数数字 n	1 半字节
虚部符号（0 除外）	1 半字节
虚部指数大小	5 半字节
虚部指数数字 1	1 半字节
...	...
虚部指数数字 n	1 半字节
虚部符号（0 除外）	1 半字节

乍看之下它可能很复杂，实际上非常简单。无限精度复数就是前导码 02660 后跟两个不带前导码的无限精度实数。第一个无限精度实数是复数的实部，第二个是虚部。前面的无限精度实数说明已经完整解释了这种格式，下面再做一个简单示例。

要表示无限精度复数 (6432.2, -14.57)，先写入前导码 02660 并反转其半字节，得到：

| #06620

然后创建表示实部的无限精度实数 6432.2，但忽略它的前导码。上一示例已经完成这一步，应得到：

| #B00002234607000019

再创建表示虚部的无限精度实数 -14.57，同样忽略其前导码，应得到：

| #A0000754197000029

把这三个部分放在一起，得到：

| #06620 B00002234607000019 A0000754197000029

就是这么简单。

57.15 符号矩阵（仅 HP 49）

前导码： 02686

结束码： 0312B

大小： 15 半字节起

TYPE： 29

内存中的结构：

字段	长度
前导码（02686）	5 半字节
对象 1	长度不定
...	...
最后一个对象	长度不定
结束码（0312B）	5 半字节

符号矩阵对象很有意思：它与列表非常相似，但其中至少必须有一个对象，而且可包含的对象类型受到限制。除此以外，以及前导码不同之外，它与列表对象完全相同。

符号矩阵允许包含的对象类型包括：另一个符号矩阵，用于构成二维结构；实数；复数；整数；或代数对象。也可以把其他类型的对象放进符号矩阵，但 HP 49 的内置例程并非为处理这些对象而设计，因此不建议这样做。

58 原始代码对象

这些内容并不真正以对象形式存在，因为除非通过指针，否则无法把它们放到堆栈上。这些 PCO 的前导码指向自身地址加 5 的位置，例如：

```
前导码（abcde）
在 abcde 处：对象地址
-----+
          |
<-----+
```

如图所示，PCO 的定义方式如下：在 PCO 地址加 5 的位置，可以找到一个五半字节地址。

该地址处是机器语言代码，没有任何前导码，也没有大小信息。HP 大量使用这种结构；例如，[SWAP](#)、[DUP](#)、[DROP](#) 等所有堆栈命令都使用 PCO 编码。

使用 System RPL 编程时，可以调用 ROM 中称为“primitives”的这些 PCO 对象；这里的“primitive”并不是数学中的原函数。

可以用 [SYSEVAL](#) 命令调用 PCO。把一个二进制整数放到堆栈上，然后输入 [SYSEVAL](#) [\[ENTER\]](#)。

警告： 在确认该二进制整数指向安全入口点之前，切勿这样操作，否则计算器内存可能被清空。

第六部分 编写程序

59 实现循环

循环有多种实现方式，可用于不同目的。循环可以用于“等待”或减慢程序，因为程序有时运行得太快；也可以单纯把一组指令重复若干次。

要实现循环，必须用某个位置保存数值。可以使用 A、B、C、D 等寄存器，甚至可以使用 P。

这里使用寄存器 C 作为循环计数器：

```
LC FF
*LOOP
C=C-1 B
GONC LOOP
A=DAT1 A
D0=D0+ 5
PC=(A)
@
```

这是一个简单程序。先装入数值 `FF`，再使用 `B` 字段循环。程序每循环一次，就用 `B` 字段从计数寄存器中减去 1。循环持续到进位标志置位。

使用进位标志时，需要记住一件重要的事：循环执行次数会比计数器的值多 1 次。为什么？

`FF` 被装入 `B` 字段。每次循环都会减去 1，但读到零时进位标志仍未置位，因此还会再循环一次。下一次，`B` 字段中是 `#00h`，减去 1 后进位标志置位，循环才停止。所以，上述循环会执行 `#FFh + 1` 次，也就是 `#100h` 次。

在程序中使用进位标志实现循环时，不要忘记在开始前从计数器值中减去一个单位。因为循环会多执行一次，这样才能得到正确的循环次数。

下面是使用 `P` 作为计数器的示例。这里编写一个程序，让 `HP` 的蜂鸣器发出轻微的“咔嗒”声。

要打开蜂鸣器，需要使用 `Saturn` 处理器的 `OUT` 寄存器。向 `OUT` 发送 `#800h`，蜂鸣器就会打开；要关闭它，则向 `OUT` 寄存器发送 `#000h`。

以下是发出咔嗒声的示例程序：

```
LC 800
OUT=C
*LOOP
P=P+1
GONC LOOP
C=0 X
OUT=C
A=DAT0 A
D0=D0+ 5
PC=(A)
@
```

编译程序，把它存入变量，将 `HP` 的背面靠近耳朵，然后连续按几次相应按键运行程序。您会听到轻微的咔嗒声。通过改变时间并反复开关扬声器，可以产生声音。`ROM` 中有一个例程，可以按您给出的频率（赫兹）和持续时间（毫秒）发声。

请注意，上述代码只适用于配有真正 `Saturn` 处理器的计算器。在本书编写时，`HP 48GII` 和 `HP 49G+` 中模拟的 `Saturn+` 处理器无法用这种方式模拟蜂鸣器。

60 从键盘读取输入

要读取按键，必须考虑到按键在内部沿若干线路排列。程序会向某些线路送出电流，再查看输出，以检查是否有按键按下。按下按键会“闭合”一个电路，程序便能检测到它。

这引出了“数值对”的概念。首先向 `OUT` 发送一个特定值，然后读取 `IN`，最后根据已发送的 `OUT` 值，把 `IN` 与出现的值进行比较，从而判断是否有按键按下。

下面是 `HP 48` 的键盘布局。每个按键名称下有两个数：左侧数值写入 `OUT`；如果该键被按下，就会在 `IN` 寄存器中读到右侧的对应值。

按键	OUT/IN	按键	OUT/IN	按键	OUT/IN
A	002/0010	B	100/0010	C	100/0008
D	100/0004	E	100/0002	F	100/0001
MTH	004/0010	PRG	080/0010	CST	080/0008
VAR	080/0004	(UP)	080/0002	NXT	080/0001
'	001/0010	STO	040/0010	EVAL	040/0008
(LEFT)	040/0004	(DOWN)	040/0002	(RIGHT)	040/0001
SIN	008/0010	COS	020/0010	TAN	020/0008
(SQRT)	020/0004	(POWER)	020/0002	(INV)	020/0001
ENTER	010/0010	+/-	010/0008	EEX	010/0004
DEL	010/0002	(BACKSPACE)	010/0001	(ALPHA)	008/0020
7	008/0008	8	008/0004	9	008/0002
(DIVIDE)	008/0001	(LEFT-SHIFT)	004/0020	4	004/0008
5	004/0004	6	004/0002	(MULTIPLY)	004/0001
(RIGHT-SHIFT)	002/0020	1	002/0008	2	002/0004

按键	OUT/IN	按键	OUT/IN	按键	OUT/IN
3	002/0002	(SUBTRACT)	002/0001	ON	/8000
0	001/0008	.	001/0004	SPC	001/0002
(ADD)	001/0001				

下面是 HP 49 的键盘布局，标注方式与上面的 HP 48 布局相同：

按键	OUT/IN	按键	OUT/IN	按键	OUT/IN
A	020/0001	B	020/0002	C	020/0004
D	020/0008	E	020/0010	F	020/0020
APPS	020/0080	MODE	010/0080	TOOL	008/0080
(UP)	040/0008	(LEFT)	040/0004	(RIGHT)	040/0001
VAR	004/0080	STO	002/0080	NXT	001/0080
(DOWN)	040/0002	HIST	010/0040	EVAL or CAT	008/0040
' or EQW	004/0040	SYMB	002/0040	(BACKSPACE)	001/0040
(POWER)	010/0020	(SQRT)	008/0020	SIN	004/0020
COS	002/0020	TAN	001/0020	EEX	010/0010
+/-	008/0010	X	004/0010	(INV)	002/0010
(DIVIDE)	001/0010	(ALPHA)	080/0008	7	008/0008
8	004/0008	9	002/0008	(MULTIPLY)	001/0008

按键	OUT/IN	按键	OUT/IN	按键	OUT/IN
(LEFT-SHIFT)	080/0004	4	008/0004	5	004/0004
6	002/0004	(SUBTRACT)	001/0004	(RIGHT-SHIFT)	080/0002
1	008/0002	2	004/0002	3	002/0002
(ADD)	001/0002	ON	/8000	0	008/0001
.	004/0001	SPC	002/0001	ENTER	001/0001

60.1 检查一次按键

编写一个程序，让它一直循环到按下 [SPC] 键为止：

```
INTOFF
% 关闭键盘中断
*LOOP
% LOOP 从这里开始
LC 001
% 在 HP 48 上把 #001h 装入 C(A)；HP 49 上改为 LC 002
OUT=C
% 发送该值，使 SPC 所在线路通电
GOSBVL =CINRTN
% =CINRTN 在 HP 48 上位于 01160，在 HP 49 上位于 00212
?CBIT=0 1
% C 是否包含 #0002h？HP 49 上改为 ?CBIT=0 0
GOYES LOOP
% 没有？跳到 LOOP
INTON
% 允许键盘中断
A=DAT0 A
% 返回 RPL
D0=D0+ 5
PC=(A)
@
```

这里有几点需要说明。首先，使用了 `INTOFF` 和 `INTON`，用来告诉 HP，键盘将由代码自行处理。源代码末尾务必使用 `INTON`。其次，查看 `[SPC]` 键对应的数值对，会发现 HP 48 上为 `001/0002`，HP 49 上为 `002/0001`。可以把 `0002` (HP 48) 或 `0001` (HP 49) 装入 A，再比较寄存器 C 与寄存器 A。但如果只考虑第一个半字节，十六进制的 `#0002h` 等于二进制的 `#0010b`，十六进制的 `#0001h` 等于二进制的 `#0001b`。因此，只需在 HP 48 上检查第 1 位是否为零，或在 HP 49 上检查第 0 位是否为零。如果为零，说明尚未按键，于是再次循环。请记住，第一位编号为零，第二位编号为一。还有其他检查按键的方法。下面的方法会使用逻辑运算符以及“掩码”的概念。

60.2 检查组合按键

检测同时按下多个按键时，掩码是一个很有用的概念。下面编写一个程序，当 `[1]` 和 `[9]` 同时按下时退出。

在 HP 48 上，`1` 键与 `9` 键的输出值分别为 `#002h` (`#0010b`) 和 `#008h` (`#1000b`)；有趣的是，在 HP 49 上二者恰好对调。对这两个值执行 `OR`，得到 `#Ah` (`#1010b`)。

可以用计算器简化这一步。只需把 `#2h` 和 `#8h` 放到堆栈上，再按 `[MTH] [F] [NXT] [A] [B]`，或直接输入 `OR`。这会对堆栈的前两个栈执行 `OR`。

注意：必须处于 HEX 模式，才能得到要放入程序的值。用 `[MTH] [F] [A]` 进入 HEX 模式。

IN 值也用 `OR` 得到。两个数值为 `#8h` 和 `#2h`，因此 `#8h OR #2h = #Ah`。

源代码如下：

```
INTOFF
*LOOP
LC 00A
OUT=C
GOSBVL =CINRTN
% =CINRTN 在 HP 48 上位于 01160，在 HP 49 上位于 00212
LA 000A
?C#A A
GOYES LOOP
INTON
A=DAT0 A
D0=D0+ 5
PC=(A)
@
```

60.3 读取整个键盘

如果希望检查是否按下了任意按键，该怎么办？只需对所有输出掩码执行 `OR`，然后检查从键盘读取的值是否不为零。

但 `[ON]` 键比较特殊：它直接连接到 Saturn 处理器的一个引脚。要测试是否按下 `[ON]`，必须执行 `C=IN`，再检查第 15 位是否为零。如果为零，说明已按下 `[ON]`。

所有输出掩码的 `OR` 掩码是：

`#1h OR #2h OR #4h OR #8h OR #10h OR #20h OR #40h OR #80h OR #100h`

结果为 `#1Ffh`。

用于测试是否按下任意按键的源代码如下：

```
INTOFF
*LOOP
LC 1FF
OUT=C
GOSBVL =CINRTN
% =CINRTN 在 HP 48 上位于 01160，在 HP 49 上位于 00212
?C=0 A
GOYES LOOP
INTON
A=DAT0 A
D0=D0+ 5
PC=(A)
@
```

这里有一个问题：程序一启动就会退出。开始循环之前，必须先清空键盘缓冲区。下面这段代码可以清空键盘缓冲区：

```

D1=(5) "adrKEYBUFFER"+2
% 如果没有入口表, 应写成 D1= 047DF
A=DAT1 A
D1=A
C=0 A
DAT1=C A
RTNCC

```

当然, 如果使用 D1, 就必须先保存它的值。

于是得到:

```

INTOFF
% 关闭键盘中断
*LOOP
LC 1FF
% 所有按键的掩码
OUT=C
GOSBVL =CINRTN
% =CINRTN 在 HP 48 上位于 01160, 在 HP 49 上位于 00212
?C=0 A
% 没有按键按下?
GOYES LOOP
% 则继续循环
INTON
LC(6) "adrKEYBUFFER"+2
% 指向键盘缓冲区指针的地址; 如果没有入口表, 则使用 LC 0047DF
CD1EX
% 把旧 D1 保存到 C
A=DAT1 A
% 把指向键盘缓冲区的指针读入 D1
D1=A
A=0 A
DAT1=A A
% 清空缓冲区
D1=C
% 恢复 D1
A=DAT0 A
% 退出
D0=D0+ 5
PC=(A)
@

```

它运行得非常快, 因此需要配合类似下面的 RPL 程序; 其中 `WAITK` 是保存这段代码的变量:

```
<< 0 WAIT DROP WAITK >>
```

把该程序放到堆栈上, 快速按下 `[EVAL]`, 忙碌指示符应会熄灭, 而 RPL 程序仍留在堆栈上。按任意键, 程序就会退出。

由此可见, 代码执行速度非常快。

60.4 读取 `[ON]` 键

`[ON]` 键很特殊, 因为它直接连接到处理器的一个引脚。要测试它, 仅执行 `INTOFF` 还不够。必须禁用所有中断, 因此需要把 ST 的标志 15 置零。稍后查看中断处理程序的反汇编代码时将看到, 如果该位为零, 就不会执行中断。实际情况稍复杂一些, 但留待后文介绍。中断很复杂, 因此会放在最后一部分讨论。

检查 `[ON]` 的代码如下:

```

ST=0 15
% 禁用所有中断
*LOOP
GOSBVL =CINRTN
% =CINRTN 在 HP 48 上位于 01160, 在 HP 49 上位于 00212
?CBIT=0 15
% 是否按下 [ON]?
GOYES LOOP
% 如果没有, 则继续循环

```

```

ST=1 15
% 允许所有中断
GOSUB REINT
% 启用中断
A=DAT0 A
% 退出
D0=D0+ 5
PC=(A)
*REINT
RTI
% 退出中断服务例程
@

```

注意！ 使用 `ST=0 15` 时不要出错，因为一旦执行它，就无法使用 `[ON]-[C]` 或 `[ON]-[A]-[F]`。如果程序把自己困在无限循环中，就必须使用 HP 底部的小孔执行硬复位，而且可能丢失部分内存。

之所以必须使用 `ST=0 15` 和 `ST=1 15`，是因为否则无法检查是否按下 `[ON]`。

还有另一种检查组合按键的方法：逐一检查每个按键。机器语言运行速度很快，因此能够检测同时按下多个按键。下面的程序会一直循环，直到同时按下 `[A]` 和 `[F]`。

程序先等待 `[A]`；一旦检测到它，立刻检查 `[F]`。程序运行得足够快，可以检测到两个按键同时按下：

```

INTOFF
% 关闭中断
*LOOP
LC 002
% [A] 的输出值
OUT=C
GOSBVL =CINRTN
% =CINRTN 在 HP 48 上位于 01160，在 HP 49 上位于 00212
LA 0010
% [A] 的输入值
?C#A A
% 如果没有按下 [A]
GOYES LOOP
% 则继续循环
LC 100
% [F] 的输出值
OUT=C
GOSBVL =CINRTN
% =CINRTN 在 HP 48 上位于 01160，在 HP 49 上位于 00212
?CBIT=0 0
% 第 0 位是否为 0?
GOYES LOOP
% 如果是，说明未按下 [F]，返回等待 [A]
INTON
% 恢复中断
LC (6) "adrKEYBUFFER"+2
% 指向键盘缓冲区指针的地址；如果没有入口表，则使用 LC 0047DF
CD1EX
% 把旧 D1 保存到 C
A=DAT1 A
% 把指向键盘缓冲区的指针读入 D1
D1=A
A=0 A
DAT1=A A
% 清空缓冲区
D1=C
% 恢复 D1
A=DAT0 A
% 退出
D0=D0+ 5
PC=(A)
@

```

61 操作堆栈

前面说过，堆栈以四个空半字节 #00000h 结束。这句话只说对了一部分，完整情况稍复杂一些。

HP 48 在内部用以下代码计算堆栈深度：

```
LC 806FD
% 48G 系列和 HP 49 使用 806FD；S/SX 使用 7057E
CD1EX
A=DAT1 A
C=A-C A
GOSUBL =DIV5
% =DIV5 位于 06A8E；该例程把 C(A) 除以 5
C=C-1 A
% 减去 1，得到堆栈深度
```

这里 D1 指向堆栈的栈 1。CD1EX 把 D1 中的地址移到 C，并让 D1 指向 #806FDh。该地址处用五个半字节存储命令行起始地址。随后把该地址装入 A，并恢复 D1。然后计算：

堆栈大小 * 5 = D1 地址 - 命令行地址

因此，需要把结果 C(A) 除以五。ROM 中地址 #06A8Eh 处有一个执行此运算的例程，它会使用寄存器 A、C 和 D。

除以五后再减去一，就得到堆栈深度。

为什么要做这一切？只检查堆栈中的 #00000h 难道还不够吗？虽然堆栈末尾确实包含 #00000h，但地址 #00000h 本身也可能位于堆栈上，因此堆栈可能如下所示：

```
4:
3:          4
2:    <External> ← 这就是 #00000h!
1:          3
```

因此，堆栈上可能有两个或更多 #00000h。这就是为什么如果确实想知道堆栈深度，就必须计算它，而不能只搜索堆栈上的第一个 #00000h。

如果愿意，可以让 HP 显示这样的堆栈。先把三个数放到堆栈上，然后编译以下代码：

```
D1=D1+ 5
% 移到栈 2
C=0 A
DAT1=C A
% 把栈 2 置为 #00000h
D1=D1-5
% 返回栈 1
GOVLNG =LOOP
% =LOOP 在 HP 48 上位于 2D564，在 HP 49 上位于 05149；它会返回 RPL
```

把这段代码存入变量，并在已有三个栈项的堆栈上运行。您看到了什么？

栈 2 中出现了 <External>，它实际上就是 #00000h。

按 [DEL] 清空整个堆栈。不要对 <External> 执行 EVAL，因为这会让 HP 跳转到 #00000h 并执行热启动。HP 只会重新启动，并没有危险，但仍不要轻率尝试。

下面是计算堆栈真实深度的代码：

```
CD1EX
% 把 D1 放入 C
D1= 806FD
% 让 D1 指向 #806FDh
A=DAT1 A
% 读取命令行起始地址
C=A-C A
% 计算 D1 地址减命令行地址
GOSUBL =DIV5
% =DIV5 位于 06A8E；该例程把 C(A) 除以 5
C=C-1 A
% 再减去一
```

C 中保存堆栈的真实深度，该例程使用寄存器 A、B、C 和 D1。

61.1 Drop

编写 `DROP` 非常简单。这里不需要知道堆栈的真实深度，只需读取栈 1 的值。如果该值为零，堆栈“应当”为空；如果不等于零，就丢弃一个栈项。

你知道 D1 指向堆栈地址。每次递增 D1，堆栈大小都会减小；每次递减 D1，堆栈都会变大。堆栈只包含地址，每个地址宽五个半字节，因此从堆栈读取时使用 A 字段。

把 D1 增加 5，就完成一次 `DROP`。代码如下：

```
C=DAT1 A
?C=0 A
GOYES QUIT
D1=D1+ 5
D=D+1 A
*QUIT
GOVLNG =LOOP
% =LOOP 在 HP 48 上位于 2D564，在 HP 49 上位于 05149；它会返回 RPL
@
```

这里先读取栈 1 的内容。如果它为 0，就退出。在 2D564（HP 48）或 05149（HP 49）处有以下代码：

```
A=DAT0 A
D0=D0+ 5
PC=(A)
```

`GOSBVL =LOOP` 在内存中占 7 个半字节，而通常的三条指令需要 11 个半字节。在这里使用跳转而不是三条指令，可以节省 4 个半字节。

61.2 Swap

由于堆栈只包含地址，实现 `SWAP` 只需交换栈 1 与栈 2 的地址。首先必须确认堆栈深度至少为二；如果满足条件，就交换这两个地址。

可以这样编写：

```
C=DAT1 A
% 读取栈 1 的地址
?C=0 A
% 如果为空，则退出
GOYES QUIT
D1=D1+ 5
% 指向栈 2
A=DAT1 A
% 把栈 2 地址读入 A
D1=D1- 5
% 返回栈 1
?A=0 A
% 如果 A 为 0，说明没有栈 2
GOYES QUIT
% 因此退出
DAT1=A A
% 否则把 A 写入 DAT1，也就是把栈 2 地址移到栈 1
D1=D1+ 5
% 指向栈 2
DAT1=C A
% 把 C 写入此处，也就是把栈 1 地址移到栈 2
D1=D1- 5
% 让 D1 返回栈 1
*QUIT
GOVLNG =LOOP
% =LOOP 在 HP 48 上位于 2D564，在 HP 49 上位于 05149；它会返回 RPL
@
```

请试一试。堆栈为空或只有一个对象时，它什么也不做；有两个对象时，它会交换二者。

HP 自身如何执行 `SWAP`？

调用 `SWAP` 时，系统会调用一个 RPL 程序。该程序包含两个调用：第一个调用检查堆栈上是否有两个对象地址；如果没有，就产生错误。否则，程序调用 `SWAP` 代码，而该代码不再检查，因为前一段代码已经完成检查。

这种方法更好，因为 HP 能够处理堆栈上的 #00000h，并把它与另一个对象交换。我们的代码会把任何 #00000h 都识别为堆栈末尾，因此如果堆栈中包含 #00000h，它不会执行任何操作。

当然，“普通”HP 用户的堆栈上不应出现 <External>。即使出现，它也很可能不是求值后会执行热启动的 #00000h。但是，作为 User RPL 基础的 System RPL 大量使用 External，因此必须能够对 External 执行 SWAP 或其他任何堆栈操作。

62 示例程序：数字之和

下面把目前所学的内容综合起来，编写一个简单程序。

该程序从堆栈的栈 1 取得一个实数，计算它所有数字之和，并把结果作为实数返回栈 1。程序还会做简单的参数检查：把对象的前导码与标准实数前导码比较，以确认栈 1 中确实是实数；如果不匹配，就产生错误。

下面是 Arnaud Amiel 编写的程序清单，感谢 Arnaud：

```
GOSBVL =SAVPTR
% =SAVPTR 位于 0679B，用于保存寄存器 D0、D1、B 和 D
A=DAT1 A
% A 保存指向栈 1 的指针
AD1EX
% D1 指向栈 1
A=DAT1 A
% 把前导码装入 A(A)
LC(5)DOREAL
% DOREAL 是 02933，即实数前导码；把它装入 C(A)
?C#A A
GOYES error
% 如果二者不同，则产生错误
D1=D1+ 5
% 指向数值
A=DAT1 W
% 把数值装入 A
SETDEC
% 实数以 BCD 格式存储
P=4
% M 字段保存 12 位数字：计数器 P=16-12
C=0 W
% 清零 C，用于保存结果

*NextDigit
ASR W
% 把最后一位移动到 X 字段
ASR X
% 把该数字移动到 X 字段的最低有效位
ASR X
C=C+A X
% 把它累加到计数器
P=P+1
% 又处理了一位数字.....
GONC NextDigit
% 如果进位，则说明全部 12 位已经处理完，且 P=0

A=0 X
% 清零 A，用于保存指数
?C=0.X
GOYES finished
% 如果所有数字都已移出 X，则结束
CSRC
% 把数字移出 X
?C=0 X
GOYES finished
% 如果所有数字都已移出 X，则结束
```

```

CSRC
% 把数字移出 X
A=A+1 X
% 指数加一；两位数结果时 X=1
?C=0 X
GOYES finished
% 如果所有数字都已移出 X，则结束
CSRC
% 把数字移出 X
A=A+1 X
% 指数加一；三位数结果时 X=2

*finished
CSRC
% 把最后一位从 S 移到 M
C=C+A X
DAT1=C W
% 覆盖栈 1 中的实数
SETHX
% 返回十六进制模式
GOVLNG =GETPTRLOOP
% =GETPTRLOOP 位于 05143；它装入此前保存的寄存器并退出

*error
LC(5)argtypeerr
% argtypeerr 是 00202，即“Bad Argument Type”错误消息编号
GOVLNG =GPErrjmpC
% =GPErrjmpC 在 HP 49 上位于 266DB，在 HP 48 上位于 10F40；它会带错误退出
@

```

该程序通过直接替换栈 1 所指地址处内存中的半字节，用数字之和替换栈 1 中的数值。这会产生一些有趣的副作用。HP 把从 -9 到 9 的实整数存储在 ROM 中，因此无法用 `DAT1=C W` 指令改写它们。使用 -1 至 -9 的负实整数运行此程序时，会发现程序仍返回原来的负数，对内存没有任何影响。输入其他负实数时，程序会以正数形式返回各数字之和。在这些情况下，数值存储在 RAM 而不是 ROM 中，因此 `DAT1=C W` 指令会按预期工作。

有一些方法可以修复此问题。可以把答案压入堆栈的栈 1，使输入上移到栈 2；先把要压入的实数放入 `A(W)`，再使用 `GOSBVL =PUSH%LOOP` 退出程序，其中 `PUSH%LOOP` 在 HP 48 上位于 `2A23D`，在 HP 49 上位于 `2F899`。要在读取输入时把它从堆栈移除，可以调用 `GOSBVL =POP1%`；`POP1%` 在 HP 48 上位于 `29FDA`，在 HP 49 上位于 `2F636`，它将从堆栈弹出该实数并放入 `A(W)`。

最好的解决办法是为新的实数分配内存，然后让栈 1 改为指向该位置。不过，所有这些方案都会让代码更复杂，因此这个简单示例中不再给出。

63 示例程序：完全平方数

该程序同样由 Arnaud Amiel 编写，用于计算所有十位完全平方数，其中使用的不同数字不超过三种。这样的数共有 48 个，其中 47 个使用三种不同数字，另一个只使用两种不同数字；在 HP 49G+ 上，程序大约 5 秒即可把全部 48 个数压入堆栈。

如果知道平方数序列为 $U(n)$ ，算法就很容易理解：

$$U(n+1) - U(n) = 2 + (U(n) - U(n-1))$$

程序采用完整 MASD 语法编写，包含 `SAVE` 等宏以及使用花括号（{ 和 }）的类高级语言结构，因此要用 HP-ASM 汇编，需要做少量修改。

```

SAVE SETDEC
% 保存寄存器，并使用十进制数
LA 1111088889000
% 把最大的无效十位平方数装入 A(M)，因此末尾附加 000
C=0.M LC 66665000 B=C.M
% 把 U(n)-U(n-1) 装入 B，并同样附加 000
LC 33333 RSTK=C
% 必须测试 SQRT(9999999999)-SQRT(1111111111)=66666 个平方数
% 需要使用所有寄存器，因此把计数器保存在 RSTK 上
% 这是安全的，因为程序不使用任何 GOSUB

```

```

*NextNum
C=RSTK C+1.A
% 将测试下一个平方数，因此取出计数器并递增
SKIPNC { P=0 SETHEX LOADRPL }
% 如果已经统计完所有平方数，就退出
RSTK=C
% 把计数器保存在 RSTK 上
B+1.M B+1.M A+B.M C=A.M
% 在这里计算平方数
% 两次使用 B+1，因为 B+2 不会考虑 DEC 模式
% 把被测试的数保存在 A 中，并测试 C
P=5 A=0.XS B=0.XS D=0.XS
% 要测试 10 位数字，因此 P=15-10
% 在 XS 中测试这些数字；A(XS)、B(XS) 和 D(XS) 是三个一位存储位置，所以先将其清零
*TestNum
P+1 GOC GoodNum
% 已无错误地检查全部 10 位，所以这是一个合格数
CSR.W
% 把 M 的最后一位移到 XS 进行测试
?C=0.XS GOYES NextNum
% 如果该数字是 0，它就不是解；转去计算下一个平方数
?A#0.XS { A=C.XS GOTO TestNum }
% 存储位置 1 中是否已有内容？如果没有，就把该数字存入其中，再去测试下一位
?A=C.XS GOYES TestNum
% 如果该数字已在存储位置 1 中，就去测试下一位
?B#0.XS { B=C.XS GOTO TestNum }
% 对存储位置 2 执行相同操作
?B=C.XS GOYES TestNum
?D#0.XS { D=C.XS GOTO TestNum }
% 对存储位置 3 执行相同操作
?D=C.XS GOYES TestNum
GOTO NextNum
% 如果运行到这里，说明出现了三种以上不同数字

*GoodNum
R1=A.M
% 使用 PUSH% 会改变所有寄存器，因此先备份
C=B.M R2=C.M
% 备份  $U(n)-U(n-1)$ 
ASL.M ASL.M
% 十位数字原本右对齐，但必须改为左对齐
LA 009
% 这是十位数的指数
A=0.S
% 确保符号正确
GOSBVL =PUSH%
% 把 A 作为实数压入堆栈
SAVE
% 保存指针
SETDEC
% PUSH% 会切换到 HEX，因此返回十进制模式
A=R1.M
% 恢复备份
C=R2.M B=C.M
% 恢复
GOTO NextNum
% 转去检查下一个数
@

```

64 创建用于生成宏的程序

您已经知道，可以把程序的各部分放入多个文件，并使用 `'` 符号建立链接。不过，如 HP-ASM 文档所述，也可以创建宏。请记住，宏是预先编译的内容，可以包含在任意代码对象中。

例如，假设要编写一个程序，在菜单栏显示 **YES** 和 **NO** 选项；按下 **YES** 时返回 `<1h>`，按下 **NO** 时返回 `<0h>`。可以创建一个内部含有 **YES** 和 **NO** 字样的 GROB，由程序把它显示在菜单区域，再等待按键。

因此，如果希望在代码中放入 GROB，其实无需包含整个对象，只需放入所需数据。ASM Flash 和 HP-ASM 能够在程序中的任意位置包含预编译对象。下面编写一个 `→ MACRO` 程序，它完成两项工作：

79. 如果向它提供代码对象，就移除前导码和大小字段，并把包含代码的字符串放到堆栈上。

80. 如果向它提供 GROB 对象，就移除前导码、大小字段以及后续用于编码行数和列数的 10 个半字节。

例如，与其每次都在程序末尾编写：

```
A=DAT0 A
D0=D0+ 5
PC=(A)
```

不如编译这段代码，再对它使用 `→ MACRO`。然后把它命名为 `'RPL'`，存入 HOME 的 `Macro` 目录。以后每次要在程序中使用它，只需输入以下内容，ASM Flash/HP ASM 就会把 `RPL` 的内容放到该位置：

```
...
/RPL
...
```

如果没有入口表，这种方法尤其便于避免记忆 ROM 例程地址。与其记住 HP 48 上包含 RPL 退出代码的 `#2D564h`，HP 49 上为 `#05149h`，不如直接编写以下代码并对其使用 `→ MACRO`：

```
GOVLNG 2D564
@
```

把它存为 `'RPL'`，此后即可在源代码中使用 `/RPL`。任何代码片段都可以这样处理。

下一节给出带注释的源代码。该代码已经过测试，应当可以工作。

不过，有些汇编器内置了部分地址和功能的名称。例如，Meta Kernel 内置了返回 RPL 的命令。

64.1 `→MACRO.S`

```
!0-15
!PC

C=DAT1 A
% 读取栈 1 的内容
?C#0 A
% 如果不为空，则开始
GOYES BEGIN
LA 00201
% 否则产生错误；A(A) 中保存错误代码
GOVLNG =Errjmp
% =Errjmp 位于 05023，用于产生错误

*BEGIN
% 从这里开始
R1=C A
% 把栈 1 的地址保存到 R1(A)
GOSBVL =SAVPTR
% =SAVPTR 位于 0679B，用于保存寄存器 D0、D1、B 和 D
C=R1 A
% 把栈 1 地址放入 C
B=C A
% 再保存到 B；原因见下文
D0=C
% D0 指向对象的第一个半字节
GOSBVL =SKIPOB
% =SKIPOB 位于 03019；它调用 SKIP，使 D0 指向对象之后的第一个半字节
CD0EX
% C 中包含该地址
C=C-B A
```

```

% 计算末地址减起始地址，因此 C 为对象长度
R2=C A
% R2 (A) 中保存对象长度，单位为半字节
C=R1 A
% 取回保存在 R1 (A) 中的对象地址
D0=C
% 让 D0 指向该地址
A=DAT0 A
% 读取对象前导码
LC 02DCC
% 02DCC 是代码对象前导码
?A#C A
% 如果不是代码对象
GOYES GROB?
% 检查是否为 GROB
% Code
C=R1 A
% C 为对象地址
C=C+10 A
% 跳过代码对象的前导码和大小，共 10 个半字节
R1=C A
% 把结果存回 R1 (A)
C=R2 A
% 此时 C 为字符串需要分配的长度
C=C-10 A
% 减去 10，因为少复制 10 个半字节
R2=C A
% 把结果存回 R2 (A)

*ALLOCATE
GOSBVL =MAKE$N
% =MAKE$N 位于 05B7D；它预留一个长度为 C (A) 的字符串
CD0EX
% C 为新预留字符串第一个半字节的地址
D1=C
% 让 D1 指向该地址；后面使用的 COPY 例程从 D0 复制到 D1
C=R1 A
% C 是“修正后”的对象地址；修正是指跳过前导码和未使用字段
D0=C
% 让 D0 指向该地址
C=R2 A
% C 为要复制的半字节数量；R2 是用 SKIPOB 计算的长度
GOSBVL =MOVEDOWN
% =MOVEDOWN 位于 0670C；这是 COPY DOWN 例程，后文会与其他例程一起说明
GOSBVL =GETPTR
% =GETPTR 位于 067D2，用于恢复寄存器 D0、D1、B 和 D
C=R0 A
% R0 中保存预留字符串的地址
D=D-1 A
% 从空闲 RAM 中减去 5 个半字节
GOC EXIT
% 如果进位，说明内存不足以把对象压入堆栈
D1=D1- 5
% 向堆栈增加一个对象，即 D1-5
DAT1=C A
% 写入对象地址，也就是把对象压入堆栈
*EXIT
% 这里是通常的返回位置
GOVLNG =LOOP
% =LOOP 在 HP 48 上位于 2D564，在 HP 49 上位于 05149；它会返回 RPL

```

```

*GROB?
% 检查对象是否为 GROB
LC 02B1E
% 02B1E 是 GROB 前导码
?A=C A
% 如果是 GROB, 就更新 R1(A) 和 R2(A)
GOYES OK
GOVLNG =GETPTRLOOP
% =GETPTRLOOP 位于 05143; 它恢复已保存的寄存器, 并使用通常的 RPL 返回方式退出
*OK
% 此时对象是 GROB, 因此:
C=R1 A
% 取得 GROB 对象的起始地址
C=C+16 A
% 先加 16
C=C+4 A
% 合计加 20, 跳过前导码、大小以及表示行数和列数的十个半字节
R1=C A
% 写回 R1(A)
C=R2 A
% C 为计算出的长度
C=C-16 A
% 减去 16
C=C-4 A
% 合计减去 20
R2=C A
% 写回 R2(A)
GOTO ALLOCATE
% 预留、复制并退出

@

```

本程序使用 R1(A) 和 R2(A)。不使用 R0, 因为用于预留字符串的例程会把预留字符串的地址返回到 R0。

首先读取堆栈的栈 1。如果它为空, 就把 #00201h 装入 A(A)。地址 #05023h 处的例程会使用 A(A) 中的错误代码产生错误; 所有用到的例程都在这段代码的说明之后介绍。

如果栈 1 不为空, 就计算对象长度。这里用 R1(A) 保存第一个对象的地址, 因为稍后把对象复制到预留字符串时还会用到它。程序还使用位于 #03019h 的 SKIPOB 例程, 下文将介绍该例程。

执行完成后, D0 指向对象之后的第一个半字节。随后使用交换指令, 再按以下公式计算对象长度: $\text{长度} = \text{末地址} - \text{起始地址}$ 。R2(A) 用于保存对象长度, 稍后复制对象时需要该值。

然后取回对象指针, 检查其前导码是否为代码对象前导码。如果是, 就更新 R1(A) 和 R2(A): 把跳过的半字节数加到 R1(A), 代码对象为 10, GROB 为 10; 再从 R2(A), 也就是要复制的半字节数量中减去该值。要修改 → MACRO 使其处理所需对象类型, 只需增加测试, 并以相同方式更新 R1(A) 和 R2(A)。

完成后, 取回字符串长度, 并使用 ROM 中地址 #05B7Dh 处的子程序预留空间; 该子程序会在字符串对象中分配 C(A) 个半字节。返回时, R0(A) 中保存字符串地址, 稍后将它放到堆栈上; D0 则指向字符串内部。

接下来只需使用 ROM 中的 COPY DOWN 复制例程, 下文列出了全部相关例程; 不过它是从 D0 复制到 D1。预留字符串返回时, D0 指向字符串, 因此要让 D0 改为指向原对象, 让 D1 指向预留字符串, 再把需要复制的半字节数量放入 C(A)。

复制完成后, 把对象放到堆栈上。这里使用 GOC, 因此如果连五个可用半字节都没有, 程序就不会压入对象, 而是直接退出。下一次垃圾回收会移除已经预留的字符串。

64.2 本程序使用的 ROM 例程

64.2.1 #5023h, 也称 Errjmp 或 ERROR_A

把通常的错误代码放入 A(A), 再使用 GOVLNG 调用该例程。调用时不得仍有已保存的寄存器; 如果需要恢复寄存器, 请先恢复, 再调用此例程。

64.2.2 #679Bh, 也称 SAVPTR 或 SAVE_REG

下面反汇编该例程, 看看它如何工作。以下是 SAVE_REG 带注释的源代码:

```

CD0EX
% 把 D0 地址移到 C
D0= 8072F
% 让 D0 指向 #8072Fh, 见下文
DAT0=C A
% 把 D0 写入该处, 也就是保存 D0
D0= 806F8
% 让 D0 指向 #806F8
CD1EX
% 把 D1 地址移到 C
D1=C
% 再把该地址写回 D1
DAT0=C A
% 从而保存 D1
D0= 806F3
% 让 D0 指向 #806F3h
C=B A
% 把 B(A) 移到 C(A)
DAT0=C A
% 把 B(A) 保存到 #806F3h
D0= 807ED
% 让 D0 指向 #807EDh
C=D A
% 把 D(A) 移到 C(A)
DAT0=C A
% 把 D(A) 保存到 #807EDh
RTNCC
% 返回并清除进位标志

```

请注意, 返回时寄存器 D0 已丢失, 但 D1、B 和 D 仍保持原值。可以看到, HP 把 D0、D1、B 和 D 保存到内存中的若干地址, 这些位置专为此用途保留。本文稍后会介绍称为“系统区”的内存区域, 其中预留了用于保存寄存器的空间。恢复寄存器时, 见下文, HP 会从保存位置读取。正因如此, 只能保存寄存器一次; 新值会覆盖旧值。

64.2.3 #3019h, 也称 SKIPOB (Skip Object)

这个例程稍复杂一些。

调用前, 让 D0 指向对象的第一个半字节。返回时, D0 指向对象之后的第一个半字节, 也就是跳过了 D0 所指对象; 进位标志被清除, ST1 被清除, P 被设为 0。该例程还会使用 D0、A 和 C, 因此如果其中任何寄存器保存着确实需要的内容, 请先另行保存。

64.2.4 #05B7Dh, 也称 MAKE\$N 或 RES_STR (REServe STRing)

该 ROM 例程使用 D0、D1、A、B、C、D 和 R0。

调用前, 把要预留的字符数放入 C(A)。如果内存不足, 就产生 *Insufficient Memory* 错误; 调用该例程前应先保存寄存器, 因为任何错误都会恢复它们。

如果临时内存区中有足够空间, 子程序就会返回; 此时 D0 指向字符串内部的第一个半字节, 字符串地址保存在 R0(A)。退出程序时, 要把该地址放到堆栈上, 从而把字符串放入堆栈。

64.2.5 #0670Ch, 也称 MOVEDOWN 或 COPY DOWN

这里调用该子程序, 是因为 ROM 中的内存处理例程编写得非常好。内存中有两个复制数据的例程: COPY DOWN 和 COPY UP。

二者有什么区别? 使用 COPY DOWN 时, 会从 D0 向 D1 复制 C(A) 个半字节, 并用 + n 移动 D0 和 D1, 其中 n 是例程设置的值。COPY UP 也从 D0 复制到 D1, 但会从被复制区域的末尾开始, 复制过程中 D0 和 D1 递减。这里使用 COPY DOWN, 是因为要从一个对象复制到另一个对象, 方向都是从各对象开头到末尾。如果在内存中反向复制, 就应使用 COPY UP。

调用 COPY DOWN 或 COPY UP 时, D0 必须指向读取数据的位置, D1 必须指向写入数据的位置。C(A) 用于给出要复制的半字节数量。

这两个例程, COPY DOWN 位于 #0670Ch, COPY UP 位于 #066B9h, 都会使用 D0、D1、A 和 C。

64.2.6 #067D2h, 也称 GETPTR 或 GET_REG

该例程恢复此前保存的寄存器。ROM 中的代码如下：

```
D0= 807ED
% D 的保存位置
C=DAT0 A
D=C A
D0= 806F3
% B 的保存位置
C=DAT0 A
B=C A
D0= 806F8
% D1 的保存位置
C=DAT0 A
D1=C
D0= 8072F
% D0 的保存位置
C=DAT0 A
D0=C
RTNCC
```

当然，也可以从 067D2 之后的某个地址开始调用。这是什么意思？很简单。

如果只想恢复 D0，或只恢复 B 和 D0，可以调用该例程，但不从开头进入。这同样适用于所有例程：只要最终会执行 RTN，就可以自行选择起始位置。

下面是与上面相同的代码，但加入了地址：

```
#067D2h      D0= 807ED
% D 的保存位置：HP 49 上为 80E9B
#067D9h      C=DAT0 A
#067DCh      D=C A
#067DEh      D0= 806F3
% B 的保存位置
#067E5h      C=DAT0 A
#067E8       B=C A
#067EAh      D0= 806F8
% D1 的保存位置
#067F1h      C=DAT0 A
#067F4h      D1=C
#067F7h      D0= 8072F
% D0 的保存位置：HP 49 上为 8076B
#067FEh      C=DAT0 A
#06801h      D0=C
#06804H      RTNCC
```

如果只想恢复 D0，只需调用 GOSBVL 067F7。

65 更多内置 ROM 例程

程序员还可以使用许多其他例程，但相关文档比较零散。调用前建议先保存工作寄存器，因为大多数例程会使用它们，不过不会改动暂存寄存器。最好的资料来源由 Mika Heiskanen 和 Carsten Dominik 汇编。Mika 的列表可从以下位置下载：

```
http://www.hpcalc.org/details.php?id=1783
→ http://www.hpcalc.org/hp48/programming/entries/ent_srta.zip
```

Carsten 的列表位于以下地址：

```
http://staff.science.uva.nl/~dominik/hpcalc/entries

http://www.hpcalc.org/details.php?id=5146
→ http://www.hpcalc.org/hp49/programming/entries/edb.zip

http://www.hpcalc.org/details.php?id=5475
→ http://www.hpcalc.org/hp48/programming/entries/entriesdb48pdf.zip

http://www.hpcalc.org/details.php?id=5476
→ http://www.hpcalc.org/hp49/programming/entries/entriesdb49pdf.zip
```

Joe Horn 制作了一个交叉引用，用来比较 HP 48 与 HP 49 的入口；对于同时为两种计算器编写代码的人，这会非常有用。可从以下位置下载：

| <http://www.hpcalc.org/details.php?id=3248>
→ <http://www.hpcalc.org/hp49/programming/entries/xref4849.zip>

请注意，这些文档中的大多数入口都用于 System RPL，但也列出了相当多的汇编例程。

查看上述列表即可发现，入口点数量非常庞大，即使只统计可在汇编语言代码中使用的入口也是如此。由于每个入口点名称都映射到各自的内存地址，如果经常离开 PC 进行开发，强烈建议在计算器上安装入口点表。当然，也可以打印入口点列表，并记住最常用的入口。

66 HP 48 内存管理

衷心感谢 Christoph Giesselink 编写本节。

Saturn 芯片具有 4 位数据总线和 20 位地址总线。因此，可寻址区域为 $2^{20} = 1\,048\,576$ 个半字节，按十六进制从 #00000h 编号到 #FFFFFFh。前面已经说明，一个字节包含两个半字节，所以把 1 048 576 个半字节除以 2，得到 524 288 字节，即 512KB。

66.1 菊花链基础

第三代和第四代 Saturn 芯片，即 S 系列中的 Clarke 和 G 系列中的 Yorke，内置六个内存控制器。它们的分配如下：

设备	S/SX 上的功能	G/G+/GX 上的功能
NCE1 (ROM)	ROM (256KB)	ROM (512KB)
HDW	I/O 寄存器 (32B)	I/O 寄存器 (32B)
NCE2 (RAM)	RAM (32KB)	RAM (32KB、128KB)
CE1	Port1 (32KB、128KB)	存储体切换器
CE2	Port2 (32KB、128KB)	Port1 (32KB、128KB)
NCE3	未使用	Port2 (最大 4MB)

内存控制器通过一种称为菊花链配置的技术相互连接。当 Saturn 向总线发送 CONFIG 命令时，链中的第一个控制器接收该信息。某个控制器完全配置后，会把命令继续传递给链中的下一个控制器。通过这种方式，可以配置所有控制器。

菊花链顺序如下：

| SATURN → HDW → NCE2 → CE1 → CE2 → NCE3 → NCE1

内存控制器可以配置大多数已分配模块的大小及其地址位置。如果两个设备配置了相互重叠的地址范围，就会选择访问优先级较高的设备，而优先级较低的设备被覆盖。这就是“covered（被覆盖）”一词的由来。按优先级控制的设备访问顺序不同于菊花链顺序，它由专门的仲裁机制实现。如果请求的地址没有配置任何设备，NCE1（ROM）控制器会响应。

设备优先级如下：

81. HDW，最高优先级
82. NCE2
83. CE2
84. CE1
85. NCE3
86. NCE1，最低优先级

如果想绕过覆盖低优先级设备的某个设备，就必须使用 UNCNFG 命令“unconfig”，也就是取消配置高优先级设备。这样就能访问所需设备。使用完目标设备后，应重新配置被取消配置的设备。

66.2 设备配置

CPU 复位后，或执行 RESET 命令后，除 ROM 外的所有设备都处于未配置状态。

设备配置顺序如下：

87. HDW：地址
88. NCE2：大小 + 地址

- 89. CE1: 大小 + 地址
- 90. CE2: 大小 + 地址
- 91. NCE3: 大小 + 地址

NCE2、CE1、CE2 和 NCE3 的大小值与地址值必须是 #01000h 的倍数，也就是 #04096d 个半字节或 2KB，因此可用的最小地址范围是 2KB。HDW 的大小始终为 64 个半字节，所以基地址必须是 #00040h (#00064d) 的倍数。所有设备都应配置，即使未使用也是如此；未使用的设备应按最小大小 2KB 配置。

重要： 如果两个模块配置在同一地址，会按优先级取消配置，而不是按配置顺序的相反次序取消配置。

66.3 RESET、CONFIG、UNCNFG 和 C=ID

设备配置由四条命令控制。

66.3.1 RESET

取消配置所有设备。

66.3.2 CONFIG

使用 C(A) 中的数据配置设备。除 HDW 外，所有设备都必须调用两次 CONFIG。原因是第一次指定设备在 Saturn 地址区域中呈现的大小，第二次指定设备在该区域中呈现的基地址。

66.3.2.1 大小

大小参数按下式计算：

```
#100000h - #<module_size> = #<size_we_give_it>
```

因此，如果要给模块设置 32KB (#10000h) 的大小，就必须求：

```
#100000h - #<module_size> = #10000h
```

这意味着模块大小为 #F0000h，第一次执行 CONFIG 时必须把该值放入 C(A) 寄存器。代码如下：

```
LC F0000
% 32KB
CONFIG
% 设置设备大小
```

为什么要使用这样奇怪的数？再看一个例子。希望在地址 #00000h 配置一张 128KB 卡，并把 384KB 作为其大小。

```
#100000h - #<size_we_give_it> = #<module_size>
= #100000h - #C0000h
= #40000h
```

代码如下：

```
LC 40000
% 384KB
CONFIG
% 设置设备大小
LC 00000
% 地址
CONFIG
% 设置设备地址
```

这张 128KB 卡会配置在 #00000h 至 #3FFFFh，并镜像到 #80000h 至 #BFFFFh。为什么会出现镜像？

原因在于大小参数。第一个 CONFIG 参数并不是真正的大小，而是地址线的“无关”掩码。值为 1 的位表示“比较这条地址线”，值为 0 的位表示“忽略这条地址线”。

内存管理器使用地址线 A19 至 A12。未连接的地址线 A11 至 A0 会自动视为“无关”，其内容被忽略。下面把 384KB (#40000h) 的大小信息写成二进制，并用 x 代替零表示“无关”。

```
A19 A18 A17 A16 A15 A14 A13 A12 A11 A10 A9 A8 A7 A6 A5 A4 A3 A2 A1 A0
X 1 X X X X X X X X X X X X X X X X X = #40000h
```

可以看到，只有 A18 有效。在所用基地址 #00000h 中，A18 被清零，因此所有 A18 为零的地址都会选择该设备，对应的地址范围为 #00000h 至 #3FFFFh 以及 #80000h 至 #BFFFFh。其他所有地址的 A18 均置位。由于实际大小小于配置大小，128KB 卡会被镜像。

66.3.2.2 地址

地址参数告诉设备控制器，应在哪个位置看到该设备。但这里有一些影响。地址线 A11 至 A0 未连接，某些地址位还可能是“无关”位；根据大小信息，这些位都会被忽略，因此大小和地址必须是 #01000h 的倍数。

示例：希望把一张 128KB 卡放在 #A0100h。实际使用哪个地址区域？

```
LC C0000
% 128KB
CONFIG
% 设置设备大小
LC A0100
% 地址
CONFIG
% 设置设备地址
```

只需把地址写成二进制，再与大小信息中的“无关”掩码执行 AND。结果就是该区域的首地址。

```
1010 0000 0001 0000 0000b = #A0100h
& 1100 0000 0000 0000 0000b = #C0000h
-----
= 1000 0000 0000 0000 0000b = #80000h
```

在此示例中，128KB 设备配置在 #80000h 至 #BFFFFh。

下一个示例说明，当配置大小小于设备实际大小，而且地址不在内存大小边界上时会发生什么。

示例：希望把一张 128KB 卡中的 64KB 放在 #A0000h。会看到哪一部分？

```
LC E0000
% 64KB
CONFIG
% 设置设备大小
LC A0000
% 地址
CONFIG
% 设置设备地址
```

128KB 卡配置在 #A0000h 至 #BFFFFh，但在地址 #A0000h 看到的是该卡的 64KB 半字节区域。这是因为 128KB 芯片的地址线 A16 至 A0 直接连接到 Saturn 芯片的地址线，中间经过 Saturn 芯片内部的半字节到字节转换。因此，内存控制器无法移动 128KB 卡的位置。请记住，所有内存模块都只能从其自身大小的边界开始。

配置 HDW 控制器时无需提供大小信息，只需执行一次 CONFIG 指定起始地址。

66.3.3UNCNFG

该命令取消配置由 C(A) 中地址指定的设备。通常使用已显露设备在该地址上的基地址，也就是该地址处优先级最高的设备，但不要忘记前面 CONFIG 部分关于大小信息的说明。

示例：取消配置位于 #A0000h、处在 64KB 窗口中的一张 128KB 卡。

```
LC A0000
% 已配置设备的地址
% 也可以像上面那样使用 LC A0100，但不建议这样做
UNCNFG
% 取消配置设备
```

66.3.4C=ID

取得下一个必须配置的设备信息，并把它放入 C 寄存器：

92. 如果没有待配置设备，C(A) 中得到 #00000h。
93. 如果有一个待配置设备，则必须检查 C(B) 的内容：

C(B) 的内容	C(A) 的内容
#01h	NCE3 的大小
#03h	NCE2 的大小
#05h	CE1 的大小
#07h	CE2 的大小
#19h	HDW 的地址
#F2h	NCE3 的地址
#F4h	NCE2 的地址
#F6h	CE1 的地址

C(B) 的内容	C(A) 的内容
#F8h	CE2 的地址

除 HDW 外，把 C 的最后两个半字节清零，也就是执行 C=0 B，即可得到当前内存设备上次使用的大小或地址。当前内存地址为 HDW 时，只需清除最后六位。

示例：在下一个未配置设备原来的地址空间中重新配置它。

```
*CONFIGURE
C=ID
% 取得下一个要配置的设备
?C=0 A
% 是否所有设备都已配置?
GOYES QUIT
A=C B
% 保存设备信息
C=0 B
% 清除设备信息，以取得大小或地址
CONFIG
?ABIT=0 7
% 这是否为大小信息?
GOYES CONFIGURE
% 如果是，则继续配置地址
*QUIT
```

66.4 RESET 后的 CONFIG

执行 RESET 后，HP 48 的默认配置是什么？下面举例。希望按以下方式配置一台 HP 48GX：

- 94. I/O 寄存器位于 #00100h。
- 95. 128KB 系统 RAM 位于 #80000h，大小 128KB。
- 96. 插槽 1 中的 128KB RAM 卡位于 #C0000h，大小 128KB。
- 97. 插槽 2 中的 512KB RAM 卡位于 #C0000h，大小 128KB。
- 98. 存储体切换器位于 #7F000h，大小 2KB。

这就是在端口 1 和端口 2 都插有卡时，HP 48GX 的默认映射。

```
RESET
% 取消配置所有设备
LC 00100
% I/O 寄存器地址
CONFIG
% HDW 的地址
LC C0000
% 128KB 系统 RAM
CONFIG
% NCE2 的大小
LC 80000
% 系统 RAM 地址
CONFIG
% NCE2 的地址
LC FF000
% 存储体切换器大小 2KB
CONFIG
% CE1 的大小
LC 7F000
% 存储体切换器地址
CONFIG
% CE1 的地址
LC C0000
% RAM 卡的 128KB 大小和地址
CONFIG
% CE2 的大小
CONFIG
```

```
% CE2 的地址
CONFIG
% NCE3 的大小
CONFIG
% NCE3 的地址
```

现在所有设备都已配置。执行 `C=ID` 时，`C(A)` 寄存器将返回 `#00000h`。

66.5 UNCNFG 示例

HP 48GX 的 ROM 在若干地址处会被覆盖。沿用上一示例的配置。在下一个示例中，希望从 ROM 的 `#FF000h` 读取五个半字节。该地址处的 ROM 被插槽 1 和插槽 2 中的 RAM 卡覆盖，因此必须从菊花链中移除内存控制器 CE2 和 NCE3。读取数据后，还要重新插入被移除的内存控制器。这里假设已经选择地址线 A19；存储体切换器一节会进一步说明。

```
LC C0000
% 128KB RAM 卡的地址
UNCNFG
% 取消配置插槽 1 中的卡，即 CE2
UNCNFG
% 取消配置插槽 2 中的卡，即 NCE3
D0= FF000
% 读取地址
A=DAT0 A
% 读取 ROM 内容
CONFIG
% 配置 CE2 的大小
CONFIG
% 配置 CE2 的地址
CONFIG
% 配置 NCE3 的大小
CONFIG
% 配置 NCE3 的地址
```

这很简单，但这里假定插入了两张 RAM 卡。如果不知道是否插有 RAM 卡，该怎么办？必须重写前面的示例。

```
LC C0000
% 128KB RAM 卡的地址

% 取消配置插槽 1 中的卡
D0= 8052F
% D0= (=CONFTAB)+#4
C=DAT0 S
?C=0 S
% 插槽 1 中是否有卡？
GOYES QUIT1
% 没有
UNCNFG
% 取消配置插槽 1 中的卡，即 CE2
*QUIT1

% 取消配置插槽 2 中的卡
D0= 80531
% D0= (=CONFTAB)+#6
C=DAT0 S
?C=0 S
% 插槽 2 中是否有卡？
GOYES QUIT2
% 没有
UNCNFG
% 取消配置插槽 2 中的卡，即 NCE3
*QUIT2

D0= FF000
% 读取地址
```

```

A=DAT0 A
% 读取 ROM 内容

% 重新配置所有未配置的卡
*CONFIGURE
C=ID
% 取得下一个要配置的设备
?C=0 A
% 是否所有设备都已配置?
GOYES QUIT
C=0 B
% 清除设备信息，以取得大小或地址
CONFIG
GOTO CONFIGURE
*QUIT

```

66.6 默认配置设置

66.6.1 HP 48S/SX 内存配置

地址范围	内容
#00000h 至 #000FFh	256 个半字节的 ROM
#00100h 至 #0013Fh	64 个半字节的 I/O RAM，覆盖 64 个半字节的 ROM
#00140h 至 #6FFFFh	约 223KB ROM
#70000h 至 #7FFFFh	32KB RAM，覆盖 32KB ROM
#80000h 至 #BFFFFh	插槽 1 中的 128KB 卡
#C0000h 至 #FFFFFFh	插槽 2 中的 128KB 卡；覆盖 2KB 未使用的 NCE3，即 #D0000h 至 #D0FFFh

66.6.2 HP 48G 内存配置

地址范围	内容
#00000h 至 #000FFh	256 个半字节的 ROM
#00100h 至 #0013Fh	64 个半字节的 I/O RAM，覆盖 64 个半字节的 ROM
#00140h 至 #7DFFFh	约 251KB ROM
#7E000h 至 #7EFFFh	空插槽 1，按 2KB 大小配置；空插槽 2 被空插槽 1 的 2KB 覆盖；为插槽 1 分配的 2KB 还覆盖 2KB ROM
#7F000h 至 #7FFFFh	存储体切换器，分配 2KB，覆盖 2KB ROM
#80000h 至 #8FFFFh	32KB RAM，覆盖 32KB ROM
#90000h 至 #FFFFFFh	ROM 的剩余部分，约 224KB

66.6.3 HP 48G+ 和 GX（无 RAM 卡）内存配置

地址范围	内容
#00000h 至 #000FFh	256 个半字节的 ROM
#00100h 至 #0013Fh	64 个半字节的 I/O RAM，覆盖 64 个半字节的 ROM
#00140h 至 #7DFFFh	约 251KB ROM
#7E000h 至 #7EFFFh	空插槽 1，按 2KB 大小配置；空插槽 2 被空插槽 1 的 2KB 覆盖；为插槽 1 分配的 2KB 还覆盖 2KB ROM
#7F000h 至 #7FFFFh	存储体切换器，分配 2KB，覆盖 2KB ROM
#80000h 至 #BFFFFh	128KB RAM，覆盖 128KB ROM
#C0000h 至 #FFFFFFh	128KB ROM，未被覆盖

66.6.4 插槽 1 中有 32KB 时的 HP 48GX 内存配置

地址范围	内容
#00000h 至 #000FFh	256 个半字节的 ROM
#00100h 至 #0013Fh	64 个半字节的 I/O RAM, 覆盖 64 个半字节的 ROM
#00140h 至 #7DFFFh	约 251KB ROM
#7E000h 至 #7EFFFh	空插槽 2, 按 2KB 大小配置; 为插槽 2 分配的 2KB 在此覆盖 2KB ROM
#7F000h 至 #7FFFFh	存储体切换器, 分配 2KB, 覆盖 2KB ROM
#80000h 至 #BFFFFh	128KB RAM, 覆盖 128KB ROM
#C0000h 至 #CFFFFh	插槽 1 中的 32KB 卡, 覆盖 32KB ROM
#D0000h 至 #FFFFFFh	96KB ROM, 未被覆盖

66.6.5 插槽 1 中有 128KB 时的 HP 48GX 内存配置

地址范围	内容
#00000h 至 #000FFh	256 个半字节的 ROM
#00100h 至 #0013Fh	64 个半字节的 I/O RAM, 覆盖 64 个半字节的 ROM
#00140h 至 #7DFFFh	约 251KB ROM
#7E000h 至 #7EFFFh	空插槽 2, 按 2KB 大小配置; 为插槽 2 分配的 2KB 在此覆盖 2KB ROM
#7F000h 至 #7FFFFh	存储体切换器, 分配 2KB, 覆盖 2KB ROM
#80000h 至 #BFFFFh	128KB RAM, 覆盖 128KB ROM

地址范围	内容
#C0000h 至 #CFFFFh	插槽 1 中的 128KB 卡, 覆盖 128KB ROM

66.6.6 插槽 1 和插槽 2 中均有 128KB 时的 HP 48GX 内存配置

地址范围	内容
#00000h 至 #000FFh	256 个半字节的 ROM
#00100h 至 #0013Fh	64 个半字节的 I/O RAM, 覆盖 64 个半字节的 ROM
#00140h 至 #7EFFFh	约 253KB ROM
#7F000h 至 #7FFFFh	存储体切换器, 分配 2KB, 覆盖 2KB ROM
#80000h 至 #BFFFFh	128KB RAM, 覆盖 128KB ROM
#C0000h 至 #CFFFFh	插槽 1 中的 128KB 卡, 覆盖插槽 2 的 128KB; 二者同时覆盖 128KB ROM

66.7 GX 存储体切换器及其他 ROM 内容

从 HP 48S 系列到 HP 48G 系列, 最大的变化是 ROM 容量翻倍, 以及插槽 2 可以使用最大 4MB 的存储卡。对于超过 128KB 的卡, 系统采用“存储体切换”技术, 把它们划分为每个 128KB 的端口。这里的“存储体”是什么? 在此意义上, 存储体是由多个设备共享的地址区域。与前面介绍的“覆盖技术”不同, 所有共享设备都使用同一地址区域。系统使用所谓的存储体切换器来切换这些存储体。插槽 2 中的存储卡被划分为若干 128KB 存储体。

HP 增加了一条地址线 A19, 用于更大的 ROM, 又增加了一个用于存储体切换器的内存控制器。为了节省芯片引脚, 地址线 A19 与 NCE3 控制线采用复用方式, 也就是共用同一个芯片引脚。选择 ROM 上部 256KB 时需要地址线 A19, 因此 NCE3 线变成了插槽 2 存储卡的“片选”线。

存储体切换器本身必须为插槽 2 的存储芯片生成五条高位地址线 A21-A17, 还必须生成使能信号 BEN, 以便在 A19/NCE3 引脚用作地址线 A19 时取消选择该存储器。它通过外部“锁存器”实现。从存储体切换器的内存区域读取某个值时, 地址 A6-A1 会被锁存。

地址线内容按以下方式保存:

输入地址线	锁存到
A1	A17
A2	A18
A3	A19
A4	A20
A5	A21
A6	BEN

如果 BEN=0，端口 2 始终禁用；如果 BEN=1，端口 2 由 NCE3 信号控制。

66.7.1 在 A19 与 NCE3 之间切换

这里假设 I/O 寄存器配置在 #00100h，存储体切换器配置在 #7F000h，系统 RAM 配置在 #80000h。用于在 A19 与 NCE3 之间切换的控制位 DA19 位于地址 #00129h 的第 3 位。问题在于，该寄存器在写入和读取时含义不同，因此不能从该地址读取数据、修改 DA19 位，再写回原处。

不过，系统会把上次写入的值保存在系统 RAM 的一个全局变量中。访问 DA19 和 BEN 时请注意次序。

示例：切换到上部 ROM 访问。

```
D0= 7F000
% BEN=0, 禁用端口 2
C=DAT0 B
D0= 8069A
% D0=(5)LINECOUNTg
C=DAT0 B
% 取得 LINECOUNT 的内容
CBIT=1 7
% DA19=1, 启用 ROM
D0= 00128
% D0=(5)LINECOUNT
DAT0=C B
```

示例：切换到端口 2 访问。

```
D0= 8069A
% D0=(5)LINECOUNTg
C=DAT0 B
% 取得 LINECOUNT 的内容
CBIT=0 7
% DA19=0, 禁用 ROM
D0= 00128
% D0=(5)LINECOUNT
DAT0=C B
D0= 7F040
% BEN=1, 启用端口 2
C=DAT0 B
```

66.7.2 选择存储体

现在已经了解硬件的工作方式，并知道可以通过从内存控制器读取来控制高位地址线。选择存储体的公式为：

```
#<memory_address> = #7F000 + #40 + 2 * n
```

其中，#7F000 是存储体切换器的基地址，#40 表示 BEN 信号置位，n 是端口 2 中所选的存储体编号，范围为 0 至 31。

示例：要选择一张 512KB 卡的存储体 2，应执行：

```
D1= 7F044
A=DAT1 B
```

读出的字节必须丢弃。

66.7.3 插槽 2 中使用 4MB 卡时的无效卡数据

在插槽 2 中装有 4MB 卡的 GX 上，总会出现消息 `Warning: Invalid Card Data`。这是 HP 48GX 固件中的错误。

下面对该错误的说明基于 Christoph Giesselink 的个人实验，与 HP 官方给出的解释相矛盾。

问题是：使用以下代码读取时，地址总线上最后出现的地址是什么？

```
D1= 7F044  
A=DAT1 B
```

该地址内容会被锁存。读取一个字节要读两个半字节，因此上述代码读取 #7F044 和 #7F045 处的半字节。由于芯片内部地址总线的 A0 没有引出，可以忽略读取地址的最低有效位。因此，锁存的应当是地址 #7F044。

但实验表明，在该地址执行字节读取时，实际读取了地址 #7F044、#7F045 和 #7F046。因此锁存的是 #7F046，而这正是下一个存储体的地址。

这与 4MB 卡有什么关系？请记住，要选择存储体 31，公式为：

```
#7F000 + #40 + 2 * 31 = #7F07E
```

执行字节读取访问时，锁存的地址是 #7F080。但在该地址上 BEN=0，所以不再选择插槽 2 中的 RAM。

那么，为什么 256KB 卡等可以正常工作？256KB 卡有两个 128KB 存储体，而且只使用锁存地址总线中的 A17。

尝试访问存储体 0：

```
D1= 7F040  
A=DAT1 B
```

锁存 #7F042：BEN=1、A17=1，因此选择第二个存储体。

尝试访问存储体 1：

```
D1= 7F042  
A=DAT1 B
```

锁存 #7F044：BEN=1、A17=0，因此选择第一个存储体。

一切都能正常工作，因为系统使用同一种方式读写存储卡数据。

能否利用这些信息向 4MB 卡的存储体 31 写入数据？已知存储体 0 始终保存在第二个 128KB 区域，而最后一个存储体保存在第一个 128KB 区域。由于 BEN 信号不存在，无法读取 #7F07E。但可以使用以下代码：

```
D1= 7F03E  
A=DAT1 B
```

此时锁存 #7F040：BEN=1，A17-21=0，因而选择第一个存储体。

那么，能否通过写入而不是读取来启用某个存储体？这是个困难的问题，答案取决于插槽 2 存储卡上写保护开关的状态。

要理解这一点，必须回顾 S 系列计算器的历史。在 SX 计算器上，两个插槽连接到 Clarke MMU（内存管理单元）的 CE1 和 CE2。每个输出引脚都由一个特殊输入引脚控制，该输入连接到存储卡的写保护开关。如果 MMU 检测到卡处于“写保护”状态，尝试写入存储卡时就不会产生片选信号，但读取仍能正常工作。

出于某种原因，Hewlett Packard 在 G 系列中把存储卡移到了 CE2 和 NCE3。但 NCE3 片选输出引脚不受这种用于阻止写入的特殊输入引脚控制。因此，GX 存储卡必须在卡上配备专门的“写保护”控制逻辑。不过，操作系统仍应知道插槽 2 中的卡是否处于写保护状态。插槽 2 存储卡的写保护开关连接到 CE1 的空闲控制输入。在 GX 上，存储体切换器触发器连接到 CE1，因此插槽 2 存储卡的写保护开关会控制对存储体切换器触发器的写入。

前面已经讨论读取条件，下面看看向存储体切换器写入时会发生什么。

如果存储卡未处于“写保护”状态，就会锁存预期值。

示例：

```
D1= 7F040  
DAT1=A B
```

字节写入命令会写入 #7F040 和 #7F041 处的两个半字节，因此锁存 #7E041。

如果存储卡处于“写保护”状态，就无法写入锁存器。请记住，Saturn CPU 使用四位数据总线，而外部模块使用八位数据总线，因此 Clarke/Yorke 芯片内部有一个专门单元，负责这种半字节与字节之间的转换。如果要向外部存储设备写入两个半字节，而且这两个半字节位于同一字节地址，只需一个写周期即可保存数据。但如果只想修改这种按字节寻址存储器中的一个半字节，控制器就必须执行读取/修改/写入周期：读取字节，修改其中一个半字节，再把修改后的字节写回。由于存储体切换器处于“写保护”状态，写周期被禁用，但读周期仍会修改锁存器内容。被锁存的地址是执行半字节读取时的最高地址。

示例：

```
D1= 7F040  
DAT1=A B
```

这段代码不起作用；因为没有读周期，锁存数据不会发生变化。

示例：

```
D1= 7F041  
DAT1=A B
```

这段代码可以工作；因为修改的是 #7E042、#7E043 字节中的半字节，所以锁存 #7E042。
可以看到，写入比读取带来更多问题，因此请使用已公布的公式，通过字节读取实现存储体切换。

67 HP 49 内存管理

67.1 概述

HP 49G 使用 Intel TE28F160-S5100 2MB 闪存芯片。HP 49G+ 最初使用 Silicon Storage Technology SST39VF160-70-4C-EK 2MB 闪存芯片，但该芯片于 2003 年 12 月 8 日结束生命周期，之后是否改用其他芯片尚不清楚。两种计算器都使用 512KB SRAM 芯片；HP 49G+ 上曾见到 BSI BS616LV4010EC-70，HP 49G 上曾见到 Samsung KM684000BLT-5 和 Hyundai HY628400。

从程序员的角度看，具体使用哪种芯片并不重要；只需知道闪存 ROM 为 2MB，RAM 为 512KB。闪存 ROM 在内部划分为 64KB 存储体，但对程序员而言，这一细节大多被抽象为 128KB 存储体。

与 HP 48 一样，HP 49 仍受 20 位寻址限制，因此 Saturn 在任一时刻只能看到 512KB。这意味着 HP 49 的内存管理与 HP 48 有许多共同点。为了处理总共 2.5MB 的内存，HP 49 需要比 HP 48 更高级的存储体切换系统，但内存仍划分为 128KB 存储体。

由于存储体大小为 128KB，闪存 ROM 被划分为 16 个存储体。第一个存储体又分为两半。前 64KB 保留为“引导扇区”，它是只读的，HP 49G 通过硬件强制保证这一点。第二个 64KB 在 ROM 版本 1.18 及以下可供用户存储，在更高 ROM 版本中则由系统用于 CAS 的一部分。其余 15 个 128KB 存储体有些由系统使用，有些可供用户使用，具体数量取决于 ROM 版本。

RAM 分为两个端口。第一个端口，即端口 0，与 HOME 目录共享，大小为 256KB。第二个端口，即端口 1，不共享。在 HP 49G 上，它同样为 256KB。在 HP 49G+ 上，端口 1 只有 128KB，因为底层

操作系统 Kinpo OS 和仿真层 Saturn emulator 使用剩余的 128KB。这部分保留内存中有一些可供 ARM 程序员使用，但不在本书讨论范围内。

HP 49G+ 操作系统还会占用两个额外的 128KB 闪存存储体，因此可供用户使用的闪存存储体更少。

下表显示“传统”ROM 上 ROM 与 RAM 存储体的用途。这里所说的传统 ROM 是指：1.20 以下版本仅用于 HP 49G，1.20 及更高版本仅用于 HP 49G+。

ROM 版本	ROM 存储体 0	系统 ROM 存储体	用户 ROM 存储体	RAM 端口 0	RAM 端口 1
1.05 - 1.18	一半引导区 / 一半用户区	接下来的 7 个存储体	最后 8 个存储体	2 个存储体	2 个存储体
1.19 Beta 6	一半引导区 / 一半系统区	接下来的 7 个存储体	最后 8 个存储体	2 个存储体	2 个存储体
1.20 +	一半引导区 / 一半系统区	接下来的 8 个存储体	最后 7 个存储体	2 个存储体	1 个存储体

可以看到，HP 49 中总共有 20 个内存存储体，包括 ROM 和 RAM，合计正是前面提到的 2.5MB。任一时刻只能看到 4 个存储体，因此系统必须根据当时的需要，把所需存储体选择性映射到这四个可寻址存储体之一。

在 HP 49G 的 .flash 闪存更新文件中，每个存储体都有自己的名称。内置 ROM 更新工具使用这些名称，决定把传输来的各存储体写入闪存的哪个位置。ROM 存储体 0 的前半部分称为 Part0，后半部分称为 FS。第一个完整的 128KB 存储体称为 System，其余存储体依次称为 Part1、Part2、Part3、Part4、Part6、Part7 等。请注意，没有名为 Part5 的存储体。

用表格最能清楚表示内存的寻址方式。表中每一行表示一个 128KB 可寻址空间存储体，四个存储体合计达到 Saturn 的 512KB 上限。

地址范围	映射内容
#00000h 至 #3FFFFh	映射到闪存 ROM 前四个存储体之一
#40000h 至 #7FFFFh	映射到任意一个闪存 ROM 存储体，或 RAM 端口 1 的任意一个存储体
#80000h 至 #BFFFFh	通常映射到 RAM 端口 0 的第一个存储体
#C0000h 至 #FFFFFFh	通常映射到 RAM 端口 0 的第二个存储体

可以看到，在任一时刻，ROM 前四个存储体之一始终处于映射状态，而 RAM 端口 0 的两个存储体通常也都处于映射状态。ROM 的最后 12 个存储体，或者 ROM 前四个存储体中尚未映射的另外三个，都必须共享第二段地址空间。

67.2 硬件细节

HP 49G 使用与 HP 48G 系列相同的第四代 Saturn 芯片，代号 Yorke。为了更好地理解后续各节，应先阅读“HP 48 内存管理”一节的基础内容。这里使用名称相同的内存控制器 NCE1、HD、NCE2、CE1、CE2 和 NCE3，但第一个控制器在菊花链配置中的顺序例外。

从软件角度看，HP 49G+ 与 HP 49G 相似；但底层硬件不同，尽管仿真层对其进行了抽象，本节内容仍并非全部适用。Christoph Giesselink 慷慨地编写了本节余下内容。

67.3 A19 与 NCE3 共用

从 HP 48 的说明中已经了解到，A19 和 NCE3 硬件信号在 Yorke 芯片上使用同一连接端。该引脚的行为由 `LINECOUNT` 寄存器中的 DA19 位控制。为避免 HP 49G 出现问题，这个共用引脚

始终用作内存控制器 NCE3，而从不用作地址线 A19。这也意味着单个模块的最大可寻址大小为 256KB，即 $2^{19}/2$ 。

67.4 内存控制器

设备	HP 49G 上的功能
NCE1	闪存芯片，只读，2MB
HDW	I/O 寄存器，32B
NCE2	RAM，256KB
CE1	存储体切换器
CE2	RAM，128KB
NCE3	RAM / 闪存芯片，可读写，128KB

67.4.1 NCE1（闪存芯片）

这是 2MB 闪存芯片的内存控制器。由于该控制器是为 ROM 访问而设计的，无法通过它执行写入。为了让芯片完整的 2MB 容量能够出现在 Saturn 芯片的 512KB 地址区域内，这 2MB 被划分为 16 个 128KB 存储体，由存储体切换器控制。

地址范围	内容
#00000h 至 #3FFFFh	前四个 128KB 存储体之一
#40000h 至 #7FFFFh	十六个可寻址 128KB 存储体之一
#80000h 至 #BFFFFh	#00000h 至 #3FFFFh 区域的镜像
#C0000h 至 #FFFFFFh	#40000h 至 #7FFFFh 区域的镜像

请记住，这是优先级最低的控制器，任何其他内存控制器都可以覆盖它。

67.4.2 HDW

I/O 寄存器控制器。

67.4.3 NCE2（RAM）

该控制器管理 512KB RAM 中的一块 256KB 区域。包含 `HOME` 目录和端口 0 数据的 RAM 部分通常配置在 Saturn 地址区域的 #80000h 至 #FFFFFFh。

67.4.4 CE1（存储体切换器）

存储体切换器像 HP 48GX 一样，由六位锁存器实现。与 HP 48GX 的存储体切换器一样，使用读取命令设置时也会出现写入错误数据的异常行为。与 HP 48GX 不同的是，HP 49G 允许用写入命令设置存储体切换器锁存器，而且会按预期工作。这是因为 HP 49G 上相应的存储卡检测线设置为可读写。

六位锁存器的低两位在 #00000h 至 #3FFFFh 地址区域内选择四个，即 2^2 个闪存存储体之一。高四位在 #40000h 至 #7FFFFh 地址区域内控制十六个，即 2^4 个可能的闪存存储体。六位锁存器连接到地址线 A1-A6。

A1	→ 位 0，选择 #00000h - #3FFFFh 区域的存储体
A2	→ 位 1，选择 #00000h - #3FFFFh 区域的存储体
A3	→ 位 0，选择 #40000h - #7FFFFh 区域的存储体
A4	→ 位 1，选择 #40000h - #7FFFFh 区域的存储体
A5	→ 位 2，选择 #40000h - #7FFFFh 区域的存储体

A6 → 位 3, 选择 #40000h - #7FFFFh 区域的存储体

与 HP 48 系列不同, 该控制器默认不配置。

示例: 把 #00000h 设置为闪存存储体 2, 把 #40000h 设置为闪存存储体 12, 编号从零开始。

```
LC FF000
% 2KB
CONFIG
% 设置 CE1, 即存储体切换器的大小
LC 3F000
% 地址
CONFIG
% 设置 CE1, 即存储体切换器的设备地址
LC 64
% 1100100b
% 1100 10 0
% =12 =2 =地址线 A0
D0=C
DAT0=A B
% 用存储体 12 和存储体 2 的信息设置锁存器
UNCNFG
% 取消配置 CE1, 即存储体切换器
```

67.4.5 CE2 (RAM)

该控制器管理 512KB RAM 中的一块 128KB 区域, 并包含端口 1 数据的一半。默认情况下它未配置。

67.4.6 NCE3 (RAM / 闪存芯片)

该控制器管理 512KB RAM 的最后一块 128KB 区域, 并包含端口 1 数据的另一半。默认情况下它也未配置。

NCE3 的另一个用途是允许写入连接到 NCE1 的闪存芯片。因此, 必须设置 LCR 寄存器中的 LED 位。

示例: 准备 NCE3, 以便在 #40000h 处写入闪存。

```
LC FF000
% 2KB
CONFIG
% 设置 CE1, 即存储体切换器的大小
LC 3F000
% 地址
CONFIG
% 设置 CE1, 即存储体切换器的设备地址
LC FF000
% 2KB
CONFIG
% 设置 CE2, 即 RAM 1 的大小
LC 3E000
% 地址
CONFIG
% 设置 CE2, 即 RAM 1 的设备地址
LC C0000
% 128KB
CONFIG
% 设置 NCE3, 即 ROM 的大小
LC 40000
% 地址
CONFIG
% 设置 NCE3, 即 ROM 的设备地址
D1= 0011C
% 装入 LCR, 即 LED Control Register
LC 8
% 打开 LED
DAT1=C 1
```

执行这段代码后，就能在 #40000 至 #7FFFF 的内存区域中，对由存储体切换器高四位选定的 128KB 闪存芯片存储体进行完整的读写访问。请记住，闪存芯片不能像普通 RAM 一样使用。向芯片写入时，必须采用制造商专用的特殊命令集，这不属于本教程的内容。

67.5 恢复存储体切换器的内容

示例：把存储体切换器的内容恢复为操作系统保存的值。

```
LC FF000
% 2KB
CONFIG
% 设置 CE1，即存储体切换器的大小
LC 3F000
% 地址
CONFIG
% 设置 CE1，即存储体切换器的设备地址
D0= 860B8
% =CurROMBank2，取得上次写入的 BS 地址 [A]
C=DAT0 B
% CE1，即存储体切换器的地址 3F0xxH
D0=C
% 重新装入 BS 地址
DAT0=C B
% 执行一次虚拟写入，重新装入 BS
UNCNFG
% 取消配置 CE1，即存储体切换器
```

68 图形

衷心感谢 Christoph Giesselink 编写本节。请注意，本节中的部分数值假定屏幕高度为 64 像素。

68.1 概述

采用 Clarke 或 Yorke 芯片的计算器，包括 38G、39G/40G、48S/SX/G/G+/GX 和 49G，使用分辨率为 131×64 像素的液晶显示器 LCD。LCD 由 Clarke/Yorke 芯片和两个商用列驱动芯片控制。

频率为 4096 Hz 的中断每次重绘下一显示行。共有 64 行，由 4096 Hz 中断刷新，因此整个屏幕的刷新率为 $4096 \text{ Hz} / 64 \text{ 行} = 64 \text{ Hz}$ 。每隔 244 μs 刷新一条显示线，每次大约需要 22-23 μs 。系统采用 UMA (Unified Memory Architecture，统一内存架构) 布局，也就是 CPU 和显示控制器的 RAM 不分离。优点是 CPU 能完全控制屏幕，无需把数据从 CPU 复制到显示内存。缺点是刷新屏幕时必须暂停 CPU，以保证显示控制器能够访问 RAM，这会降低 CPU 速度。

屏幕由若干 I/O 寄存器控制。本节假定 I/O 寄存器映射到 #00100，这是通常设置。屏幕只使用图形对象 GROB，也就是说，显示内存中的每一位都代表屏幕上的一个像素。

示例：把最后更新行的编号读入 A(B)。

```
LC 3F
% 6 个行计数位的掩码
D0=(5) "LINECOUNT"
% =LINECOUNT 位于 00128
A=DAT0 B
% 取得最后更新的行
A=A&C B
% 清除 DA19 和 M32 位
```

该行计数器是递减计数器。最上方行的编号为 #3Fh，最下方行的编号为 #0h。这段代码主要用于同步输出。

示例：等待最后一行绘制完成。

```
LC 3F
% 6 个行计数位的掩码
D0=(5) "LINECOUNT"
% =LINECOUNT 位于 00128
*LOOP
A=DAT0 B
% 取得最后更新的行
A=A&C B
```

```
% 清除 DA19 和 M32 位
?A#0 B
% 不为零?
GOYES LOOP
% 则等待
```

68.2 主显示区 / 菜单显示区

硬件把显示区域分为两部分：“主显示”区域和“菜单显示”区域。这两个区域有什么区别？屏幕上部是主显示区。该区域中的 GROB 数据可以大于或小于实际可见区域。无需改变 GROB 数据，只要重新编程显示控制器，就能移动显示内容。屏幕底部的菜单区宽度固定为 131 像素。这种设计对 HP 计算器非常有用，因为可以滚动或修改主显示内容，而不影响菜单区。

通过 `LINECOUNT` 寄存器，可以配置主显示区和菜单显示区各使用多少行。由于内部原因，主显示区不能设置为零行。把 `#0` 或 `#1` 作为 `=LINECOUNT` 参数，与使用 `#3F` 效果相同，都会让主显示区使用全部 64 行。

示例：把“主显示”区域设为 56 行高，把“菜单显示”区域设为 8 行高。

```
D0=(5) "LINECOUNT"
% =LINECOUNT 位于 00128
LC 37
% 主显示区 56 行
DAT0=C B
```

可以看到，必须编程的是“主显示”区域的最后一行，因此不可能完全取消“主显示”区域。但把 63 写入 `LINECOUNT` 寄存器会清除“菜单显示”区域。

显示控制器的所有地址都按字节对齐。这意味着每条显示线都从偶数地址开始。每个显示区都有各自的基地址，指向一个 GROB 数据字段，因此 GROB 数据字段必须从偶数地址开始。主显示区的基地址位于 `#00120h-#00124h`，即 `=DISP1CTL`；菜单显示区的基地址位于 `#00130h-00134h`，即 `=DISP2CTL`。这些寄存器为 W/O（只写），必须指向 RAM 中显示 GROB 的开头。显示 GROB 只是普通 GROB 对象的数据字段，用于表示像素状态。

因为这些寄存器和另外两个显示寄存器都是“只写”的，操作系统会在 RAM 中使用所谓的“影子”寄存器，保存上次写入的值，以便恢复原内容。假定采用默认 RAM 布局，影子寄存器的地址如下：

寄存器	影子名称	38G	48SX	39G/40G/48Gx/49Gx
<code>=DISP1CTL #00120</code>	<code>=DISP1CTLg</code>	<code>#F062C</code>	<code>#7050E</code>	<code>#8068D</code>
<code>=LINENIBS #00125</code>	<code>=LINENIBSg</code>	<code>#F0631</code>	<code>#70513</code>	<code>#80692</code>
<code>=LINECOUNT #00128</code>	<code>=LINECOUNTg</code>	<code>#F0639</code>	<code>#7051B</code>	<code>#8069A</code>
<code>=DISP2CTL #00130</code>	<code>=DISP2CTLg</code>	<code>#F0634</code>	<code>#70516</code>	<code>#80695</code>

并非所有入口点表都包含这些入口点，因此即使已经安装入口点表，也可能仍需手工输入地址。

示例：恢复“主显示”区域指针。

```
D0=(5) "DISP1CTLg"
% =DISP1CTLg 在 HP 48G 系列和 HP 49 上位于 8068D
C=DAT0 A
D0=(5) "DISP1CTL"
% =DISP1CTL 位于 00120
DAT0=C A
```

68.3 主显示区域设置

“主显示”区域中的 GROB 尺寸可能与屏幕尺寸不同，因此必须调整显示控制寄存器，才能得到正确画面。

控制主显示区有三个主要寄存器：

地址	名称	功能
<code>#00120h - #00124h</code>	<code>=DISP1CTL</code>	显示起始地址（W/O）
<code>#00100h</code>	<code>=BITOFFSET</code>	GROB 左边距（R/W）
<code>#00125h - #00127h</code>	<code>=LINENIBS</code>	GROB 右边距（W/O）

`DISP1CTL` 定义 GROB 位图案的基地址。请记住，起始地址必须为偶数。使用 `BITOFFSET` 寄存器的低三位 `[OFF2 OFF1 OFF0]`，最多可以把 GROB 向左滚动 7 像素，即 2^3-1 。

示例：把“主显示”区域内容向左滚动 2 像素。

```

D0=(5) "BITOFFSET"
% =BITOFFSET 位于 00100
LC A
% DON 位 + 左移 2 像素
DAT0=C 1

```

怎样才能把显示内容滚动超过 7 像素？很简单。要滚动 8 像素，只需把屏幕基地址加 2，因为 $2 * 1 \text{ 半字节} = 2 * 4 \text{ 位} = 8 \text{ 位} = 8 \text{ 像素}$ 。

示例：把“主显示”区域内容向左滚动 13 像素，即 8+5 像素。

```

D0=(5) "DISP1CTLg"
% =DISP1CTLg 在 HP 48G 系列和 HP 49 上位于 8068D
C=DAT0 A
% 请记住，必须使用影子副本
C=C+1 A
% 8 像素
C=C+1 A
D0=(5) "DISP1CTL"
% =DISP1CTL 位于 00120
DAT0=C A
D0=(5) "BITOFFSET"
% =BITOFFSET 位于 00100
LC D
% DON 位 + 左移 5 像素
DAT0=C 1

```

现在还必须调整 LINENIBS 寄存器，以设置 GROB 的右边距。要显示一条 131 像素的行，必须计算每行需要多少半字节。

```

半字节数 = (CEIL(131 / 8) * 8) / 4 = 34

```

每条显示线必须预留 34 个半字节。为什么是 34 而不是 33，即 $\text{CEIL}(131 / 4)$ ？如前所述，显示地址按字节对齐，因此必须用像素数 131 除以一个字节中的位数，也就是像素数 8。

```

131 / 8 = 16.375

```

当然不能预留 0.375 字节，所以总共必须预留 17 字节。

```

CEIL(131 / 8) = 17 字节

```

17 字节可以控制 136 个像素，即 $17 * 8$ ，所以每行必须预留 34 个半字节，即 $136 / 4$ 。但这里又出现一个问题：当显示内容向左滚动超过 5 像素，即 $136 - 131$ 时，为了正确显示整行，需要更多“额外”半字节，因为此时行宽已经超过预留的 136 像素，例如可见 131 像素再加向左移动 6 像素。以下公式考虑了这一点。

```

下一行地址 = 当前行地址 + 34 + 右边距 + FLOOR(左边距 / 4)

```

其中：

```

下一行地址 - 当前行地址 = 显示线的半字节数

```

对于本例中宽 131 像素且左边距不移动的显示线，右边距寄存器应装入：

```

LINENIBS = 右边距
           = 下一行地址 - 当前行地址 - 34 - FLOOR(左边距 / 4)
           = 34           - 34           - FLOOR(0 / 4)
           = 0

```

示例：配置 $159 * 64$ 的 GROB，并向左滚动 3 像素。

```

半字节数 = (CEIL(159 / 8) * 8) / 4 = 40

```

```

LINENIBS = 40 - 34 - FLOOR(3/4) = 6

```

```

D0=(5) "BITOFFSET"
% =BITOFFSET 位于 00100
LC B
% DON 位 + 左移 3 像素
DAT0=C 1
D0=(5) "LINENIBS"
% =LINENIBS 位于 00125
LC(3) 6
% 右边距
DAT0=C 3

```

示例：仍配置 $159 * 64$ 的 GROB，但这次向左滚动 4 像素。

```
半字节数 = (CEIL(159 / 8) * 8) / 4 = 40
```

```
LINENIBS = 40 - 34 - FLOOR(4/4) = 5
D0=(5) "BITOFFSET"
% =BITOFFSET 位于 00100
LC(1) B
% DON 位 + 左移 3 像素
DAT0=C 1
D0=(5) "LINENIBS"
% =LINENIBS 位于 00125
LC(3) 5
% 右边距：这里使用 #4 也可以
DAT0=C 3
```

由于显示控制器接口按字节对齐，内部按下式计算下一行地址：

```
下一行地址 = 当前行地址 + (34 + 右边距 + FLOOR(左边距 / 4) * 2) & #FFE
```

切勿使用该公式计算右边距；它只用于演示。因此，上一示例中的右边距也可以使用 #4。例如，HP 函数 `=WINDOWRIGHT` 就利用了这种行为。当 `BITOFFSET` 的值从 3 变为 4 时，它不会重新计算右边距值，而是始终在此处把右边距值减 2。

下面还有一个来自 Jean-Yves Avenard 的巧妙示例，用于把屏幕中的“主显示”区域上下翻转：

```
LINENIBS = 右边距
          = 下一行地址 - 当前行地址 - 34 - FLOOR(左边距 / 4)
          = -34          - 34          - FLOOR(0 / 4)
          = -68
          = #FBC (三半字节宽)
```

程序：

```
GOSBVL =SAVPTR
% =SAVPTR 位于 0679B
D0=(5) "DISP1CTLg"
% =DISP1CTLg 在 HP 48G 系列和 HP 49 上位于 8068D
A=DAT0 A
LC FBC0074E
% #0074E = 1870 = 55 * 34，即主显示区末尾的偏移量
C=C+A A
D0=(5) "DISP1CTL"
% =DISP1CTL 位于 00120
DAT0=C 8
GOVLNG =GETPTRLOOP
% =GETPTRLOOP 位于 05143
```

68.4 菜单显示区域设置

“菜单显示”区域中的 GROB 宽度必须始终为 131 像素，即 34 个半字节，因为无法调整其左右边距。y 方向尺寸通过写入 `LINECOUNT` 寄存器控制。

控制菜单显示区有一个主要寄存器：

地址	名称	功能
#00130h - #00134h	=DISP2CTL	菜单起始地址（W/O）

`DISP2CTL` 定义 GROB 位图案的基地址。请记住，起始地址必须为偶数。

68.5 显示 I/O 寄存器概览

本节按数值顺序说明显示 I/O 寄存器的各个位。

```
(R/W) = 读/写
(R/O) = 只读
(W/O) = 只写
```

#00100h =BITOFFSET (R/W)

位 3 `[DON]`

这是屏幕使能标志；必须设置该位，才能启用屏幕刷新。清除该位后，屏幕关闭，CPU 也不再受 4096 Hz 刷新中断影响。如果想提高计算器速度，只需关闭屏幕。

位 2-0 `[OFF2 OFF1 OFF0]`

这些位控制主显示区的左边距。该数值是屏幕的垂直偏移量。如果要滚动超过七个像素，就必须把主显示区的基址地址增加二。当位 2 置位，也就是使用大于三的垂直偏移量时，还必须调整右边距值。

#00101 =CONTRAST (R/W)

位 3-0 `[CON3 CON2 CON1 CON0]`

这是五位对比度值的低四位。数值越大，对比度越深。

#00102 =DTEST (R/W)

位 3-1 `[VDIG LID TRIM]`

内部控制位，应当全部为 0。

位 0 `[CON4]`

这是五位对比度值的最高位。

#00103 =DSPCTL (W/O)

位 3-0 `[LRT LRTD LRTC BIN]`

内部控制位，应当全部为 0。随意试验这些控制位可能损坏屏幕。请务必注意。

#0010B =ANNCTRL (R/W)

位 3-1 `[LA4 LA3 LA2 LA1]`

这是六位指示符值的低四位。每一位表示一个指示符；0 表示关闭，1 表示打开。

```
LA4 = Alert
LA3 = Alpha
LA2 = Right Shift
LA1 = Left Shift
```

#0010C =ANNCTRL+1 (R/W)

位 3 `[AON]`

这是指示符使能标志。设置该位即可启用指示符。指示符与屏幕 DON 位相互独立，因此可以关闭屏幕，同时继续使用指示符。

位 2 `[XTRA]`

该位未使用。

位 1-0 `[LA6 LA5]`

这是六位指示符值的高两位。

```
LA6 = Transmitting
LA5 = Busy
```

示例：打开 Alpha 指示符，并关闭其他所有指示符。

```
D0=(5) "ANNCTRL"
LC 84
% AON 或 LA3
DAT0=C B
```

#00120 =DISP1CTL (W/O)

这是五半字节的主显示起始地址，最低有效位在前。更多信息请参阅第 68.2 节和第 68.3 节。

#00123 =LINENIBS (W/O)

这是三半字节的主显示右边距配置寄存器，为有符号值，最低有效位在前。更多信息请参阅第 68.3 节。

#00128 =LINECOUNT (W/O)

位 3-0 `[LC3 LC2 LC1 LC0]`

这是六位屏幕高度值的低四位。更多信息请参阅第 68.2 节。

#00128 =LINECOUNT (R/O)

位 3-0 `[LC3 LC2 LC1 LC0]`

这是六位行绘制值的低四位。更多信息请参阅第 68.1 节。

#00129 = LINECOUNT+1 (W/O)

位 3 `[DA19]`

地址线 19 控制。请保持该位不变。

位 2 `[M32]`

未使用。请保持该位不变。

位 1-0 `[LC5 LC4]`

这是六位屏幕高度值的高两位。更多信息请参阅第 68.2 节。

#00129 = LINECOUNT+1 (R/O)

位 3 `[DA19]`

地址线 19 控制值。

位 2 `[M32]`

未使用。

位 1-0 `[LC5 LC4]`

这是六位行计数器值的高两位。更多信息请参阅第 68.1 节。

#00130 = DISP2CTL (W/O)

这是五半字节的菜单显示起始地址，最低有效位在前。更多信息请参阅第 68.2 节和第 68.4 节。

目前就介绍到这里。希望您喜欢本书，也请期待未来的第 4 版。

69 延伸阅读

本书旨在全面介绍汇编语言编程，但它仍不能作为您唯一的编程资料来源。大多数用汇编语言编写的程序仍会包含一些 System RPL 或 User RPL 代码来完成某些操作，因此掌握这些语言同样重要。此外，多年来还有其他汇编语言教程问世；如果您对本书中的某个主题感到困惑，它们可以提供不同视角。下面汇总了一份推荐的延伸阅读清单。

69.1 内部开发工具

前文已经提到过这份资料，但它足够重要，值得再次说明。了解如何在 HP 49 上使用 MASD 的最佳资料，是 MASD 程序员 Cyrille de Brébisson 编写的文档。虽然其中有些部分不太容易理解，但它全面介绍了 MASD 的全部独特功能。

<http://www.hpcalc.org/details.php?id=2986>
→ <http://www.hpcalc.org/hp49/docs/programming/masddocs.zip>

HP 49 还内置了库创建器；开发库一节曾作简要介绍，以下文档则提供了更完整的说明：

<http://www.hpcalc.org/details.php?id=2980>
→ <http://www.hpcalc.org/hp49/docs/programming/crlib.zip>

库创建器还可以扩展内置菜单，以下文档对此作了说明：

<http://www.hpcalc.org/details.php?id=2981>
→ <http://www.hpcalc.org/hp49/docs/programming/extprg.zip>

编写自己的程序并把它打包成库时，可能希望加入可通过帮助目录访问的文档。库创建器支持这一功能，以下文件提供了说明：

<http://www.hpcalc.org/details.php?id=2979>
→ <http://www.hpcalc.org/hp49/docs/programming/addhelp.zip>

69.2 汇编语言

HP 最初编写的 SASM 文档介绍了如何使用 HP 语法为 Saturn 编程，以下位置提供文本格式版本：

<http://www.hpcalc.org/details.php?id=1746>
→ <http://www.hpcalc.org/hp48/docs/programming/sasm.zip>

Matthew Mastracci 编写了一份全面介绍 Saturn 架构和 HP 48 硬件的文档，可从以下地址取得纯文本版本：

<http://www.hpcalc.org/details.php?id=1753>

→ <http://www.hpcalc.org/hp48/docs/programming/saturnd.zip>

Peter Geelhoed 提供了大量用汇编语言编写的简单示例程序，所有学习 Saturn 汇编的人都会发现它很有用。以下位置提供 PDF 版本：

<http://www.hpcalc.org/details.php?id=5007>

→ <http://www.hpcalc.org/hp49/docs/programming/mltut.zip>

69.3 System RPL

HP 最初编写的 [RPLMAN](#) 文档介绍了如何用 System RPL 为 HP 48 编程，以下位置提供文本格式版本：

<http://www.hpcalc.org/details.php?id=1743>

→ <http://www.hpcalc.org/hp48/docs/programming/rplman.zip>

前文还提到过 Eduardo Kalinowski 和 Carsten Dominik 的优秀著作 *Programming in System RPL*。它以 PDF 形式提供完整的 System RPL 编程教程和参考资料：

<http://www.hpcalc.org/details.php?id=5142>

→ http://www.hpcalc.org/hp49/docs/programming/progsysrpl_pdf.zip

69.4 User RPL

HP 48 和 HP 49 系列随附的使用说明书很好地介绍了 User RPL，但另有大量更深入的资料。

HP 的 *48G Series Advanced User's Reference* 已经绝版，但仍可下载。它是一份接近 40MB、超过 700 页的大型 PDF 文档，涵盖 User RPL 编程的各个方面，并收录全部 User RPL 命令的参考资料。

<http://www.hpcalc.org/details.php?id=6036>

→ <http://www.hpcalc.org/hp48/docs/misc/hp48gaur.zip>

HP 的 *49G Advanced Users Guide* 虽然不如 HP 48G 的 AUR 易读，但提供了 HP 49 新增 User RPL 命令的参考资料，可从以下位置下载 PDF 版本：

<http://www.hpcalc.org/details.php?id=2998>

→ http://www.hpcalc.org/hp49/docs/misc/49g_aug.zip

One-Minute Marvels 由 Richard Nelson 和 Wlodek Mier-Jedrzejowicz 编写，收录 100 个 User RPL 程序，每个程序只需一分钟即可输入。对于想学习 User RPL 技巧的人，这是一套很好的示例。PDF 文档位于：

<http://www.hpcalc.org/details.php?id=1691>

→ <http://www.hpcalc.org/hp48/docs/programming/lminmarv.zip>

70 索引

- 绝对地址, 29, 82
加法, 14, 47, 48, 49
可寻址空间, 33
ALU, 39
指示符, 44
ASM Flash, 24
存储体切换器, 127, 133
BCD, 20, 20, 80, 86
大端序, 34
二进制, 10, 11, 12
二进制编码十进制: 见 BCD
移位, 57, 58, 58
布尔, 21
错误
 4MB 卡, 134
 进位, 49
 R=IN, 78
总线, 81
存储卡, 128
循环处理, 36
菊花链, 127
递减, 50
设备, 127
指令控制符, 26, 29
除法, 17, 52
入口, 30, 40
 adrKEYBUFFER, 115, 115, 116
 AINRTN, 78
 AllowIntr, 81
 ANNCTRL, 44, 45
 argtypeerr, 120
 CINRTN, 78, 113, 114, 115, 116
 DisableIntr, 80
 DIV5, 117, 117
 DOREAL, 119
 Errjmp, 89, 122, 124
 GETPTR, 91, 93, 123, 126
 GETPTRLOOP, 120, 124, 142
 GETTEMP, 90, 92
 GPErrjmpC, 120
 MAKE\$N, 123, 125
 MOVEDOWN, 123, 125
 POP1%, 120
 PUSH%, 121
 PUSH%LOOP, 120
入口点: 见“入口”
扩展 BCD, 21
字段, 33, 34, 35
 A, 34
 B, 35
 F1-F7: 见“用户定义”
 M, 35
 S, 35
 用户定义, 36
 W, 34
 WP, 36
 X, 35
 XS, 35
标志, 37
 进位, 37
 HST, 38, 79
 ST, 37, 78
闪存, 137, 138
图形, 139
灰度, 94
哈希表, 106
十六进制, 17, 18, 18
HP-ASM, 24, 24, 25
I/O 寄存器, 139
指令
 C=ID, 129
 CONFIG, 128
 RESET, 128
 UNCNFG, 129
指令清单
 ?A#0 f, 74
 ?A#B f, 75
 ?A#C f, 75
 ?A<=B f, 76
 ?A<B f, 75
 ?A=0 f, 74
 ?A=B f, 74
 ?A=C f, 74
 ?A>=B f, 75
 ?A>B f, 75
 ?ABIT=0 n, 76
 ?ABIT=1 n, 76
 ?B#0 f, 74
 ?B#C f, 75
 ?B<=C f, 76
 ?B<C f, 75
 ?B=0 f, 74
 ?B=C f, 74
 ?C=0 f, 74
 ?C=D f, 75
 ?C>=A f, 75
 ?C>A f, 75
 ?CBIT=0 n, 76
 ?CBIT=1 n, 76
 ?D#0 f, 74
 ?D<=C f, 76
 ?D<C f, 75
 ?D=0 f, 74
 ?D>=C f, 76
 ?D>=D f, 75
 ?D>C f, 75
 ?HST=0 n, 79
 ?HST=1.n, 83
 ?MP=0, 79
 ?MP=1, 83
 ?P# n, 76
 ?P= n, 76
 ?SB=0, 79
 ?SB=1, 83
 ?SR=0, 79
 ?SR=1, 83
 ?ST=0 n, 78
 ?ST=1 n, 78
 ?XM=0, 79
 ?XM=1, 83
 A=0 f, 42
 A=1 f, 52
 A=-A f, 54
 A=A!B f, 55
 A=A!C f, 55
 A=A%A f, 53
 A=A%B f, 53
 A=A%C f, 53
 A=A&B f, 56
 A=A&C f, 56
 A=A*A f, 51
 A=A*B f, 51
 A=A*C f, 51
 A=A/B f, 52
 A=A/C f, 52
 A=A^A f, 57
 A=A^B f, 56
 A=A^C f, 56
 A=A+1 f, 46
 A=A+A f, 48
 A=A+B f, 48
 A=A>B f, 59
 A=A>C f, 60
 A=A-1 f, 50
 A=-A-1 f, 54
 A=A-B f, 50
 A=A-c f, 53
 A=A-C f, 50
 A=B f, 45
 A=-B f, 55
 A=-B-1 f, 54
 A=B-A f, 50
 A=C f, 45
 A=-C f, 55
 A=-C-1 f, 54
 A=D f, 46
 A=-D f, 55
 A=-D-1 f, 54
 A=DAT0 f, 68
 A=DAT0 n, 69
 A=DAT1 f, 68
 A=DAT1 n, 69
 A=IN, 78
 A=PC, 72
 A=R0, 61
 A=R0 f, 61
 A=R1, 61
 A=R1 f, 61
 A=R2, 61
 A=R2 f, 61
 A=R3, 61
 A=R3 f, 62
 A=R4, 61
 A=R4 f, 62
 ABEX f, 45
 ABIT=0 n, 43
 ABIT=1 n, 43
 ACCESSSD, 82
 ACEX f, 45
 AD0EX, 64
 AD0XS, 68
 AD1EX, 64
 AD1XS, 68
 APCEX, 72
 AR0EX, 62
 AR0EX f, 62
 AR1EX, 62
 AR1EX f, 62
 AR2EX, 62

SAVPTR, 90, 92, 119, 122, 124, 142	?B>=C f, 75	A=A+c f, 49	AR2EX f, 62
SKIPOB, 122, 125	?B>C f, 75	A=A+C f, 48	AR3EX, 62
	?C#0 f, 74	A=A+D f, 48	AR3EX f, 62
	?C#D f, 75	A=A<A f, 59	AR4EX, 62
	?C<=A f, 76	A=A<B f, 59	AR4EX f, 62
	?C<A f, 75	A=A<C f, 59	ARMSAT, 83
		A=A>A f, 60	ARMSYS, 83
ASL f, 57	BSRB f, 58	C=-D-1 f, 54	D=-C f, 55
ASLC, 57	BSRC, 58	C=DAT0 f, 68	D=C%D f, 52
ASR f, 57	BUSCB, 81	C=DAT0 n, 69	D=C*D f, 51, 51
ASRB, 58	BUSCC, 81	C=DAT1 f, 68	D=C/D f, 52, 52
ASRB f, 58	BUSCD, 81	C=DAT1 n, 69	D=C^D f, 57
ASRC, 58	C+P+1, 76	C=ID, 81	D=C<D f, 58
AUTOTEST, 83	C=0 f, 42	C=IN, 78	D=C>D f, 59, 59
B=0 f, 42	C=1 f, 52	C=P n, 76	D=-C-1 f, 54
B=1 f, 52	C=A f, 45	C=PC, 72	D=C-D f, 51
B=A f, 45	C=-A f, 55	C=R0, 61	D=-D f, 55
B=-A f, 55	C=-A-1 f, 54	C=R0 f, 62	D=D!C f, 55
B=-A-1 f, 54	C=A-C f, 51	C=R1, 61	D=D%C f, 53
B=-B f, 54	C=B f, 45	C=R1 f, 62	D=D%D f, 53
B=B!A f, 55	C=-B f, 55	C=R2, 61	D=D&C f, 56
B=B!C f, 55	C=-B-1 f, 54	C=R2 f, 62	D=D*C f, 51
B=B%A f, 52	C=-C f, 54	C=R3, 61	D=D*D f, 52
B=B%B f, 53	C=C!A f, 55	C=R3 f, 62	D=D/C f, 52
B=B%C f, 53	C=C!B f, 55	C=R4, 61	D=D^C f, 57
B=B&A f, 56	C=C!D f, 56	C=R4 f, 62	D=D^D f, 57
B=B&C f, 56	C=C%A f, 52	C=RSTK, 77	D=D+1 f, 46
B=B*A f, 51	C=C%B f, 53	C=ST, 78	D=D+A f, 48
B=B*B f, 51	C=C%C f, 53	CBIT=0 n, 43	D=D+B f, 48
B=B*C f, 51	C=C%D f, 53	CBIT=1 n, 43	D=D+c f, 49
B=B/A f, 52	C=C&A f, 56	CD0EX, 64	D=D+C f, 48
B=B/C f, 52	C=C&B f, 56	CD0XS, 68	D=D+D f, 48
B=B^A f, 56	C=C&D f, 56	CD1EX, 64	D=D<C f, 59
B=B^B f, 56	C=C*A f, 51	CD1XS, 68	D=D<D f, 59
B=B^C f, 56	C=C*B f, 51	CDEX f, 45	D=D>C f, 60
B=B+1 f, 46	C=C*C f, 51	CLRST, 78	D=D>D f, 60
B=B+A f, 48	C=C*D f, 51	CLRHST, 79	D=D-1 f, 50
B=B+B f, 48	C=C/A f, 52	CLRST, 78	D=-D-1 f, 54
B=B+c f, 49	C=C/B f, 52	CONFIG, 81	D=D-c f, 53
B=B+C f, 48	C=C/D f, 52	CONFIGD, 82	D=D-C f, 50
B=B+D f, 48	C=C^A f, 56	CPCEX, 72	D0= hex, 63
B=B<A f, 59	C=C^B f, 56	CPEX, 76	D0=A, 64
B=B<B f, 59	C=C^C f, 57	CR0EX, 62	D0=AS, 64
B=B<C f, 59	C=C^D f, 57	CR0EX f, 63	D0=C, 64
B=B>A f, 59	C=C+1 f, 46	CR1EX, 62	D0=CS, 64
B=B>B f, 59	C=C+A f, 48	CR1EX f, 63	D0=D0- n, 64
B=B>C f, 60	C=C+B f, 48	CR2EX, 62	D0=D0+ n, 64
B=B-1 f, 50	C=C+c f, 49	CR2EX f, 63	D1= hex, 63
B=-B-1 f, 54	C=C+C f, 48	CR3EX, 62	D1=A, 64
B=B-A f, 50	C=C+D f, 48	CR3EX f, 63	D1=AS, 64
		CR4EX, 62	

B=B-c f, 53	C=C<A f, 59	CR4EX f, 63	D1=C, 64
B=B-C f, 50	C=C<B f, 59	CRTMP, 83	D1=CS, 64
B=C f, 45	C=C<C f, 59	CSL f, 57	D1=D1-n, 64
B=-C f, 55	C=C<D f, 59	CSLC, 57	D1=D1+n, 64
B=-C-1 f, 54	C=C>A f, 59	CSR f, 57	DAT0=A f, 69
B=C-B f, 51	C=C>B f, 59	CSRB, 58	DAT0=A n, 69
B=D f, 46	C=C>C f, 60	CSRB f, 58	DAT0=C f, 69
B=-D f, 55	C=C>D f, 60	CSRC, 58	DAT0=C n, 69
B=-D-1 f, 54	C=C-1 f, 50	CSTEX, 78	DAT1=A f, 69
BCEX f, 45	C=-C-1 f, 54	D=0 f, 42	DAT1=A n, 69
BEEP2, 83	C=C-A f, 50	D=1 f, 52	DAT1=C f, 69
BIGAPP?, 83	C=C-B f, 50	D=A f, 45	DSL f, 57
BSL f, 57	C=C-c f, 53	D=B f, 46	DSLC, 58
BSLC, 57	C=C-D f, 50	D=-B f, 55	DSR f, 57
BSR f, 57	C=D f, 45	D=-B-1 f, 54	DSRB, 58
BSRB, 58	C=-D f, 55	D=C f, 46	DSRB f, 58
DSRC, 58	R3=A, 60	小端序, 34	库数据, 95
GETTIME, 82	R3=A f, 61	逻辑运算, 21, 55	链接数组, 108
GOC label, 71	R3=C, 60	AND, 22, 56	列表, 99
GOLONG label, 71	R3=C f, 61	NAND, 23	局部名称, 92
GONC label, 71	R4=A, 60	NOR, 23	长复数, 108
GOSBVL label, 73	R4=A f, 61	NOT, 23	长实数, 94
GOSLOW, 83	R4=C, 60	OR, 22, 55	小字体, 97
GOSUB label, 73	R4=C f, 61	XOR, 23	实数, 85
GOSUBL label, 73	REFRESHD, 83	逻辑 XOR, 56	RPL 程序, 100
GOTO label, 71	REMOFF, 82	循环, 47	字符串, 86
GOVLNG label, 72	REMON, 82	LSB, 13, 34	符号矩阵, 111
HSCREEN, 83	RESET, 81	宏, 122	系统二进制整数, 94
HST=0 n, 79	RESETOS, 83	MASD, 27, 28, 29	标记对象, 102
HST=1.n, 83	RPL2, 82	掩码, 19	单位, 102
INTOFF, 80	RSI, 80	内存, 127	用户二进制整数: 见“二进制整数”
INTON, 80	RSTK=C, 77	读取, 68, 69	XLIB, 94
KEYDN, 83	RTI, 73	写入, 69, 69	ZINT, 96
LA, 41	RTN, 73	MiniWriter, 26	ON 键, 115
LC, 41	RTNC, 73	模块, 127	反码, 15, 53
MIDAPP?, 83	RTNCC, 73	取模, 52	操作码, 41, 42
MOVEDN, 83	RTNNC, 73	最高有效位: 见 MSB	指针, 63, 64
MOVEUP, 83	RTNSC, 73	MSB, 13, 34	原始代码对象, 111
MP=0, 79	RTNSXM, 73	乘法, 16, 51	寄存器, 31
MP=1, 83	SB=0, 79	负数, 14	IN, 38, 77
NATIVE?, 83	SB=1, 83	半字节, 11	OUT, 38, 77
NOP3, 82	SERIAL, 83	半字节移位, 57	P, 75
NOP4, 82	SETDEC, 80	对象, 84	指针寄存器, 32
NOP5, 82	SETFLDn, 83	代数对象, 101	保存寄存器, 33, 60
OFF, 82	SETHEX, 80	数组, 98	工作寄存器, 33, 60
OUT=C, 77	SETLNED, 83	备份, 107	ROM 存储体, 136
OUT=CS, 77	SETOFFD, 83	二进制整数, 92	RSTK, 37, 77
OUTBYT, 83	SETTIME, 83	字符, 95	Saturn+, 39, 40, 81
P= n, 76	SHUTDN, 81	代码对象, 95	有符号数, 14
P=C n, 76	SR=0, 79	复数, 98	

P=P+1, 76	SR=1, 83	目录, 103	堆栈, 117
P=P-1, 76	SREQ?, 81	扩展指针, 96	粘滞位, 38, 57, 58, 79, 80
PC=(A), 72	ST=0 n, 78	扩展实数: 见“长实数”	StringWriter, 26
PC=(C), 72	ST=1 n, 78	闪存指针, 97	子例程, 72
PC=A, 72	ST=C, 78	字体, 97	减法, 14, 50, 53
PC=C, 72	UNCNFG, 81	全局名称, 89	符号对象: 见“代数对象”
PORTTAG?, 83	UNCNFGD, 82	图形, 93, 139	TED, 26
R0=A, 60	WSCREEN, 83	GROB: 见“图形”	测试, 70, 73
R0=A f, 60	XM=0, 79	无限精度复数, 110	位测试, 76
R0=C, 60	XM=1, 83	无限精度整数: 见 ZINT	比较测试, 75
R0=C f, 61	中断, 80	无限精度实数, 109	相等测试, 74
R1=A, 60	跳转, 70	整数: 见 ZINT	零测试, 74
R1=A f, 60	条件跳转, 70	库, 105	补码, 15, 54
R1=C, 60	无条件跳转, 71		UFL, 26
R1=C f, 61	组合按键, 114		
R2=A, 60	键盘, 112		
R2=A f, 60	标签, 47		
R2=C, 60	最低有效位: 见 LSB		
R2=C f, 61			